

I♥A.I. 제이펍의 인공지능 시리즈 01
Jpub's A.I. Series

기초부터 고급 이론까지
체계적으로 정리하여
알기 쉬운 딥 러닝 교과서

Deep Learning

딥러닝

비제대로 시작하기

오카타니 타카유키 지음 / 심호섭 옮김

Jpub
제이펍

Deep Learning

© Takayuki Okatani 2015

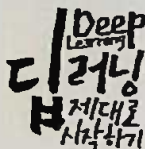
All rights reserved.

Original Japanese edition published by KODANSHA LTD.

Korean translation rights arranged with KODANSHA LTD.

through Shin Won Agency Co.

이 책의 한국어판 저작권은 신원 에이전시를 통한 저작권자와의 독점 계약으로 제이펍에 있습니다.
저작권법에 의해 한국 내에서 보호를 받는 저작물이므로 무단 전재와 복제를 금합니다.



초판 1쇄 발행 2016년 10월 10일 4쇄 발행 2018년 10월 10일

자문이 오카타니 타카유키

옮긴이 심효섭

펴낸이 강성두

펴낸곳 주식회사 제이펍

출판신고 2009년 11월 10일 제406-2009-000087호

주소 경기도 파주시 회동길 159 3층 3-B호

전화 070-8201-9010 / 팩스 02-6280-0405

홈페이지 www.jpub.kr / 원고투고 jeipub@gmail.com

독자문의 readers.jpub@gmail.com / 교재문의 jeipubmarketer@gmail.com

편집부 이종무, 황해나, 최병찬, 이 승, 이주원 / 소통·기획팀 민지환 / 회계팀 김유미

용지 신승지류유동 / 인쇄 한승인쇄 / 재본 광우제책사

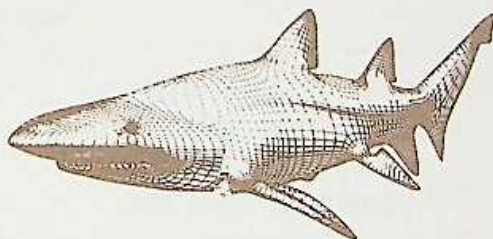
ISBN 979-11-85890-59-3 (93000)

값 20,000원

- ※ 이 책은 저작권법에 따라 보호를 받는 저작물이므로 무단 전재와 무단 복제를 금지하며,
이 책 내용의 전부 또는 일부를 이용하려면 반드시 저작권자와 제이펍의 서면동의를 받아야 합니다.
- ※ 잘못된 책은 구입하신 서점에서 바꾸어 드립니다.

제이펍은 독자 여러분의 아이디어와 원고 투고를 기다리고 있습니다. 앞으로 제이펍이 하는 아이디어를 원고로 있으신
분들은 쪽지나 이메일, 전화, 팩스, 소셜 미디어 등 다양한 방법으로 연락주세요.
jeipub@gmail.com

Deep
Learning
딥러닝
이제대로
시작하기



오카타니 타카유키 지음 / 심효섭 옮김

Jpub
제이퍼블

※ 드리는 말씀

- 이 책에 기재된 내용을 기반으로 한 운용 결과에 대해 저자, 역자, 소프트웨어 개발자 및 제공자, 제이펍 출판사는 일체의 책임을 지지 않으므로 양해 바랍니다.
- 이 책에 등장하는 각 회사명, 제품명은 일반적으로 각 회사의 등록 상표 또는 상표입니다. 본문 중에는 ™, ©, ®, 마크 등이 표시되어 있지 않습니다.
- 이 책에서 사용하고 있는 제품 버전은 독자의 학습 시점이나 환경에 따라 책의 내용과 다룰 수 있습니다.
- 일본 책의 번역서이다 보니 현재 국내에서 두루 사용하고 있는 영어식 용어와 다르게 표현된 것들이 있습니다. 그러한 용어들은 기존 용어와의 혼동을 피하고자 처음 나올 때 영어를 병기하였으며, 기본적으로는 대한수학회 용어집을 많이 참조하였음을 알려 드립니다.
- 본문에서 컬러로 봐야 하는 그림은 출판사 웹사이트(www.jpub.kr)의 이 책 소개 페이지에서 확인하실 수 있습니다.
- 책 내용과 관련된 문의사항은 옮긴이나 출판사로 연락해 주시기 바랍니다.
 - 옮긴이: flourscent@gmail.com
 - 출판사: readers.jpub@gmail.com



차례

움긴이 머리말 x

머리말 xii

베타리더 후기 xiv

CHAPTER 1

시작하며

_1

1.1 신경망 연구의 역사 3

1.1.1 다층 신경망에 대한 기대와 실망 3

1.1.2 다층 신경망의 사전훈련 4

1.1.3 자질에 대한 학습 6

1.1.4 딥 러닝의 융성 7

1.2 이 책의 구성 9

CHAPTER 2

앞먹임

신경망

_11

2.1 유닛의 출력 13

2.2 활성화 함수 15

2.3 다층 신경망 18

2.4 출력층의 설계와 오차함수 21

2.4.1 학습의 열개 21

2.4.2 회귀 22

2.4.3 이진 분류 23

2.4.4 다클래스 분류 25

확률적 경사 하강법

_29

3.1	경사 하강법	31
3.2	확률적 경사 하강법	33
3.3	'미니배치'의 이용	35
3.4	일반화 성능과 과적합	36
3.5	과적합을 완화시키는 방법	38
	3.5.1 규제화 38	
	3.5.2 가중치 감쇠 39	
	3.5.3 드롭아웃 40	
3.6	학습을 위한 트릭	43
	3.6.1 데이터 정규화 43	
	3.6.2 데이터 확장 45	
	3.6.3 여러 신경망의 평균 45	
	3.6.4 학습률의 결정 방법 46	
	3.6.5 모멘텀 48	
	3.6.6 가중치의 초기화 49	
	3.6.7 샘플의 순서 50	

역전파법

_53

4.1	기울기 계산의 어려움	55
4.2	2층으로 구성된 신경망의 계산	57
4.3	다층 신경망으로 일반화	60
4.4	경사 하강법의 전체 알고리즘	63
	4.4.1 출력층의 델타 63	
	4.4.2 순전파와 역전파의 행렬 계산 65	
	4.4.3 기울기의 차분근사 계산 67	
4.5	기울기 소실 문제	68

자기부호화기

_71

5.1	개요	73
5.2	자기부호화기의 설계	74
5.2.1	출력층의 활성화 함수와 오차함수	74
5.2.2	가중치 공유	75
5.3	자기부호화기의 동작	76
5.3.1	데이터를 나타내는 특징을 학습	76
5.3.2	주성분 분석과의 관계	78
5.4	희소 규제화	80
5.4.1	데이터의 과완비한 표현	80
5.4.2	최적화	83
5.4.3	희소 규제화의 효과	85
5.5	데이터의 백색화	87
5.6	딥 뉴럴넷의 사전훈련	92
5.7	그 외의 자기부호화기	94
5.7.1	심층 자기부호화기	94
5.7.2	디노이징 자기부호화기	95

합성곱 신경망

_99

6.1	단순 세포와 복잡 세포	101
6.2	전체적인 구조	104
6.3	합성곱	105
6.3.1	정의	105
6.3.2	합성곱의 작용	106
6.3.3	패딩	107
6.3.4	스트라이드	108
6.4	합성곱층	109
6.5	풀링층	112
6.6	정규화층	115
6.6.1	국소 콘트라스트 정규화	115
6.6.2	단일 채널 이미지의 정규화	116
6.6.3	다채널 이미지의 정규화	118
6.7	기울기의 계산	119
6.8	실제 예: 물체 유형 인식	121

재귀 신경망

_135

7.1	연속열 데이터의 분류	137
7.2	RNN의 구조	139
7.3	순전파 계산	142
7.4	역전파 계산	144
7.5	장·단기 기억	147
	7.5.1 RNN의 기울기 소실 문제	147
	7.5.2 LSTM의 개요	148
	7.5.3 순전파 계산	149
	7.5.4 역전파 계산	151
7.6	입력과 출력의 연속열 길이가 다른 경우	153
	7.6.1 은닉 마르코프 모델	153
	7.6.2 커넥션리스트 시계열 분류	154

볼츠만 머신

_161

8.1	데이터의 생성 모델	163
8.2	볼츠만 머신	164
	8.2.1 확률적 구조	164
	8.2.2 학습	166
8.3	깁스 샘플링	168
8.4	은닉 변수를 갖는 볼츠만 머신	170
	8.4.1 확률적 구조	170
	8.4.2 학습	171
8.5	제한 볼츠만 머신	173
	8.5.1 확률적 구조	173
	8.5.2 조건부 분포	174
	8.5.3 RBM과 자기부호화기	175
8.6	RBM의 학습	176
	8.6.1 깁스 샘플링을 사용한 기울기 계산	176
	8.6.2 대조적 발산(CD)	178
	8.6.3 CD의 실제 적용	180
	8.6.4 지속적 CD	182

8.7. 그 외의 유닛	183
---------------------	-----

8.7.1 가우시안 유닛	183
---------------	-----

8.7.2 ReLU	184
------------	-----

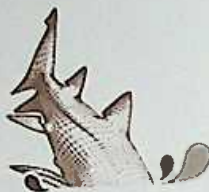
8.8. 딥 빌리프 네트워크	186
------------------------	-----

8.9. 딥 볼츠만 머신	188
----------------------	-----

8.10. 성능 비교	191
--------------------	-----

참고문헌	194
------	-----

찾아보기	199
------	-----



웁긴이 머리말

인공지능의 대부라 불리는 제프리 힌턴(Geoffrey Hinton)이 2006년 사전훈련에 대한 발표를 딥 러닝의 본격적인 시작으로 본다면, 올해가 꼭 10년째입니다. 그동안 딥 러닝은 영상이나 언어 처리 등 다양한 곳에서 해당 분야 최신 기법이 가졌던 성능 최고 기록을 차례차례 경신해 왔으며, 이제는 일반인에게도 그리 낯설지 않은 단어가 되었습니다. 그래서인지 딥 러닝을 배우고자 하는 분들을 최근 온/오프라인에서 자주 뵙습니다. 하지만 누구나 결정판이라 인정할 만한 교과서가 아직 나와 있지 않았습니다. 벤지오(Bengio) 교수의 교과서 원고가 인터넷에 공개되어 있긴 하지만 현시점에 서도 아직 미출간 상태입니다. 딥 러닝 자체가 다양한 신경망 기법을 뭉뚱그려 부르는 명칭이기 때문에 학습의 시작점이 될 만한 서적이 없었습니다.

솔직히 밝히자면, 일본에서 발간된 기술 서적을 처음부터 끝까지 읽어 본 것은 저도 이 책이 처음입니다. 제가 이전에 읽어 봤던 일본 서적은 대개는 역사나 사회과학 관련 책이었습니다. 그래서 일본 서적이라 하면 본질이 왜곡되는 경우가 잦은 핸드북 정도의 인상을 받고 있었습니다. 그러나 이 책은 제가 인공지능을 배우면서 시행착오를 겪었거나, '이걸 이렇게 배웠다면 좀 더 편했을 텐데'와 같은 많은 아쉬움을 해소해 주어 일본 책에 대한 인상까지 달리해 준 책입니다. 딥 러닝의 범주에 속하는 다양한 신경망 기법을 기본적인 원리부터, 시행착오가 있지 않고서는 알기 어려운 팁까지 잘 녹아 있고, 입문자뿐만 아니라 업무에 활용하려는 현업 개발자 모두에게 유용

한 구성을 택하고 있습니다. 그래서 딥 러닝에 관심 많은 분들에게 출발점이자 큰 그림이 되어 줄 지도와 같은 책이라는 생각이 들어 이 책의 번역을 맡게 되었습니다.

먼저, 이 책은 크게 딥 러닝 등장 이전과 이후에 해당하는 두 부분으로 나뉩니다. 1장부터 4장은 딥 러닝 등장 이전을, 5장부터 8장은 그 이후에 소개된 기법을 다룹니다. 전반부는 인공 신경망에 대한 기본적인 이해와 굴곡 많았던 연구사부터 출발합니다. 그런 다음, 딥 러닝 이전까지의 전통적인 인공 신경망 학습법과 함께 다시 그 한계점과 원인을 설명합니다. 후반부는 딥 러닝 기법에 속하는 자기부호화기, 합성곱 신경망, 재귀 신경망, 볼츠만 머신 등 여러 기법에 대해 이들이 해결할 수 있는 문제 유형과 함께 동작 원리에 대한 설명, 그리고 사용법을 차례차례 소개합니다. 구체적인 구현 예제가 없는 게 조금 아쉬운 부분이지만, 기본적 이론에 대한 이해는 충분히 얻을 수 있을 것입니다.

번역하면서 가장 신경 쓴 부분은 기존에 제가 영어로만 학습하였던 내용에 대해서 정확한 역어를 택하는 것이었습니다. 영어로만 알고 있던 용어나 기존에 알지 못하던 일본식 수학 용어에 대해서도 대한수학회 용어집이 많은 도움이 되었습니다. 그리고 출간 전에 미처 발견하지 못한 오류를 지적해 주시고 여러 제안을 해 주신 베타리더들께도 감사의 인사를 드립니다. 마지막으로 부모님과 아내 정이, 그리고 좋은 책의 번역을 맡겨 주시고 초보 역자를 잘 이끌어 주신 제이펍 장성두 실장님께 감사를 드립니다.

옮긴이 심효섭



머리말

딥 러닝은 다층 신경망을 사용한 기계학습 방법론인데, 최근 음성이나 영상 또는 언어를 대상으로 하는 인공지능의 여러 문제에 대해 다른 방법론을 압도하는 높은 성능을 보여 많은 연구자와 기술자들이 관심을 보이고 있다. 인공지능에 대한 기대가 최근 전례 없이 높아지고 있는 것도 바로 딥 러닝의 성공이 있기 때문이다.

신경망에 대한 연구 자체는 80년대부터 이미 왕성하게 진행되고 있었다. 그러나 90년대 중반부터 불과 얼마 전까지 연구에 대한 관심이 극도로 떨어지던 'AI winter'라 불리던 시기가 있었다. 이 시기가 막 시작될 무렵 발전한 서포트 벡터 머신과 비교하면 신경망은 그 성능을 끌어내기 위해 수많은 파라미터를 조절해야 했고, 이론적인 뒷받침이 부족했던 결점이 있었기 때문에 상당수가 신경망에 대한 관심을 잃어갔다.

이 겨울과 같은 시기를 견디고 이제 다시 신경망이 주목받고 있다. 그 계기가 된 것은 다층 신경망 학습에 대한 힌턴(Hinton)의 연구였다. 그리고 90년대에 비해 컴퓨터의 성능이 크게 향상되었고, 웹이나 검색엔진 등의 발전에 따라 대규모 훈련 데이터의 수집이 쉬워진 것도 높은 성능을 달성할 수 있게 하였다. 최근에는 일반 매스컴에서도 딥 러닝의 장밋빛 미래가 종종 언급되고 있다. 연구가 더 진전되면 언젠가는 사람에 필적할 만한 인공지능을 실현할 수 있을 거라는 이야기가 있는 반면, 인공지능이 사람에게 위협이 될 거라는 우려도 나오고 있다.

그러나 지금의 열광은 앞서 이야기한 'AI winter'의 반작용이라 할 만한 부분이 있다. 아무도 주목하지 않았던 신경망이 몇 가지 어려운 문제를 보기 좋게 해결해 보인 것이다. 관련 분야 연구자들은 상당히 큰 충격을 받았다. 그러나 다시 잘 생각해 보면 할 수 있는 것이 꽤 늘었지만, 사람의 지능 수준을 목표로 한다면 아직 이루지 못하는 게 압도적으로 많은 것도 사실이다. 열광과 무관심이 교차하던 과거에 비해 지금은 신경망에 대한 기대가 너무 높아진 시기일지도 모른다.

상황이 상황이니만큼 바로 지금 한 발 한 발 나아가는 착실한 연구 개발이 필요하다. 딥 러닝의 높은 성능은 누구나 아는 것이 되었지만, 이론적인 이해는 아직 이를 쫓아가지 못하고 있다. 예를 들면, '층수가 늘어나면 왜 높은 성능이 나오는가'라는 물음에 대해 수궁할 만한 대답이 아직 나와 있지 않다. 각 과제에 대한 효과도 해 보지 않으면 알 수 없으며, 잘 안 되는 경우도 그 이유조차 아직 알 수 없다. 이런 점만 보자면, 80년대에서 발전된 것이 그다지 없다고도 말할 수 있다. 연구의 진전을 계속 바랄 뿐이다.

마지막으로, 이 책을 쓰면서 도움을 받은 분들에게 감사드린다. 아소 히데키 님과 야스다 무네키 님으로부터는 이 책의 초고 단계부터 많은 조언과 도움을 받았다. 고단샤 사이언티픽의 요코야마 신고 님은 집필 시작부터 출판에 이르기까지의 모든 지원을 해 주었다. 또, 시리즈 편집자인 스기야마 마사시 님께는 뛰어난 집필진과 함께 일할 귀중한 기회를 주신 것에 이 자리를 빌려 다시 감사드린다.

오카타니 타카유키



베타리더 후기

강대명(카카오)

딥 러닝의 기초를 어느 정도 알고 있는 분들에게는 딥 러닝의 원리를 수학적으로 잘 설명해 주는 좋은 책인 것 같습니다. 머신 러닝에 대한 아주 기초적인 지식은 있다고 생각하고 딥 러닝 책을 선택했는데, 저에게는 다소 어려웠습니다. 기본적으로 관련 수학 지식과 딥 러닝에 대한 이해가 선행되어야 할 것 같습니다. 그리고 용어의 한글화가 많이 되어 있었는데, 영문 병행 표기 등을 통해 한글 용어에 낯선 독자들도 접근하기 쉽게 수정되었으면 좋겠습니다.

김지연(Eastern Michigan University)

딥 러닝에 대한 책을 처음 접해 보았습니다. 생각보다 많은 수식과 수학적 지식이 있어야 해서 그런지 날카로운 피드백을 하지 못한 것 같아 조금 아쉽네요. 마지막 베타리딩까지 이렇게 끝내고 나니 배운 게 더 많은 것 같아 감사할 따름입니다. 베타리딩이 작게나마 도움이 되어 더 많은 분이 재미있는 도서를 읽게 되길 바랍니다!

박재유(KAIST 소프트웨어대학원)

구글이나 바이두 등의 선두주자들은 최근 저마다 딥 러닝 프레임워크를 개발하여 오픈소스로 공개했습니다. 그러나 무작정 딥 러닝 코드를 작성하기 이전에 반드시

수학적인 배경(확률론과 통계분석)에 대한 선수 지식을 이해해야 자유로운 응용이 가능합니다. 이 책은 먼저 다양한 학습 알고리즘을 직관적으로 소개하고, 그 내면의 수학적 정리를 친절하게 설명함으로써 독자 여러분이 사전훈련이나 파라미터 선정에 대한 감을 잡으실 수 있도록 안내합니다. 출간 전에 용어에 대한 영문 병행 표기 등을 철저히 하여 이해를 최대한 돕도록 하는 것이 좋을 것 같습니다.

서진구(삼성전자)

기반 알고리즘들의 수학적 유도 과정을 통해 딥 러닝에 대해 깊이 이해할 수 있게 해주는 책입니다. 머신 러닝 및 통계에 대한 기초 지식이 있는 분들이 딥 러닝의 이론적 배경을 배우기에 적당해 보입니다.

손은주(한양대학교)

이번 베타리딩 도서를 통해 딥 러닝의 과정과 원리를 이해하고 나니 기계학습 오픈소스 라이브러리를 사용하는 데 많은 도움이 되었습니다. 곁에 두고 자주 읽으며 학습할 만한 좋은 책이라는 생각이 듭니다. 이번 베타리딩에서는 개발보다는 수학 공부를 한 느낌이지만, 베타리딩이 아니었다면 만나지 못했을 좋은 도서를 접하게 되어 기쁩니다.

이주형(LG전자)

최근 트렌드를 이끄는 딥 러닝의 기본 모델들 하나하나를 근간이 되는 수식과 해설을 통해 잘 묘사하고 있습니다. 딥 러닝의 최신 트렌드나 실제 구현 방법 등을 알기에 적합한 책은 아니지만, 이론적 기초를 차근차근 닦기에는 이보다 친절하게 정리해 놓은 책은 없는 것 같습니다.

전태균(세트렉아이)

딥 러닝에 대한 영문 강의 자료와 튜토리얼은 쉽게 구할 수 있지만, 쉽고 체계적으로 접근할 수 있는 국문 자료는 아직 많이 부족합니다. 베타리딩을 통해 이 책을 보면서 느낀 점은 많은 분이 궁금해하던 딥 러닝의 동작 원리에 대한 기반 이론을 콤팩트

하게 담고 있습니다. 이 책은 딥 러닝의 이론적 체계가 필요한 개발자 및 학생들에게
유익할 것 같습니다.

최경호(강원대학교)

서점에서 만나게 되면 딥 러닝 코너에서 가장 얇은 책이 될 것 같습니다. 얇은 책임
에도 수식만으로 가득 차 있는 다른 책들과는 달리 친절한 그림이 많아 이해를 돕는
데 많은 도움이 되었습니다. 제가 본 영역에서는 그림이나 수식의 오류는 없었으며,
딥 러닝에 관심 있는 학부생이나 비전공자들이 보기에 정말 좋은 책이 될 것 같습니
다. 일반적으로 우리말 번역이 어려워 영어를 그대로 사용하는 전문 용어들을 한글
로 표현하려는 역자의 노력이 좋아 보였습니다. 다만, 일본 책을 옮긴 것이다 보니 베
타리딩을 할 당시에는 우리나라에서 잘 사용하지 않는 용어가 일부 있었는데, 가능
하면 이를 바로잡아 출간하였으면 합니다.

한상곤(우분투 한국 커뮤니티 소속 마이크로소프트 MVP)

마이크로소프트의 봇 프레임워크(Bot Framework)나 구글의 텐서플로(TensorFlow)를 활
용해서 인공지능 봇(Bot)을 쉽게 만들 수 있게 되었습니다. 그런데 가끔은 딥 러닝의
원리가 궁금할 때가 있는데, 그 궁금증을 이 책이 해결해 주었습니다. 딥 러닝 발전
사와 함께 알고리즘에 사용되는 수식을 글로 쉽게 풀어낸 솜씨 덕에 한결 쉽게 이해
할 수 있었습니다. 오랫동안 기다려서 진행된 베타리딩이었는데, 너무 재미있고 즐겁
게 읽을 수 있어서 좋았습니다(제가 전공이 인공지능이라 그렇지도). 제이펍에서 펴내는 다
음 인공지능 서적도 기대하겠습니다!

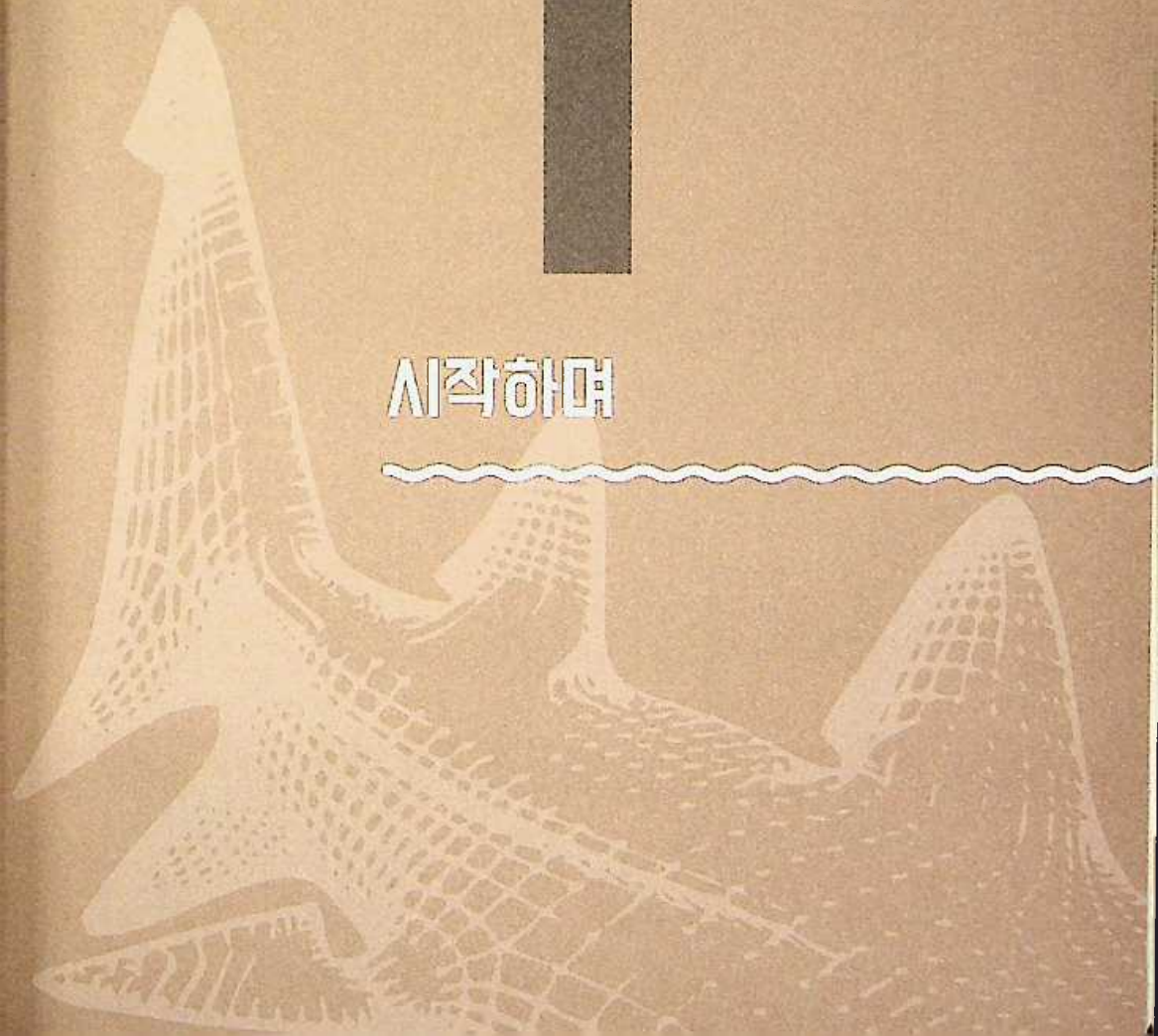


제이펍은 책에 대한 애정과 기술에 대한 열정이 뜨거운 베타리더들로 하여금
출간되는 모든 서적에 사전 검증을 시행하고 있습니다.

CHAPTER

1

시작하며





1.1 신경망 연구의 역사

1.1.1 다층 신경망에 대한 기대와 실망

높은 수준의 정보 처리를 위해 생물의 신경 회로망을 모방한 인공 신경망(이하 신경망)이 오랫동안 연구되었다. 그러나 이 연구의 역사는 평탄하지 않았다. 처음 연구가 시작^[24, 48, 57]된 1940년대부터 지금까지 이 연구는 두 번이나 봄을 이루었다가 열기가 식어버리는 과정을 반복했다. 그중에서도 다층 신경망의 학습법인 역전파법(backpropagation)의 고안^[59]을 계기로 한 두 번째 봄은 80년대 후반에서 90년대에 초반까지 연구에 큰 진전을 이루었다. 그러나 이 시기의 연구 봄도 90년대 후반에는 완전히 끝나버리고, 그 후 최근까지 신경망에 관한 연구는 매우 저조하였다^[64].

80년대 중반부터 일어났던 이 두 번째 봄이 끝나게 된 원인으로는 두 가지를 꼽을 수 있다.

첫 번째로, 역전파법에 의한 신경망의 학습이 그림 1-1과 같은 2층 정도의 신경망에서는 잘 이루어졌으나 그보다 층수가 많은 신경망에서는 기대했던 결과를 얻을 수 없었기 때문이다. 훈련 샘플의 학습은 가능해도 일반화 성능이 오르지 않는, 즉 과적합(또는 과학습, overfitting)이 벽이 되었다. 역전파법이란, 샘플에 대한 신경망의 오차(error cost, 목표 출력과 실제 출력의 차이)를 다시 출력층에서부터 입력층으로 거꾸로 전파시켜 각 층의 가중치의 기울기를 계산하는 방법이다. 이 계산을 수행할 때 입력층에서 멀리 떨어진 ‘깊은(deep)’ 층에 이르게 되면 기울기가 급속히 작아지거나 너무 커져 발산(發散)해 버리는, 소위 기울기 소실(vanishing gradient) 문제라 불리는 현상이 일어나게 된다. 이 현상이 다층 신경망의 학습을 곤란하게 하는 원인이었다.

두 번째로, 신경망은 학습을 위한 여러 파라미터로 층수나 유닛의 수를 갖는데, 이 파라미터가 최종적으로 어떻게 성능으로 이어지는지를 알 수 없었다. 즉, 성능을 이끌어 내기 위해 이들 파라미터를 어떻게 결정해야 하는지에 대한

노하우는 있지만 이론이 없었다^[64]. 이 점 때문에 신경망 기법은 보다 나중에 제안된 머신 러닝(기계학습) 기법보다 뒤떨어진다는 평가를 받았고, 이러한 이유로 90년대 후반부터 신경망에 대한 연구는 완전히 식어버리게 되었다.

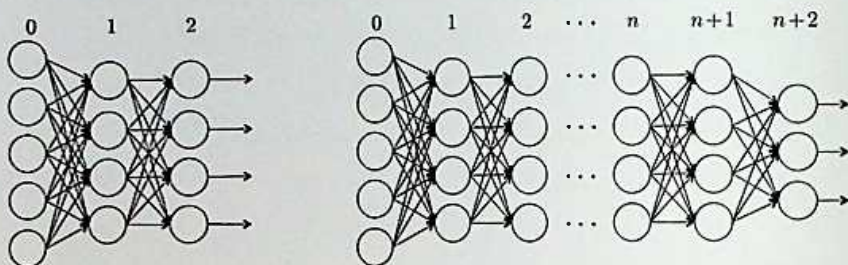


그림 1-1 '얕은' 신경망(2층)과 '깊은(딥)' 신경망(다층)

하지만 첫 번째 이유, 그러니까 여러 층으로 구성된 신경망의 학습이 어려웠다는 점에 대해서는 당시에도 한 가지 예외가 있었다는 사실을 지적하고자 한다. 층과 층 사이의 결합 밀도가 높은 다층 신경망의 학습이 어렵다는 것은 분명하였으나, 이미지를 대상으로 하는 신경망인 합성곱 신경망(Convolutional Neural Network, CNN)은 80년대 후반에는 무려 다섯 개 층으로 구성되는 신경망의 학습에도 성공한 예가 있었다^[42]. CNN은 특정한 이미지 처리를 수행하는 층이 여러 개 쌓인 구조를 갖는, 층간의 결합 밀도가 낮은 신경망이다. 후쿠시마 구니히코(Kunihiko Fukushima)의 연구진이 제안했던 네오코그니트론(Neocognitron)^[17]을 시발점으로, 얀 러쿤(Yann LeCun)의 연구진이 역전파법을 이용한 학습을 다시 도입하여 완성시켰다^[43]. 당시 러쿤이 제안했던 CNN은 필기 인식에 응용해서 얻은 높은 성능으로 잘 알려졌지만, 신경망에 대한 연구 열기가 식었던 당시 분위기와 맞물려 점차 관심을 잃어갔다.

1.1.2 다층 신경망의 사전훈련

그 후 90년대 후반부터 2000년대 초반까지 신경망 전반에 대한 관심도가 매우 낮았지만, 제프리 힌튼(Geoffrey Hinton)의 연구진이 발표한 딥 빌리프 네트워크(Deep

Belief Network, DBN)의 연구^[28]가 신경망 연구의 활기를 되살리는 돌파구가 되었다*. DBN은 일반적인 신경망과 유사하게 다층 구조를 갖는 그래프 모델(graph model)이다. 이 모델의 동작은 확률적으로 기술되며, 주로 데이터의 생성 모델로 쓰였다. 이에 대한 학습 과정은 일반적인 신경망과는 다른 원리에 기초하고 있었으며, 층수가 늘어나면 DBN의 학습도 역시 곤란해졌다.

힌턴은 이에 대한 대책으로, 먼저 각 층을 제약 볼츠만 머신(Restricted Boltzmann Machine, RBM)이라는 단층 신경망으로 분해한 뒤, 탐욕 알고리즘(Greedy Algorithm)과 같은 아이디어로 이들 RBM에 입력층과의 거리가 가까운 순으로 차례차례 비지도(unsupervised) 학습을 수행하는 방법을 제안하였다. 그리고 이 방법을 통해 DBN을 잘 학습시킬 수 있다는 것을 보였다. 나아가, 이 방법으로 학습시킨 DBN은 일반적인 앞먹임 신경망(feed-forward neural network)으로 다시 전환할 수 있으며, 이렇게 전환된 신경망은 여러 층으로 구성되어도 기존의 방법으로 성공적인 학습이 가능하다는 것을 발견하였다. 이와 같이, 사용하려는 신경망을 학습시키기 전에 층 단위의 학습을 거치는 것으로 더 나은 초깃값을 얻는 방법을 사전훈련(pre-training)이라 부른다.

그 후 DBN이나 RBM이 아닌, 그보다 더 단순한 자기부호화기(autoencoder)**를 사용하여도 다층 신경망의 사전훈련이 가능하다는 사실이 보고되었다^[4]. 자기부호화기는 입력으로부터 계산되는 출력이 입력 자체와 비슷해지도록 훈련되는 신경망이다. 다시 말해, 자기부호화기의 목표 출력은 입력 그 자체이며, 따라서 비지도 학습으로 학습이 이루어진다. 예를 들어, 임의의 단층 신경망을 가정하고 그 입력을 정방향으로 전파시켜 한 번 출력을 구한 뒤, 이번에는 이것을

* **참고** 2006년 Hinton의 논문에서 사전훈련이라 부르는 방법의 최초 아이디어가 제안되었는데, 이 문헌에서 제안된 최초의 사전훈련은 직층 RBM을 DBN으로 변환하는 형태로 이루어졌다.

** **참고** 일반적으로 국내에서는 autoencoder를 '오토인코더'라 부르는 경우가 많은데, 인공지능을 공부하면서 이해를 방해했던 몇 안 되는 원문 용어 중 하나였다. 왜냐하면 'auto'를 보통 '자동'이나 '스스로'로 해석하지 'self'로 생각하기가 어렵기 때문이다. 그래서 이 책에서는 autoencoder를 자기부호화기로 번역하였음을 밝힌다.

역방향으로 전파시켜 입력 측으로 되돌리는 조작을 한다고 하면, 이를 하나의 자기부호화기라고 할 수 있다. 자기부호화기를 사용하면, DBN과 같은 방법으로 다층 신경망의 사전훈련이 가능하다고 알려져 있다. 목적으로 하는 다층 신경망을 단층으로 분할한 뒤, 입력층과 가까운 순서대로 각 층을 위에서 설명한 자기부호화기로 보고 비지도 학습을 실시해서 각 층의 파라미터의 초깃값을 얻는다. 그 다음, 전체 신경망에 대한 지도 학습(supervised learning)을 실시한다. 다층 신경망은 파라미터를 랜덤하게 초기화하면 학습이 잘 되지 않지만, 이렇게 층마다 비지도 학습 형태의 사전훈련을 거친 뒤 얻은 파라미터를 초깃값으로 사용하면, 학습이 잘 된다는 사실이 알려지게 되었다.

1.1.3 자질에 대한 학습

이미지나 음성 등과 같은 자연계의 데이터는 일반적으로 고차원 공간에 존재하지만, 그 공간 전체에 걸쳐 무작위로 존재하는 것이 아니라 복잡하지만 어떤 강한 경향을 갖는 형태로 흩어져 있다고 볼 수 있다. 예를 들면, 풍경을 찍은 자연 이미지는 백색잡음(무작위로 생성한 신호로부터 변환한 이미지)과는 성질이 무척 다르다. 사전훈련의 성공 이후에 연구자들의 관심이 향한 곳은, 다층 신경망이 이와 같은 자연계 데이터를 학습했을 때 이러한 데이터가 갖는 구조가 신경망의 다층 구조 내에 어떤 식으로 포착되거나 표현될 것인가였다.

자연 이미지로부터 잘라낸 패치(작은 영역)의 집합을 대상으로 희소 부호화(sparse coding)를 통해 각 유형에 대한 사전(기저)을 학습시키면, 포유류 뇌의 초기(전단계*) 시각 피질(visual cortex)에서 추출한 듯한 가버 웨이블릿(Gabor Wavelet)과 같은 형태의 기저를 얻을 수 있다는 것이 이전부터 알려져 있었다^[53]. 자기부호화기에 잉여성을 갖는 기저 몇 가지의 조합으로 입력을 표현한다는 희소 부호화의 아이디어를 도입하면, 다층 신경망에는 학습에 의해 흥미로운 계층 구조가

* **역사** 시각 처리 단계의 앞 단계에 해당한다는 의미다. 원시적과는 다르다.

형성된다는 사실이 알려져 있다. 예를 들면, 이홍락(Honglak Lee) 교수의 연구진은 자연 이미지의 패치를 2층 이상의 신경망(RBM을 겹친 것)으로 학습하면, 영장류의 시각 피질 V2 영역에서 관찰되는 특징^[36]과 유사한 특징을 얻을 수 있다는 사실을 보고한 바 있다^[45]. 또, CNN의 구조를 추가한 DBN을 사용하여 사물을 찍은 자연 이미지를 학습하면, 이미지의 특징이 신경망의 층과 대응하는 계층성을 갖는 형태로 학습된다는 사실도 보고되었다^[46]. Quoc Viet Le(스탠포드 대학교 교수)는 이와 같은 구조를 갖는 더 큰 규모의 신경망으로 자기부호화기를 구성하고, 유튜브의 동영상에서 무작위로 선택한 100만 장의 이미지를 학습시키면, 특정한 물체에만 선택적으로 반응하는 유닛^{*}이 자연적으로 생성된다는 것을 보인 바 있다^[41].

1.1.4 딥 러닝의 융성

그 후, 음성 인식이나 이미지 인식에 대한 벤치마크 테스트를 통해 다층 신경망의 유효성이 확인되고, 이전의 성능 기록을 속속 갱신하게 되었으며, 딥 러닝(deep learning, 심층학습)의 유효성이 널리 알려지게 되었다(이후에 딥 러닝이 대상으로 하는 다층 신경망을 딥 뉴럴넷이라 한다).

그러나 딥 러닝이라 뭉뚱그려 부르는 것도 실제로는 몇 가지 서로 다른 방법론의 집합으로, 문제에 따라 다르게 사용된다는 점에 주의해야 한다. 이를테면, 음성 인식에서는 층과 층 사이의 유닛이 모두 결합한 신경망이 많이 사용되며, 위에서 다뤘던 사전훈련을 거치는 것이 일반적이다. 그와는 다르게 이미지 인식에서는 CNN이 주류를 이루며, 이들 CNN은 일반적으로 사전훈련을 필요로 하지 않는다. 또, 자연 언어 처리나 음성 인식의 일부 과제에서는 재귀 신경망(Recurrent Neural Network, RNN)이 사용된다. 겉만 보면 모두 다층 신경망이라고

* 뇌의 물체를 인식하는 작용과 그 내부 표현에 대한 신경과학계의 오래된 가설인 '할머니 세포'와 같은 것. 여기서 할머니 세포란, 피질에 있을 것으로 가정되는, 할머니 얼굴처럼 매우 구체적인 대상에 반응하는 세포에 대한 이름을 말한다.

할 수 있지만, 그 열개와 학습 방법에 있어서는 상당한 차이가 있다.

그러나 여러 분야에서 최근 다층 신경망이 이렇게 높은 성능을 발휘할 수 있었던 배경에는 공통적인 이유가 있다. 먼저, 실제 세계의 문제는 매우 복잡하여 이를 풀기 위해서는 그 복잡도에 걸맞은 규모의 신경망이 필요하다. 그리고 그렇게 큰 신경망을 과적합 없이 학습시키려면 (사전훈련이 아무리 유용하다 하더라도) 일정한 규모 이상의 데이터가 필요하다. 지금은 90년대와 상황이 달라져서 웹을 비롯한 인프라가 잘 갖추어져 있고 충분한 양의 학습 데이터를 수집할 수 있다. 마찬가지로, 90년대와 비교하면 GPU나 멀티코어 프로세서의 발달, 나아가 이를 대규모로 묶은 클러스터에서 볼 수 있듯이 컴퓨터의 연산 능력이 비약적으로 향상되었다¹⁾. 이러한 요소가 딥 뉴럴넷이 성공하기 위한 필요조건이었다고 할 수 있을 것이다.

또, 사전훈련은 딥 러닝 붐의 계기가 되었지만, 최근의 경향으로는 필수적인 것만은 아니다. 주로 이미지 인식에 한정되어 사용되기는 하지만, CNN은 처음부터 사전훈련을 필요로 하지 않는다. 현재 쓰이고 있는 CNN은 80년대 말에 얀 러쿰의 연구진이 제안했던 신경망 구조와 학습 방법을 거의 그대로 이어받고 있다. 음성 인식 등에서 주류가 되는 전결합형 신경망에서도 드롭아웃처럼 층 단위로 이뤄지는 사전훈련에 의지하지 않고도 기울기 소실 문제나 과적합을 피할 수 있는 방법이 몇 가지 제안된 바 있다. 게다가, 가중치의 초깃값을 신중히 선택하는 것만으로도 이전의 예상보다 학습이 잘 이루어지는 결과가 보고되고 있다. 어떻게 보면, 최근의 딥 뉴럴넷의 성공의 본질은 실제 세계의 대규모 문제를 상대로 그저 다층 신경망을 시험 삼아 적용해 보았다가 예기치 못한 성능 향상을 얻은 것이라 볼 수도 있다.

1.2 이 책의 구성

2장에서는 앞먹임 신경망을 다룬다. 이는 입력에서 출력까지 정해진 한 방향으로만 정보가 전달되는 신경망으로, 가장 기본적인 널리 응용되는 신경망이다.

3장에서는 앞먹임 신경망의 학습 방법을 설명한다. 특히, 딥 뉴럴넷에서 기본적인 학습 방법으로 활용되는 확률적 경사 하강법에 초점을 맞추고 기초 이론과 방법을 설명한다. 이때 최적화하는 대상이 되는 오차함수의 최소화를 위해 신경망의 파라미터에 대한 미분(오차 기울기)을 구해야 한다. 4장에서는 이 오차 기울기를 구하기 위한 방법으로 역전파법을 설명한다. 5장은 자기부호화기를 다룬다. 이는 비지도 학습을 하는 신경망으로, 주로 데이터를 잘 나타내는 자질(feature)을 학습하고 데이터에 대한 좋은 표현을 얻는 것을 목적으로 한다. 열쇠가 되는 희소 규제화나 데이터 백색화에 대해서도 설명한다. 또, 자기부호화기를 이용한 딥 뉴럴넷의 사전훈련 방법도 다룰 것이다.

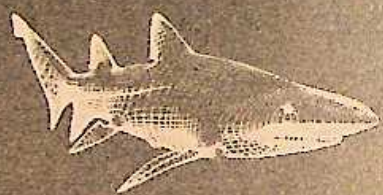
6장에서는 이미지에 대한 응용에서 빼놓을 수 없는 합성곱 신경망(CNN)을 다룬다. 합성곱층과 풀링층 등 특수한 구조를 갖는 층에 대해서 자세히 설명하고 구체적인 응용 예를 소개한다. 7장은 재귀 신경망(Recurrent Neural Network, RNN)을 다룬다. 이는 가변길이 연속열 데이터를 효율적으로 다루기 위한 신경망이다. 재귀 신경망을 위한 기본적인 구성과 학습 방법을 설명하고, 음성 인식, 필기 인식 등의 성공적으로 응용되고 있는 장·단기 기억(Long Short-Term Memory, LSTM)과 입력 연속열과 길이가 다른 연속열을 추정할 수 있는 커넥서니스트 시계열 분석법(CTC) 등을 설명한다. 그리고 8장에서는 볼츠만 머신을 다룬다. 다른 장에서 다른 신경망과 달리, 유닛 간에 양방향성 결합을 가지며 그 거동이 확률적으로 기술되는 것이 특징이다. 데이터의 생성 모델로 이용되는 것 외에도 앞먹임 신경망으로 전환되어 쓰이기도 한다.

1. ...
 2. ...
 3. ...
 4. ...
 5. ...
 6. ...
 7. ...
 8. ...
 9. ...
 10. ...
 11. ...
 12. ...
 13. ...
 14. ...
 15. ...
 16. ...
 17. ...
 18. ...
 19. ...
 20. ...
 21. ...
 22. ...
 23. ...
 24. ...
 25. ...
 26. ...
 27. ...
 28. ...
 29. ...
 30. ...
 31. ...
 32. ...
 33. ...
 34. ...
 35. ...
 36. ...
 37. ...
 38. ...
 39. ...
 40. ...
 41. ...
 42. ...
 43. ...
 44. ...
 45. ...
 46. ...
 47. ...
 48. ...
 49. ...
 50. ...
 51. ...
 52. ...
 53. ...
 54. ...
 55. ...
 56. ...
 57. ...
 58. ...
 59. ...
 60. ...
 61. ...
 62. ...
 63. ...
 64. ...
 65. ...
 66. ...
 67. ...
 68. ...
 69. ...
 70. ...
 71. ...
 72. ...
 73. ...
 74. ...
 75. ...
 76. ...
 77. ...
 78. ...
 79. ...
 80. ...
 81. ...
 82. ...
 83. ...
 84. ...
 85. ...
 86. ...
 87. ...
 88. ...
 89. ...
 90. ...
 91. ...
 92. ...
 93. ...
 94. ...
 95. ...
 96. ...
 97. ...
 98. ...
 99. ...
 100. ...

2

앞먹임 신경망

앞먹임 신경망은 가장 기본적인데서 가장 많이 사용되는 신경망이다. 이 장에서는 신경망의 구성 요소와 회귀나 분류를 분류 등 문제의 유형마다 다른 출력층에 대한 실제 방편과 오차전수의 선택 등 구체적인 사항을 설명한 것이다.



2.1 유닛의 출력

앞먹임 신경망(feedforward neural network)*은 층 모양으로 늘어선 유닛이 인접한 층(layer)들과만 결합하는 구조를 가지며, 정보가 입력 측으로부터 출력 측으로 한 방향으로만 흐르는 신경망이다. 문헌에 따라서는 다층 퍼셉트론(multi-layer perceptron)이라 부르기도 한다**.

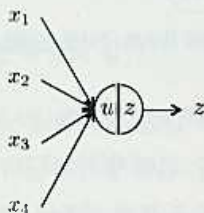


그림 2-1 유닛 한 개의 입출력. $u = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$, $z = f(u)$

신경망을 구성하는 각 유닛은 그림 2-1과 같이 복수의 입력을 받아 하나의 출력을 계산한다. 그림 2-1은 네 개의 입력 x_1, x_2, x_3, x_4 를 받지만, 이 유닛이 받는 총 입력 u 는 다음과 같이 된다.

$$u = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

각 입력마다 다른 가중치(weight) w_1, w_2, w_3, w_4 를 곱한 값을 모두 합하고, 이에 바이어스(bias)***라 불리는 값을 더한 값이 된다. 이 유닛의 출력 z 는 총 입력 u 에 대한 활성화 함수(activation function)라 불리는 함수 f 의 함수값이다.

* 피드포워드 뉴럴넷이라는 표기도 많이 쓰인다.

** 다층 퍼셉트론이라는 호칭을 의도적으로 피하는 경우도 있다. 단층 퍼셉트론을 그냥 다층화한 것이 아니냐는 오해를 살 수 있기 때문인데, 이는 원래 제안되었던 퍼셉트론은 임출력 함수가 스텝형 함수로 제한되어 있지만, 앞먹임 신경망은 그런 제한이 없다는 점에서 사실과 다르다.

*** 바이어스는 의미 그대로 어떤 '편견'을 학습 결과에 반영시키는 역할을 한다. 좀 더 구체적인 동작은 신경망의 유닛 내에서 활성화 함수를 왼쪽 혹은 오른쪽으로 이동시켜 변화시키는 것과 같은 결과를 얻게 된다. 이와 달리 가중치는 활성화 함수의 곡선을 좌우로 늘리거나 줄이는 것과 같은 결과를 낳게 된다.

$$z = f(u)$$

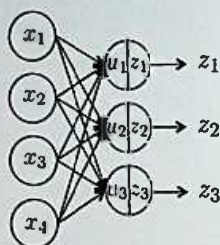


그림 2-2 두 개의 층으로 구성된 신경망. 식 2-1을 참조

앞먹임 신경망에서는 그림 2-2에서처럼 이러한 유닛이 층 모양으로 늘어서 있어 층과 층 사이에만 결합을 갖는다. 왼쪽 층의 유닛의 출력이 오른쪽 층의 유닛의 입력이 되는 형태로, 이러한 결합을 통해 왼쪽에서부터 오른쪽으로만 한 방향으로 정보가 전달된다. 그림 2-2의 신경망에는 오른쪽 층의 세 개의 유닛이 각각 왼쪽 층의 네 개의 유닛으로부터 출력 x_1, x_2, x_3, x_4 를 입력으로 받는다. 유닛 간의 결합은 모두 합하여 $3 \times 4 = 12$ 개이며, 그 하나하나의 결합마다 각각 다른 가중치 w_{ji}^* 가 주어진다. 식으로 나타내면, 세 개의 유닛이 받는 입력은 각각 다음과 같이 계산된다.

$$u_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4 + b_1$$

$$u_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + w_{24}x_4 + b_2$$

$$u_3 = w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + w_{34}x_4 + b_3$$

식 2-1a

식 2-1b

식 2-1c

여기에 다시 활성화 함수를 적용하여 아래 식과 같은 출력이 된다.

$$z_j = f(u_j) \quad (j = 1, 2, 3)$$

식 2-2

* **용어** 1번째 층의 i번째 유닛의 출력이 l + 1번째 층의 j번째 유닛의 입력으로 전달될 때 곱해지는 값을 말한다.

첫 번째 층의 유닛을 $i = 1, \dots, I$, 두 번째 층의 유닛을 $j = 1, \dots, J$ 라고 할 때, 첫 번째 층의 유닛의 출력으로부터 두 번째 층의 유닛의 출력이 결정되는 연산은 다음과 같이 일반화할 수 있다($j = 1, \dots, J$).

$$u_j = \sum_{i=1}^I w_{ji}x_i + b_j$$

$$z_j = f(u_j)$$

벡터와 행렬 표기법을 사용하면 다음과 같다.

$$\mathbf{u} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{z} = \mathbf{f}(\mathbf{u})$$

식 2-3a

식 2-3b

아래와 같이 각 행렬과 벡터를 정의하면 위의 식과 같이 나타낼 수 있다.

$$\mathbf{u} = \begin{bmatrix} u_1 \\ \vdots \\ u_J \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_I \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_J \end{bmatrix}, \quad \mathbf{z} = \begin{bmatrix} z_1 \\ \vdots \\ z_J \end{bmatrix},$$

$$\mathbf{W} = \begin{bmatrix} w_{11} & \dots & w_{1I} \\ \vdots & \ddots & \vdots \\ w_{J1} & \dots & w_{JI} \end{bmatrix}, \quad \mathbf{f}(\mathbf{u}) = \begin{bmatrix} f(u_1) \\ \vdots \\ f(u_J) \end{bmatrix}$$

2.2 활성화 함수

유닛의 활성화 함수로는 통상적으로 단조증가(monotone increasing)하는 비선형함수가 사용된다. 활성화 함수에는 여러 가지가 있지만, 가장 많이 사용되는 것이 로지스틱 시그모이드 함수(logistic sigmoid function) 또는 로지스틱 함수(logistic function)라 불리는 것으로, 다음과 같은 식으로 표현된다.

$$f(u) = \frac{1}{1 + e^{-u}}$$

이 책에서는 로지스틱 함수라 부르겠다. 이 함수는 실수 전체를 정의역으로 가지며, 그림 2-3에서 보듯이 (0, 1)을 치역으로 갖는다. 로지스틱 함수 대신에 유사한 쌍곡선 정접함수를 사용하는 경우도 있다.

$$f(u) = \tanh(u)$$

함수의 치역은 (-1, 1)로 달라지지만, 식 $\tanh(u) = (e^u - e^{-u}) / (e^u + e^{-u})$ 에서 볼 수 있듯이 로지스틱 함수와 비슷한 성질을 갖는다. 두 함수 모두 입력의 절댓값이 포화하여 일정값을 가지며, 그 사이의 값에 대해서는 출력이 서서히 그리고 매끄럽게 변화하는 특징을 갖는다. 그래서 이들을 일반적으로 시그모이드 함수(sigmoid function)로 묶어 부른다. 이러한 특징은 생물의 신경세포가 갖는 성질*을 모델링한 것이다.

최근에는 이러한 활성화 함수 대신에 램프 함수(ramp function, rectified linear function)가 자주 사용된다^[18, 38, 52]. 또는 간단하게 줄여 rectifier라고도 부른다.

$$f(u) = \max(u, 0)$$

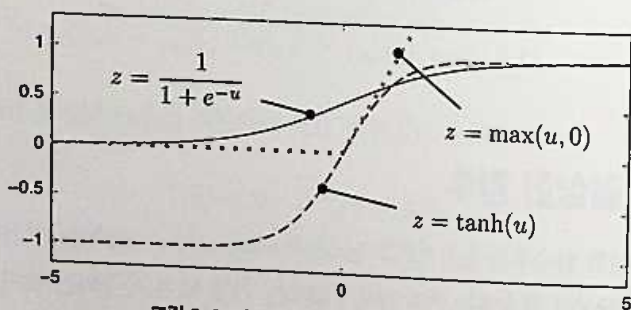


그림 2-3 자주 쓰이는 활성화 함수와 그 모양

* 고등생물의 신경세포 안에서 정보는 전기적인 펄스의 시간적 밀도로 표현되어 세포 간에 전달된다. 이 시간적 밀도는 최소가 0이며, 전기화학적인 제약상 결정되는 상한치를 최댓값으로 갖는다.

램프 함수는 그림 2-3과 같이 $z = u$ 인 선형함수 중 $u < 0$ 인 부분을 $f(u) = 0$ 으로 바꾼 단순한 함수다. 단순하고 계산량이 적으며, 더불어 위에서 설명한 두 함수보다도 학습이 빠르고, 최종 결과도 더 좋은 경우가 많기 때문에 현재 가장 많이 사용되고 있다. 시그모이드 함수를 사용할 때는 입력값의 변동 폭에 주의할 필요가 있지만(너무 커질 경우 출력 대부분이 0이나 1 둘 중 하나가 된다), 램프 함수를 사용하면 그런 일이 일어나지 않는다(예를 들면, 바이어스가 0이라면 입력을 두 배로 할 때 출력도 그대로 두 배가 된다). 또한, 이 함수를 갖는 유닛을 ReLU(Rectified Linear Unit)라고 표기하기도 한다. 이 함수의 성질에 대해서는 8.7.2절에서 다시 다룰 것이다.

지금까지 설명한 함수가 가장 표준적인 것들이지만, 이 외에도 몇 가지 활성화 함수가 더 있다. 그 중 하나가 선형사상(linear mapping) 혹은 항등사상(identity mapping)이다.

$$f(u) = u$$

식 2-4

신경망에서는 각 유닛의 활성화 함수가 비선형성을 갖는 것이 본질적으로 중요하지만, 부분적으로 선형사상을 사용하는 경우가 있다. 예를 들면, 2.4.2절에서 설명할 회귀 문제(regression problem)를 위한 신경망에서는 출력층에서 항등사상을 사용한다. 또, 클래스 분류를 목적으로 하는 신경망에서는 출력층의 활성화 함수로 대개 소프트맥스 함수를 사용한다. 이렇게 출력층에서 사용되는 활성화 함수에 대해서는 2.4.4절에서 다시 설명할 것이다.

또한, 로지스틱 함수를 부분적으로 직선에 근사한 다음과 같은 함수를 쓰는 경우가 있다.

$$f(u) = \begin{cases} -1, & \text{if } u < -1 \\ u, & \text{if } -1 \leq u < 1 \\ 1, & \text{if } u \geq 1 \end{cases}$$

직선의 조합을 통해 비선형성을 표현하고 있는 점에서는 램프 함수나 최대화

출력함수와 비슷하지만, 치역에 제한이 가해져 있다는 점에서는 시그모이드 함수에 가깝다고도 할 수 있다.

또한, 램프 함수와 관계 깊은 맥스아웃(maxout)¹⁹⁾이라는 함수¹⁹⁾가 있다. 이 활성화 함수를 갖는 유닛 하나는 K 개의 유닛을 하나로 합친 것과 같은 구조를 갖는다. 이들 K 개 하나하나가 서로 다른 가중치와 바이어스를 가지므로 각각의 총 입력을 $u_{j1}, \dots, u_{jk}$ 별로 따로 계산한 후, 그중 최댓값을 이 유닛의 출력으로 한다.

$$u_{jk} = \sum_i w_{jik} z_i + b_{jk} \quad (k = 1, \dots, K)$$

$$f(u_j) = \max_{k=1, \dots, K} u_{jk}$$

각 유닛이 이 활성화 함수를 가지는 신경망은 각종 벤치마크 테스트에서 램프 함수를 사용한 신경망을 뛰어넘는 성능을 보였다. 다만, 파라미터 수가 일반적인 유닛의 K 배이기 때문에 그리 널리 쓰이지는 않는다. 6장에서 설명할 CNN이 갖는 최대 풀링도 실질적으로는 같은 연산을 하는 구조다.

2.3 다층 신경망

그림 2-2의 신경망에 또 하나의 층을 추가하여 그림 2-4(a)에 보이는 것과 같은 2층 구조의 신경망을 생각해 볼 수 있다*. 정보는 왼쪽에서 오른쪽 한 방향으로 전달되며, 이 순서대로 각 층을 $l = 1, 2, 3$ 으로 표기한다. $l = 1$ 인 층을 입력층(input layer)이라고 하며, $l = 2$ 는 중간층(internal layer) 또는 은닉층(hidden layer), $l = 3$ 인 층을 출력층(output layer)이라고 부른다.

* 이 책에서는 신경망의 층수에서 입력층을 제외하고 세도록 한다. 그림, 그림 2-4(a), (b)의 신경망의 층수는 2가 된다.

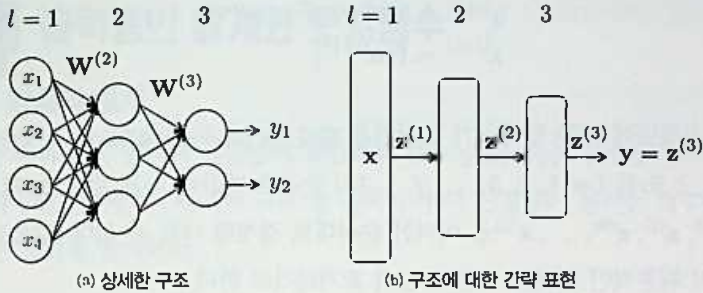


그림 2-4 중간층에 세 개의 유닛을 갖는 2층 신경망과 이 신경망 각 층의 입출력

각 층에 속한 유닛의 입출력을 구별하기 위해 각 변수의 우측에 층의 번호를 붙여 $u^{(l)}$ 이나 $z^{(l)}$ 처럼 쓰기로 하자. 이 표기법을 사용하면 중간층 ($l=2$)의 유닛의 출력은 식 2-3에 따라 다음과 같이 계산할 수 있다.

$$\begin{aligned} u^{(2)} &= W^{(2)}x + b^{(2)} \\ z^{(2)} &= f(u^{(2)}) \end{aligned}$$

$W^{(2)}$ 은 입력층과 중간층 사이의 가중치로, 그림 2-4의 신경망에서는 3×4 행렬이 된다. 또, $b^{(2)}$ 는 중간층에 속한 유닛에 주어진 바이어스다. 다음으로, 출력층 ($l=3$)에 속하는 유닛의 출력은 x 를 중간층의 출력 $z^{(2)}$ 으로 치환하여 다음과 같이 된다.

$$\begin{aligned} u^{(3)} &= W^{(3)}z^{(2)} + b^{(3)} \\ z^{(3)} &= f(u^{(3)}) \end{aligned}$$

$W^{(3)}$ 은 중간층과 출력층 간의 가중치로, 그림 2-4의 신경망에서는 2×3 행렬이다. $b^{(3)}$ 은 출력층의 유닛에 주어진 바이어스다.

여기까지의 내용을 임의의 층수를 갖는 신경망으로 일반화할 수 있다. 층 $l+1$ 의 유닛의 출력 $z^{(l+1)}$ 은 바로 이전 층 l 의 유닛의 출력 $z^{(l)}$ 로부터 다음과 같이 계산할 수 있다.

$$\begin{aligned} u^{(l+1)} &= W^{(l+1)}z^{(l)} + b^{(l+1)} \\ z^{(l+1)} &= f(u^{(l+1)}) \end{aligned}$$



따라서 신경망에 대한 입력 x 가 주어졌을 때(층 $l = 1$ 의 유닛의 출력을 $z^{(1)} = x$ 라 함) 식 2-5a, 2-5b를 $l = 1, 2, 3, \dots, L - 1$ 의 순서로 계산해 가는 것으로, 각 층의 출력 $z^{(1)}, z^{(2)}, z^{(3)}, \dots, z^{(L)}$ 을 이러한 순서대로 결정해 나갈 수 있다. 앞으로는 신경망의 최종적인 출력을 다음과 같이 표기하기로 한다.

$$y \equiv z^{(L)}$$

그리고 위의 내용에서는 간소한 표기를 위해 한 가지의 활성화 함수 f 만 사용한 것처럼 되어 있지만, 각 층에서 서로 다른 활성화 함수를 사용하여도 무방하다. 특히, 출력층 유닛의 활성화 함수는 해결하려는 문제의 유형에 따라 일반적으로 중간층과는 다른 활성화 함수를 사용한다.

이렇게 앞먹임 신경망에서는 주어진 입력 x 에 대하여 입력층에서 출력층 방향으로, 앞에서 표기한 계산을 반복하는 형태로 정보를 전파하며 출력 y 를 계산한다. 이런 관계는 함수 $y = y(x)$ 로 나타낼 수 있다. 이 함수의 내용을 결정하는 것은 각 층 사이의 가중치 $W^{(l)}(l = 2, \dots, L)$ 과 유닛의 바이어스 $b^{(l)}(l = 2, \dots, L)$ 이다. 앞으로 이러한 값을 신경망의 파라미터라고 부르겠다. 이들 값을 어떻게 변화시키는데 따라 다양한 함수를 표현할 수 있게 된다. 이 점을 표현하기 위해 $y(x; W^{(2)}, \dots, W^{(L)}, b^{(2)}, \dots, b^{(L)})$ 이나 또는 신경망의 파라미터의 모든 성분으로 구성되는 벡터 w 를 정의하여 간결하게 $y(x; w)$ 로 표기한다.

2.4 출력층의 설계와 오차함수

2.4.1 학습의 열개

상술한 바와 같이 앞먹임 신경망이 표현하는 함수 $y(\mathbf{x}; \mathbf{w})$ 는 신경망의 파라미터 $\mathbf{w}$ 를 바꾸면 변화한다. 따라서 $\mathbf{w}$ 를 잘 선택하여 이 신경망이 원하는 함수를 얻을 수 있는 방법을 알아보자.

목적으로 하는 함수는 그 구체적인 형태를 알 수 없고, 함수의 입력과 출력의 쌍만을 여러 개 갖고 있다고 하자. 다시 말해, 하나의 입력 $\mathbf{x}$ 에 대하여 바람직한 출력을 $\mathbf{d}$ 라고 할 때 이러한 입출력의 쌍이 여러 개 주어졌다고 하자.

$$\{(\mathbf{x}_1, \mathbf{d}_1), (\mathbf{x}_2, \mathbf{d}_2), \dots, (\mathbf{x}_N, \mathbf{d}_N)\}$$

그리고 이 입출력 쌍 $(\mathbf{x}, \mathbf{d})$ 하나하나를 훈련 샘플(training samples)이라 부르고, 그 집합을 훈련 데이터(training data)라고 부르자.

이때 지금 생각하고 있는 신경망의 $\mathbf{w}$ 를 조정하여 이들 입출력 쌍을 표현할 수 있을지를 생각해 보자. 달리 말하면, 모든 입출력 쌍 $(\mathbf{x}_n, \mathbf{d}_n)$ ($n = 1, \dots, N$) 입력 $\mathbf{x}_n$ 이 주어진 신경망의 출력 $y(\mathbf{x}_n; \mathbf{w})$ 이 최대한 $\mathbf{d}_n$ 과 가까워지도록 $\mathbf{w}$ 를 조정한다. 이러한 과정을 학습이라고 부른다.

이때 신경망이 나타내는 함수와 훈련 데이터와의 가까운 정도 ($y(\mathbf{x}_n; \mathbf{w}) \approx \mathbf{d}_n$), 즉 거리를 어떻게 측정할 것인가가 중요하다. 그 거리의 척도를 앞으로는 오차함수(error function)*라고 부르기로 한다. 앞먹임 신경망으로는 주로 회귀나 클래스 분류 문제를 다루지만, 문제의 종류에 따라 오차함수 및 신경망의 출력층에 대한 설계를 바꾸어 사용할 수 있다. 표 2-1에 문제의 유형에 적합한 활성화 함수와

* 손실함수(loss function)라고도 부른다. 머신 러닝에서는 오히려 손실함수라는 명칭을 일반적으로 사용하지만, 이 책에서는 다른 명칭과 잘 맞는지를 고려해서 오차함수라 부르기로 한다. erf(x)로 표기되는 같은 이름의 특수함수가 있지만, 이것과는 완전히 별개다.

오차함수의 목록을 볼 수 있다. 이제부터는 각각을 순서대로 설명하겠다.

표 2-1 문제의 유형에 따라 사용되는 활성화 함수 및 오차함수의 종류

문제의 유형	출력층에 쓰이는 활성화 함수	오차함수
회귀	항등사상	제곱오차(식 2-6)
이진 분류	로지스틱 함수	식 2-8
다클래스 분류	소프트맥스 함수	교차 엔트로피(식 2-11)

2.4.2 회귀

회귀(regression)란, 출력이 연속값을 갖는 함수를 대상으로 훈련 데이터를 잘 재현하는 함수를 찾는 것을 말한다. 이때 목적으로 하는 함수와 같은 치역을 갖는 함수를 출력층의 활성화 함수로 골라야 한다. 예를 들면, 목표함수의 치역이 $[-1 : 1]$ 인 경우는 출력층의 활성화 함수에는 쌍곡선 정접함수가 적당하며, 목표함수가 임의의 실수를 $(-\infty$ 부터 ∞ 까지) 갖는 경우는 항등사상이 적합하다.

이렇게 출력층의 활성화 함수를 고르고 나서 신경망의 출력 $y(x_i)$ 가 훈련 데이터의 목표 출력 d_i 에 가능한 한 근접하도록 할 방법을 생각해 보자. 이를 위해서는 먼저 '거리'의 척도(오차 함수)를 정해야 하는데, 보통은 아래 식과 같이 정의되는 제곱오차(squared error)를 사용한다*.

$$\|d - y(x; w)\|^2$$

훈련 데이터의 모든 샘플 $n = 1, \dots, N$ 에 대해 제곱오차를 합한 후 이를 반으로 나누는 값을 계산하고, 이 값이 최소가 되는 w 를 선택한다. 참고로, 2로 나누는 것은 이를 미분할 때 2가 곱해지는 것을 상쇄하기 위해서다.

* 훈련 데이터의 목표 출력 d 가 갖는 통계적인 성질에 따라 고르는 것이 이상적이다. d 의 실제 값에 가우스 분포를 따르는 노이즈가 더해졌다고 보는 경우는 제곱오차가 오차함수로서 가장 적합하다.

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \| \mathbf{d}_n - \mathbf{y}(\mathbf{x}_n; \mathbf{w}) \|^2$$

식 2-6

이 함수 $E(\mathbf{w})$ 가 회귀에서 가장 일반적으로 쓰이는 오차함수다.

2.4.3 이진 분류

다음으로는 입력 $\mathbf{x}$ 를 내용에 따라 두 종류로 구별하는 문제를 생각해 보자. 예를 들면, 사람의 얼굴 사진이 주어졌다고 하자. 그로부터 어떤 특정한 방법으로 특징 벡터 $\mathbf{x}$ 를 추출했을 때 $\mathbf{x}$ 를 기반으로 남녀를 구별하는 것과 같은 경우를 들 수 있다. $d = 0$ 은 여자, $d = 1$ 을 남자라고 한다면, 종류를 이진 변수 $d \in \{0, 1\}$ 로 나타내게 된다. 이러면 문제를 입력 $\mathbf{x}$ 로부터 d 의 값을 추정하는 것으로 볼 수 있다.

이 문제를 형식화하는 방법에는 몇 가지가 있지만, 여기서는 $\mathbf{x}$ 가 정해져 있을 때 $d = 1$ 이 되는 사후확률 $p(d = 1|\mathbf{x})$ 를 모델화하는 방법을 사용한다. 주어진 $\mathbf{x}$ 에 대한 d 의 추정은 이 모델을 사용하여 사후확률을 계산한 뒤, 그 값이 0.5를 넘으면 $d = 1$, 그렇지 않으면 $d = 0$ 으로 판단한다.

이 사후확률 $p(d = 1|\mathbf{x})$ 를 모형화하는 데 신경망을 사용한다. 이 신경망은 출력층에 유닛을 단 하나만 가지며, 그 활성화 함수는 로지스틱 함수 $y = 1 / (1 + \exp(-u))$ 를 갖는다. 이 신경망 전체의 입출력함수 $y(\mathbf{x}; \mathbf{w})$ 를 사후확률의 모델로 한다(모델 = 수식이므로).

$$p(d = 1|\mathbf{x}) \approx y(\mathbf{x}; \mathbf{w})$$

신경망의 파라미터 $\mathbf{w}$ 를 변경함으로써 다양한 사후확률을 (신경망의 자유도 범위 내에서) 표현할 수 있을 것이다.

파라미터 w 는 훈련 데이터 $\{(x_n, d_n) | n = 1, \dots, N\}$ 를 사용하여 모형으로부터 얻은 사후분포 $p(d|x; w)$ 가 데이터가 주는 사후분포와 가장 잘 들어맞도록 정한다. 구체적으로 밝히면, 최대우도법(maximum likelihood estimation)을 통해 추정된 파라미터를 사용한다.

거듭제곱의 성질을 이용한 방법으로, 사후분포 $p(d|x; w)$ 을 $d = 1$ 과 $d = 0$ 의 사후분포를 이용하여 다음과 같이 풀 수 있다.

$$p(d|x) = p(d=1|x)^d p(d=0|x)^{1-d} \quad \text{식 2-7}$$

$p(d=1|x) = y(x; w)$ 이고, 또 $d=1$ 이 아니면 $d=0$ 이므로 $p(d=0|x) = 1 - y(x; w)$ 을 유도할 수 있다. 최대우도법은 이 모델을 기반으로 w 의 데이터에 대한 우도(likelihood)를 구하여 그 우도를 가장 크게 하는 w 를 고르는 것이다. w 의 우도는 다음과 같이 주어진다.

$$L(w) \equiv \prod_{n=1}^N p(d_n|x_n; w) = \prod_{n=1}^N \{y(x_n; w)\}^{d_n} \{1 - y(x_n; w)\}^{1-d_n}$$

로그함수는 단조성을 가지므로 결과는 같다. 따라서 이 우도의 로그를 취하여 (최대화 대신 최소화를 하기 위해) 부호를 바꾼 식 2-8을 오차함수로 삼는다.

$$E(w) = - \sum_{n=1}^N [d_n \log y(x_n; w) + (1 - d_n) \log \{1 - y(x_n; w)\}] \quad \text{식 2-8}$$

위의 논의에서는 출력층 유닛의 활성화 함수에 로지스틱 함수를 선택하였다. 이것은 다음과 같다. 먼저, 사후확률 $p(d=1|x)$ 은 조건부확률의 정의로부터 다음과 같이 나타낼 수 있다.

$$p(d=1|x) = \frac{p(x, d=1)}{p(x, d=0) + p(x, d=1)}$$

위의 식과 같이 정의하면 사후확률 $p(d=1|\mathbf{x})$ 은 u 의 로지스틱 함수와 같다.

$$u \equiv \log \frac{p(\mathbf{x}, d=1)}{p(\mathbf{x}, d=0)}$$

지금까지 본 유도과정과는 달리, 이진 분류를 다음 장에서 설명할 다클래스 분류(multi-class classification)의 특수한 경우로 보고 다클래스 분류 방법으로 유도하는 것도 가능하다. 그 경우에 출력층은 유닛을 두 개 가지며, 활성화 함수는 소프트맥스를 사용하게 된다. 신경망으로부터 얻을 수 있는 결과는 같다.

2.4.4 다클래스 분류

클래스 분류란, 입력 $\mathbf{x}$ 를 내용에 따라 유한개의 클래스로 분류하는 문제다. 그림 2-5에 나오는 손글씨 인식의 예를 들어 보자. 숫자가 하나 쓰인 이미지 하나가 주어졌을 때, 그 숫자가 0~9 중 어떤 것인가를 정확히 답하는 것이 목표다. 이미지의 모든 픽셀값(간단히 [0:1]이라고 하자)을 성분값으로 갖는 벡터를 입력 $\mathbf{x}$ 라고 하자(이미지의 크기가 32×32 픽셀이라면 $\mathbf{x}$ 의 성분 수는 1024). 이러한 $\mathbf{x}$ 가 주어졌을 때 해당 이미지가 0~9 중 어느 클래스에 속하는지 분류하는 것이 목표다.

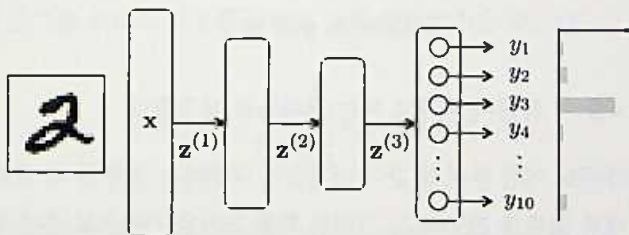


그림 2-5 클래스 분류 문제의 예. 숫자 필기 인식. 입력 이미지 $\mathbf{x}$ 가 0에서 9까지의 숫자 중 어느 것이지를 판별한다. 소프트맥스 층은 0에서 9까지의 숫자 중 입력 이미지가 어느 것에 가까운지를 나타내는 $p_1, \dots, p_{10}$ 의 수치로 출력한다.

이러한 다클래스 분류를 대상으로 할 때, 신경망의 출력층에 분류하려는 클래스 수 L 과 같은 수의 유닛을 구성하고, 이 층의 활성화 함수를 다음과 같이

선택한다. 출력층 $l = L$ 의 각 유닛 $k(= 1, \dots, K)$ 의 총 입력은 바로 아래층 $l = L - 1$ 의 출력으로부터 $u_k^{(L)} (= \mathbf{W}^{(L)}\mathbf{z}^{(L-1)} + \mathbf{b}^{(L)})$ 으로 주어진다. 이것으로부터 다시 출력층의 k 번째 유닛의 출력을 다음과 같이 한다.

$$y_k \equiv z_k^{(L)} = \frac{\exp(u_k^{(L)})}{\sum_{j=1}^K \exp(u_j^{(L)})} \quad \text{식 2-9}$$

이 함수를 **소프트맥스 함수(softmax function)**라고 부른다. 이런 방법으로 결정되는 출력 $y_1, \dots, y_K$ 는 총합이 항상 1이 되는 ($\sum_{j=1}^K y_k = 1$)이라는 것에 주의하라. 그리고 소프트맥스 함수도 활성화 함수의 하나로 취급되지만, 그 외의 활성화 함수에서 유닛 k 의 출력 z_k 는 해당 유닛으로 들어오는 총 입력 u_k 로부터만 결정되는 것과 대조적으로, 소프트맥스 함수에서는 이 층 모든 유닛의 총 입력 $u_1, \dots, u_K$ 으로부터 결정되는 점이 다른 함수들과 다르다*.

그럼, 이제 분류하려는 클래스를 $C_1, \dots, C_K$ 라고 하였을 때, 위와 같이 구성한 출력층의 유닛 k 의 출력 $y_k (= z_k^{(L)})$ 은 주어진 입력 $\mathbf{x}$ 가 클래스 C_k 에 속할 확률을 나타내는 것으로 해석한다.

$$p(C_k|\mathbf{x}) = y_k = z_k^{(L)} \quad \text{식 2-10}$$

그리고 입력 $\mathbf{x}$ 를 이 확률이 최대가 되는 클래스로 분류한다.

다클래스 분류에도 이진 분류와 같이 신경망이 구현하는 함수를 각 클래스의 사후확률에 대한 함수로 간주하고, 그러한 확률 모형을 기반으로 훈련 데이터에 대한 신경망 파라미터의 우도를 평가하여 그 우도를 최대화한다. 지금 입력 $\mathbf{x}$ 와 그 정답 클래스의 쌍이 훈련 데이터로 주어진다고 하자. 이때 신경망의 목표

* 소프트맥스 함수는 대부분 출력층에서만 사용되지만, RBF라 불리는 신경망에서는 중간층에서도 사용된다.

출력을 이진 값을 K 개 열거한 벡터 $\mathbf{d}_n = [d_{n1} \dots d_{nK}]^T$ 로 표현한다고 하자. $\mathbf{d}_n$ 의 각 성분은 대응하는 클래스가 정답 클래스일 때 1이 되며, 그 외에는 0을 갖는다. 예를 들면, 그림 2-5에 손글씨 숫자(10개 클래스)를 분류하는 문제에서는 입력 $\mathbf{x}_n$ 이 숫자 '2'이고, 그에 따라 정답 클래스가 C_3 일 때* 이 입력의 목표 출력은 다음이 된다.

$$\mathbf{d}_n = [0010000000]^T$$

이런 식으로 부호화하였을 때 사후분포는 다음과 같이 나타낼 수 있다.

$$p(\mathbf{d}|\mathbf{x}) = \prod_{k=1}^K p(C_k|\mathbf{x})^{d_k}$$

이 사후확률의 표기에 식 2-10을 적용하면, 훈련 데이터 $\{(\mathbf{x}_n, \mathbf{d}_n) | n = 1, \dots, N\}$ 에 대한 $\mathbf{w}$ 의 우도를 다음 식으로 유도할 수 있다.

$$L(\mathbf{w}) = \prod_{n=1}^N p(\mathbf{d}_n|\mathbf{x}_n; \mathbf{w}) = \prod_{n=1}^N \prod_{k=1}^K p(C_k|\mathbf{x}_n)^{d_{nk}} = \prod_{n=1}^N \prod_{k=1}^K (y_k(\mathbf{x}; \mathbf{w}))^{d_{nk}}$$

이 우도에 로그를 취하여 부호를 반전한 것을 오차함수로 삼는다.

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K d_{nk} \log y_k(\mathbf{x}_n; \mathbf{w})$$

식 2-11

이 함수를 교차 엔트로피(cross entropy, negative log likelihood)라고 한다.

그리고 위에서는 식 2-9와 같이 소프트맥스 함수를 출력층의 활성화 함수로 선택하여 그 출력을 각 클래스의 확률로 간주하였으나, 이것은 다음과 같이

* C_1 이 숫자 0에 해당하고, C_2 가 1, C_3 가 2(나머지도 마찬가지)다.

다르게 생각할 수 있다. 먼저, 클래스 C_k 의 사후확률은 정의로부터 다음과 같이 쓸 수 있다.

$$p(C_k|x) = \frac{p(x, C_k)}{\sum_{j=1}^K p(x, C_j)}$$

여기서 $u_k \equiv \log(p(x, C_k))$ 라고 하면, 이 확률은 다음과 같이 바꿔 쓸 수 있다.

$$p(C_k|x) = \frac{\exp(u_k)}{\sum_{j=1}^K \exp(u_j)}$$

이 식은 입력 $u_1, \dots, u_K$ 에 대한 소프트맥스 함수와 같다는 것을 알 수 있다. 그리고 소프트맥스 함수는 다음과 같이 각각의 입력 $u_1^{(L)}, \dots, u_K^{(L)}$ 모두에 정수 u_0 를 더하여도 그 출력이 변하지 않는다.

$$\frac{\exp(u_k^{(L)} + u_0)}{\sum_{j=1}^K \exp(u_j^{(L)} + u_0)} = \frac{\exp(u_k^{(L)}) \exp(u_0)}{\sum_{j=1}^K \exp(u_j^{(L)}) \exp(u_0)} = \frac{\exp(u_k^{(L)})}{\sum_{j=1}^K \exp(u_j^{(L)})}$$

이와 같은 잉여성*으로 인해 신경망의 가중치를 학습할 때 출력층의 가중치 $W^{(L)}$ 가 잘 수렴하지 않고 학습이 느리게 진행되는 단점이 있으므로, 대개의 경우는 출력층에 몇 가지 제약을 추가하게 된다. 하나는 최적화를 실행할 때의 가중치 감쇠(3.5.2절 참조)를 이용하는 방법이다. 가중치에 제약이 가해지므로 잘 수렴하지 않는 현상이 해결된다. 다른 하나는 출력층의 유닛 중 하나를 적당히 선택하여 그 유닛의 입력을 (실제 값과 상관없이) 강제로 0으로 바꾸는 방법이다. 앞에서 언급한 것과 같이 무작위로 선택한 유닛 k 의 입력 u_k 가 0이 되는 u_0 (바이어스 유닛) 값을 가진 것과 같다고 생각하면, 이러한 자의적인 변경도 무방하다는 것을 알 수 있다. 선택된 유닛에 연결된 가중치도 잘 반영되게 된다.

* **적부** 함수의 입력이 바뀌는데도 출력이 변하지 않는 성질을 말한다.

3

확률적 경사 하강법

안덕의 신경망의 서론 학습법을 한 러닝에서 특히 자주 쓰이는 확률적 경사 하강법에 초점을 맞추어 설명하겠다. 기본적인 알고리즘에 대하여 다루고, 딥 뉴럴넷을 학습시키는 데 가장 큰 걸림돌이 되는 과적합(overfitting)을 설명하고, 이를 완화시키는 방법을 알아보자. 더불어 학습을 더 잘 관찰시킬 수 있도록 하는 노하우를 몇 가지 소개한다.



3.1 경사 하강법

2.4절에서 다루었던 바와 같이 앞먹임 신경망의 학습은 주어진 다음의 훈련 데이터로부터 계산되는 오차함수 $E(\mathbf{w})$ 를 신경망의 파라미터(가중치와 바이어스) $\mathbf{w}$ 에 대하여 최소화하는 과정이다.

$$\mathcal{D} = \{(\mathbf{x}_1, \mathbf{d}_1), \dots, (\mathbf{x}_N, \mathbf{d}_N)\}$$

2장을 요약하자면, 회귀 문제에서는 오차함수가 다음과 같으며,

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \|\mathbf{d}_n - y(\mathbf{x}_n; \mathbf{w})\|^2$$

목표 출력 $\mathbf{d}_n$ 은 임의의 실숫값을 성분으로 갖는 벡터였다. 다클래스 분류 문제에서는 다음과 같고,

$$E(\mathbf{w}) = - \sum_{n=1}^N \sum_{k=1}^K d_{nk} \log y_k(\mathbf{x}_n; \mathbf{w})$$

목표 출력 $\mathbf{d}_n = [d_{n1} \dots d_{nK}]^T$ 는 클래스 수와 같은 수의 성분을 가진 벡터다. 그리고 정답 클래스에 대응하는 성분 하나만 1이고 나머지 성분은 0을 값으로 취하였다.

학습의 목표는 선택한 $E(\mathbf{w})$ 에 대하여 최솟값을 주는 $\mathbf{w} = \operatorname{argmin}_{\mathbf{w}} E(\mathbf{w})$ 를 구하는 것이지만, $E(\mathbf{w})$ 는 일반적인 경우 볼록함수(convex function)가 아니므로 전역 극소점(global minimum)을 직접 구하는 것은 통상적으로 불가능하다. 그래서 그 대신 $E(\mathbf{w})$ 의 국소 극소점(local minimum) $\mathbf{w}$ 를 구하는 것을 생각해 보자. $E(\mathbf{w})$ 의 국소 극소점은 보통 여러 개 존재하므로 이렇게 구한 극소점 중 하나가 우연히 $E(\mathbf{w})$ 의 전역 극소점일 가능성은 높지 않다. 그렇지만 그 국소 극소점 $\mathbf{w}$ 에서의

$E(w)$ 의 값이 충분히 작다면, 목적으로 하는 클래스 분류나 회귀 문제를 나름대로 잘 풀 수도 있다.

국소 극소점 하나는 어떤 초기 점을 출발점으로 하고 w 를 되풀이하여 갱신하는 반복 계산을 통해 구할 수 있다. 이러한 방법에는 여러 가지가 있지만, 그중에서도 가장 간단한 방법이 경사 하강법(gradient descent method)이다. 경사 하강법의 기울기(gradient)란, 다음에 해당하는 벡터다(M 은 w 의 성분 수이다).

$$\nabla E \equiv \frac{\partial E}{\partial w} = \left[\frac{\partial E}{\partial w_1} \cdots \frac{\partial E}{\partial w_M} \right]^T$$
식 3-1

경사 하강법은 현재의 w 를 음의 기울기 방향($-\nabla E$)으로 조금씩 움직이는 것을 여러 번 반복한다. 즉, 현재의 가중치를 $w^{(t)}$, 움직인 후의 가중치를 $w^{(t+1)}$ 이라고 하면, 다음과 같이 갱신한다.

$$w^{(t+1)} = w^{(t)} - \epsilon \nabla E$$
식 3-2

여기서 ϵ 는 w 의 갱신량의 크기를 결정하는 값으로, 학습률(learning rate)이라고 부른다. 초깃값 $w^{(1)}$ 을 적당히 결정한 뒤, 식 3-2를 $t = 1, 2, \dots$ 에 대하여 되풀이하여 계산하는 것으로 $w^{(2)}, w^{(3)}, \dots$ 을 얻을 수 있다.

이렇게 계산되는 $w^{(t)}$ 는 ϵ 이 충분히 작다면 t 의 증가에 따라 $E(w^{(t)})$ 를 반드시 감소시킨다. 따라서 t 를 크게 하면(반복 계산을 계속 하면) 언젠가는 극소점에 도달할 수 있게 된다(다만, $E(w)$ 의 모양과 ϵ 의 크기에 따라서는 $E(w)$ 가 오히려 증가할 수도 있다).

한편, ϵ 의 값이 너무 작으면 w 가 한 번 갱신될 때 변화하는 양이 너무 적어지므로 반복 횟수가 많이 필요하고, 따라서 학습에 걸리는 시간이 늘어나게 된다. 앞으로 설명하겠지만, 학습률 ϵ 을 결정하는 것은 학습에서 매우 중요한 요소로, 실제 학습이 잘 진행될지에 대한 열쇠가 된다.

비선형함수의 최소화 방법 중에서 경사 하강법은 가장 단순한 방법이다. 같은 최소화 방법에도 목적함수(objective function)의 2차 미분을 이용하는 뉴턴법 등이 있지만, 이 방법들에 비해 목표하는 최솟값으로의 수렴이 느리다는 특징이 있다. 또, 학습률 ϵ 을 결정하는 것이 학습을 크게 좌우하므로 뉴턴법이나 그 파생 방법(quasi-newton 등)을 사용할 수 있다면 구태여 경사 하강법을 쓸 이유는 없다. 다만, 문제의 크기가 큰 경우에는 목적함수의 2차 미분에 대한 계산이 어려우므로 간신히 1차 미분(즉, 기울기) 정도밖에 계산하지 못하는 경우도 적지 않다. 그러한 경우에는 경사 하강법은 사용할 수 있는 몇 안 되는 유효한 방법 중 하나가 된다. 딥 뉴럴넷이야말로 이러한 경우에 해당하는 대표적인 예라고 할 수 있다^[44].

3.2 확률적 경사 하강법

3.1절에서는 모든 훈련 샘플 $n = 1, \dots, N$ 에 대하여 계산되는 오차함수 $E(\mathbf{w})$ 를 최적화하는 것을 다루었다. 회귀와 클래스 분류 어느 쪽이든 $E(\mathbf{w})$ 는 각 샘플 한 개에 대해서만 계산되는 오차 E_n 의 합으로 주어진다.

$$E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$$

$\mathbf{w}$ 업데이트 식 3-2에서는 이 $E(\mathbf{w})$ 의 기울기를 사용한다. 이 방법을 배치학습(batch learning)^{*}이라 부르기로 하자.

이와 대조적으로 샘플의 일부, 극단적으로는 샘플 하나만을 사용하여 파라미터를 업데이트하는 방법이 있다. 이 방법은 확률적 경사 하강법(stochastic gradient descent)^{**}

* 에포크 학습(epoch learning)도 같은 의미로 사용되는 말이다.

** 또는 순차적 경사 하강법(sequential gradient descent)이라고도 한다.

으로 불리며, 머릿글자를 따서 SGD로 줄여 쓴다. 이 방법에서 w 의 업데이트는 하나의 샘플 n 에 대해 계산한 오차함수 $E_n(w)$ 의 기울기 ∇E_n 을 계산하여 다음과 같이 w 를 업데이트한다.

$$w^{(t+1)} = w^{(t)} - \epsilon \nabla E_n$$

다음의 $w^{(t+1)}$ 을 업데이트할 때에는 다른 샘플 n' 을 선택한다. 그리고 그 샘플로 같은 방법을 이용하여 기울기 $\nabla E_{n'}$ 을 계산한 후에 w 를 업데이트한다. 이렇게 샘플을 매번 다르게 추출하여 w 를 업데이트하며 진행한다.

딥 뉴럴넷을 학습시키는 데는 배치 형태의 경사 하강법보다 확률적 경사 하강법이 더 많이 쓰이는데, 이는 확률적 경사 하강법이 다음과 같은 장점을 가지기 때문이다. 먼저, 훈련 데이터에 잉여성이 있을 때 계산 효율이 향상되고 학습이 빨리 진행된다. 극단적인 예를 들자면, 가지고 있는 훈련 데이터를 그대로 복사하여 샘플 수를 두 배로 늘리는, 소위 '물타기'를 했다고 하자. 두 배로 늘어난 이 훈련 데이터를 사용하여 배치학습을 시행하면 학습 결과의 성능에는 영향이 없지만, 기울기 ∇E_n 을 계산하는 비용은 두 배로 늘어난다. 그러나 확률적 경사 하강법을 사용하면 이 데이터의 '물타기'는 학습 결과, 계산량 양쪽 모두 영향을 주지 못한다.

더욱이 확률적 경사 하강법을 사용하면 반복 계산이 바람직하지 않은 (오차함수의 값이 상대적으로 그렇게 작지 않은) 국소 극소점에 갇히는 위험을 줄일 수 있다.

배치학습의 경우에는 최소화하려는 목적함수는 항상 같은 $E(w)$ 이므로, 바람직하지 않은 국소 극소점에 한번 갇혀 버리면 두 번 다시 빠져나올 수 없게 된다. 그러나 확률적 경사 하강법을 사용할 때는 목적함수 $E_n(w)$ 가 w 를 업데이트할 때마다 달라지므로 그런 위험은 상대적으로 적다. 반복할 때마다 무작위로 샘플을 선택하면 그 효과를 최대화할 수 있다고 알려져 있으며, 확률적 경사 하강법의 이름은 이 점에서 유래했다.

확률적 경사 하강법의 그 외의 이점으로는, 파라미터의 업데이트 크기가 작은 상태로 학습이 진행되므로 학습의 경과를 좀 더 자세히 들여다보면서 진행할 수 있다는 점과 온라인 학습, 즉 훈련 데이터의 수집과 최적화가 동시에 진행될 수 있다는 점을 들 수 있다.

3.3 '미니배치'의 이용

큰 규모의 신경망 학습은 대규모의 계산 비용이 들기 때문에, 수치 계산을 효율적으로 하기 위해서는 컴퓨터가 갖춘 행렬 계산 자원을 이용하는 것이 필수불가결하다. 이를 위해 샘플 한 개 단위가 아니라 몇 개의 샘플을 하나의 작은 집합으로 묶은 집합 단위로 가중치를 업데이트한다. 이렇게 복수의 샘플을 묶은 작은 집합을 미니배치(minibatch)라고 부른다.

구체적인 방법은 다음과 같다. 먼저, 하나의 미니배치를 $\mathcal{D}_t$ 라 부르기로 하자. 이것은 위에 설명한 바와 같이 몇 개의 샘플 집합을 가리키며, 첨자는 t 번째 갱신마다 샘플 집합이 변화한다는 것을 나타낸다. 그리고 $\mathcal{D}_t$ 이 포함하는 모든 샘플에 대한 오차를 계산하여 그 기울기 방향으로 파라미터를 업데이트한다.

$$E_t(\mathbf{w}) = \frac{1}{N_t} \sum_{n \in \mathcal{D}_t} E_n(\mathbf{w})$$

$N_t = |\mathcal{D}_t|$ 은 이 미니배치가 포함하는 샘플의 개수다. 그리고 이렇게 $E_t(\mathbf{w})$ 를 N_t 로 정규화하면, 미니배치의 크기(샘플 수)를 변화시켰을 때 그에 맞춰 학습률을 조정해야 하는 번거로움을 피할 수 있다.

통상적으로, 미니배치 $\mathcal{D}_t(t = 1, 2, \dots)$ 은 학습을 시작하기 전에 결정하여 고정해 둔다. 미니배치의 크기를 결정하는 통계적인 방법이 따로 있지는 않지만,

확률적 경사 하강법의 장점과 병렬 계산 자원의 유효한 이용을 비교하여 대체로 10~100개 샘플 전후로 결정하는 경우가 많다. 다클래스 분류 문제에서는 미니배치 간의 가중치 업데이트 값을 일정하게 하기 위해 미니배치마다 각 클래스의 샘플이 하나 이상 들어가도록 하는 것이 이상적이다. 따라서 클래스 수가 10~100개 정도이면서 각 클래스의 출현 빈도가 서로 같을 때는 분류하려는 클래스 수와 같은 크기의 미니배치를 생성하는 것이 좋다^[26]. 클래스 수가 그 이상이 되면 샘플을 랜덤으로 섞은 후에 미니배치를 분할한다.

그리고 미니배치의 크기를 너무 작게 잡는 것은 확률적 경사 하강법의 장점을 완전히 활용하지 못하게 하므로 별로 좋지 않다. 크기를 크게 할수록 미니배치마다 계산되는 기울기가 일정해진다. 따라서 w 의 업데이트 양이 안정되기 때문에 학습률 ϵ 을 크게 잡을 수 있고, 그만큼 학습이 빨라지는 효과가 있다. 그러나 일반적으로는 미니배치의 크기를 N 배로 늘려도 ϵ 을 N 배로 할 수 있는 것은 아니어서 종합적인 계산 속도(단위시간당 업데이트 횟수)는 오히려 느려진다^[26].

3.4 일반화 성능과 과적합

지금까지는 훈련 데이터에 대한 오차(오차함수의 값)의 최소화에 대하여 논하였다. 학습의 진짜 목적은 이미 주어진 훈련 데이터가 아니라, 앞으로 주어질 ‘미지의’ 샘플 x 에 대한 정확한 추정을 가능토록 하는 것이다. 전자인 훈련 데이터에 대한 오차를 훈련 오차(training error), 후자의 오차, 정확하게는 샘플 모집단에 대한 오차에 대한 기댓값을 일반화 오차(generalization error)로 부르며, 두 가지를 구분한다. 일반화 오차가 작도록 하는 것이 목표이지만, 일반화 오차는 통계적인 기댓값이므로 훈련 오차처럼 계산할 수가 없다. 그러므로 훈련 데이터와 다른 별도의 샘플 집합을 준비한 후, 이 샘플 집합에 대해서 훈련 오차와 같은 방법으로 계산한 오차를 기준으로 삼도록 한다. 이 책에서는 이를 목적으로

준비하는 데이터를 테스트 데이터(test data), 테스트 데이터에 대한 오차를 테스트 오차(test error)라 부를 것이다.

학습에 의한 파라미터 변화에 따라 훈련 오차 및 테스트 오차가 어떻게 변화하는지를 그림 3-1에 시각화하여 나타내었다. 이러한 곡선을 학습곡선(learning curve)이라 부른다. 이 두 그림은 훈련 오차와 테스트 오차의 전형적인 거동(behaviour)을 나타내고 있다. (여기서 거동이란, 어떤 변수를 변화시키는 데에 따라 다른 변수가 어떤 패턴으로 변화하는지를 뜻함.) 일반적으로 훈련 오차는 파라미터의 업데이트를 반복함에 따라 대개 단조적으로 감소한다. 그에 비해 테스트 오차는 그림 3-1 왼쪽에서 볼 수 있듯이, 학습 초기에는 훈련 오차와 같이 감소하다가 학습 도중에 자주 훈련 오차와 많이 달라진 값을 갖게 된다. 심한 경우에는 같은 그림 오른쪽에서 볼 수 있듯이, 훈련 오차가 감소하고 있는 와중에도 테스트 오차가 증가하는 경우도 있다. 이렇게 훈련 오차와 일반화 오차가 동떨어진 값을 갖는 상태가 되는 것을 과적합(overfitting) 또는 과잉적합, 혹은 과학습(overlearning)이라 부른다.

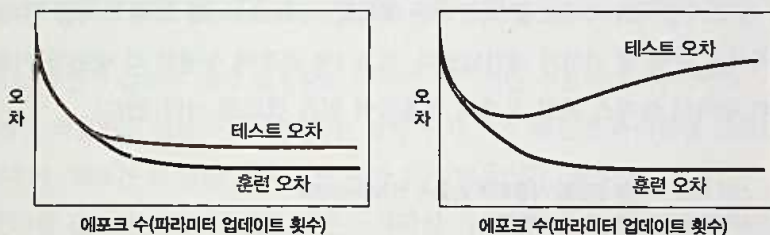


그림 3-1 학습곡선의 이미지. 파라미터의 업데이트 횟수에 따른 훈련 오차(일반화 오차의 목표가 되는)와 테스트 오차의 변화 양상. 학습이 진행됨에 따라 훈련 오차는 단조감소하지만, 테스트 오차도 반드시 그렇게 되지는 않는다.

테스트 오차가 일반화 오차와 매우 가까운 값을 가진다면, 테스트 오차가 최소가 되는 상태가 이상적이다. 위에서 설명한 바와 같이 파라미터의 업데이트에 따라 테스트 오차가 증가한다면 더 이상의 학습은 오히려 방해가 되기 때문에 그 시점에서 학습을 종료한다. 이를 조기종료(early stopping) 또는 조기종단이라 부르며, 예전부터 알려진 신경망 학습을 위한 기술 중 하나다.

어떻게든 이런 과적합을 일으키지 않고 테스트 오차(≒ 일반화 오차)를 보다 작게 하는 것이 신경망 학습의 열쇠다. 신경망의 층수, 유닛 수와 같은 구조 또는 학습률 등의 하이퍼 파라미터(hyper parameter)를 적절히 조절하고, 나아가 뒤에서 설명할 규제화 방법이나 학습을 위한 트릭을 이용하여 이를 실현하도록 한다.

3.5 과적합을 완화시키는 방법

3.5.1 규제화

과적합이란, 학습 시에 오차함수의 값이 작은 국소 극소점에 갇힌 상황이라고 해석할 수 있다. 신경망의 자유도(주로 가중치의 수)가 높을수록 그럴 가능성이 높다고 할 수 있다. 다만, 신경망의 자유도는 그 표현 능력과 직결된다. 따라서 과적합을 해결하기 위해 단순히 신경망의 자유도를 낮추는 것은, 풀려는 문제에 비교하여 과도한 자유도를 가질 경우를 제외하면 그다지 바람직하지 않다. 그래서 학습 시에 가중치의 자유도를 제약하는 규제화(regularization)에 의해 과적합 문제를 완화시키는 방법 몇 가지가 제안되었다. 표 3-1에 이후에 설명할 각 방법과 이를 손글씨 숫자의 클래스 분류 문제에 적용하여 얻은 결과를 나타내었다.

표 3-1 서로 다른 규제화 방법을 사용하여 손글씨 인식(mnist)의 정확도 비교(문헌 [67]에서 발췌)

규제화 종류	테스트 데이터 분류 오차(%)
가중치 감쇠	1.62
가중치 상한	1.35
드롭아웃 + 가중치 감쇠	1.25
드롭아웃 + 가중치 상한	1.05

3.5.2 가중치 감쇠

가장 간단한 규제화 방법은 가중치에 어떤 제약을 가하는 것이다. 예전부터 자주 사용된 방법이 아래와 같이 오차함수에 가중치의 제곱합(norm의 제곱)을 더한 뒤, 이를 최소화하는 방법이다.

$$E_l(\mathbf{w}) \equiv \frac{1}{N_l} \sum_{n \in \mathcal{D}_l} E_n(\mathbf{w}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

λ 는 이 규제화의 강도를 제어하는 파라미터로, 일반적으로 $\lambda = 0.01 \sim 0.00001$ 정도의 범위 내에서 선택한다. 이 항을 추가하여 학습 시에 더 작은 가중치가 선호되게 된다. 실제로, 경사 하강법의 업데이트 식은 다음과 같다.

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \epsilon \left(\frac{1}{N_l} \sum \nabla E_n + \lambda \mathbf{w}^{(t)} \right)$$

그러므로 가중치는 자신의 크기에 비례하는 속도로 항상 감쇠하도록 업데이트된다. 이러한 점 때문에 이 방법을 가중치 감쇠(weight decay)라 부른다.

그리고 가중치 감쇠는 대개 신경망의 가중치 $\mathbf{W}^{(l)}$ 에만 적용하며, 바이어스 $\mathbf{b}^{(l)}$ 에는 적용하지 않는다. 바이어스는 전체 수가 적기 때문에 과적합을 일으키기 어려우며, 때로는 큰 값을 가져야 할 때가 있기 때문이다. 위에서는 표기의 간략화를 위하여 바이어스에도 같은 규제화를 적용한 것과 같이 쓰여 있으므로 주의해야 한다($\mathbf{w}$ 는 2.3절에서 정의한 바와 같이 각 층의 가중치와 바이어스를 모두 포함하기 때문이다).

이와는 별도로 가중치 값의 상한을 통해 가중치를 제약하는 방법(가중치 상한)이 있다. 이것은 각 유닛의 입력 측 결합의 가중치에 대해서 그 제곱합의 최댓값을 제약하는 방법이다. 즉, 층 l 의 유닛 j 가 있고, 이 유닛이 층 $l-1$ 의 유닛 $i = 1, \dots, I$ 의 출력을 입력으로 받는다고 할 때 그 사이의 가중치 w_{ji} 가 다음을 만족하도록 가중치를 제약한다.

$$\sum_i w_{ji}^2 < c$$

이 부등식이 만족하지 않는 경우에는 $w_{j1}, \dots, w_{jl}$ 에 미리 정한 (1보다 작은) 상수를 곱하여 부등식을 만족하도록 한다. 이 방법은 비교적 최근에 제안된 방법으로서^[66] 제곱합(가중치 감쇠)보다 뛰어난 효과를 가지며, 앞으로 설명할 드롭아웃과 함께 사용하면 특히 높은 효과를 갖는다는 사실이 보고된 바 있다^[67].

3.5.3 드롭아웃

드롭아웃(drop-out)은 다층(multi-layer) 신경망의 유닛 중 일부를 확률적으로 선택하여 학습하는 방법이다^[67]. 비교적 최근에 제안되었으나, 다양한 곳에서 매우 좋은 결과를 얻을 수 있어서 현재 널리 사용되고 있다. 드롭아웃은 신경망의 학습 과정과 학습 후의 추론 과정을 다음 내용과 같이 수정한다.

학습 시에는 중간층과 입력층 각 층의 유닛 중 미리 정해 둔 비율 p 만큼을 선택하고 선택되지 않은 유닛을 무효화, 다시 말해 원래 존재하지 않았던 것처럼 취급한다(즉, 이들 층의 유닛은 확률 p 로 존재한다). 그리고 그림 3-2와 같이 선택된 유닛만으로 구성된 '가상의' 신경망을 평소와 같이 최적화한다. 즉, 훈련 샘플을 입력하는 앞먹임 계산을 하고 오차를 구한 뒤, 이어 역전파 계산을 하고 오차의 기울기를 구하여 가중치를 업데이트한다. 그리고 가상의 신경망을 구성하는 유닛은 가중치를 업데이트할 때마다 다시 무작위로 선택한다. 미니배치를 적용하고 있다면 미니배치 단위로 유닛을 다시 선택하게 된다. 유닛을 선택하는 확률은 각 층마다 다르게 적용해도 상관없다. 예를 들면, 표 3-1의 실험에서는 입력층에 $p = 0.9$, 은닉층에 $p = 0.5$ 를 적용하고 있다.

학습이 끝난 뒤 추론 시에는 모든 유닛을 사용하여 앞먹임 계산을 한다. 단, 드롭아웃에서 무효화된 유닛은 그림 3-3과 같이 일률적으로 출력을 p 배로 한다(출력 층의 가중치를 p 배로 생각하는 것과 같다). 이렇게 하는 것은 이 층에서 추론

시의 유닛 수가 학습 시에 비해 $1/p$ 배 된 것과 같기 때문에 이를 보상하기 위함이다.

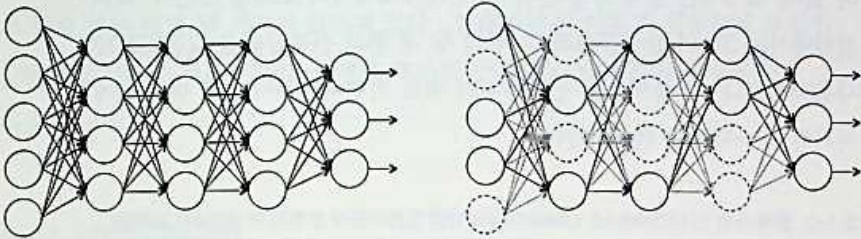


그림 3-2 학습 시의 드롭아웃 과정. 왼쪽 신경망에서 중간층과 입력층의 유닛을 미리 정해 둔 확률 p (보통 0.5 정도로 설정)로 무작위 선택하여 선택되지 않은 신경망의 가중치만 업데이트한다. 업데이트에서 배제할 유닛은 가중치를 업데이트할 때마다 다시 선택한다.

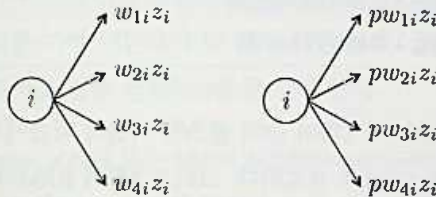


그림 3-3 추론 시의 드롭아웃 과정. 추론 시에는 모든 유닛을 사용하지만, 드롭아웃의 대상이 되었던 층의 유닛은 오른쪽 그림과 같이 모든 출력을 p 배로 한다(또는 출력의 가중치를 p 배로 한다).

이런 과정을 통해 드롭아웃이 목적하는 바는 학습 시에 신경망의 자유도를 강제적으로 낮추고 과적합을 회피하는 것이다. 또, 이 과정은 (유닛이 무작위로 선택되어 무효화되는) 드롭아웃을 거친 신경망을 여러 개 훈련한 후, 이 여러 개의 신경망으로부터 얻은 결과값의 평균을 내는 것과 같은 효과(3.6.3절 참조)가 있다고도 볼 수 있다. 다수의 신경망의 평균을 내면 추론의 정확도가 일반적으로 좋아진다고 알려져 있으며, 드롭아웃은 같은 효과를 보다 적은 계산 비용으로 얻을 수 있는 방법이라 볼 수 있다. 특히, 클래스 분류의 출력층에 사용되는 소프트맥스 층에 대해서는 이러한 방법이 출력의 기하평균을 내는 것과 같음을 알 수 있다^[67].

6장에서 설명할 CNN에서는 원래 드롭아웃을 신경망 상층부의 전결합층에만 적용하였으나^[39, 76], 합성곱층에 적용하여도 성능을 향상시키는 것이 확인된 바 있다. 표 3-2는 물체 유형 분류 인식 태스크에서의 효과를 보인다. 표의 실험에서는 3층의 합성곱층과 그 뒤의 두 개 층의 전결합층을 갖는 CNN이 사용되었으나, 각 층의 p 는 입력층부터 위로 가면서 0.9~0.5의 범위 내에서 서서히 작아지도록 선택되었다.

표 3-2 물체 유형 인식(CIFAR-10, CIFAR-100)에 대한 드롭아웃의 효과(문헌 [67]에서 발췌함)

드롭아웃 적용 범위	테스트 데이터에서의 정확률(%)
합성곱 신경망 + 최대 풀링	15.60
합성곱 신경망 + 확률적 풀링	15.13
합성곱 신경망 + 최대 풀링 + 드롭아웃(전결합층)	14.32
합성곱 신경망 + 최대 풀링 + 드롭아웃(모든 층)	12.61

또, 8장에서 다룰 RBM이나 DBM 등의 볼츠만 머신의 학습에도 드롭아웃이 (사전훈련과 미세조정 양쪽 모두에서) 유효하다. 그리고 8장의 RBM이나 5장의 자기부호화기의 학습에 드롭아웃을 적용하면, 두 경우 모두 희소적(대부분의 값이 0인 벡터/매트릭스의 형태로 표현된다는 의미) 특징이 자동 학습된다는 결과가 보고되고 있다*. 즉, 5.4절에서 말하는 희소 규제화를 거친 경우에 얻어진 것과 유사한 특징을 희소 규제화 없이도 얻을 수 있다는 이야기다. 이처럼 드롭아웃에는 매력적인 성질이 있다.

드롭아웃 외에도 신경망의 일부를 학습 시에 랜덤으로 무효화하는 유사한 방법이 있는데, 드롭커넥트^[72]나 확률적 최대 풀링^[76]이 그것이다. 이들도 나름대로 유효하게 작용하는 경우가 보고되어 있지만, 실제 사용의 편의와 적용 범위로 볼 때 드롭아웃이 훨씬 뛰어나다고 할 수 있다.

* 학습 시 오차함수에 희소 규제화 항을 더한 것을 목적함수로 최적화하는 것.

3.6 학습을 위한 트릭

학습 시에 적용하는 것만으로도 일반화 성능을 향상시키거나 학습을 빨리 진행할 수 있게 하는 방법 몇 가지가 알려져 있다. 그 대부분은 이론적 뒷받침이 부족한 노하우에 가까운 것들이지만, 실제로 효과를 인정받는 것이 많아 오래전부터 사용됐다^[26, 44].

3.6.1 데이터 정규화

훈련 데이터에 어떤 경향이 포함될 때 이 경향이 학습에 방해가 될 수 있다. 이때는 데이터에서 경향을 제거하기 위한 전처리를 거치게 된다. 이런 전처리는 신경망에서만뿐만 아니라 머신 러닝 전반에서 일반적으로 이루어진다. 그리고 훈련 데이터에 어떤 전처리를 가해 변환해야 할 때도 당연히 테스트 데이터 및 추론 시에 입력되는 데이터에도 같은 전처리(변환)를 해야 한다.

데이터 정규화(normalization of data) 또는 데이터 표준화(standardization of data)는 가장 기본적인 전처리로, 각 샘플 $\mathbf{x}_n$ 을 선형 변환하여 그 성분 단위의 평균과 분산을 구한다(그림 3-4). $\mathbf{x}_n$ 의 각 성분을 x_{ni} 로 나타낼 때, 먼저 각 성분 x_{ni} 에서 해당 성분의 학습 데이터 전체에서의 평균 $\bar{x}_i \equiv (\sum_{n=1}^N x_{ni})/N$ 을 빼서 다음으로 변환한 뒤,

$$x_{ni} \leftarrow x_{ni} - \bar{x}_i$$

변환된 샘플의 평균이 0이 되도록 한다(그림 3-4(b)). 그런 다음, 평균을 빼고 난 데이터의 각 성분 x_{ni} 를 그 표준편차

$$\sigma_i \equiv \sqrt{\frac{1}{N} \sum_{n=1}^N (x_{ni} - \bar{x}_i)^2}$$

로 다음과 같이 나눈다.

$$x_{ni} \leftarrow \frac{x_{ni} - \bar{x}_i}{\sigma_i}$$

이렇게 두 번의 변환을 거친 샘플은 각 성분의 평균이 0, 분산이 1이 된다(그림 3-4(c)). 데이터에 따라서는 특정한 i 성분 (x_{ni})만 분산이 0이나 매우 작은 경우가 있어서 위에 설명한 변환을 그대로 적용하면 문제가 발생할 수 있다. 그때는 σ_i 대신, 이를테면 $\max(\sigma_i, \epsilon)$ (ϵ 은 충분히 작은 값)으로 나눈다. 또, 변동이 적은 성분에 원래 의미가 그리 크지 않은 경우에는 분산을 따로 계산하지 않는 경우도 있다.

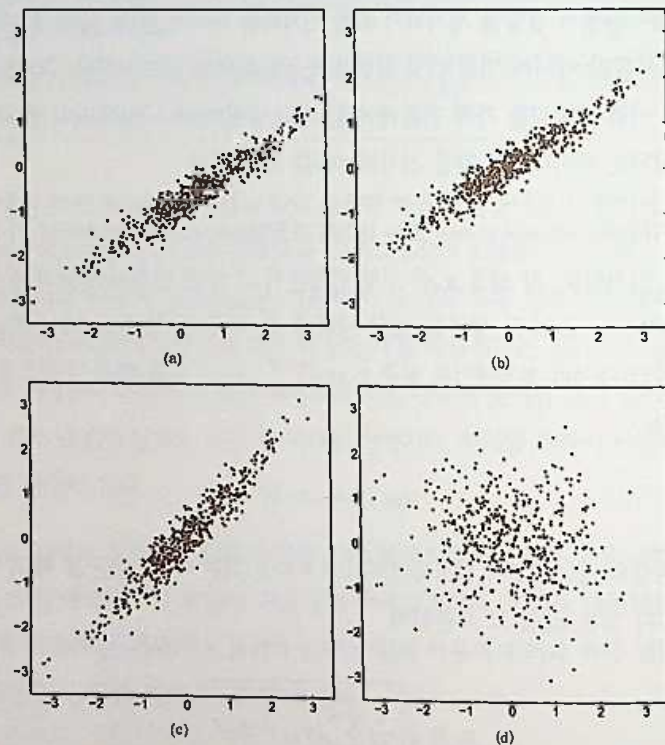


그림 3-4 훈련 데이터의 정규화. (a) 원래 샘플 집합. (b) 평균이 0이 되도록 변환한 집합. (c) 성분 단위 분산이 1이 되도록 변환한 집합. (d) 백색화: 성분 간의 상관이 0이 되도록 변환한 집합(5.5절 참조).

위와 같은 정규화 방법 외에 전처리에서 샘플의 성분 단위의 상관을 제거하는 백색화가 있다. 이것은 5.5절에서 다시 설명하겠다.

3.6.2 데이터 확장

훈련 데이터의 부족은 과적합을 일으키는 가장 큰 원인이다. 대량의 샘플을 수집하는 것은 대개 많은 비용이 들며, 때로는 아예 불가능한 경우도 적지 않다. 이럴 때 이미 확보한 샘플 데이터를 일정하게 가공하여 양적으로 '물타기'하는 방법이 데이터 확장(data augmentation)이다. 샘플 x 에 어떤 변형을 가하여 x' 을 만들고 이를 새로운 샘플로 취급한다. 지도 학습에서는 보통 (x, d) 로부터 (x', d) 를 만들게 된다.

데이터 확장은 샘플의 분포 양상을 예상할 수 있는 경우에 특히 유효하다. 이미지 데이터가 그 전형적인 예로, 이를테면 6.8절에서 설명할 물체 유형 인식에서는 같은 카테고리의 물체 이미지가지만 하면 이미지에 어느 정도 변화를 가해도 타당한 훈련 샘플이라고 할 수 있다. 이미지의 평행 이동, 거울상 반전, 회전 등의 기하학적 변환을 가하거나, 또는 (픽셀의) 명암값이나 색에 변동을 가한 결과 이미지를 새로운 샘플로 삼는다. 가우스 분포를 따르는 랜덤 노이즈를 일괄적으로 적용하는 것도 유효한 방법이다. 이것은 이미지에만 한정되지 않고 예전부터 사용된 기법 중 하나다.

3.6.3 여러 신경망의 평균

여러 개의 서로 다른 신경망을 조합하면 일반적으로 추정의 정확도를 향상시킬 수 있다. 구체적으로, 같은 입력을 여러 개의 신경망에 입력하여 얻어진 출력의 평균을 응답으로 삼는다. 각 신경망은 같은 문제를 풀기 위해 훈련된 것으로, 입력층과 출력층의 구성은 물론 같은 것이다. 이런 방법을 모델 평균(model averaging)이라고 하는데, 신경망뿐 아니라 머신 러닝 전반에서 유용한 아이디어다.

조합할 신경망은 같은 데이터로 훈련한 서로 구조가 다른 것들로 구성하거나, 구조가 같아도 서로 다른 초기값으로 초기화한 것으로 구성한다. 또, 입력에 서로 다른 변환*을 가해 두고 각각 다른 신경망에 입력하여 결과를 평균 내는 방법도 있다¹⁰⁾. 어떤 경우를 택해도 학습 시에 각 신경망은 서로 독립적으로 훈련시킨다.

모델 평균은 많은 경우에 있어 실제로 유효하다는 것이 경험적으로 알려져 있다. 예를 들면, 10클래스를 갖는 분류 문제에서 단일 신경망으로 얻은 전 클래스의 평균 정답률이 70% 정도였다면, 10개 정도의 신경망으로 모델 평균을 적용하면 75~80%까지 오르는 경우가 드물지 않다. 다만, 복수의 신경망을 학습시켜야 하므로 시간 및 연산량이 증가하는 것이 단점이다. 그리고 3.5.3절에서 설명한 드롭아웃은 신경망 하나를 사용해서 실질적으로는 여러 개의 신경망에 모델 평균을 적용한 것과 같은 효과가 있다고 여겨지고 있다.

3.6.4 학습률의 결정 방법

경사 하강법에서는 파라미터의 업데이트 정도를 학습률을 통해 조절한다. 이 학습률을 어떻게 설정할지는 학습의 성패를 좌우할 정도로 매우 중요한 요소다. 학습률의 결정 방법에 대한 연구는 여러 가지가 있으며, 또 자동으로 결정하는 방법도 제안된 바 있다. 그러나 아직도 사람이 직접 (시행착오를 통해) 값을 설정하는 것이 일반적이라 할 수 있다.

학습률을 결정하기 위한 정석과 같은 방법이 두 가지 있다. 하나는 학습 초기에 값을 크게 설정했다가 학습의 진행과 함께 학습률을 점점 줄여가는 방법이다. 예를 들면, 학습률을 파라미터 업데이트 횟수에 비례해서 작아지도록 ($\epsilon = \epsilon_0 / at$) 제어한다. 또는 처음에 상수 $\epsilon = \epsilon_0$ 을 설정하고 어느 정도 학습이 진행되면 그 값을 1/10로 줄여($\epsilon = \epsilon_0 / 10$) 파라미터 업데이트를 계속하며, 때에

* 문헌 [10]에서는 영상 인식을 대상으로 입력 이미지의 콘트라스트를 강조 혹은 저하시키거나 해상도를 낮추는 등의 변환을 실험하고 있다.

따라서 이틀 몇 번 반복한다.

학습률을 결정하는 두 번째 방법은 신경망의 모든 층에서 같은 학습률을 사용하는 것이 아니라 층마다 서로 다른 값을 사용하는 것이다. 각 층의 가중치 업데이트 속도가 되도록 비슷하게 가중치를 설정하는 것이 좋다고 알려져 있으며^[44], 이것은 로지스틱 함수처럼 치역이 제약된 활성화 함수를 사용할 때 특히 중요하다. 신경망 구조에 따라서 다를 수 있으므로 일반화시켜 말할 수는 없지만, 예를 들어 출력 방향에 가까운 얇은 층에서는 학습률을 작게 잡고, 입력에 가까운 깊은 층에는 크게 잡는 경우가 있다. 같은 식으로, CNN처럼 가중치 공유를 할 때 학습률을 가중치가 공유되는 수의 제곱근에 비례하도록 설정하면, 가중치의 업데이트 속도를 대체로 비슷하게 맞출 수 있다*. 단, 이 방법은 램프 함수를 사용할 때는 적합하지 않다.

학습률을 자동적으로 결정하는(변화시키는) 방법도 몇 가지 있는데, 이 중 AdaGrad^[13]가 가장 많이 사용된다. 오차함수의 기울기를 $g_t \equiv \nabla E_t$, 이 벡터의 성분을 $g_{t,i}$ 라 나타낼 때 일반적인 경사 하강법에서는 업데이트 양의 성분 i 가 $-\epsilon g_{t,i}$ 이지만, AdaGrad에서는 다음과 같이 된다.

$$-\frac{\epsilon}{\sqrt{\sum_{t'=1}^t g_{t',i}^2}} g_{t,i}$$

직관적으로 보면, 자주 나타나는 기울기의 성분보다 드물게 나타나는 기울기 성분을 더 중시해서 파라미터를 업데이트하는 것이다. 이 방법은 꽤나 유효해서 현재는 거의 일반적으로 사용되고 있다.

* 공유되는 가중치 w_{ji} 는 오차함수 $E(w)$ 에 공유되는 횟수만큼 반복적으로 나타난다. 따라서 그 기울기 $\partial E(w)/\partial w_{ji}$ 는 공유되는 수만큼 더해진다. 이 각각의 기울기가 서로 독립적으로 가우스 분포를 따른다고 보는 간단한 모형을 상정하면, 그 합의 값이 들쭉날쭉한 정도가 그만큼 멀해지므로 이를 다시 보상하기 위한 방법이다.

3.6.5 모멘텀

경사 하강법의 수렴 성능을 향상시키기 위한 방법 중 하나로, 모멘텀(momentum)이라는 방법이 있다. 이것은 가중치의 업데이트 값에 이전 업데이트 값의 일정 비율을 더하는 방법으로, 미니배치 $t-1$ 에 대한 가중치 업데이트 값을 $\Delta \mathbf{w}^{(t-1)} \equiv \mathbf{w}^{(t-1)} - \mathbf{w}^{(t-2)}$ 라고 하면, 미니배치 t 에 대한 업데이트를 다음과 같이 한다.

$$\mathbf{w}^{(t)} = \mathbf{w}^{(t-1)} - \epsilon \nabla E_t + \mu \Delta \mathbf{w}^{(t-1)}$$

식 3-5

μ 는 더할 값을 비율로 제어하는 하이퍼 파라미터(hyper-parameter)이며, μ 는 0.5 ~ 0.9 사이의 값으로 정한다.

그림 3-5와 같이 오차함수가 깊은 골짜기 같은 형상을 가지며, 그 골짜기 바닥이 비교적 평평한 경우에는 경사 하강법의 효율이 매우 떨어진다고 알려져 있다. 골짜기가 깊으므로 골짜기 바닥을 약간만 빠져나오면 골짜기와 직교하는 방향으로 큰 기울기가 생긴다. 이 결과, 가중치는 매번 골짜기와 직교하는 방향으로 업데이트되어서 경로가 지그재그 모양이 되고 골짜기 바닥을 정상적으로 탐색하지 못하게 된다.

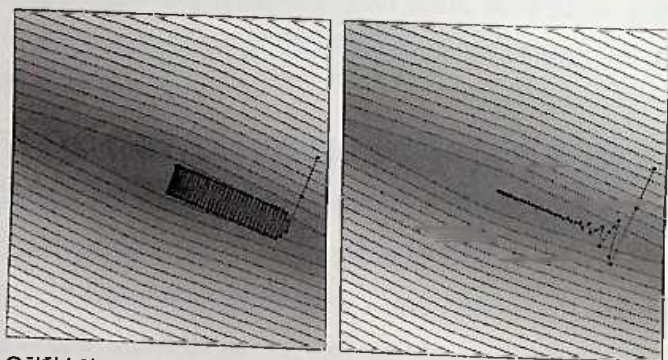


그림 3-5 오차함수의 골짜기 부분의 탐색. 좌측: 기울기 방향으로 100회 업데이트한 경우 (모멘텀 적용하지 않음). 우측: 왼쪽과 같은 초기값으로부터 모멘텀을 적용하여 40회 업데이트한 경우($\mu = 0.5$, 학습률은 그대로). 모멘텀 항의 적용을 통해 탐색의 효율이 향상되었음을 알 수 있다.

모멘텀은 이러한 문제를 해결하고 골짜기 방향을 따라 골짜기 바닥을 효율적으로 탐색할 수 있게 해준다. 식 3-3은 다음과 같이 다시 쓸 수 있으며,

$$\Delta \mathbf{w}^{(t)} = \mu \Delta \mathbf{w}^{(t-1)} - \epsilon \nabla E_t$$

$-\epsilon \nabla E_t$ 를 '가중치의 업데이트 양의 업데이트 양'으로 생각할 수 있다. 업데이트 양이 과거의 업데이트 양의 가중평균이 되므로 골짜기와 직교하는 방향의 업데이트 양은 자연스럽게 평균화 중에 상쇄되어 가중치가 수정되는 경로가 지그재그 모양이 되지 않고 골짜기 방향으로 바닥을 따라갈 수 있게 된다. 만약 ∇E_t 가 잠시 일정해진다면, $\Delta \mathbf{w}^{(t)} \rightarrow 1/(1 - \mu) \cdot (-\epsilon \nabla E_t)$ 로 수렴한다. $\mu = 0.9$ 로 설정했다면, 이 설정은 학습률을 10배로 설정한 것과 같은 효과가 있다. 그러나 모멘텀 항을 사용하지 않고 단순히 학습률을 10배로 설정했다면, 경로가 훨씬 더 심한 지그재그 모양이 된다는 것에 주의하도록 한다.

3.6.6 가중치의 초기화

학습을 시작하기 위해서는 가중치의 초기값을 결정해야 한다. 가장 일반적인 방법은 가우스 분포로부터 랜덤값을 생성하여 초기값으로 삼는 방법 ($w_{ji} \sim N(0, \sigma^2)$)이다. 이 가우스 분포의 표준편차 σ 의 선택은 일반적으로 학습의 결과에도 영향을 미친다. 예를 들어, 표준편차를 크게 잡고 초기값을 들쭉날쭉하게 하면 초기 학습은 빠르게 진행되지만, 오차함수의 감소가 일찌감치 멈춰버리는 경향이 있다. 그리고 바이어스의 초기값은 통상 0으로 설정한다.

유닛의 활성화 함수로 로지스틱 함수 등 치역에 상하한이 있는 함수를 쓰는 경우, 그 유닛의 입력에 대한 가중치를 초기화하는 σ 의 범위도 필연적으로 제약받는다. σ 의 값이 적절하지 못하면 활성화 함수의 출력이 그 치역 내에서 적절한 범위로 들어오지 못하기 때문이다(지금, 이 층의 유닛에 대한 입력의 변동은 정규화되었다고 가정한다). 먼저, σ 가 너무 작으면 0으로 채운 초기값을 사용하는 것과 차이가 없게 된다. 반대로, σ 가 너무 크면 유닛의 입력에 대한 총합 $u_j = \sum_i w_{ji}x_i$ 이 너무

들쭉날쭉해서 유닛의 출력이 치역의 최댓값 아니면 최솟값을 갖는 경우가 너무 많게 된다. 그러면 오차함수의 w_{ji} 에 대한 (4장에서 설명하겠지만, 이 유닛의 활성화 함수의 1차 미분 $f'(u_j)$ 에 의존적이므로) 미분이 0에 가까운 값만을 갖게 되어 학습이 잘 되지 않는다.

위의 내용에 따르면, 유닛의 총 입력 $u_j = \sum_i w_{ji}x_i$ 가 적절한 분산을 갖도록 σ 를 정하는 것이 좋다고 할 수 있다. 가중치를 고정된 상수로 보고, 또 유닛의 각 입력 x_i 이 서로 독립이라고 가정하면, u_j 의 분산은 $V(u_j) = \sum_{i=1}^M w_{ji}^2 V(x_i)$ 가 된다. M 은 이 유닛의 입력 쪽 결합의 수다. 입력 x_i 의 변동을 정규화되어 있다고 할 때 ($V(x_i) = 1$), 총 입력의 분산을 $V(u_j) = \sigma_u^2$ 하려면 다음과 같이 하면 된다.

$$\sigma = \sigma_u / M^{\frac{1}{2}}$$

그리고 활성화 함수로 원점에서 대칭인 성질을 갖는, 이를테면 $\tanh$ 함수를 사용할 때에도 이 방법으로 (위에서 다룬) 훈련 샘플의 정규화를 모든 중간층의 출력에 대해서도 위의 식이 성립하도록 가중치의 초기값을 정할 수 있다.

이 외에도 사전훈련을 통해 초기값을 정하는 방법이 있으며, 딥 뉴럴넷에서는 이쪽이 더 일반적이다. 이에 대해서는 5.6절에서 자세히 다루겠다.

3.6.7 샘플의 순서

확률적 경사 하강법을 쓸 때 훈련 샘플을 어떤 순서로 추출할 것인지에 대해서 자유롭게 선택할 수 있다. 미니배치를 쓸 때 어떤 샘플을 조합하여 미니배치를 구성할 것인지와 구성한 미니배치를 어떤 순서대로 제시할 것인지에 대해서도 임의로 구성해도 무방하다. 일반적으로는 신경망이 '익숙하지 않은' 샘플을 먼저 보이는 전략이 학습에 가장 유리하다고 할 수 있다. 즉, 아직 잘 학습되지 않은 샘플을 먼저 제시하는 방법이 있다^[44]. 이 방법은 특히 유형별로 샘플 수에 편차가 있는 클래스 분류 문제에 좋은 결과를 보이곤 한다. 하지만 훈련 데이터 중에

오답. 그러니까 잘못된 목표 출력이 포함된 경우에는 오히려 역효과가 나니 주의가 필요하다.

그러나 딥 뉴럴넷을 대상으로 하는 최근의 케이스에서는 클래스 간의 샘플 수에 편차가 없도록 서플링한 샘플을 기계적으로 조합하여 미니배치를 구성하고, 미니배치를 구성한 순서대로 반복하여 신경망에 입력하는 경우가 많다. 대규모 신경망과 대량의 훈련 샘플을 어떻게 효율적으로 다룰지에 중점을 더 두는 경우가 많다고 할 수 있다.

4

역전파법

양극의 전경통의 막층에서는 카중층과 타이어층에 대한
유체압축의 미분을 계산해야 한다. 역전파법(backpropagation)은
이 미분을 효율적으로 계산하는 방법이다. 4장에서는 이 방법의
이해대어를 설명하고 계산식을 유도해 볼 것이다.



4.1 기울기 계산의 어려움

3장에서 설명한 경사 하강법을 실행하기 위해서는 오차함수 $E(\mathbf{w})$ 의 기울기 $\nabla E = \partial E(\mathbf{w})/\partial \mathbf{w}$ 를 계산해야 한다. 기울기 벡터의 각 성분은 각 층의 결합 가중치와 각 유닛의 바이어스에 대한 오차함수의 편미분 $\partial E/\partial w_{ji}$ 와 $\partial E/\partial b_j$ 이다*. 그러나 이들 미분을 계산하는 것은 중간층, 특히 입력이 가까운 깊은 층의 파라미터일수록 까다로워진다. 예를 들어 보자. 하나의 샘플 $\mathbf{x}_n$ 에 대한 제곱오차 $E_n = 1/2 \|\mathbf{y}(\mathbf{x}_n) - \mathbf{d}_n\|^2$ 을 l 번째 층의 가중치 $w_{ji}^{(l)}$ ($\mathbf{w}^{(l)}$ 의 한 성분)으로 미분해 보겠다. 일단, 다음과 같이 된다.

$$\frac{\partial E_n}{\partial w_{ji}^{(l)}} = (\mathbf{y}(\mathbf{x}_n) - \mathbf{d}_n)^T \frac{\partial \mathbf{y}}{\partial w_{ji}^{(l)}}$$

그런 다음, 우변의 미분 $\partial \mathbf{y}/\partial w_{ji}^{(l)}$ 를 구해야 하지만, 그러려면 함수 $\mathbf{y}(\mathbf{x})$ 안에 $w_{ji}^{(l)}$ 가 다음과 같이 활성화 함수가 몇 겹이나 적용되는 깊은 식 안에 있기 때문에 미분의 연쇄법칙을 여러 번 적용하는 식으로 풀어야 한다.

$$\begin{aligned} \mathbf{y}(\mathbf{x}) &= \mathbf{f}(\mathbf{u}^{(L)}) \\ &= \mathbf{f}(\mathbf{W}^{(L)}\mathbf{z}^{(L-1)} + \mathbf{b}^{(L)}) \\ &= \mathbf{f}(\mathbf{W}^{(L)}\mathbf{f}(\mathbf{W}^{(L-1)}\mathbf{z}^{(L-2)} + \mathbf{b}^{(L-1)}) + \mathbf{b}^{(L)}) \\ &= \mathbf{f}(\mathbf{W}^{(L)}\mathbf{f}(\mathbf{W}^{(L-1)}\mathbf{f}(\dots \mathbf{f}(\mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}) \dots)) + \mathbf{b}^{(L)}) \end{aligned}$$

이 때문에 프로그래밍하기가 매우 까다로운 동시에 계산 비용도 크게 증가하게 된다. 역전파법(backpropagation)은 이 문제를 해결하는 방법이다.

* 이전에 정의한 대로, L 층의 앞먹임 신경망에서 $\mathbf{w}$ 는 $\mathbf{W}^{(2)}, \dots, \mathbf{W}^{(L)}$ 와 $\mathbf{b}^{(2)}, \dots, \mathbf{b}^{(L)}$ 의 모든 성분을 열거한 벡터다.

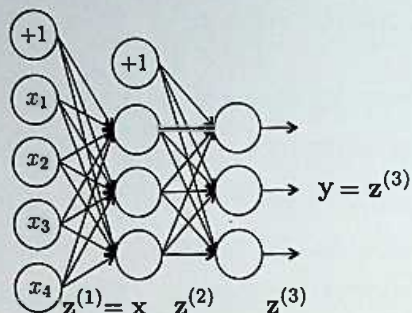


그림 4-1 항상 1을 출력하는 유닛을 각 층마다 하나씩 추가하여 식 4-1과 같이 바이어스를 가중치에 포함하여 표기한다.

앞으로, 표기를 간소화하기 위해 그림 4-1에서 보는 것과 같이 항상 +1을 출력하는 특수한 0번째 유닛을 각 층마다 추가하여 바이어스 b_j 를 그 유닛과 각각의 유닛의 결합에 대한 가중치 $w_{0j}^{(l)} = b_j^{(l)}$ 로 보기로 한다. 다시 말해, l 층의 유닛의 입력에 $l-1$ 층의 0번째 유닛이 항상 $z_0^{(l-1)} = 1$ 로 하는 것으로 다음과 같이 간단하게 나타낼 수 있게 된다.

$$u_j^{(l)} = \sum_{i=1}^n w_{ji}^{(l)} z_i^{(l-1)} + b_j = \sum_{i=0}^n w_{ji}^{(l)} z_i^{(l-1)}$$

3장에서 설명했듯이, 경사 하강법에 의한 최소화 대상은 한 개 샘플 $\mathbf{x}_n$ 에 대한 오차 $E_n(\mathbf{w})$ 의 전 샘플에 대한 총합 $E(\mathbf{w}) = \sum_{n=1}^N E_n(\mathbf{w})$ (배치학습의 경우), 또는 미니배치(소수의 샘플 집합)에 대한 합 $E_l(\mathbf{w}) = \frac{1}{N_l} \sum_{n \in \mathcal{D}_l} E_n(\mathbf{w})$ 이었다. 합의 미분은 미분의 합과 같으므로, 어떤 경우에도 하나의 샘플 n 에 대한 오차 $E_n(\mathbf{w})$ 를 미분할 수 있다면 목적함수의 기울기를 계산할 수 있게 된다. 앞으로는 표기의 편의를 위해 각각의 샘플을 가리키는 인덱스 n 을 생략할 것이다($\mathbf{x}_n \rightarrow \mathbf{x}$ 와 같이). 단, 오차함수에는 어떤 샘플의 오차인지 명시하기 위해 $E_n = E_n(\mathbf{w})$ 와 같이 나타낸다.

4.2 2층으로 구성된 신경망의 계산

그림 4-1에 나온 2층 신경망을 가정해 보자. 회귀 문제에 적용하는 것을 염두에 두고, 출력층의 활성화 함수를 식 2-4에 나온 항등사상으로 선택한다. 이때 중간층의 유닛은 임의의 활성화 함수 f 를 갖는다고 하자. 이 신경망에서는 입력 $\mathbf{x} = [x_1 \ x_2 \ x_3 \ x_4]^T$ 이 출력 층으로 다음과 같이 전파된다. 입력층의 출력을 언제나처럼 $z_i^{(1)} = x_i$ 라고 하면, 중간층의 출력은 다음이 된다.

$$z_j^{(2)} = f(u_j^{(2)}) = f\left(\sum_i w_{ji}^{(2)} z_i^{(1)}\right)$$

식 4-2

출력층의 활성화 함수는 항등사상이므로 그 출력은 다음과 같다.

$$y_j(\mathbf{x}) = z_j^{(3)} = u_j^{(3)} = \sum_i w_{ji}^{(3)} z_i^{(2)}$$

식 4-3

이 신경망의 오차함수로 다음의 제곱오차를 골랐다면,

$$E_n = \frac{1}{2} \|\mathbf{y}(\mathbf{x}) - \mathbf{d}\|^2 = \frac{1}{2} \sum_j (y_j(\mathbf{x}) - d_j)^2$$

가중치에 대한 미분 $\partial E_n / \partial w_{ji}^{(3)}$ 와 $\partial E_n / \partial w_{ji}^{(2)}$ 는 어떻게 계산해야 할까? 먼저, 출력층의 가중치에 대한 미분 $\partial E_n / \partial w_{ji}^{(3)}$ 는 비교적 쉽게 다음과 같이 계산할 수 있다.

$$\frac{\partial E_n}{\partial w_{ji}^{(3)}} = (\mathbf{y}(\mathbf{x}) - \mathbf{d})^T \frac{\partial \mathbf{y}}{\partial w_{ji}^{(3)}}$$

위의 식과 같이 전개한 뒤, 우변의 $\partial \mathbf{y} / \partial w_{ji}^{(3)}$ 는 벡터이지만, 식 4-3에 의해 성분 j 의 값만 $z_i^{(2)}$ 이고 나머지 성분은 0인 다음과 같은 형태가 된다.

$$\frac{\partial y}{\partial w_{ji}^{(3)}} = [0 \dots 0 \ z_i^{(2)} \ 0 \dots 0]^T$$

따라서 다음과 같이 구할 수 있다.

$$\frac{\partial E_n}{\partial w_{ji}^{(3)}} = (y_j(x) - d_j) z_i^{(2)}$$

식 4-4

다음으로, 중간층의 기울기에 대한 미분 $\partial E_n / \partial w_{ji}^{(2)}$ 를 계산하도록 하겠다. $w_{ji}^{(2)}$ 는 $u_j^{(2)} = \sum_i w_{ji}^{(2)} z_i^{(1)}$ 와 같이 $u_j^{(2)}$ 안에만 존재한다. 그러므로 E_n 을 $u_j^{(2)}$ 로 미분하여 연쇄법칙을 적용하면, $\partial E_n / \partial w_{ji}^{(2)}$ 은 다음과 같이 된다.

$$\frac{\partial E_n}{\partial w_{ji}^{(2)}} = \frac{\partial E_n}{\partial u_j^{(2)}} \frac{\partial u_j^{(2)}}{\partial w_{ji}^{(2)}}$$

식 4-5

$u_j^{(2)} = \sum_i w_{ji}^{(2)} z_i^{(1)}$ 인 점을 이용하면, 식 4-5의 우변 두 번째 항을 곧바로 다음 식과 같이 계산할 수 있다.

$$\frac{\partial u_j^{(2)}}{\partial w_{ji}^{(2)}} = z_i^{(1)}$$

식 4-5의 우변 첫 번째 항인 $\partial E_n / \partial u_j^{(2)}$ 는 다음과 같이 계산한다. 그림 4-2와 같이 중간층 유닛 j 의 입력 $u_j^{(2)}$ 는 이 유닛과 연결된 출력층 유닛 $k (= 1, 2, 3)$ 에 전달되므로 이 세 개의 유닛 각각의 총 입력 $u_k^{(3)}$ 에 영향을 미치게 된다. E_n 은 $u_k^{(3)}$ ($k = 1, 2, 3$)의 함수이므로 E_n 의 $u_j^{(2)}$ 에 대한 미분은 연쇄법칙을 적용하여 출력층의 세 유닛에 대한 합으로 나타낼 수 있다.

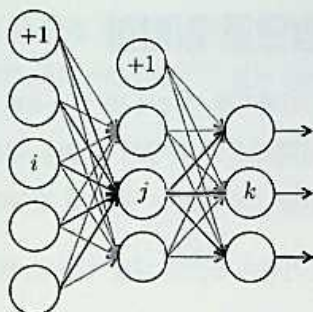


그림 4-2 중간층 유닛 j 의 입력 $u_j^{(2)}$ 는 출력층의 각 유닛 $k(k = 1, 2, 3)$ 의 총 입력 $u_k^{(3)}$ 에 영향을 미친다. 그러므로 전자에 대한 후자의 미분이 존재한다.

$$\frac{\partial E_n}{\partial u_j^{(2)}} = \sum_k \frac{\partial E_n}{\partial u_k^{(3)}} \frac{\partial u_k^{(3)}}{\partial u_j^{(2)}}$$

식 4-6

우변의 $\partial E_n / \partial u_k^{(3)}$ 는 $E_n = 1/2 \sum_k (y_k(x) - d_k)^2 = 1/2 \sum_k (u_k^{(3)} - d_k)^2$ 이므로 다음과 같이 계산할 수 있으며,

$$\frac{\partial E_n}{\partial u_k^{(3)}} = u_k^{(3)} - d_k$$

식 4-7

$\partial u_k^{(3)} / \partial u_j^{(2)}$ 은 $u_k^{(3)} = \sum_j w_{kj}^{(3)} f(u_j^{(2)})$ 이므로 다음과 같이 계산한다.

$$\frac{\partial u_k^{(3)}}{\partial u_j^{(2)}} = w_{kj}^{(3)} f'(u_j^{(2)})$$

이들을 다시 식 4-5에 대입하면 다음을 얻을 수 있다.

$$\frac{\partial E_n}{\partial w_{ji}^{(2)}} = \left(f'(u_j^{(2)}) \sum_k w_{kj}^{(3)} (u_k^{(3)} - d_k) \right) z_i^{(1)}$$

식 4-8

4.3 다층 신경망으로 일반화

2층 신경망인 경우의 오차 기울기를 식 4-4 및 식 4-8과 같이 계산할 수 있었다. 이를 임의의 층수를 갖는 신경망으로 확장해 보겠다. 구체적으로는 2층 신경망의 중간층 가중치에 대한 계산을 일반화하는 것이 될 것이다.

먼저, 식 4-5를 l 번째 층의 가중치 $w_{ji}^{(l)}$ 에 대한 식으로 보면 다음이 된다.

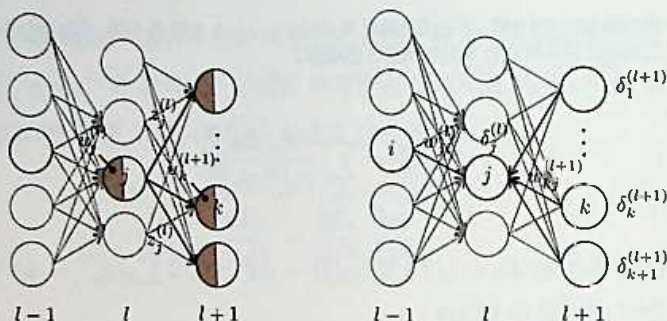


그림 4-3 좌측: 유닛의 총 입력의 변화가 오차함수에 미치는 영향은 항상 다음 층의 모든 유닛의 총 입력을 경유한다. 우측: 엘타의 역전파($l+1$ 에서 l 층으로)의 양상과 $\partial E / \partial w_{ji}^{(l)}$ 가 계산되는 과정($l-1$ 층과 l 층 사이).

$$\frac{\partial E_n}{\partial w_{ji}^{(l)}} = \frac{\partial E_n}{\partial u_j^{(l)}} \frac{\partial u_j^{(l)}}{\partial w_{ji}^{(l)}}$$

식 4-9

식 4-9의 우변 첫 번째 항을 보도록 하자. 그림 4-3의 좌측 그림을 보면, $u_j^{(l)}$ 의 변동이 E_n 에 미치는 영향은 해당 유닛 j 의 출력 $z_j^{(l)}$ 를 통해 $l+1$ 번째 층의 각 유닛 k 의 총 입력 $u_k^{(l+1)}$ 을 변화시키는 것만을 통해 일어난다는 것을 알 수 있다. 따라서 각 $u_k^{(l+1)}$ 를 거치도록 연쇄법칙을 적용하여 다음과 같이 분해할 수 있다.

$$\frac{\partial E_n}{\partial u_j^{(l)}} = \sum_k \frac{\partial E_n}{\partial u_k^{(l+1)}} \frac{\partial u_k^{(l+1)}}{\partial u_j^{(l)}}$$

식 4-10

이렇게 분해한 뒤, 우변의 각 항을 계산해 나간다. 식 4-10의 양변에 l 번째 층과 $l+1$ 번째 층의 입력에 대한 미분 $\partial E_n / \partial u_j^{(l)}$ 이 있는 것에 착안하여 식 4-11로 한다.

$$\delta_j^{(l)} \equiv \frac{\partial E_n}{\partial u_j^{(l)}}$$

식 4-11

이 책에서는 이 값을 델타(delta)라고 부르겠다. 델타는 각 층 l 의 각 유닛 j 마다 정의되는 것에 주의하라.

그 다음에는 $u_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} z_j^{(l)} = \sum_j w_{kj}^{(l+1)} f(u_j^{(l)})$ 인 사실로부터 $\partial u_k^{(l+1)} / \partial u_j^{(l)} = w_{kj}^{(l+1)} f'(u_j^{(l)})$ 가 된다는 것을 이용하여 식 4-10을 다음과 같이 쓸 수 있다.

$$\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} \left(w_{kj}^{(l+1)} f'(u_j^{(l)}) \right)$$

식 4-12

식 4-12는 $\delta_j^{(l)}$ 을 $\delta_k^{(l+1)}$ ($k = 1, 2, \dots$)으로부터 계산할 수 있음을 의미한다. 다시 말해, 더 위층인 $l+1$ 층의 유닛에 대한 델타가 주어지면 l 층의 델타는 식 4-12로 계산할 수 있게 된다. 그림 4-3 오른쪽 그림에 식 4-12에 의한 델타의 계산 과정을 나타내었다. 이 식은 임의의 중간층 l 에 대해서 성립하므로 l 을 $l+1$ 로 바꿔 써도 성립한다. 이 점으로부터 $\delta_k^{(l+1)}$ 도 마찬가지로 $\delta_1^{(l+2)}, \delta_2^{(l+2)}, \dots$ 로부터 계산할 수 있는 것이다. 이렇게 출력층 L 층까지 거슬러 올라가 맨 처음 출력층의 유닛에 대한 델타가 계산되어 있으면, 식 4-12를 반복 계산하여 어떤 층이든 델타를 계산할 수 있다. 이때 델타가 출력층부터 입력층 방향, 그러니까 순전파와는 반대 방향으로 전파되는데, 이것이 역전파법이라는 이름의 유래가 된다.

식 4-9의 첫 번째 항은 이런 방법으로 델타를 계산하여 구하면 되며, 두 번째 항인 $\partial u_j^{(l)} / \partial w_{ji}^{(l)}$ 는 $u_j^{(l)} = \sum_i w_{ji}^{(l)} z_i^{(l-1)}$ 와 같은 관계로부터 다음과 같이 쉽게 계산할 수 있다.

입력: 훈련 샘플 x_n 과 목표 출력 d_n 의 쌍 하나.

출력: 오차함수 $E_n(w)$ 의 각층 l 의 파라미터에 대한 미분 $\partial E_n / \partial w_{ji}^{(l)} (l = 2, \dots, L)$.

1. $z^{(1)} = x_n$ 로 하여 각각의 층 $l (= 2, \dots, L)$ 의 유닛의 입력 $u^{(l)}$ 과 출력 $z^{(l)}$ 을 순서대로 계산한다.
2. 출력층의 델타 $\delta_j^{(L)}$ 를 구한다. (통상적으로 $\delta_j^{(L)} = z_j^{(L)} - d_j$ 이다.)
3. 각 중간층 $l (= L - 1, L - 2, \dots, 2)$ 에서의 델타 $\delta_j^{(l)}$ 를 출력층부터 가까운 순서대로 식 4-12와 같이 계산한다. (역전파)
4. 각 층 $l (= 2, \dots, L)$ 의 파라미터 $w_{ji}^{(l)}$ 에 대한 미분을 식 4-13과 같이 계산한다.

그림 4-4 오차 역전파를 통해 오차 기울기(가중치에 대한 오차의 미분)를 계산하는 절차. 미니배치의 조작을 포함하는 상세한 내용은 4.4절을 참조할 것.

$$\frac{\partial u_j^{(l)}}{\partial w_{ji}^{(l)}} = z_i^{(l-1)}$$

따라서 이렇게 계산한 값과 식 4-11의 $\delta_j^{(l)}$ 를 식 4-9에 대입하면, 목적하는 미분은 다음으로 나타낼 수 있다.

$$\frac{\partial E_n}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} z_i^{(l-1)}$$

식 4-13

이렇게 $l - 1$ 번째 층의 유닛 i 와 l 번째 층의 유닛 j 를 잇는 결합의 가중치 $w_{ji}^{(l)}$ 에 대한 미분은, 유닛 j 에 대한 $\delta_j^{(l)}$ 와 유닛 i 의 출력 $z_i^{(l-1)}$ 의 곱에 지나지 않는다는 것을 알 수 있다. 그림 4-3 오른쪽 그림에는 이 미분을 계산하는 데 관여되는 유닛과 그 사이의 결합이 진하게 표시되어 있다.

델타 $\delta_j^{(l)}$ 는 위와 같이 출력층에서 입력층 방향으로 반복해서 식 4-12를 계산하면 구할 수 있다. 역전파되는 최초 값은 출력층의 델타 $\delta_j^{(L)}$ 가 주어지는데, 이것은 간단하게 다음과 같이 계산한다.

$$\delta_j^{(L)} = \frac{\partial E_n}{\partial u_j^{(L)}}$$

구체적인 계산은 오차함수의 종류에 따라 다르며, 이 부분은 4.4.1절에서 다루겠다.

지금까지 다룬 내용을 정리하자면, 어떤 훈련 샘플 $(\mathbf{x}, \mathbf{d})$ 가 주어졌을 때 이 샘플에 대한 오차함수 E_n 의 기울기는 그림 4-4에 나타난 순서대로 계산할 수 있다. 그리고 앞부분에서 밝힌 바와 같이, 미니배치 등 여러 개의 훈련 샘플에 대한 오차의 총합 $E = \sum_n E_n$ 의 기울기는 이 절차를 훈련 샘플 $(\mathbf{x}_n, \mathbf{d}_n)$ 마다 계산하여 구한 기울기를 합하면 구할 수 있다.

$$\frac{\partial E}{\partial w_{ji}^{(l)}} = \sum_n \frac{\partial E_n}{\partial w_{ji}^{(l)}}$$

4.4 경사 하강법의 전체 알고리즘

4.4.1 출력층의 델타

역전파 계산의 시작점은 출력층의 델타 $\delta_j^{(L)} = \partial E_n / \partial u_j^{(L)}$ 이다. 이를 계산하는 방법은 사용하고 있는 오차함수나 출력층의 활성화 함수가 무엇이냐에 따라 다르지만, 대표적인 경우를 살펴해보도록 하겠다.

문제 유형은 회귀 문제, 오차함수는 제곱오차 $E_n = 1/2 \|\mathbf{y} - \mathbf{d}\|^2 = 1/2 \sum_j (y_j - d_j)^2$ 이고, 출력층 활성화 함수는 항등사상 $y_j = z_j^{(L)} = u_j^{(L)}$ 로 할 경우, 출력층의 유닛 j 의 델타 $\delta_j^{(L)}$ 는 다음과 같다.

$$\delta_j^{(L)} = u_j^{(L)} - d_j = z_j^{(L)} - d_j = y_j - d_j$$

이진 분류 문제이고, 오차함수가 $E_n = -[d \log y + (1 - d) \log(1 - y)]$, 출력층의 활성화 함수는 로지스틱 함수 $y = 1/(1 + \exp(-u))$ 인 경우에는 뒤에 설명할 로지스틱 함수의 미분을 사용하여 다음과 같이 된다.

$$\begin{aligned}\delta^{(L)} &= -\left[-\frac{d}{y} \cdot \frac{dy}{du} + \frac{1-d}{1-y} \left(-\frac{dy}{du}\right)\right] \\ &= -[d(1-y) - (1-d)y] = y - d\end{aligned}$$

여러 클래스를 갖는 분류 문제이고, 출력층 활성화 함수로 소프트맥스 함수를 사용하며, 오차함수를 교차 엔트로피로 사용하는 경우,

$$E_n = -\sum_k d_k \log y_k = -\sum_k d_k \log \left(\frac{\exp(u_k^{(L)})}{\sum_i \exp(u_i^{(L)})} \right)$$

출력층 유닛 j 의 델타 $\delta_j^{(L)} = \partial E_n / \partial u_j^{(L)}$ 는 다음과 같이 유도할 수 있다.

$$\begin{aligned}\delta_j^{(L)} &= -\sum_k d_k \frac{1}{y_k} \frac{\partial y_k}{\partial u_j^{(L)}} \\ &= -d_j(1 - y_j) - \sum_{k \neq j} d_k(-y_j) = \sum_k d_k(y_j - d_j) \\ &= y_j - d_j\end{aligned}$$

이때 마지막 부분의 등호는 $\sum_k d_k = 1$ (여러 클래스의 분류 문제에서 목표 출력 d 가 갖는 성질)을 이용한 것이다.

이렇게 회귀, 분류 두 가지 유형에서 모두 출력층 L 의 유닛 j 의 델타 $\delta_j^{(L)}$ 는 신경망의 출력 y_j 와 목표 출력 d_j 의 차가 된다.

4.4.2 순전파와 역전파의 행렬 계산

순전파와 역전파, 그리고 파라미터 업데이트의 각 계산은 층 단위의 행렬 계산으로 나타낼 수 있다. 이러한 표기는 이를 코드로 옮길 때 유용하며, GPU나 CPU의 멀티코어를 이용한 병렬 계산을 고려하면 더욱 중요하다*. 그래서 아래에서는 미니배치를 적용한 확률적 경사 하강법의 모든 계산을 행렬 표기법으로 나타내겠다.

이번 장에서는 지금까지 각 층의 유닛 j 의 바이어스를 가중치 w_{j0} 으로 나타내는 파라미터를 정의해 왔었다. 지금부터는 바이어스와 가중치를 다시 구별하여 $i \geq 1$ 인 w_{ji} 를 (j, i) 성분으로 갖는 행렬을 $\mathbf{W}$ 라 하고, 바이어스는 이와 별도로 w_{j0} 를 j 번째 성분으로 갖는 벡터 $\mathbf{b}$ 로 나타내기로 한다.

미니배치는 N 개의 샘플로 구성되며, 이들 샘플을 열벡터로 갖는 행렬을 $\mathbf{X} = [\mathbf{x}_1 \cdots \mathbf{x}_N]$ 로 나타낸다**. 샘플 $\mathbf{x}_n$ 을 입력한 시점의 l 층의 각 유닛에 대한 총 입력을 성분으로 갖는 벡터를 $\mathbf{u}_n^{(l)}$, 그리고 활성화 함수를 적용한 각 유닛의 출력을 성분으로 갖는 벡터를 $\mathbf{z}_n^{(l)}$ 으로 나타낸다. 그리고 이것들을 열벡터로 갖는 행렬을 각각 $\mathbf{U}^{(l)} = [\mathbf{u}_1^{(l)} \cdots \mathbf{u}_N^{(l)}]$, $\mathbf{Z}^{(l)} = [\mathbf{z}_1^{(l)} \cdots \mathbf{z}_N^{(l)}]$ 로 표기한다.

위의 표기를 사용하면 순전파 계산은 $\mathbf{Z}^{(l)} \equiv \mathbf{X}$ 로 보고, $l = 2, \dots, L$ 에 대해 다음 두 식에 대한 반복 계산으로 이루어진다(식 2-5를 여러 개의 샘플로 확장한 것).

$$\mathbf{U}^{(l)} = \mathbf{W}^{(l)} \mathbf{Z}^{(l-1)} + \mathbf{b}^{(l)} \mathbf{1}_N^T$$

식 4-14a

$$\mathbf{Z}^{(l)} = f^{(l)}(\mathbf{U}^{(l)})$$

식 4-14b

$\mathbf{1}_N$ 은 1을 N 개 열거한 벡터이며, $f^{(l)}(\cdot)$ 은 행렬의 각 성분에 활성화 함수를 적용하고 그 결과를 같은 크기의 행렬로 반환하는 함수다. 그리고 6장에서 설명할 CNN처럼 층간 결합 밀도가 낮거나 서로 다른 결합이 같은 가중치를 가지는

* 행렬 곱의 병렬 계산은 실용화된 라이브러리를 사용하면 자동적으로 상당 부분의 병렬화가 가능하다.

** 다음 번 파라미터 업데이트 시에는 N 개의 다른 샘플로 이루어진 미니배치를 사용한다.

경우에도 순전파 계산은 같은 식으로 나타낼 수 있다. 단, 이런 경우에는 $\mathbf{W}$ 가 서로 결합이 없는 유닛 사이의 가중치가 0이 되거나, 유닛 i 와 j , i '와 j '이 각각 같은 가중치를 갖는 결합으로 연결되어 있다면 $w_{ji} = w_{j'i'}$ 이 된다.

$\mathbf{x}_n$ 에 대한 출력 $\mathbf{y}_n$ 을 이어 늘어놓은 행렬 $\mathbf{Y} = [\mathbf{y}_1 \cdots \mathbf{y}_N]$, 이에 대응하는 목표 출력을 늘어놓은 것을 $\mathbf{D} = [\mathbf{d}_1 \cdots \mathbf{d}_N]$ 라고 하자. 또 l 층의 각 유닛에 대한 델타 $\delta_j^{(l)}$ 를 요소로 갖는 행렬을 $\Delta^{(l)}$ 로 나타낸다. $\Delta^{(l)}$ 그 각각의 열은 미니배치를 구성하는 각 샘플 $n = 1, \dots, N$ 에 대응하고, 각각의 행은 l 층의 각 유닛에 대응한다. 델타에 대한 역전파 계산은 $\Delta^{(l)} = \mathbf{Y} - \mathbf{D}$ 를 구한 후에 $l = L - 1, \dots, 2$ 의 순서대로 다음 계산을 반복하면 된다.

$$\Delta^{(l)} = f^{(l)'}(\mathbf{U}^{(l)}) \odot (\mathbf{W}^{(l+1)\top} \Delta^{(l+1)})$$

식 4-15

$f^{(l)'}(\mathbf{U})$ 는 $\mathbf{U}$ 의 각 성분에 l 층의 활성화 함수의 미분 $f'(\cdot)$ 을 적용하여 얻은 행렬이다. $\odot$ 은 행렬의 대응 성분끼리의 곱, 그러니까 $\mathbf{C} = \mathbf{A} \odot \mathbf{B}$ 는 $c_{ij} = a_{ij}b_{ij}$ 를 의미한다. 식 4-15는 모든 샘플과 모든 유닛에 대해 식 4-12를 정리해서 표기한 것이다. 각각의 활성화 함수에 대한 도함수를 표 4-1에 정리해 두었다.

표 4-1 활성화 함수와 그 도함수

활성화 함수	$f(u)$	$f'(u)$
로지스틱 함수	$f(u) = 1/(1 + e^{-u})$	$f'(u) = f(u)(1 - f(u))$
쌍곡선 정점함수	$f(u) = \tanh(u)$	$f'(u) = 1 - \tanh^2(u)$
램프 함수	$f(u) = \max(u, 0)$	$f'(u) = \begin{cases} 1 & u \geq 0 \\ 0 & u < 0 \end{cases}$

다음으로, 앞에서 계산한 $\Delta^{(l)}$ 을 이용하여 오차함수의 기울기를 계산한다. 가중치 $w_{ji}^{(l)}$ 에 대한 오차함수 $E = \frac{1}{N} \sum_{n=1}^N E_n(\mathbf{W})$ 의 미분을 (j, i) 성분으로 갖는 행렬을 $\partial \mathbf{W}^{(l)}$, 바이어스 $b_j^{(l)}$ 에 대한 미분을 j 성분으로 갖는 벡터를 $\partial \mathbf{b}^{(l)}$ 로 나타내면, 각 층 $l = 2, \dots, L$ 에 대해 층에 속하는 유닛에 대한 이들 값을 층마다 병렬로(순전파 혹은 역전파 계산처럼 층에 따른 순서의 제약 없이) 계산할 수 있다.

$$\begin{aligned}\partial \mathbf{W}^{(l)} &= \frac{1}{N} \Delta^{(l)} \mathbf{Z}^{(l-1)\top} \\ \partial \mathbf{b}^{(l)} &= \frac{1}{N} \Delta^{(l)} \mathbf{1}_N\end{aligned}$$

우변의 행렬곱은 현재 미니배치의 샘플에 대한 오차함수의 합이

$E = \sum_{n=1}^N E_n(\mathbf{W})$ 이므로, 각 샘플 n 에 대한 E_n 의 미분의 합을 구하게 된다.

마지막으로, 계산한 기울기를 이용하여 파라미터를 업데이트한다. 기울기가 하강하는 방향으로 업데이트하려면 $\mathbf{W}^{(l)}$ 과 $\mathbf{b}^{(l)}$ 의 업데이트 값을 다음으로 결정하고,

$$\begin{aligned}\Delta \mathbf{W}^{(l)} &= -\epsilon \partial \mathbf{W}^{(l)} \\ \Delta \mathbf{b}^{(l)} &= -\epsilon \partial \mathbf{b}^{(l)}\end{aligned}$$

각각 다음과 같이 업데이트한다.

$$\begin{aligned}\mathbf{W}^{(l)} &\leftarrow \mathbf{W}^{(l)} + \Delta \mathbf{W}^{(l)} \\ \mathbf{b}^{(l)} &\leftarrow \mathbf{b}^{(l)} + \Delta \mathbf{b}^{(l)}\end{aligned}$$

가중치 감쇠(3.5.2절) 및 모멘텀(3.6.5절)을 적용하는 경우에는 업데이트 값이 다음과 같이 된다.

$$\begin{aligned}\Delta \mathbf{W}^{(l)} &= \mu \Delta \mathbf{W}^{(l)'} - \epsilon (\partial \mathbf{W}^{(l)} + \lambda \mathbf{W}^{(l)}) \\ \Delta \mathbf{b}^{(l)} &= \mu \Delta \mathbf{b}^{(l)'} - \epsilon \partial \mathbf{b}^{(l)}\end{aligned}$$

이때 $\Delta \mathbf{W}^{(l)'}$ 및 $\Delta \mathbf{b}^{(l)'}$ 는 이전 반복 때의 업데이트 값이다.

4.4.3 기울기의 차분근사 계산

최적화 계산은 위와 같은 행렬 계산으로 나타낼 수 있으나, 신경망의 구조나 활성화 함수에 따라서는 이를 실행하기 위한 코드가 상당히 복잡해진다.

오차함수의 기울기를 올바르게 계산하고 있는지를 검증하기 위해 기울기를

수치미분으로도 계산한 뒤, 두 가지를 비교해 보는 것이 좋다. 오차함수 E 의 기울기에 대한 차분근사(difference approximation)는 다음과 같이 계산할 수 있다.

$$\frac{\partial E}{\partial w_{ji}^{(l)}} = \frac{E(\dots, w_{ij}^{(l)} + \varepsilon, \dots) - E(\dots, w_{ij}^{(l)}, \dots)}{\varepsilon}$$

이때 $E(\dots, w_{ij}^{(l)} + \varepsilon, \dots)$ 는 $E(\mathbf{w})$ 의 변수 $\mathbf{w}$ 안에서 $w_{ij}^{(l)}$ 에만 ε 을 더한 것이다. $\varepsilon \rightarrow 0$ 인 극한을 취하면, 위의 식은 편미분 $\partial E(\mathbf{w})/\partial w_{ij}^{(l)}$ 와 같다.

ε 은 근사의 정밀도를 확보하기 위해 충분히 작은 값으로 정해야 하지만, 너무 작은 값을 정해도 오차가 커지기 때문에(자릿수 버림이나 반올림 등에 의해) 조심스럽게 결정해야 한다. 이때 컴퓨터의 입실론 값(machine epsilon)*이 ε_c 일 때 가중치 w_{ji} 의 크기에 맞춰 다음으로 정하는 것이 일반적이다.

$$\varepsilon \equiv \sqrt{\varepsilon_c} |w_{ji}|$$

이때 우변 값이 0이 되는 경우에는 $\varepsilon \equiv \sqrt{\varepsilon_c}$ 로 재설정한다. 더 복잡한 방법을 통해 근사치의 정밀도를 높이는 방법도 있다. 이를 포함한 수치미분에 대한 좀 더 엄밀한 기술은 수치계산 문헌^[35, 55]을 참조하기 바란다.

4.5 기울기 소실 문제

4.4.2절에서 보았듯이, 순전파 및 역전파 계산은 모두 층 단위의 행렬 계산으로 나타낼 수 있으며, 식의 형태가 매우 닮았다. 그러나 이 두 가지 사이에는 중요한 차이가 있다. 순전파는 비선형 계산인데 비해, 역전파는 선형 계산이라는 차이다.

* 컴퓨터의 부동소수점 표현으로 나타낼 수 있는 '1보다 큰 최소의 수와 1의 차이'를 머신 입실론이라 부른다.

식 4-14에 나온 순전파 계산에서는 각 층에 대한 입력은 유닛이 갖는 활성화 함수를 경유하기 때문에, 활성화 함수가 비선형이라면 이 층의 입출력의 관계도 비선형성을 갖는다. 그 결과, 로지스틱 함수를 예로 들자면 각 층의 출력은 항상 $[0, 1]$ 의 범위로 제약되며, 값이 지나치게 커져서 발산해 버리는 일은 일어나지 않는다. 한편, 식 4-15에 나온 역전파 계산은 선형 계산이다. 선형 계산을 여러 번 반복하는 형태로, 델타가 출력층에서 입력층으로 전해지는 전체 계산도 선형 계산이 된다. 그 결과, 각 층의 가중치의 값이 크면 델타가 각 층을 거쳐 전달되는 도중에 급속하게 커지거나(발산), 혹은 반대로 기울기가 작으면 급속하게 작아져 0이 되어 버린다. 어떤 경우든 가중치의 업데이트가 잘 안 되고, 나아가 학습 자체가 어렵게 된다.

이 문제를 기울기 소실 문제(vanishing gradient problem)라 부른다. 2층 정도의 얇은 신경망에서는 이 델타의 발산이나 소실이 큰 문제가 되지 않지만, 그보다 많은 층을 갖는 신경망에서는 심각한 문제라서 학습을 진행하는 데 가장 큰 장애가 된다. 3.6절에서 설명한 트릭 중 몇 가지는 이 문제에 어느 정도 유효하게 작용하지만, 충분한 해결책이 되지는 못한다.

이렇게 기울기 소실 문제는 여러 층을 갖는 신경망의 학습을 방해하는 큰 장애물이며, 이것을 어떻게 해결하는가가 신경망 연구에서 오랜 숙원이었다. 이를 회피하는 방법으로 제안된 것이 5.6절에서 설명할 사전훈련이라는 방법이다. 이 방법은 이렇듯 어려웠던 다층 신경망의 학습을 가능케 하였고, 다시 연구자들의 관심을 신경망으로 돌리는 계기가 되었다.

5

자기부호화기

자기부호화기는 목표 출력 없이 입력만으로 구성된 훈련 데이터로 비지도 학습을 수행하여 데이터의 특징을 잘 나타내는, 더 나은 표현을 얻는 것이 목표인 신경망이다. 변 비트율들의 사전훈련, 즉 그 가중치의 중은 초깃값을 얻는 목적으로 이용된다. 이런 경우에는 자기부호화기의 기초와 목적을 설명한 후에 표현의 과잉피생과 학습 문제의 훈련 데이터의 배색화 등 몇 가지 중요한 관련 사항에 대해서도 다룬다.



5.1 개요

그림 5-1(a)와 같은 단층의 앞먹임 신경망을 생각해 보자. 지금까지와 마찬가지로, 입력 x 가 그대로 입력층의 출력력이 되어 출력층에서 나오는 출력 y 는 다음과 같다.

$$y = f(Wx + b)$$

그림 5-1(b)와 같이 이 신경망의 출력층에 출력층에서 본 거울상을 반복시켜서 입력층 그리고 출력층과 같은 수의 유닛을 갖는 2층의 신경망을 만든다. 새롭게 추가된 층의 가중치를 $\bar{W}$, 바이어스를 $\bar{b}$ 라고 하자. 이렇게 만들어진 2층의 신경망은 첫 번째 층에서는 위의 식에 따라 입력 x 를 y 로 변환하고, 다음 층에서는 이렇게 얻어진 y 를 입력 x 와 같은 공간으로 되돌리는 변환을 행한다. 변환의 결과를 $\hat{x}$ 로 쓰고, y 로부터 $\hat{x}$ 로의 변환을 다음과 같이 쓰기로 하자.

$$\hat{x} = \bar{f}(\bar{W}y + \bar{b})$$

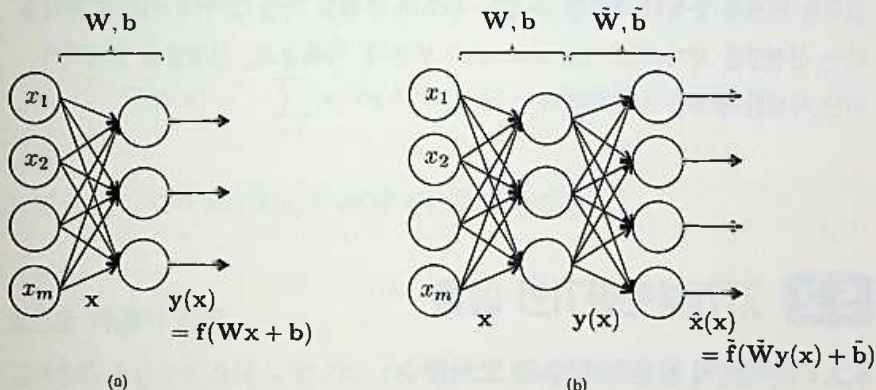


그림 5-1 간단한 신경망과 이것의 거울상으로 만든 2층 자기부호화기

여기서 $\bar{f}$ 는 새로 추가한 층의 활성화 함수로, 일반적으로 가장 첫 층의 활성화 함수 f 와 다른 것을 사용해도 무방하다. 이러한 두 가지의 변환을 합하면, x 로부터 출력 $\hat{x}$ 로의 변환을 다음으로 쓸 수 있다.

$$\hat{x}(x) = \tilde{f}(\tilde{W}f(Wx + b) + \tilde{b})$$

이러한 2층 신경망의 가중치와 바이어스를 잘 조절하여 입력 x 에 대하여 출력 $\hat{x}$ 가 원래의 입력 x 와 최대한 가까워지도록 하는 것을 생각해 보자. 정확하게는 다음의 훈련 데이터가 주어졌을 때,

$$x_1, x_2, \dots, x_N$$

각 샘플 x_n 을 입력했던 시점의 신경망의 출력 $\hat{x}_n$ 이 x_n 에 평균적으로 가까워지도록 학습한다. 구체적으로 말하자면, 5.2절에서 보일 오차함수를 3장에서 설명한 확률적 경사 하강법으로 최소화한다.

우리의 관심사는 이렇게 훈련한 신경망에 어떤 입력 x 를 주었을 때의 중간층의 출력 y 다. x 에 의해 결정되는 y 를 x 의 부호(code)라고 보고, 첫 번째 변환 $y = f(Wx + b)$ 를 부호화(encode), 두 번째 변환 $\hat{x} = \tilde{f}(\tilde{W}y + \tilde{b})$ 를 복호화(decode)라 부른다. 학습의 목표는 입력을 부호화한 뒤, 이어 다시 복호화했을 때 원래의 입력을 되도록 충실히 재현할 수 있는 부호화 방법을 찾는 것이다. 이러한 동작을 하는 신경망을 자기부호화기(autoencoder)라 부른다. 다음으로, 신경망을 설계하기 위한 자세한 방법을 소개하겠다.

5.2 자기부호화기의 설계

5.2.1 출력층의 활성화 함수와 오차함수

자기부호화기의 활성화 함수로는 중간층의 f 와 출력층의 $\tilde{f}$ 가 있다. 중간층의 f 는 자유롭게 바꿀 수 있으며, 통상적으로 비선형함수를 사용한다. 출력층의 $\tilde{f}$ 는, 신경망의 목표 출력이 입력한 x 자신이 될 수 있도록 입력 데이터의 유형에 따라 선택한다. 또, 이에 따라 x 와 $\hat{x}$ 의 거리의 척도가 되는 오차함수도 결정된다.

$\mathbf{x}$ 는 각 성분 x_i 가 실숫값을 가지며, 그 값의 범위에는 일반적으로 제약이 없다. 이때는 $\tilde{f}$ 를 항등사상으로 하는 것이 좋다. 그리고 오차함수로는 입력값과 출력값의 차에 대한 제곱합을 사용한다.

$$E(\mathbf{w}) = \sum_{n=1}^N \|\mathbf{x}_n - \hat{\mathbf{x}}(\mathbf{x}_n)\|^2$$

여기서 $\mathbf{w}$ 는 신경망의 가중치와 바이어스의 성분인 $\{\mathbf{W}, \mathbf{b}, \tilde{\mathbf{W}}, \tilde{\mathbf{b}}\}$ 를 모두 열거하여 벡터화한 것이다.

$\mathbf{x}$ 의 성분이 이진값을 갖는 경우($x_i \in \{0, 1\}$)에는 $\tilde{f}$ 로 로지스틱 함수를 사용하는 것이 자연스럽다. 오차함수로는 교차 엔트로피 $E(\mathbf{w})$ 를 사용한다.

$$E(\mathbf{w}) = \sum_{n=1}^N C(\mathbf{x}_n, \hat{\mathbf{x}}_n)$$

이때 $C(\mathbf{x}, \hat{\mathbf{x}})$ 는 다음과 같이 정의된다.

$$C(\mathbf{x}, \hat{\mathbf{x}}) = - \sum_{i=1}^D [x_i \log \hat{x}_i(\mathbf{x}) + (1 - x_i) \log \{1 - \hat{x}_i(\mathbf{x})\}]$$

x_i 및 $\hat{x}_i(\mathbf{x})$ 는 각각 $\mathbf{x}$ 와 $\hat{\mathbf{x}}(\mathbf{x})$ 의 i 번째 성분을 나타낸다.

5.2.2 가중치 공유

입력층과 중간층의 유닛 수를 각각 D_x , D_y 라고 할 때, $\mathbf{W}$, $\tilde{\mathbf{W}}$ 의 크기는 각각 $D_y \times D_x$, $D_x \times D_y$ 가 된다. 일반적으로 이 두 개의 가중치는 서로 다른 것을 사용해도 무방하지만, 다음의 경우처럼 공유하는 경우도 있다.

$$\tilde{\mathbf{W}} = \mathbf{W}^T$$

이를 가중치 공유(weight sharing, weight tying)*의 일종으로 볼 수도 있다. 어떠한 경우에 가중치를 공유하거나 하지 말아야 하는지에 대해서는 알려진 바가 없다. $\tilde{\mathbf{W}}$ 와 $\mathbf{W}$ 가 서로 관계없는 경우라도, 학습의 결과로 얻어진 두 개의 가중치가 위의 관계를 근사적으로 만족하는 경우도 자주 있다.

5.3 자기부호화기의 동작

5.3.1 데이터를 나타내는 특징을 학습

자기부호화기는 위에서 다룬 오차함수를 최소화하는 과정을 통해 신경망의 가중치와 바이어스를 결정한다. 보통, 중간층의 ($\mathbf{W}$, $\mathbf{b}$)와 출력층의 ($\tilde{\mathbf{W}}$, $\tilde{\mathbf{b}}$) 중 유용한 것은 전자뿐이다(즉, 출력층의 ($\tilde{\mathbf{W}}$, $\tilde{\mathbf{b}}$)는 사용하지 않는다). 이들 파라미터는 (데이터를 나타내는) 자질(feature)이라 불린다. $\mathbf{y}$ 의 계산은 가중치 행렬 $\mathbf{W}$ 와 $\mathbf{x}$ 의 곱셈으로 시작하지만, 이것은 $\mathbf{W}$ 의 각 행벡터와 $\mathbf{x}$ 의 내적과 같으므로, 입력 $\mathbf{x}$ 에 $\mathbf{W}$ 의 각 행벡터가 나타내는 성분이 포함된 정도를 계산해 낸다고 볼 수도 있다.

자기부호화기의 목적은 이러한 자질의 학습을 통해 샘플 $\mathbf{x}$ 의 또 ‘다른’ 표현인 $\mathbf{y}$ 를 얻는 것이다. 다만, 어떤 것이 좋은 표현인가에 대한 명확한 정의는 없다. 직관적으로 생각할 때 좋은 표현이란, $\mathbf{x}$ 를 그대로 쓰는 대신 변환된 $\mathbf{y}$ 를 사용하는 것이다. 예를 들면, 서포트 벡터 머신(Support Vector Machine, SVM)을 사용하여 클래스 분류 문제를 풀 때 $\mathbf{x}$ 를 대상으로 하는 것보다도 $\mathbf{y}$ 를 사용하면 성능이 상승한다는 얘기와 같다.

숫자 필기 인식 데이터**를 사용하여 자기부호화기를 학습한 예를 그림 5-2에 나타내었다. 샘플 $\mathbf{x}$ 는 숫자의 이미지 한 장을 그림 5-3과 같은 과정을 거쳐

* 과적합을 방지하기 위해 신경망의 자유도를 제약하는 방법 중 하나다.

** MNIST 데이터 세트(<http://yann.lecun.com/exdb/mnist>)

벡터화한 것이다. 이미지는 그레이 스케일(흑백)로, 크기는 26×26 픽셀이며, x 의 성분 수는 676이다. 이러한 샘플 1만 개*를 사용하여 학습을 수행하였다**. 그림에서 층과 층 사이에 표현된 이미지는 중간층의 각 유닛의 가중치를 이미지화한 것이다***. 이것을 보면 어수선한 패턴 속에 손글씨 숫자 형태를 띠는 성분이 도드라져 보인다. 이러한 첫 번째 층의 가중치 W 가 나타내는 자질의 조합으로 입력 이미지가 표현되며, 그로부터 두 번째 층의 가중치 $\tilde{W}$ 에 의하여 입력이 재현된다.

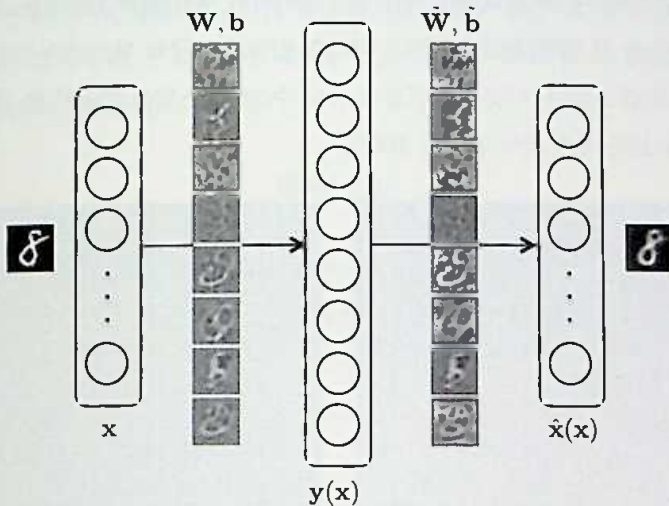


그림 5-2 자기부호화기가 동작하는 과정. 중간층의 유닛 하나의 입력 결합에 대한 가중치 $[w_{j,1}, \dots, w_{j,676}]^T$ 과 출력 결합에 대한 가중치 $[\tilde{w}_{1,j}, \dots, \tilde{w}_{676,j}]^T$ 를 각각 이미지로 시각화한 것. 중간층 100개의 유닛 중 8개를 골라 나타내었다. 그리고 그림 5-6에 가중치 모두를 시각화하였다.

- * 각각의 이미지는 서로 다른 필기체 숫자의 이미지다.
- ** 입력은 26×26 픽셀, 각 샘플로부터 모든 샘플의 평균을 뺀 것을 x 로 사용한다. 입력층과 출력층의 유닛 수는 $26 \times 26 = 676$. 중간층 유닛 수는 100이며, 활성화 함수는 중간층에 램프 함수, 출력층에 항등사상. 가중치는 $\sigma = 0.01$ 인 가우스 분포로 무작위 초기화하였으며, 학습률은 2층 모두 0.005, 바이어스는 0.01, 100개 단위로 구성된 미니배치로 확률적 경사 하강법을 수행하였다. 이때 가중치 감쇠($\lambda = 0.1$)와 모멘텀($\mu = 0.5$)을 적용하였다.
- *** 중간층의 유닛은 당연히 x 의 성분 수와 같은 수의 결합을 가진다. 그러므로 이들의 가중치를 입력과 같은 크기의 이미지로 나타내고 있다. 가중치는 음수일 값으로 가질 수도 있다. 따라서 가중치 0이 명암 256단계의 중앙에 오고 가중치의 최댓값, 최솟값이 명암 범위 상하한 값에 대응하도록 조절하였다(나머지 이미지도 같은 조절을 거침).

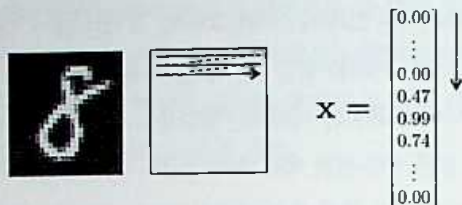


그림 5-3 이미지와 샘플 벡터 x 의 관계

같은 학습 후의 자기부호화기에 의한 입력 이미지의 재현 결과 100개를 그림 5-4에 실었다. 각 입력에 대한 재현은 각각의 입력과 조금씩 다르거나 약간 흐려진 것처럼 보인다. 이것은 중간층의 유닛 수(100개)가 입력 이미지 한 개의 자유도(676)보다 작은 것이 반영된 결과다.

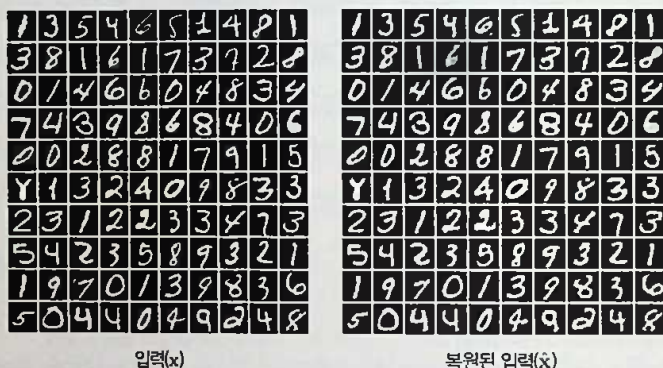


그림 5-4 MNIST 데이터 세트 중 100개의 샘플과 샘플에 대한 자기부호화기의 출력

5.3.2 주성분 분석과의 관계

자기부호화기로 어떤 표현을 학습할 수 있는지는 신경망의 구조에 따라 달라진다. 이를 결정하는 요소는 주로 중간층의 유닛의 수(y 의 성분 수와 일치함)와 해당 층에서 사용되는 활성화 함수다. 가장 단순한 경우로, 중간층의 활성화 함수 f 가 항등사상인 경우를 생각해 보자. x 가 실숫값을 갖도록 하고, 출력층의 활성화 함수 $\tilde{f}$ 로도 항등함수를 사용하자. 이때 중간층의 출력은 $y = Wx + b$ 와 같이

계산되어 전체가 나타내는 사상은 다음과 같은 선형사상이 된다.

$$\hat{\mathbf{x}} = \tilde{\mathbf{W}}(\mathbf{W}\mathbf{x} + \mathbf{b}) + \tilde{\mathbf{b}}$$

이 신경망의 중간층의 유닛 수를 늘리거나 줄였을 때 어떤 영향이 있는지 살펴보자. 입력층의 유닛 수를 D_x , 중간층의 유닛 수를 D_y 라고 하자. 먼저, 중간층이 입력층보다 큰 경우($D_y \geq D_x$)부터 살펴보도록 하자. $\mathbf{W}$ 는 $D_y \times D_x$ 행렬, $\tilde{\mathbf{W}}$ 는 $D_x \times D_y$ 행렬이고, 이들 행렬의 최대 계수는 D_x 가 된다. 이들의 곱 $\tilde{\mathbf{W}}\mathbf{W}$ 은 크기가 $D_x \times D_x$, 최대 계수는 D_x 가 되므로 $\mathbf{W}$ 와 $\tilde{\mathbf{W}}$ 의 성분을 잘 고르면 다음이 되도록 할 수 있다.

$$\tilde{\mathbf{W}}\mathbf{W} = \mathbf{I}^*$$

여기에 $\mathbf{b} = \tilde{\mathbf{b}} = 0$ 이 되도록 하면, $\mathbf{x}$ 에 상관없이 항상 입력과 출력이 일치하게 된다. 이때 $E(\mathbf{w})$ 는 항상 0이 되지만, 이것이 우리가 원하는 결과가 되는 것은 아니다. 따라서 적어도 활성화 함수가 선형함수일 때 의미 있는 결과를 얻기 위해서는 중간층의 유닛 수가 입력층보다 적은 ($D_y \leq D_x$) 것이 필요조건이 된다. 그렇게 되면 $\tilde{\mathbf{W}}\mathbf{W}$ 는 full rank가 되지 않고, $\mathbf{x} \rightarrow \hat{\mathbf{x}}$ 는 항등사상이 되지 않을 수 있다.

$D_y \leq D_x$ 인 경우, 오차함수 $E(\mathbf{w})$ 를 최소가 되게 하는 $\mathbf{W}$ 및 $\tilde{\mathbf{W}}$ 는 훈련 데이터 $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ 의 주성분 분석(Principal Component Analysis, PCA)의 결과와 실질적으로 같다¹¹⁾. 주성분 분석으로는, 데이터가 D_x 차원의 공간 안에서 퍼진 정도를 훈련 샘플 $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ 의 다음의 공분산 행렬 Φ 의 고유값, 고유벡터에 대하여 나타낸다.

$$\Phi = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^T$$

* **각주** 1)는 단위행렬을 의미한다.

데이터가 있는 저차원 부분 공간 내에서만 편재하는 경우, Φ 의 고유값의 크기순으로 선택한 고유벡터가 그 부분 공간을 나타낸다.

그럼, 고유값의 내림차순으로 Φ 로부터 D_y 개의 고유벡터를 골라 이것을 행벡터로 포함하는 행렬 U_{D_y} 을 정의한다. 이 U_{D_y} 및 훈련 샘플의 평균 $\bar{x}$ 는 다음의 최소화 문제의 해 $(\Gamma, \xi) = (U_{D_y}, \bar{x})$ 가 된다(단, Γ 는 $D_y \times D_x$ 행렬이다).

$$\min_{\xi, \Gamma} \sum_{n=1}^N \| (x_n - \xi) - \Gamma^T \Gamma (x_n - \xi) \|^2$$

식 5-1

이 점으로부터 주성분 분석, 즉 Φ 의 고유벡터는 D_y 차원 공간에 존재하는 각 샘플을 그 노름(norm)의 제곱이 최소가 되면서 가장 잘 나타내는 D_y 차원 부분 공간을 구하게 해준다고 할 수 있다.

식 5-1은 지금 논의하고 있는 자기부호화기의 오차함수와 같은 형태를 가지며, 실제로 중간층의 가중치와 바이어스를 $W = \Gamma$, $b = -\Gamma\xi$, 출력층의 가중치와 바이어스를 $\tilde{W} = \Gamma^T$, $\tilde{b} = \xi$ 로 하면 이 두 가지는 완전히 같아진다. 즉, 지금 논의하고 있는 자기부호화기는 $(\xi, \Gamma) = (\bar{x}, U_D)$ 로부터 오차함수를 최소화하는 신경망의 파라미터를 구하는 것이다. (엄밀하게 말하면, 오차함수의 최소 해에는 자유도가 있으며, 역행렬이 존재하는 임의의 $D_y \times D_y$ 행렬 Q 에 대해 $W = QU_{D_y}$ 도 최적 해가 된다.)

5.4 희소 규제화

5.4.1 데이터의 과완비한 표현

앞에서 설명한 바와 같이 자기부호화기는 입력 데이터의 자질을 학습하는 것이지만, 일반적으로 좋은 자질은 입력 데이터에서 불필요한 정보를 제거하고 그 본질만을 추출하는 것이다. 그렇다면 입력 데이터의 성분 수 D_x 보다도 그것을

부호화한(인코딩된) 부호가 갖는 성분 수 D_y 는 자연히 더 작을 것이다. 그러나 반드시 항상 그러한 것은 아니다. 이번 절에서 설명할 희소 규제화를 이용하면, 여분의 자유도를 갖는 특징이어도 입력 데이터를 잘 나타내는 자질을 얻는 것이 가능하다. 이것을 과완비(overcomplete)한 표현이라고 한다.

적어도 중간층에 활성화 함수로 선형함수를 사용한 경우, 중간층의 유닛수 D_y 가 입력층의 유닛 수 D_x 보다 많다면 무의미한 결과밖에 얻을 수 없음을 이전 절에서 설명하였다. 활성화 함수에 비선형함수를 사용하면 이러한 논의는 처음부터 성립할 수 없지만, 중간층의 자유도가 입력의 자유도보다 크다는($D_y \geq D_x$) 것은 변하지 않으며, 쓸모없는 ($x \rightarrow \hat{x}$ 가 항등사상인) 해만을 얻게 될 가능성이 높다. 이에 대하여 아래에 설명할 희소 규제화를 사용하면, 중간층의 유닛 수가 더 많은 경우($D_y > D_x$)에도 자기부호화기가 의미 있는 표현을 학습할 수 있게 된다. 이것을 희소 자기부호화기(sparse autoencoder)라고 부른다.

기본적인 아이디어는 개개의 훈련 샘플 $\mathbf{x}_n$ 을 되도록 적은 수의 중간층 유닛을 사용하여 재현할 수 있도록 파라미터를 결정하는 것이다. 입력 $\mathbf{x}$ 로부터 중간층의 출력 $\mathbf{y}$ 를 거쳐 출력 $\hat{\mathbf{x}}$ 가 계산되는 과정에 있어서, $\mathbf{y}$ 의 각 유닛 중 되도록 적은 수의 유닛만이 0이 아닌 출력치를 갖고 나머지는 출력이 0이 되도록(= 발화하지 않음) 하는 제약을 가한다.

구체적으로, 원래의 오차함수 $E(\mathbf{w})$ 에 희소 규제화 항을 추가한 $\tilde{E}(\mathbf{w})$ 을 최소화한다.

$$\tilde{E}(\mathbf{w}) \equiv E(\mathbf{w}) + \beta \sum_{j=1}^{D_y} \text{KL}(\rho \| \hat{\rho}_j)$$

식 5-2

여기서 $\hat{\rho}_j$ 는 중간층의 유닛 j 의 평균 활성화도의 추정치를 나타낸다. ρ 는 그 목표치가 되는 파라미터다. 그리고 $\text{KL}(\rho \| \hat{\rho}_j)$ 은 앞으로 설명하겠지만, 이들 두 값의 유사도를 제공한다. ρ 에 작은 값을 설정하고 식 5-2를 최소화하면

중간층의 각 유닛의 평균 활성도가 작은 ρ 에 가깝게 되고, 입력의 재현오차 $E(w)$ 가 작아지도록 w 가 정해진다. β 는 이 두 가지 목표의 밸런스를 결정하는 파라미터로, ρ 와 함께 지정한다. 그리고 평균 활성도 $\hat{\rho}_j$ 는 입력 x_n 에 대한 해당 유닛의 출력을 $y = y(x_n)$ 이라 했을 때 다음과 같이 계산한다.

$$\hat{\rho}_j = \frac{1}{N} \sum_{n=1}^N y_j(x_n) \quad \text{식 5-3}$$

이렇게 하는 것으로 각 샘플을 표현하는 데 쓰이는 중간층 유닛의 수가 적어지도록 학습이 진행된다. 이때 어디까지나 활성도의 평균값이 작아지도록 제약을 가하는 것이므로 각 샘플 x_n 마다 활성화되는 유닛은 달라도 되는 것에 주의한다. (그렇지 않으면 모든 샘플에 대해 항상 같은 유닛만 쓰이게 되므로, 그냥 유닛 수를 줄여버리는 것과 차이가 없다.)

식 5-2의 $KL(\rho \parallel \hat{\rho}_j)$ 는 다음과 같이 정의된다.

$$KL(\rho \parallel \hat{\rho}_j) = \rho \log \left(\frac{\rho}{\hat{\rho}_j} \right) + (1 - \rho) \log \left(\frac{1 - \rho}{1 - \hat{\rho}_j} \right)$$

이 식은 평균치가 각각 ρ 와 $\hat{\rho}_j$ 인 두 개의 베르누이 분포 간의 쿨백-라이블러 발산(Kullback-Leibler divergence)을 나타낸다(그림 5-5). 다시 말하면, 중간층의 유닛이 출력으로 이진값을 갖고 그 출력이 확률 ρ 로 1, $1 - \rho$ 로 0이 되는 경우, 이 유닛의 출력의 실제 분포와 이상적인 분포 사이의 유사도를 측정한다. 그리고 중간층의 출력이 이진값이 아니어도 활성화 함수에 로지스틱 함수나 램프 함수 등 함숫값이 0 이상의 값을 갖는 것을 사용한다면, (이론적인 의미는 차치하더라도) 식 5-2의 규제화는 효과가 있다.

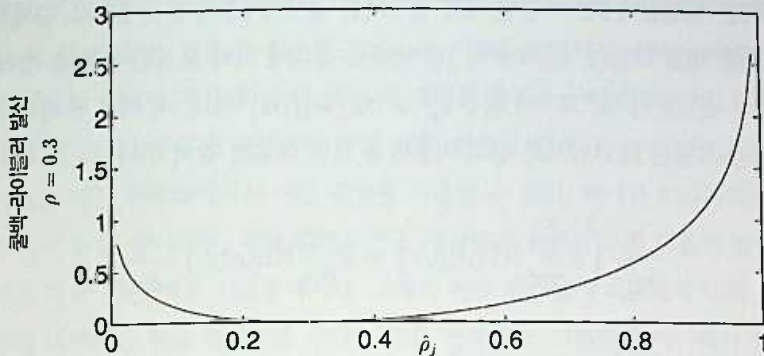


그림 5-5 평균 활성화도의 목표치 ρ 와 실측치 $\hat{\rho}_j$ 의 거리를 재는 쿨백-라이블러 발산 $KL(\rho||\hat{\rho}_j)$. $\rho = 0.3$ 일 때의 $\hat{\rho}_j$ 의 함수로 플로팅한 것이다.

5.4.2 최적화

식 5-2를 확장한 오차함수(목적함수) $\tilde{E}(\mathbf{w})$ 도 경사 하강법을 사용하여 최소화할 수 있다. 단, 추가한 희소 규제화 항은 유닛의 평균 활성화도 $\hat{\rho}_j$ 를 거쳐 $\mathbf{w}$ 에 따라 결정되므로 목적함수의 기울기를 계산할 때 이를 고려해야만 한다.

지금 논의하고 있는 희소 규제화와 비슷한 규제화로, 3.5.2절에서 설명한 가중치 감쇠가 있다. 가중치 감쇠를 적용하는 경우에 규제화 항은 $\|\mathbf{w}\|^2/2$ (단, 바이어스를 제외)이며, 이것을 오차함수에 더하여 목적함수로 삼았다. 이 항은 가중치 파라미터 그 자체에 대한 함수이므로 가중치의 업데이트 식만 수정하면 되었다. 그러나 희소 규제화 항의 경우는 해당 층 유닛의 활성화도에 대한 제약이기 때문에, 단지 그 층뿐 아니라 해당 층 아래에 있는 모든 층의 파라미터에 의해 정해지게 된다. 따라서 일반적으로는 델타의 역전파 자체를 수정해야 한다*.

* 2층 신경망만 보자면, 희소 규제화의 대상이 되는 중간층 아래에는 파라미터를 갖는 층이 없으므로 파라미터의 업데이트 식만 수정하면 된다.

아래에는 신경망이 3층 이상일 때로 일반화한 경우에 l 층의 유닛에 희소 규제화를 적용했을 때를 다룬다. l 층의 유닛 j 의 델타는 정의에 의해 오차함수를 총 입력 $u_j^{(l)}$ 으로 편미분한 것으로, 이 경우 $\delta_j^{(l)} = \partial \tilde{E}(\mathbf{w}) / \partial u_j^{(l)}$ 이다. 이 미분 중에서 $\tilde{E}(\mathbf{w})$ 가 포함된 희소 규제화 항의 미분만을 보면 다음과 같이 된다.

$$\frac{\partial}{\partial u_j^{(l)}} \left(\beta \sum_{j=1}^{D_l} \text{KL}(\rho \| \hat{\rho}_j) \right) = \beta \frac{\partial}{\partial \hat{\rho}_j} \text{KL}(\rho \| \hat{\rho}_j) \cdot \frac{\partial \hat{\rho}_j}{\partial u_j^{(l)}}$$

우변 첫 번째 미분은 다음과 같으며,

$$\frac{\partial}{\partial \hat{\rho}_j} \text{KL}(\rho \| \hat{\rho}_j) = \left(-\frac{\rho}{\hat{\rho}_j} + \frac{1 - \rho}{1 - \hat{\rho}_j} \right)$$

두 번째의 미분은 $\hat{\rho}_j = (1/N) \sum_n z_j(\mathbf{x}_n) = (1/N) \sum_n f(u_j(\mathbf{x}_n))$ 이므로 다음과 같이 된다.

$$\frac{\partial \hat{\rho}_j}{\partial u_j^{(l)}} = f'(u_j^{(l)})$$

$u_j^{(l)}$ 은 지금 대상이 되는 훈련 샘플 하나($\mathbf{x}_n$)가 입력되었을 때 해당 유닛이 받는 총 입력이다. 위의 내용을 한꺼번에 정리하면, 델타의 계산을 식 4-12 대신 다음과 같이 계산하게 된다.

$$\delta_j^{(l)} = \left\{ \sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)} + \beta \left(-\frac{\rho}{\hat{\rho}_j} + \frac{1 - \rho}{1 - \hat{\rho}_j} \right) \right\} f'(u_j^{(l)}) \quad \text{식 4-13}$$

이렇게 수정된 델타의 계산식에 따라 역전파 계산을 해서 정해진 각 값을 사용하여 $\partial \tilde{E} / \partial w_{ji}^{(l)} = \delta_j^{(l)} z_i^{(l-1)}$ 로 계산한다. 그리고 출력층은 규제화의 대상이 아니므로 (출력층의) 델타의 계산은 바뀌지 않는다.

그리고 식 5-4에서 델타를 계산할 때 유닛의 평균 활성화도 $\hat{\rho}_j$ 의 값이 필요하다. 평균 활성화도는 모든 샘플에 대한 평균이므로, 이것을 엄밀하게 구하기 위해서는 모든 샘플의 앞먹임 계산을 한 번 해서 각 유닛의 출력을 구해 둔다. 그리고 그 다음에 활성화도의 평균을 계산해야 한다. 모든 샘플을 사용하여 파라미터를 갱신하는 배치 최적화에서는 이런 방법을 사용할 수 있다. 하지만 미니배치를 사용하여 학습 중이라면, 평균 활성화도만을 계산하기 위해서 모든 샘플의 앞먹임 계산을 하는 것은 매우 비효율적이다. 그래서 이런 경우에는 미니배치 내의 모든 샘플에 대해서만 평균 활성화도를 구하는 것을 반복한다. 그러면서 미니배치 간에 다음과 같은 가중평균을 취한 값을 샘플 전체에 대한 값의 근사치로 사용한다.

$$\hat{\rho}_j^{(t)} = \lambda \hat{\rho}_j^{(t-1)} + (1 - \lambda) \hat{\rho}_j$$

$\hat{\rho}_j^{(t-1)}$ 은 이전 미니배치에서 사용한 유닛 j 의 평균 활성화도이며, $\hat{\rho}_j$ 는 현재 미니배치에서 새롭게 계산한 평균 활성화도다. λ 는 평균의 가중치로, 0.9 정도의 값을 취한다.

5.4.3 희소 규제화의 효과

위에서 설명한 바와 같은 방법으로 자기부호화기에 희소 규제화 항을 추가하여 숫자 필기 인식을 학습한 결과를 그림 5-6에 나타내었다. 해당 실험에서는 중간층의 목표 평균 활성도를 $\rho = 0.05$ 로 잡고 β 를 0.0~3.0까지 변화시켰을 때, 각 β 에 대해 학습된 특징을 그림에 표시하였다. $\beta = 0.0$, 즉 희소 규제화를 하지 않은 경우에 학습된 특징은 어수선한 패턴을 보이고 있다. 이에 비해 $\beta = 0.1$ 로 했을 때의 특징은 숫자를 분석한 '스트로크'와 같은 패턴을 보인다. 이러한 스트로크의 조합으로 임의의 숫자가 표현될 수 있으므로, 직감적으로 생각하기에는 이것들을 좋은 특징이라고 볼 수 있을 것 같다. 한편, $\beta = 3.0$ 인 경우에는 각각의 숫자가 그대로 특징으로 선택되고 만다. 규제화가 너무 강했기 때문에 단 하나의 특징으로 입력 이미지를 표현하려고 한 결과다. 이래서는 분명히

숫자의 미묘한 형상 변화를 포착할 수 없을 것이다.

자기부호화기는 입력된 각 샘플을 중간층을 구성하는 각 유닛이 '분담'하여 표현한다. 학습 시의 희소 규제화는 이런 분담의 양상을 제어하는 역할을 한다고 할 수 있다. 희소 규제화를 하지 않을 때 중간층의 유닛은 각각 제멋대로 입력을 표현하려고 한다(그림 5-6의 $\beta = 0.0$ 의 경우). 알맞은 정도로 희소 규제화가 행해지면, 입력이 갖는 구조를 효율적으로 표현할 수 있도록 중간층의 유닛이 협력하여 각각의 입력을 표현하게 된다(그림 5-6의 $\beta = 0.1$). 또, 희소 규제화가 너무 강하면 중간층 유닛이 집합으로서가 아닌, 되도록 단독으로 각각의 입력을 표현하려 한다(그림 5-6의 $\beta = 3.0$ 의 경우).

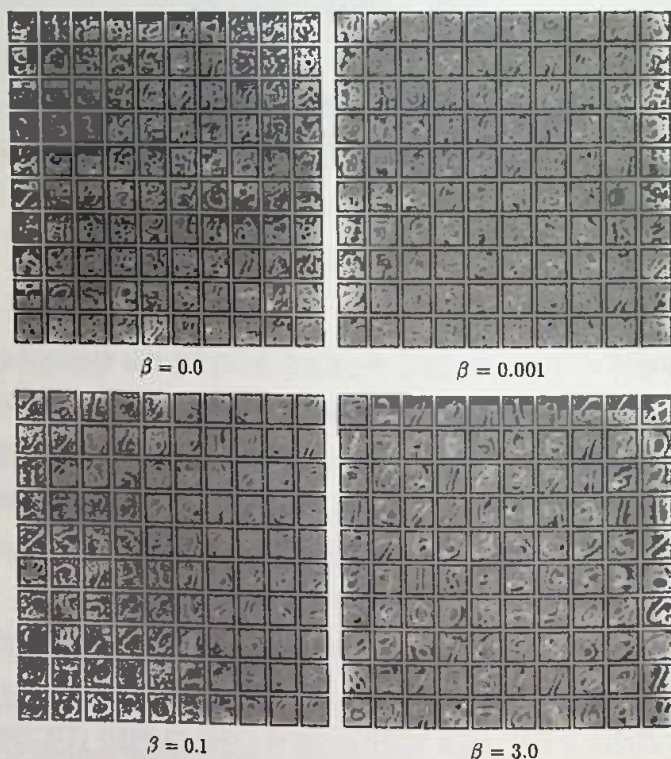


그림 5-6 희소 규제화 항의 가중치 β 에 따라 학습되는 특징의 차이

5.5 데이터의 백색화

훈련 데이터는 종종 과업을 해결하는 데 불필요한 경향을 갖는데, 이것은 학습을 방해하는 원인이 된다. 따라서 학습 전에 데이터를 처리하여 그런 데이터 내 경향을 제거하는 것은 매우 중요하다. 3.6.1절에서는 가장 기본이 되는 데이터의 정규화에 대하여 다루었지만, 이번 절에서는 그보다 더 수준 높은 과정인 백색화(whitening)에 관해 설명할 것이다. 백색화는 자기부호화기가 좋은 자질을 학습할 수 있을지를 크게 좌우할 수 있다^[11].

백색화의 목적은 훈련 샘플에서 성분 간의 상관성을 제거하는 것이다. 즉, D 차원 공간에 존재하는 샘플 $\mathbf{x} = [x_1 \cdots x_D]$ 의 임의의 두 성분 x_p 와 x_q 이 서로 상관없이 있도록 한다. (각 샘플로부터 평균을 낸 후, 각 성분을 성분 단위의 표준편차로 나누는) 정규화가 샘플의 성분 단위 처리였던 데에 비해, 백색화는 성분 간의 관계를 수정하는 처리다.

아래에는 각 샘플로부터 사전에 산출한 평균을 뺀 것을 $\mathbf{x}_1, \dots, \mathbf{x}_N$ 이라 한다. 즉, $(1/N) \sum_{n=1}^N \mathbf{x}_n = 0$ 이도록 한다. 훈련 샘플의 성분 간의 관계는 다음의 공분산행렬로 구할 수 있다.

$$\Phi_X \equiv \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n \mathbf{x}_n^T = \frac{1}{N} \mathbf{X} \mathbf{X}^T$$

여기서 $\mathbf{X} = [\mathbf{x}_1 \cdots \mathbf{x}_N]$ 이다. 이 공분산행렬 Φ 의 (p, q) 성분은 샘플 $\mathbf{x}$ 의 (p, q) 성분이 얼마나 비슷하게 변화하는지를 나타낸다. 그림 3-4(c)에서 보았듯이, 만약 성분끼리의 분산을 1로 정규화하더라도 일반적으로는 샘플이 분포하는 너비에는 성분 간의 상관이 존재하며, 이것은 Φ 의 비대각 성분이 0이 아닌 것과 등치다. 거꾸로 공분산행렬이 대각행렬이면, 그림 3-4(d)와 같이 분포의 너비가 각 축과 독립적이게 된다.

여기서 각 샘플에 $D \times D$ 행렬 $\mathbf{P}$ 에 의한 다음과 같은 선형 변환을 적용했을 때,

$$\mathbf{u}_n = \mathbf{P}\mathbf{x}_n \quad (n = 1, \dots, N)$$

변환 후의 $\{\mathbf{u}_n\}$ 의 다음의 공분산행렬이 대각행렬이 되도록 $\mathbf{P}$ 를 결정한다(D 는 샘플 $\mathbf{x}$ 가 존재하는 공간의 차원수였다).

$$\Phi_U \equiv \frac{1}{N} \sum_{n=1}^N \mathbf{u}_n \mathbf{u}_n^T = \frac{1}{N} \mathbf{U} \mathbf{U}^T$$

여기서 $\mathbf{U} = [\mathbf{u}_1 \cdots \mathbf{u}_N]$ 이다. 목표로 하는 대각행렬로 단위행렬을 선택하여, 즉 $\Phi_U = \mathbf{I}$ 로 하면 $\mathbf{P}$ 를 만족하는 식은 $\mathbf{U} = \mathbf{P}\mathbf{X}$ 를 대입하여 다음이 된다.

$$\mathbf{P}^T \mathbf{P} = \Phi_X^{-1}$$

식 5-5

식 5-5를 만족하는 $\mathbf{P}$ 는 공분산행렬 Φ_X 의 고유벡터 $\mathbf{e}_1, \dots, \mathbf{e}_D$ 를 사용하여 나타낼 수 있다. 이들 고유벡터를 열벡터로 갖는 행렬 $\mathbf{E} = [\mathbf{e}_1 \cdots \mathbf{e}_D]$ 를 정의하면, Φ_X 는 우선 고유벡터의 정의에 따라 다음과 같이 분해할 수 있다.

$$\Phi_X = \mathbf{E} \mathbf{D} \mathbf{E}^T$$

여기서 $\mathbf{D}$ 는 Φ_X 의 고유값 $\lambda_1, \dots, \lambda_D$ 를 대각성분으로 갖는 대각행렬이다. $\mathbf{E}$ 가 직교행렬이라는 것($\mathbf{E} \mathbf{E}^T = \mathbf{E}^T \mathbf{E} = \mathbf{I}$)으로부터 $\Phi_X^{-1} = \mathbf{E} \mathbf{D}^{-1} \mathbf{E}^T$ 로 쓸 수 있다. 식 5-5와 같이 이것이 $\mathbf{P}^T \mathbf{P}$ 와 같다는 사실에서 구하려는 $\mathbf{P}$ 는 다음과 같이 나타낼 수 있다.

$$\mathbf{P} = \mathbf{Q} \mathbf{D}^{-1/2} \mathbf{E}^T$$

단, $\mathbf{Q}$ 는 $\mathbf{P}$ 와 같은 크기인 임의의 직교행렬이며($\mathbf{Q}^T \mathbf{Q} = \mathbf{Q} \mathbf{Q}^T = \mathbf{I}$), $\mathbf{D}^{-1/2}$ 는 $\mathbf{D}$ 의 대각성분에 대해 제곱근의 역수를 취한 대각행렬이다. $\mathbf{Q}$ 가 임의성을 갖는 만큼 $\mathbf{P}$ 는 무수히 많은 수가 존재한다. 예를 들면, $\mathbf{Q} = \mathbf{I}$ 일 때 위의 식으로부터 얻는 $\mathbf{P}$ 는 그 무수히 많은 해 중 하나다. 공분산행렬의 고유벡터를 이용하는 것이 샘플

집합에 대한 주성분 분석(Principal Component Analysis, PCA)과 일맥상통한다는 점에서 이 $\mathbf{P}$ 를 PCA 백색화라고 부르겠다.

식 5-5를 만족하는 $\mathbf{P}$ 를 대칭행렬(즉, $\mathbf{P} = \mathbf{P}^T$)로 제한하는 방법을 생각해 볼 수 있다. 이러한 $\mathbf{P}$ 를 사용한 백색화를 영위상 백색화(Zero-phase Whitening) 또는 영위상 성분 분석(Zero-phase Component Analysis, ZCA)이라 부른다^[2]. 구하는 해는 $\mathbf{Q} = \mathbf{E}$ 로 만드는 과정에서 얻을 수 있다. 즉, 다음이다.

$$\mathbf{P}_{ZCA} = \mathbf{E}\mathbf{D}^{-1/2}\mathbf{E}^T$$

이 명칭은 $\mathbf{P}$ 의 각 행벡터를 $\mathbf{x}$ 에 적용하는 필터로 보고, 이것이 대칭인(영위상인) 것으로부터 유래한다.

그리고 어떤 백색화를 사용하는 경우에도 데이터에 따라 특정 성분의 분산이 매우 작거나 극단적으로 0이 되는 경우가 있어서 위의 $\mathbf{D}^{-1/2}$ 를 계산할 때 문제가 될 수 있다. 그런 데이터에는 작은 값 ϵ (예를 들면, 10^{-6})을 사용하여 다음과 같이 한다.

$$\mathbf{P}_{ZCA} = \mathbf{E}(\mathbf{D} + \epsilon\mathbf{I})^{-1/2}\mathbf{E}^T$$

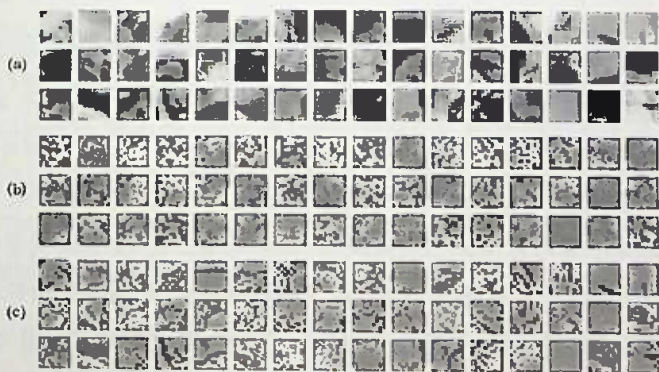


그림 5-7 자연 이미지 패치를 백색화한 예. (a) CIFAR-10의 이미지(32×32 픽셀)로부터 무작위로 50,000장을 잘라낸 10×10 픽셀 패치의 일부. (b) a의 패치를 PCA 백색화한 결과. (c) a의 패치를 ZCA 백색화한 결과. 백색화한 결과는 픽셀값 0을 회색의 중간 값으로 두고 명암값의 범위가 0에서 255까지가 되도록 변환한 것

자연의 이미지로부터 무작위로 추출한 작은 서브 이미지(패치) 집합을 백색화한 결과를 그림 5-7에 나타내었다. 그림 5-7의 (a)는 CIFAR-10*로부터 무작위로 50,000장을 추출한 10×10 서브 이미지의 일부다. 같은 그림의 (c)는 이것을 ZCA 백색화한 것이고, (b)는 PCA 백색화한 것이다. ZCA 백색화를 거친 샘플은 직류성분이 제거되어 이미지의 모서리가 강조(콘트라스트 변화가 큼)되어 있다. PCA 백색화를 거친 샘플은 고주파성분이 이미지 전체로 강조되어 원래 이미지의 공간구조가 거의 남지 않은 것처럼 보인다.

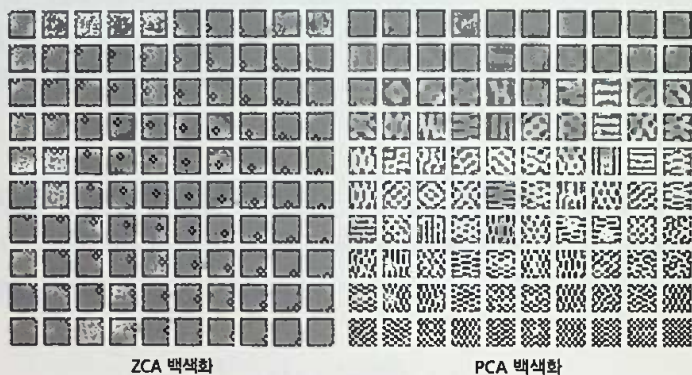


그림 5-8 ZCA 백색화와 PCA 백색화의 기저(사영행렬 $P(u = Px)$ 의 행벡터들 이미지로 시각화한 것)

이 계산을 할 때 구한 사영행렬 P 를 그 각 열벡터를 이미지화(서브 이미지로부터 x 를 만드는 순서를 뒤집어서 P 의 열벡터에 적용)한 것을 그림 5-8에 나타내었다. ZCA, PCA 백색화 모두 P 에 100개의 열벡터를 전부 표시하였다. ZCA 백색화는 각 열벡터(이미지 필터)가 하나하나 다른 픽셀을 샘플로 하고, 그 픽셀과 주위 픽셀과의 차이를 강조하는 효과를 갖고 있다(온센터(on-center)라 불린다). 이 강조 작용이 그림 5-7(c)와 같은 결과를 낳게 된다. 한편, PCA 백색화에서 각 필터는 이산코사인 변환의 기저와 매우 닮아서 각각이 특정 공간주파수 및 위상에 튜닝된 것을 알 수 있다. 자연 이미지는 일반적으로 주파수가 낮을수록 진폭이 커지는 경향이

* 물체 유형 카테고리 인식을 위한 데이터 세트다(<http://www.cs.toronto.edu/~kriz/cifar.html>).

있지만, PCA 백색화는 이러한 경향을 상쇄시켜서 고주파성분을 강조하는 작용을 한다. 이런 이유로 그림 5-7(b)와 같은 결과가 얻어진다.

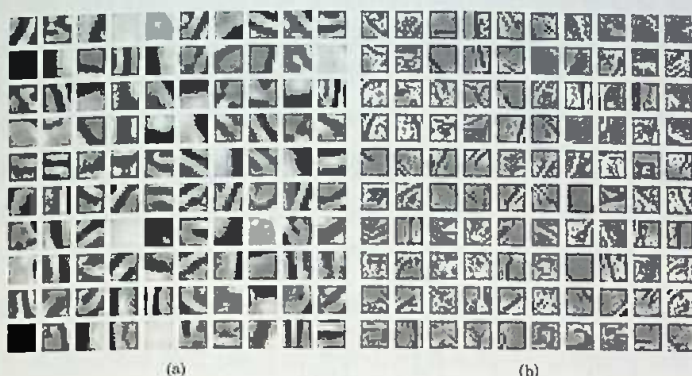


그림 5-9 데이터 백색화의 여부와 RBM 학습 후의 가중치. (a) 정규화만 한 것. (b) 영위상 백색화(ZCA)를 한 것

여기서 사용된 자연 이미지의 서브 이미지를 8장에서 설명할 제약 볼츠만 머신 (Restricted Boltzmann Machine, RBM)을 사용하여 학습한 결과를 그림 5-9에서 나타내었다. RBM은 이번 장의 자기부호화기와 거의 유사한 학습을 하지만, 다른 알고리즘을 통해 학습한다. 자세한 것은 8장으로 미루겠지만, 여기서 사용한 RBM은 가우시안-베르누이 RBM이라 불리는 것으로, 데이터의 각 성분의 평균이 0, 분산이 1인 것을 전제로 한다. 그림 5-9의 (a)는 이 전제를 만족하도록 기초적인 데이터 정규화만을 했을 때, (b)는 거기에 추가적으로 ZCA 백색화를 거치고 학습을 했을 때의 결과다*. 양쪽 모두 몇 가지 방향과 공간주파수에 튜닝된 줄무늬 모양의 명암 구조가 학습되었으나, ZCA 백색화를 거친 경우가 훨씬 섬세한 패턴이 학습된 것을 알 수 있다.

* 최소 규제화를 적용한 가우시안-베르누이 RBM을 사용하였다.

5.6 딥 뉴럴넷의 사전훈련

4.5절에서 밝힌 바와 같이, 여러 층으로 구성된 앞먹임 신경망은 기울기 소실 문제로 인해 일반적으로는 학습이 잘되지 않는다. 이것을 해결하는 방법 중 하나가 사전훈련이다^[28]. 앞먹임 신경망의 지도 학습법에서는 일반적으로 학습을 시작할 때의 초기 가중치를 랜덤값으로 초기화한다. 사전훈련법의 기본적인 아이디어는 '이 초기값을 좀 더 좋은 값으로 정하면 학습이 좀 더 잘 될 것이다'라는 것이다.

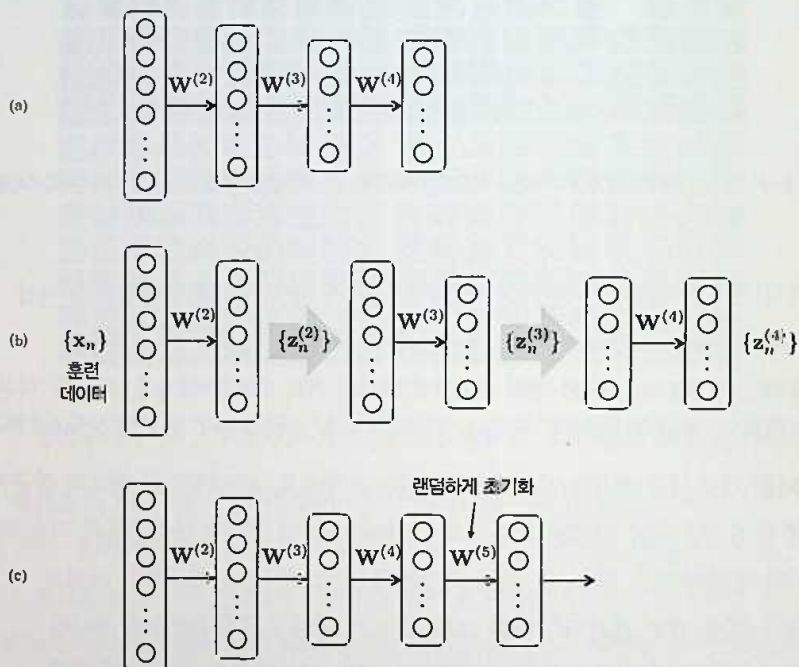


그림 5-10 적응 자기부호화기에 기초한 딥 뉴럴넷의 사전훈련. (a) 목적으로 하는 다층 신경망. (b) 각 층을 분해해서 자기부호화기로 재구성하고 가중치 W 와 바이어스(그림에서는 생략됨)를 학습. (c) (b)에서 얻은 각 층의 파라미터를 목표로 하는 신경망의 초기값으로 하고, 최상층에 가중치를 랜덤하게 초기화한 층을 추가하여 지도 학습을 실시한다.

현재 다양한 사전훈련 방법이 알려져 있으나, 가장 기본적인 것이 자기부호화기를 사용한 방법이다. 이것은 그림 5-10과 같이 먼저 목표하는 (a)의 다층 신경망을 한 층씩 여러 개의 단층 신경망으로 분할한다. 그런 다음, 분할한 단층 신경망을 입력층에서 가까운 순서대로 자기부호화기와 같은 방법으로 비지도 학습을 수행하여 각 층의 파라미터를 결정한다.

구체적인 방법은, 먼저 그림 5-10(b)의 가장 왼쪽과 같은 (원래 신경망의 입력층을 포함하는) 단층 신경망으로 자기부호화기를 구성하여 훈련 데이터 $\{x_n\}$ 을 학습시켜 이 신경망의 가중치 $W^{(2)}$ 와 바이어스 $b^{(2)}$ 를 결정한다. 다음으로, 학습한 파라미터를 이 단층 신경망에 설정한 상태에서 훈련 데이터 $\{x_n\}$ 을 여기에 입력하여 그 출력층의 표현 $\{z_n^{(2)}\}$ 를 구한다. 다음으로, 그 인접층을 포함하는 단층 신경망을 마찬가지로 자기부호화기를 만들고, 이번에는 $\{z_n^{(2)}\}$ 를 훈련 샘플로 하여 비지도 학습을 수행하여 $W^{(3)}$, $b^{(3)}$ 을 학습한다. 학습한 파라미터를 이 단층 신경망에 설정한 후, $\{z_n^{(2)}\}$ 를 입력하여 그 출력층의 표현 $\{z_n^{(3)}\}$ 을 구한다. 이후 이 과정을 층수만큼 상위층으로 올라가는 순서대로 반복한다. 이렇게 자기부호화기를 층층이 쌓은 것을 적층 자기부호화기(stacked autoencoder)라 부른다.

여기까지가 사전훈련의 과정이다. 이를 통해 각 층의 가중치와 바이어스를 얻을 수 있다. 그런 다음, 그림 5-10(c)에서처럼 사전훈련을 통해 얻은 파라미터를 원래의 신경망에 초깃값으로 설정한 뒤, 출력층을 포함하는 새로운 한 개 이상의 층을 신경망의 최상위에 추가하고, 이렇게 추가된 층의 가중치는 랜덤하게 초기화한 후에 당초의 목표였던 지도 학습을 수행한다(풀어야 하는 문제가 분류 문제라면 새로이 추가된 출력층의 활성화 함수는 소프트맥스 함수를 선택하게 된다.) 이렇게 사전훈련으로 얻은 파라미터를 초깃값으로 사용하면, 그림 5-10(c)처럼 신경망의 가중치를 랜덤값으로 초기화했을 때와 비교하여 기울기 소실 문제가 발생할 가능성이 훨씬 작고, 또 학습이 좀 더 잘 진행되는 것으로 알려져 있다.

그리고 사전훈련 후에 그림 5-10(c)와 같이 층을 추가하여 지도 학습을 수행하는 대신, 그림 5-10(a)와 같은 형태의 신경망을 ‘자질 추출기’로 이용하고 SVM 등의 일반적인 머신 러닝 방법을 사용하여 주어진 문제를 푸는 방법도 있다. 이 신경망에 x 를 입력했을 때의 최상위층의 출력 $z^{(4)}$ 를 x 의 자질값으로 간주하는 방법이다.

그러나 사전훈련이 잘 기능하는 것은 오로지 경험적인 지식일 뿐, 왜 그렇게 되는지는 이론적으로는 밝혀져 있지 않다. x 의 벡터공간 내의 데이터 $\{x_n\}$ 의 분포를 자기부호화기의 비지도 학습으로 잘 포착했기 때문이라고 추측해 볼 수는 있다. 따라서 사전훈련으로 사용하는 훈련 데이터 $\{x_n\}$ 는 목표한 지도 학습의 훈련 데이터 $\{(x_n, d_n)\}$ 의 입력 $\{x_n\}$ 을 추출하여 사용하는 것이 이치에 맞다. 또, 사전훈련만을 위한 훈련 데이터 $\{x_n\}$ 을 수집하여 사용하는 것도 가능하다. 이 경우에는 목표 출력이 불필요한 만큼 대량의 데이터를 수집하기 쉬운 이점이 있다.

5.7 그 외의 자기부호화기

5.7.1 심층 자기부호화기

5.1절부터 5.6절까지는 단층 앞먹임 신경망을 기본으로 자기부호화기를 구성하였다. 그림 5-11과 같이 여러 층의 앞먹임 신경망의 출력층을 이전 층으로 되접어 더 많은 층으로 구성된 자기부호화기를 만들 수 있다. 이러한 자기부호화기를 심층 자기부호화기(deep autoencoder)라 부른다. 여러 층으로 구성된 앞먹임 신경망의 학습이 어려웠던 것과 마찬가지로, 심층 자기부호화기의 학습도 그리 간단하지는 않다.

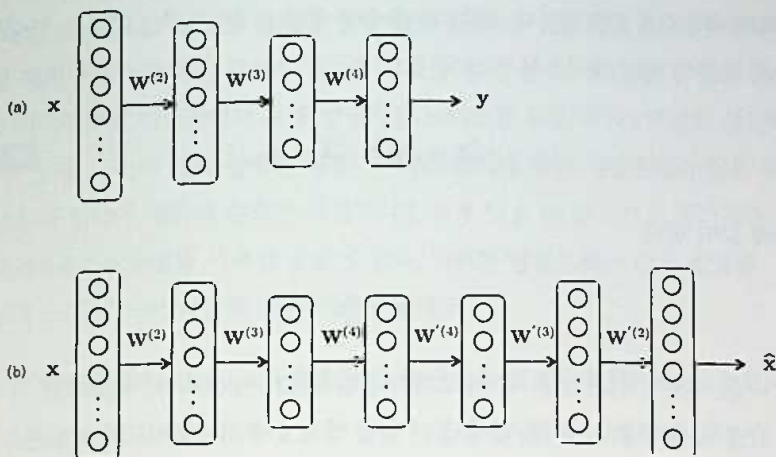


그림 5-11 심층(딥) 자기부호화기. (a) 심층 앞먹임 신경망, (b) (a)로부터 만든 심층 자기부호화기의 구조

5.7.2 디노이징 자기부호화기

자기부호화기와 같은 역할을 하는 것으로, 8장에서 설명할 제약 볼츠만 머신(RBM)이 있다. RBM도 마찬가지로, 단층구조의 신경망을 통해 비지도 학습을 수행한다. 그러나 RBM은 훈련 데이터로부터 그 확률적 생성 모델(어떤 확률분포에 따라 훈련 샘플이 생성되는가)을 학습한다. 또, 그 최적화 방법에도 확률적(랜덤)인 요소가 포함된다(자기부호화기의 최적화는 확률적 경사 하강법의 샘플 추출 시를 제외하면, 랜덤성 없이 결정론적으로 진행된다는 점에 주의한다).

이와 같이 학습 방법은 달라도 RBM의 용도나 사용법은 자기부호화기와 많이 비슷하다. 그러나 양자의 성질을 비교해 보자면, 일반적으로 RBM이 자기부호화기보다 조금 더 우수한 부분이 있다. 디노이징 자기부호화기(denoising autoencoder)^[71]는 자기부호화기를 확장한 것으로, 학습 시에 확률적인 요소를 도입하여 결과적으로는 RBM에 맞먹는 성능을 가지도록 할 수 있다.

디노이징 자기부호화기는 신경망의 구조 자체는 보통 자기부호화기와 완전히 같지만, 학습 방법이 다음과 같이 일부 다르다. 먼저, 훈련 샘플 x 를 확률적으로

(랜덤하게) 변동시켜 $\tilde{x}$ 를 얻는다. 예를 들면, x 에 평균 0, 분산 σ^2 인 가우스 분포를 따르는 다음의 랜덤 노이즈를 더하여

$$\delta x \sim N(0, \sigma^2 \mathbf{I})$$

[식 5-6]

다음과 같이 한다.

$$\tilde{x} = x + \delta x$$

그런 다음, 이렇게 만든 $\tilde{x}$ 를 자기부호화기에 입력하고, 부호화 및 복호화를 거쳐 얻은 다음의 출력이 노이즈를 더하기 전 원래 샘플 x 에 가까워지도록 학습을 진행한다.

$$\hat{x}(\tilde{x}) = \tilde{f}(\tilde{W}f(W\tilde{x} + b) + \tilde{b})$$

보통의 자기부호화기에서는 출력이 입력 자체에 가깝게 되도록 학습하기 때문에 $\tilde{x}$ 가 입력이라면 목표 출력도 항상 $\tilde{x}$ 이어야 한다. 이 부분이 일반적인 자기부호화기와의 차이다.

이 점을 제외하면 보통의 자기부호화기와 같다. 출력층의 활성화 함수가 항등사상이라면 제곱오차 $\|\tilde{x} - x\|^2$ 를, 시그모이드 함수를 사용한다면 교차 엔트로피 $C(\tilde{x}, x)$ 를 각각 오차함수로 사용하며, 이 오차가 작아지도록 신경망의 가중치와 바이어스 $\{W, b, \tilde{W}, \tilde{b}\}$ 를 확률적 경사 하강법으로 구한다. 이때 항상 x_n 을 조금 변동시킨 $\tilde{x}_n$ 을 제시하면서 원래의 x_n 이 출력되도록 신경망을 훈련시킨다. 훈련 후에는 입력을 재현할 수 있는 것뿐만 아니라 노이즈를 제거하는 능력을 기대할 수 있으며, 이것이 이 이름의 유래가 된다.

이렇게 랜덤한 변동을 가한 샘플은 $p(\tilde{x}|x)$ 의 조건부 분포로부터 샘플을 무작위로 추출하는 것이라고 볼 수 있다. 이를테면, 식 5-6의 가산적 노이즈의 경우 분포는 평균 x , 분산 σ^2 인 가우스 분포로 나타낼 수 있다.

$(p(\hat{x}|x) \sim N(x, \sigma^2 I))$. 이 분포의 선택에는 자유도가 있으므로 훈련 데이터의 발생 메커니즘과 관련된 지식이 있다면 그것을 반영시키는 것도 좋다. 예를 들어, x 가 이미지라면 이미지에서 자주 발생하는 노이즈를 포함시키는 방법을 생각해 볼 수 있다. 그러나 앞서 설명한 적층 자기부호화기에 의한 사전훈련과 같은 경우, 중간층의 입력에 대하여 유효한 사전지식은 보통 얻을 수 없으므로 일반적인 분포(변동을 주는 방법)를 사용할 수밖에 없다. 이러한 방법으로는 앞서 설명한 가산적 노이즈 외에 다음과 같은 방법이 있다.

- 마스크 형태의 노이즈. x 의 요소의 일정 비율을 랜덤하게 선택하여 0으로 한다.
- 솔트 & 페퍼 노이즈. x 의 요소의 일정 비율을 랜덤하게 선택하여 그 값을 랜덤하게 (50:50의 확률로) 상한치 혹은 하한치 중 하나로 설정한다(예를 들어, x 의 범위가 $\{0:1\}$ 이라면 0이나 1 중 하나).

6

합성곱 신경망

합성곱 신경망은 합성곱층과 풀링층이라는 특별한 두 종류의 층을 포함하는 앞막인 신경망으로, 이미지 인식에 주로 사용된다. 지금까지 다룬 앞막인 신경망과 마찬가지로, 역전파전과 선택적 경계 하강법을 사용하여 최적화를 수행한다. 합성곱 신경망의 특징은 작은感受 영역(local receptive field) 및 가중치 공유라 불리는 특별한 중간 결정을 갖는 것으로, 이 점이 지금까지 보았던 앞막인 신경망과의 차이점이다.



6.1 단순 세포와 복잡 세포

합성곱 신경망(convolutional neural network)은 주로 이미지 인식에 응용되는 앞먹임 신경망이다. 앞장에서 다뤘던 앞먹임 신경망은 모두 그림 6-1과 같이 인접층의 유닛이 모두 서로 연결된(전결합, fully-connected) 것이었다. 이에 비해 합성곱 신경망은 그림 6-1(b), 6-1(c)처럼 인접한 층과 층 사이에 특정한 유닛만이 결합을 갖는 특별한 층을 가진다. 이러한 층에서는 합성곱과 풀링이라는 이미지 처리의 기본적인 연산이 일어난다. 이것은 생물의 뇌에 있는 시각 피질(visual cortex)에 대한 신경과학적 지식으로부터 힌트를 얻어 만들어졌다. 구체적으로 말하자면, 시각 피질에 대한 신경세포의 감수 영역(receptive field)의 국소성과 단순 세포(simple cell) 및 복잡 세포(complex cell)의 존재다. 그리고 일반적으로는 감각기관으로부터 입력을 받는 신경세포에 대해서, 그 세포가 입력을 받는 감각기관상의 범위를 감수 영역이라고 부른다.

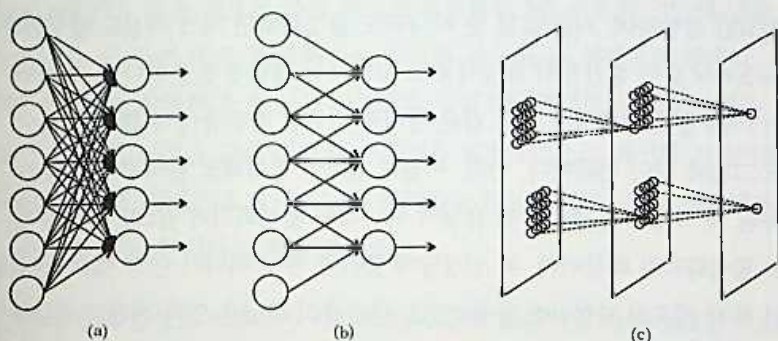


그림 6-1 앞먹임 신경망의 층간 결합의 차이. (a) 전결합층. 층마다 모든 유닛이 서로 결합한다. (b) 합성곱층과 풀링층의 구조(간략화한 표현). 위층의 유닛은 각각 정해진 소수의 아래층 유닛과 결합한다(국소 감수 영역의 구조). 또, 층간 결합은 대개의 경우 모든 상위층 유닛이 공유한다(가중치 공유). (c) 이미지를 대상으로 하는 CNN에서 합성곱층과 풀링층은 그림과 같은 구조를 갖는다.

외부로부터 눈에 들어온 빛은 망막에서 전기적인 신호로 변환된 뒤, 뇌의 시각 피질로 전달된다. 그곳에 있는 세포 중에는 망막의 특정 부분이 특정 패턴으로

입력되면 흥분하고, 그 외의 경우에는 흥분하지 않는 선택적인 행동을 보이는 세포가 있다. 특히, 망막으로부터 온 정보의 입구 역할을 하는 초기 시각 피질에 속한 신경세포에 이러한 선택적인 반응을 보이는 세포가 있다는 것이 예전부터 알려져 있었다. 예를 들면, 시야의 특정 위치에 특정한 방향이나 굽기의 선분이 제시되었을 때만 선택적으로 반응하게 된다.

이러한 세포에는 단순 세포, 복잡 세포라 불리는 두 종류의 세포가 있다^{15, 33, 34}. 두 종류 모두 특정 입력 패턴에만 반응하지만, 입력의 위치 선택성이 서로 다르다. 단순 세포는 엄격한 위치 선택성을 갖지만, 복잡 세포는 그렇게 엄격하지 않고 입력 패턴을 조금 벗어나도 반응한다. 이들의 반응은 그림 6-2와 같은 구조의 2층 신경망으로 모형화할 수 있다. 각 층의 유닛은 2차원적으로 늘어서 있어서 중간층에 속하는 유닛은 입력층의 4×4 유닛 그룹하고만 결합하고(그림 6-2 (a), (b)), 거기서 특정한 패턴이 입력되면 그에 반응하여 활성화한다. 이 패턴은 중간층의 모든 유닛이 공통적이다. 한편, 출력층에 속한 유닛은 중간층의 3×3 인 유닛 그룹하고만 결합하며, 이들 유닛 중 하나라도 활성화하면 자기 자신도 활성화한다. 그림 6-2(a)와 같이 중간층의 유닛이 4×4 의 사선 패턴에 반응한다면, 전체의 입력이 (c)와 같은 형태에서 (d)와 같은 형태로 바뀌면 중간층에서 활성화하는 유닛도 그림과 같이 변화한다. 한편, 이렇게 입력이 변화해도 출력층의 유닛은 활성화된 채 그대로다. 중간층의 유닛이 하나라도 활성화되어 있으면 출력층의 유닛도 활성화되기 때문이다. 이 신경망의 중간층 유닛 하나가 단순 세포, 출력층의 유닛이 복잡 세포의 모형이라 볼 수 있다. 중간층의 유닛은 입력 패턴의 위치에 민감하나, 출력층의 유닛은 (이 예제에서는) 3×3 범위 안에서는 둔감하다.

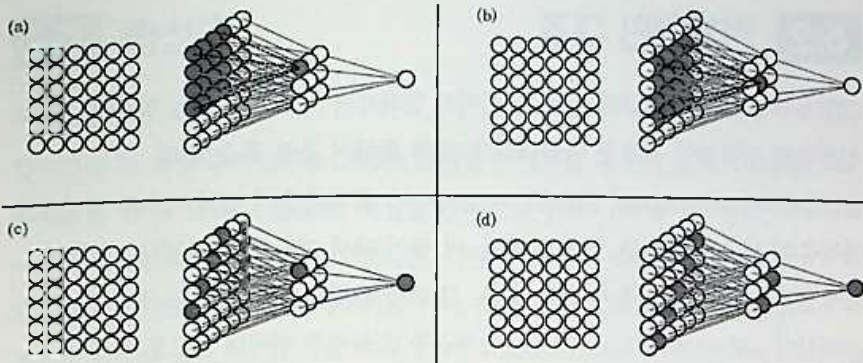


그림 6-2 단순 세포와 복잡 세포의 모형. 그림에 나온 2층 신경망의 중간층 각 유닛이 단순 세포, 출력층의 유닛이 복잡 세포에 대응한다. 중간층 각 유닛은 (a)와 (b)처럼 입력층의 $4 \times 4 = 16$ 개의 유닛하고만 결합하며, 출력층 유닛은 중간층 유닛 $3 \times 3 = 9$ 개의 유닛 중 하나가 활성화되면 활성화한다. (c)와 (d)는 입력 패턴의 위치 변화에 대해 중간층과 출력층의 상태의 변화를 보여주는 예다.

이러한 구조를 공학적인 패턴인식에 최초로 응용한 것이 네오코그니트론(Neo-Cognitron)이다^[17]. 단순 세포와 복잡 세포를 모형화한 2층 구조를 고안해 냈을 뿐만 아니라, 이 쌍이 여러 겹 반복하는 구조를 만들어 낸 점에서도 현대의 합성곱 신경망과 상통한다. 합성곱 신경망의 직접적인 뿌리로 인정받는 것은 LeNet^[42, 43]이다. 이것은 같은 구조의 신경망을 역전파법으로 계산한 기울기를 사용하여 경사 하강법으로 파라미터를 최적화한 것으로, 당시 문자 인식에서 높은 성능을 올릴 수 있다는 것을 보였다.

현재, 여러 층의 합성곱 신경망은 이미지 인식 문제 전반에서 가장 중요한 기술로 자리매김하고 있다. 한 장의 이미지로부터 그곳에 포함된 물체의 유형을 식별하는 물체 카테고리 인식(object category recognition)은 사람에게는 쉽지만 컴퓨터에는 어려운 문제 중 하나였다. 합성곱 신경망은 이러한 오랜 난제를 해결해 오고 있다. 이와 병행하여 신경과학의 분야에서는 합성곱 신경망이 영장류의 뇌에 존재하는 고차원 시각 피질과 닮은 행동을 보이는 것이 전기생리학적 실험을 통해 밝혀져 있다^[18, 75].

6.2 전체적인 구조

그림 6-3에 물체 카테고리 인식 등 이미지 인식에서 자주 사용되는 합성곱 신경망의 전형적인 구조를 나타내었다. 입력 쪽에서 출력 쪽을 향해 합성곱층(convolution layer)과 풀링층(pooling layer)이, 합성곱층이 먼저 나오고 뒤이어 풀링층이 나오는 형태로 쌍을 이루며, 이 쌍이 다시 여러 번 반복된다. 그러나 합성곱층만으로 여러 층이 반복된 후, 그 뒤에 풀링층이 한층 붙는 경우도 존재한다(그림 6-3의 $l = 1, 2, 3$ 층) 또, 합성곱층과 풀링층 뒤에다 앞으로 설명할 국소 콘트라스트 정규화(Local Contrast Normalization, LCN)층을 배치하는 경우도 있다.

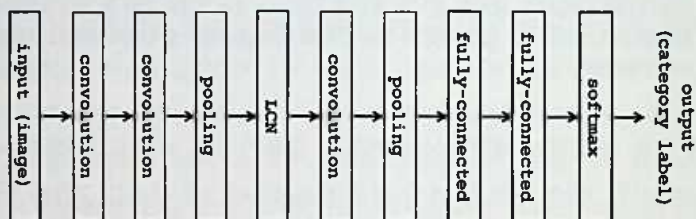


그림 6-3 물체 카테고리 인식 등 이미지 인식을 위한 전형적인 CNN의 구조. 입력에서 출력 방향으로 합성곱층과 풀링층이 짝을 이루어 반복되며, 그 뒤로 전결합층을 몇 층 거친 후 마지막에 소프트맥스 층이 배치된다. 중간에 국소 콘트라스트 정규화층이 포함되는 경우도 있다.

합성곱층과 풀링층이 반복되는 구조 뒤에는 인접한 층 사이의 유닛이 전결합한(그림 6-1(a)와 같이 모든 유닛이 결합한) 층이 배치된다. 이것을 전결합층(fully-connected layer)이라 부르며, 합성곱층과 구별한다. 전결합층은 이전 장에서 다뤘던 앞먹임 신경망의 각 층과 같은 것이다. 전결합층도 일반적으로 여러 층을 연결하여 배치한다. 마지막 출력층은 이전 장의 앞먹임 신경망과 같다. 다시 말해, 분류 작업을 목적으로 한다면 마지막 출력층은 소프트맥스 층을 사용한다.

잠시 후에 합성곱층, 풀링층, 국소 콘트라스트 정규화층에 대해 각각 순서대로 설명할 것이다. 그 전에 합성곱층에서 일어나는 합성곱의 계산을 설명하겠다.

6.3 합성곱

6.3.1 정의

강도값을 각 픽셀에 저장하는 그레이 스케일 이미지를 생각해 보자. 이미지 사이즈를 $W \times W$ 라고 했을 때 픽셀을 인덱스 (i, j) 로 나타내기로 한다. 지금까지는 인덱스를 1부터 시작했지만, 이미지(혹은 그와 비슷한 것)를 다루는 경우에는 인덱스를 0부터 시작하기로 한다. 픽셀값을 x_{ij} 라고 하고, 음수를 포함한 실수를 값으로 취하도록 하자. 그리고 필터(filter)라는 작은 크기의 이미지를 상정하고, 그 크기를 $H \times H$ 라고 하자. 필터의 픽셀은 인덱스 (p, q) ($p = 0, \dots, H-1, q = 0, \dots, H-1$)로 나타내고, 픽셀값은 h_{pq} 와 같이 쓴다. h_{pq} 역시 임의의 실수를 값으로 가진다.

이미지의 합성곱이란, 이미지와 필터 사이에 정의되는 다음과 같은 연산이다.

$$u_{ij} = \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} x_{i+p, j+q} h_{pq}$$

식 6-1

그러나 엄밀하게 말하면 이 계산은 상관(correlation)에 해당하며, 합성곱이란 원래 다음과 같은 계산을 말한다.

$$u_{ij} = \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} x_{i-p, j-q} h_{pq}$$

위와 같은 경우에 상관도 필터를 상하좌우로 반전하면 합성곱과 같은 결과를 얻게 되므로 실질적으로 두 계산 간에 차이가 없다. 그러므로 식 6-1과 같은 계산도 관용적으로 합성곱이라고 부른다.

6.3.2 합성곱의 작용

이미지의 합성곱(또는 상관)은 필터의 명암 패턴과 유사한 명암 패턴이 입력된 이미지 어디에 있는지를 검출하는 작용을 한다. 즉, 필터가 나타내는 특징적인 명암 구조를 이미지로부터 추출하는 작용을 한다.

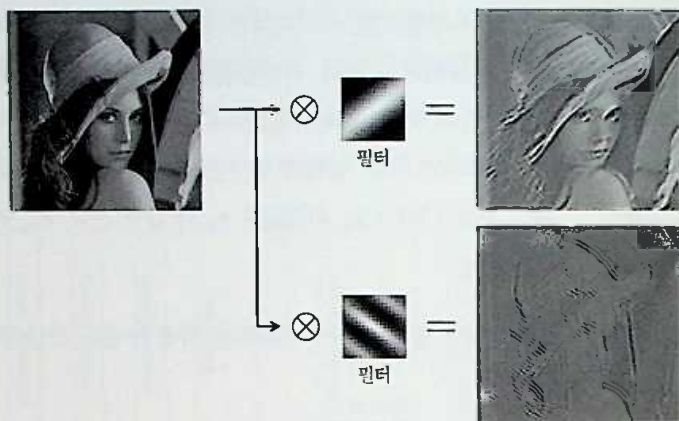


그림 6-4 필터를 합성곱한 결과의 예. 512×512 크기의 이미지에 17×17 크기의 필터 두 종류를 합성곱하였다. 필터는 보기 쉽도록 이미지보다 상대적으로 크게 확대 표시하였다. 또, 필터와 합성곱 결과 이미지는 음의 값을 갖는 픽셀이 검게, 양의 값을 갖는 픽셀은 희게 나타나도록 명암값을 조정하였다.

그림 6-4의 예제를 보자. 그림에서는 입력 이미지에 두 가지 필터를 합성곱하였다. 첫 번째 필터는 왼쪽 아래로부터 오른쪽 위로 향하는 두꺼운 선 모양의 명암 구조를 갖는다. 그 합성곱한 결과(즉, 식 6-1의 u_i)를 보면, 입력 이미지에서 유사한 구조를 갖는 부분, 이를테면 모자의 챙 부분에 강한(명암 차이가 강한) 반응이 있는 것을 알 수 있다. 두 번째 필터는 거꾸로 왼쪽 위로부터 오른쪽 아래로 향하는 선이 반복되는 구조를 갖는다. 합성곱한 결과는 어깨 부분이나 모자의 오른쪽처럼 첫 번째 필터와는 다른 부분이 강한 반응을 보이는 것이 보인다. 또, 두 번째 필터는 첫 번째 필터보다 가는 구조(높은 공간 주파수)를 가지므로 머리카락 등의 가느다란 명암 패턴에 더 강한 반응을 보이는 것도 확인할 수 있다.

6.3.3 패딩

식 6-1처럼 합성곱은 이미지에 필터를 겹쳤을 때, 이미지와 필터가 겹쳐지는 픽셀끼리 곱을 구하여 필터 전체에서 그 값을 합하는 연산이다. 따라서 이미지로부터 필터가 벗어나는 위치에 적용하는 것은 처음부터 불가능하다. 필터 전체가 이미지 내에 포함되는 상태를 유지하면서 필터를 움직여 갈 때, 합성곱 계산의 결과 이미지 크기는 $(u_{ij}$ 의 인덱스 범위) 입력 이미지의 크기보다 작아지게 된다. 이때 그 크기는 다음으로 나타낼 수 있다.

$$(W - 2 \lfloor H/2 \rfloor) \times (W - 2 \lfloor H/2 \rfloor)$$

단, $\lfloor \cdot \rfloor$ 는 소수점 이하를 버리고 정수화하는 연산자다. 그림 6-5는 8×8 크기의 입력 이미지에 크기가 3×3 인 필터를 합성곱한 결과를 나타낸다. 출력 이미지의 가로세로 화소 수는 위의 식에 따라 $8 - 2 \lfloor 3/2 \rfloor = 6$ 이 된다.

합성곱한 결과 이미지가 입력 이미지와 크기가 같으면 편리할 때가 많다. 이런 경우에는 입력 이미지의 바깥쪽에 폭 $\lfloor H/2 \rfloor$ 만큼 테두리를 둘러 크기를 늘려서 출력 이미지의 크기가 원래 입력 이미지와 같은 크기가 되도록 한다. 이 ‘테두리’ 부분의 픽셀값은 정해지지 않았으므로 모종의 방법으로 이를 결정해야 한다. 가장 일반적인 방법은 이 부분의 픽셀값을 0으로 설정하는 방법으로, 이를 제로 패딩(zero-padding)이라 부른다. 그림 6-5의 합성곱 연산에 대하여 제로 패딩을 적용한 모습을 그림 6-6에서 볼 수 있다.

CNN에서는 제로 패딩이 널리 쓰이고 있으나, 이미지 처리라는 관점에서 보면 그리 좋지 않은 부분도 있다. 제로 패딩 처리로 인해 출력 이미지의 주변부가 어둡게 되기 때문이다. 이를 막기 위해 ‘테두리’ 부분의 픽셀값을 0이 아닌 더 그럴 듯한 값으로 채우는 기법이 몇 가지 있다. 이미지가 네 변으로 타일처럼 반복되는 것처럼 생각하고 이를 정해지지 않은 부분의 픽셀값으로 사용하는 방법이나, 이미지의 맨 가장자리 픽셀값을 그대로 매워 사용하는 방법 등이 있다. 그러나 완벽한 방법은 없다.

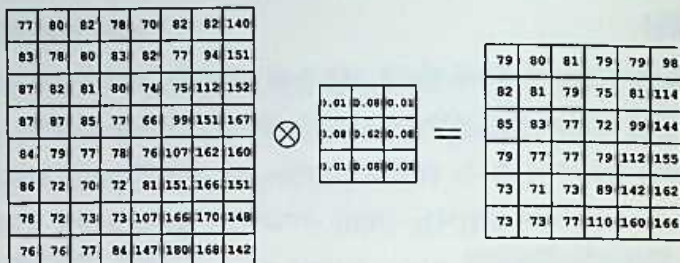


그림 6-5 크기 8×8 인 이미지와 3×3 인 필터를 합성곱하여 생성되는 이미지. 이미지 내에 필터가 완전히 포함될 수 있는 범위에서 필터를 움직이며, 각 위치마다 상관을 계산하면 출력 이미지 크기는 6×6 이 된다.

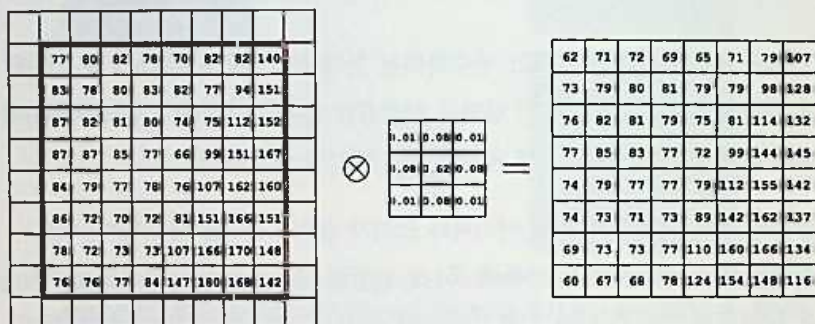


그림 6-6 크기 8×8 인 이미지 주위로 제로 패딩을 적용하여 합성곱을 한 후에도 입력과 같이 8×8 이 되도록 한 경우

6.3.4 스트라이드

여기까지는 이미지의 가로세로 방향으로 1픽셀씩 필터를 움직여가며 각 위치마다 필터와 입력 이미지 픽셀값의 곱의 합으로 계산하였다. 필터가 적용되는 위치를 1픽셀이 아니라 여러 픽셀씩 움직이며 계산하는 경우도 있다. 이 필터의 적용 위치 간격을 스트라이드(stride)라 부른다. 스트라이드 값을 s 라고 했을 때 출력 이미지의 픽셀값은 다음과 같이 계산되며,

$$u_{ij} = \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} x_{si+p, sj+q} h_{pq}$$

출력 이미지의 크기는 약 $1/s$ 배가 된다. 위에서 다룬 패딩을 거칠 때 출력 이미지의 정확한 크기는 다음과 같이 계산된다.

$$(\lfloor (W-1)/s \rfloor + 1) \times (\lfloor (W-1)/s \rfloor + 1)$$

그리고 위의 설명은 필터 적용 시작 위치를 $(i, j) = (0, 0)$ 으로 한 경우이고, 그렇지 않은 경우에는(예를 들면, $(i, j) = (1, 1)$) 크기가 그만큼 변화하게 된다.

크기가 큰 입력 이미지를 다룰 때, 합성곱층의 출력쪽 유닛 수가 너무 많아지는 것을 막기 위하여 2 이상의 스트라이드 값을 적용하는 경우가 있다. 그러나 스트라이드 값을 크게 잡는 것은 이미지의 특징을 놓칠 가능성을 의미하며, 일반적으로 성능을 악화시키는 방향으로 작용하는 경우가 많아서 가능하면 피하는 것이 좋다. 또, 뒤에 설명할 풀링층에서도 마찬가지로 스트라이드와 같은 아이디어를 적용할 수 있다. 풀링층에서는 그 목적에 걸맞게 2보다 큰 스트라이드 값을 사용하는 것이 오히려 일반적이다.

6.4 합성곱층

합성곱층은 앞에서 설명한 합성곱 연산을 하는 단층 신경망이다. 앞 절에서 설명한 예는 그레이 스케일 이미지 한 장에 하나의 필터를 합성곱하는 기본적인 것이었다. 이 기본적인 합성곱 연산을 하는 합성곱층은 그림 6-2에 나타난 2층 신경망의 중간층과 같은 것이다.

실용적인 CNN은 그레이 스케일 이미지 한 장이 아니라, 다채널 이미지에 대해 필터를 여러 개 합성곱하는 연산을 한다(그림 6-7). 다채널 이미지란, 하나의 픽셀이 여러 개의 값을 갖는 이미지를 말하며, 채널 수가 K 인 이미지의 각 픽셀은 K 개의 값을 갖는다. 예를 들면, RGB 세 가지 색으로 구성된 컬러 이미지의 채널 수

$K = 3$ 이다(그레이 스케일의 경우 $K = 1$). CNN은 이런 이미지를 입력으로 받지만, 대개 중간층(합성곱층과 풀링층)에서 그 이상의 채널 수($K = 16$ 이나 $K = 256$ 등)를 다룬다. 이미지의 가로세로 화소 수는 $W \times W$ 이고, 채널 수가 K 일 때 이 이미지의 크기를 $W \times W \times K$ 로 나타내도록 한다. 문헌에 따라서는 이러한 중간층의 출력을 이미지라 부르지 않고 맵(map)이라 부르는 경우도 있다.

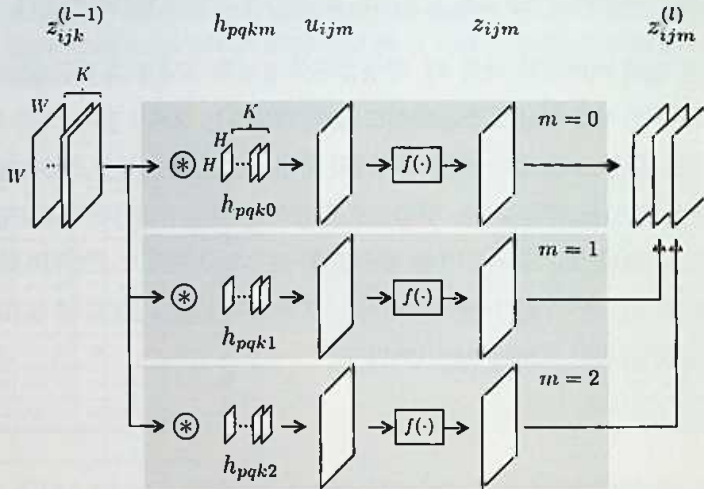


그림 6-7 합성곱층의 개요. 이 합성곱층은 K 채널을 갖는 입력 이미지에 세 가지 필터(가로세로 $H \times H$ 픽셀, 크기는 $H \times H \times K$)를 적용하여 3채널 이미지(맵)를 출력한다.

그림 6-7을 통해 합성곱층에서 일어나는 계산을 상세히 설명하겠다. 이 합성곱층은 l 번째 층에 위치하여, 바로 이전 층인 $l - 1$ 번째 층으로부터 K 채널 이미지 $z_{ijk}^{(l-1)}$ ($k = 0, \dots, K - 1$)을 입력받아 여기에 $M (= 3)$ 종류의 필터 h_{pqkm} ($m = 0, \dots, M - 1$)을 적용하는 역할을 한다.

각 필터는 입력 이미지와 같은 채널 수 K 를 가지며, 그 크기를 $H \times H \times K$ 라 한다. 그림 6-7처럼 $m = 0, 1, 2$ 의 각 필터 m 에 대하여 병렬로 계산이 실행되며, 각각 한 개 채널의 u_{ijm} 이 출력된다. 이 계산은 각 채널 k ($k = 0, \dots, K - 1$)에 대해 병렬로 이미지와 필터의 합성곱 연산을 한 후, 결과를 픽셀마다 모든 채널에서

더하는 것으로 다음과 같이 나타낼 수 있다*.

$$u_{ijm} = \sum_{k=0}^{K-1} \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} z_{i+p,j+q,k}^{(l-1)} h_{pqkm} + b_{ijm} \quad \text{식 6-2}$$

이렇게 입력 이미지의 채널 수와 상관없이 하나의 필터로부터 나오는 출력은 항상 1채널이 된다. 또, 그림에서는 생략되었지만 여기에는 바이어스가 도입되어 있다. 바이어스는 필터마다 다른 값을, 유닛에 대해서는 공통으로 사용하는 것이 일반적이다.

다음에는 이렇게 얻은 u_{ijm} 에 활성화 함수를 적용한다.

$$z_{ijm} = f(u_{ijm}) \quad \text{식 6-3}$$

그림 6-7에서 볼 수 있듯이, 이 값이 합성곱층의 최종적인 출력이며 그 다음 층으로 전달되는 값이다. 이들은 필터 수 M 과 같은 수의 채널 수를 갖는 다채널 이미지로 간주할 수 있다. 즉, 입력의 크기가 $W \times W \times K$ 일 때(합성곱층의 필터 수는 M) 출력의 크기는 $W \times W \times M$ 이 된다. 합성곱 연산의 스트라이드 값 s 가 2 이상인 경우에는 앞서 밝힌 바와 같이 출력 이미지의 가로세로 사이즈가 작아진다(약 $1/s$ 배로).

위의 계산(식 6-2와 6-3)은 층 사이의 결합에 특별한 구조가 있는 단층 신경망으로 표현할 수 있다. 먼저, 이 층의 입력과 출력 유닛 수는 각각 $W \times W \times K$ 와 $W \times W \times M$ 이다. 그리고 합성곱 연산의 국소성을 반영하여 출력층의 유닛 하나(채널 m 의 한 개 픽셀)는 입력층의 $H \times H \times K$ 개의 유닛하고만 결합한다. 이 결합의 가중치가 필터의 계수 h_{pqkm} 이다(m 은 지금 다루고 있는 채널). 이 가중치는 출력층의 같은 채널에 속하는 모든 유닛에서 같다(공유된다)는 점에 주의하도록

* 단일 채널 필터로는 입력 이미지의 국소적인 명암 구조를 추출할 수 있었다. 다채널 필터의 경우에는 그에 더하여 채널마다의 차이(이름테면, 입력 이미지에 특정한 색의 포함 여부)를 추출할 수 있다.

한다. 이 방법을 가중치 공유(weight sharing, weight tying)라 부른다. 이렇게 국소적인 결합을 가지는 것과 가중치를 공유하는 것이 합성곱층의 특징이다.

앞서 말한 바와 같이, CNN에서도 일반적인 앞먹임 신경망과 마찬가지로 경사 하강법에 의한 파라미터 최적화를 수행한다. 합성곱층의 가중치는 필터 그 자체이므로 최적화의 대상이 필터(그리고 바이어스)라는 이야기가 된다. 역전파법에 의해 합성곱층의 델타와 필터의 오차 기울기가 계산된다. 이 부분에 관한 자세한 내용은 6.7절에서 다루겠다.

6.5 풀링층

풀링층은 보통 합성곱층의 바로 뒤에 배치된다. 풀링층을 구성하는 유닛은 이번 장 처음 부분에 설명한 복잡 세포를 모형화한 것으로, 합성곱층에서 추출한 자질의 위치 감도를 약간 저하시켜 대상이 되는 자질값의 이미지 내에서의 위치가 조금씩 변화하는 경우에도 풀링층의 출력이 변화하지 않도록 해준다(그림 6-2 참조).

풀링층의 계산 내용은 다음과 같다. 크기 $W \times W \times K$ 인 입력 이미지의 픽셀 (i, j) 를 중심으로 하는 $H \times H$ 크기의 정사각형 영역을 잡아 그 영역에 포함되는 픽셀의 집합을 P_{ij} 로 나타낸다. 합성곱 연산과 마찬가지로, 적당히 패딩을 적용하면 입력 이미지의 가장자리를 포함하는 모든 점을 중심으로 하는 P_{ij} 를 만드는 것이 가능하다. 이 P_{ij} 내의 픽셀에 대하여 채널 k 마다 독립적으로 H^2 개 있는 픽셀값을 사용하여 하나의 픽셀값 u_{ijk} 를 구한다. 이 값을 구하는 방법은 몇 가지가 있으나, 이 중 최대 풀링(max pooling)은 H^2 개의 픽셀값 중 최댓값을 고르는 방법이다(그림 6-8).

$$u_{ijk} = \max_{(p,q) \in P_{ij}} z_{pqk}$$

또, 평균 풀링(average pooling)은 이들 픽셀값의 평균값을 계산하여 픽셀값으로 삼는다.

$$u_{ijk} = \frac{1}{H^2} \sum_{(p,q) \in P_{ij}} z_{pqk}$$

풀링의 계산은 입력 이미지의 각 채널마다 병렬로 실행된다. 따라서 보통 풀링층의 출력 채널 수는 입력 이미지의 채널 수와 일치한다. 또한, 이들 풀링을 포함하는 일반적인 표기로서 아래와 같은 L_p 풀링(L_p pooling)이 있다.

$$u_{ijk} = \left(\frac{1}{H^2} \sum_{(p,q) \in P_{ij}} z_{pqk}^P \right)^{\frac{1}{P}}$$

$P = 1$ 이면 평균 풀링이 되며, $P = \infty$ 이면 최대 풀링을 의미한다. 그리고 이미지 인식을 위한 응용에서는 최대 풀링이 가장 많이 쓰이고 있다.

합성곱층과 마찬가지로 풀링층에도 2 이상의 스트라이드를 설정할 수 있으며, 통상적으로 그렇게 설정한다. 그러한 경우 출력 이미지의 가로세로 크기는 입력 이미지의 크기를 $W \times W$, 풀링의 시작 지점을 (적당한 패딩을 저처) $(i, j) = (0, 0)$ 이라고 했을 때 $\lfloor (W - 1)/s \rfloor + 1$ 이 된다. 그림 6-9는 스트라이드를 2로 설정한 풀링의 계산 예를 보여준다.

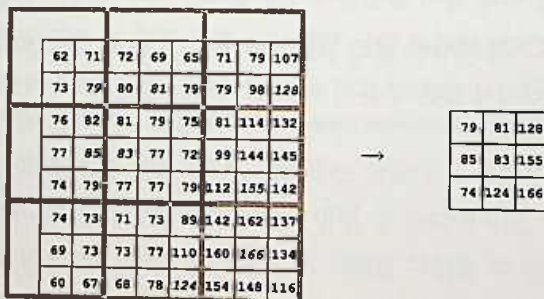


그림 6-8 풀링의 예. 풀링 사이즈는 3×3 , 스트라이드 $s = 3$, 제로 패딩을 하고 최대 풀링을 실행한 경우의 예. 굵은 선은 풀링 영역을 나타낸다.

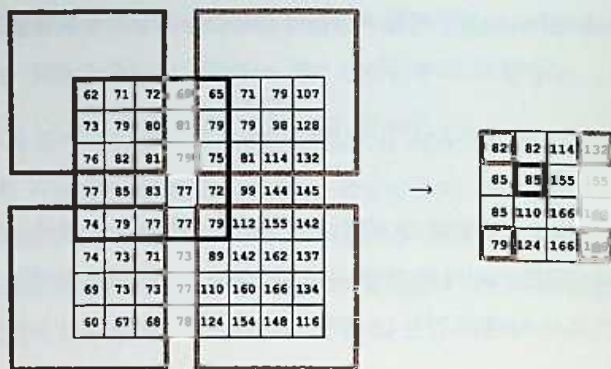


그림 6-9 풀링의 또 다른 예(풀링 영역이 출력 이미지의 인접한 픽셀에서 중복되는 경우). 풀링 사이즈는 5×5 , 스트라이드 $s = 2$, 제로 패딩을 한 후 최대 풀링을 실행한 경우의 예. 왼쪽 입력 이미지의 굵은 선은 4개의 일부 선택된 풀링 영역으로, 오른쪽 출력 이미지의 4개 픽셀에 대응한다.

풀링층의 유닛에 활성화 함수를 적용하는 것은 이론적으로는 가능하지만, 보통은 적용하지 않는다(즉, $z_{ijk} = u_{ijk}$). 풀링층도 합성곱층과 같이 2층으로 구성된 신경망으로 표현할 수 있으며, 마찬가지로 층간의 결합이 국소적으로 제한되도록 구성된다. 그러나 이 경우 결합의 가중치는 합성곱층의 필터처럼 고정값으로 되어 있어 조절할 수 없다. 따라서 풀링층에는 학습에 따라 변화할 수 있는 파라미터가 존재하지 않는다. 역전파법을 수행할 때 풀링층에는 델타에 대한 역전파 계산만을 수행한다. 자세한 것은 6.7절에서 다루도록 한다.

이번 절에서 밝힌 바와 같이 풀링층에서 수행하는 처리 자체는 단순한 것이지만, 여러 층을 갖는 CNN 내에서 갖는 역할이나 최대 풀링 및 평균 풀링의 종류에 따른 차이 등 밝혀지지 않은 부분이 아직 남아있는 상태다^[6, 7, 38, 61].

6.6 정규화층

6.6.1 국소 콘트라스트 정규화

자연물 이미지를 입력으로 하는 이미지 인식 문제에서는 입력 이미지의 전체적인 밝기나 대비의 차이를 잘 흡수해야 한다. 이미지가 촬영 시 카메라의 노출(셔터스피드, 조리개 및 이미지 센서의 게인 값 등)이나 주변의 조명에 따라 이미지 전체의 밝기나 대비가 크게 변하기 때문이다. 우리 인간은 이미지의 밝기나 대비의 차이에도 이미지에 찍힌 대상을 인식하는 것이 가능하다. 생물의 시각 시스템에는 이러한 차이를 잘 조절하는 기능이 있기 때문이다. 이것에 대응하여 이미지의 명암을 모종의 방법으로 정규화하는 방법이 필요하다.

이러한 정규화 방법으로서 먼저 이미지의 집합(훈련 데이터)에 대한 통계치를 이용하는 처리가 있다. 3.6.1절에서 다룬 정규화나 5.5절에서 설명한 백색화가 이에 해당한다. CNN에서는 학습 이미지의 픽셀 단위 평균을 구하여,

$$\bar{x}_{ijk} = \frac{1}{N} \sum_{n=1}^N x_{ijk}^{(n)}$$

대상 이미지로부터 이 평균을 뺀 다음을 CNN의 입력으로 사용하는 경우가 있다^[39].

$$x_{ijk} \leftarrow x_{ijk} - \bar{x}_{ijk}$$

단, $x_{ijk}^{(n)}$ 은 n 번째 샘플의 픽셀 (i, j) 의 채널 k 의 값을 나타낸다.

또 다른 방법은 국소 콘트라스트 정규화(local contrast normalization)가 있는데, 이 방법은 이미지 한 장 한 장에 대해 개별적으로 처리하는 방법이다. 이것은 CNN에만 쓰이는 방법은 아니고 일반적인 이미지 처리 방법 중 하나이지만, 합성곱 연산이나 풀링과 마찬가지로 한 개 층으로 이 처리를 구현할 수 있다. 역전파 계산도 가능하여 CNN에 포함시키는 것이 가능하다. 풀링층과 마찬가지로, 이 층의 가중치는 고정값이어서 학습이 가능한 파라미터는 없다.

국소 콘트라스트 정규화에는 감산 정규화(subtractive normalization)와 제산 정규화(divisive normalization) 두 가지가 있다. 이것들에 대해서는 먼저 단일 채널 이미지에 대하여 설명하고 이를 다채널로 확장하여 설명하도록 하겠다.

6.6.2 단일 채널 이미지의 정규화

단일 채널 이미지 x_{ij} 에 대하여 폴링과 마찬가지로, 픽셀 (i, j) 를 중심으로 하는 $H \times H$ 크기의 정사각형 모양의 영역 P_{ij} 를 상정하자. 감산 정규화는 입력 이미지의 각 픽셀 명암에서 P_{ij} 에 포함되는 픽셀의 명암값의 평균, 다시 말해 $\bar{x}_{ij} = \frac{1}{H^2} \sum_{(p,q) \in P_{ij}} x_{i+p,j+q}$ 를 빼는 것을 말한다^[54].

$$z_{ij} = x_{ij} - \bar{x}_{ij}$$

여기서 감산하기 위한 평균으로 가중치를 곱한 평균을 쓰는 경우도 있다.

$$\bar{x}_{ij} = \frac{1}{H^2} \sum_{(p,q) \in P_{ij}} w_{pq} x_{i+p,j+q}$$

그때 w_{pq} 는 다음과 같은 성질을 가지며,

$$\sum_{(p,q) \in P_{ij}} w_{pq} = \sum_{p=0}^{H-1} \sum_{q=0}^{H-1} w_{pq} = 1$$

영역의 중앙부에서 값이 최대가 되고 주변으로 갈수록 작아지도록 한다^[38].

영역의 중앙부를 중시하고 주변부의 영향력을 상대적으로 줄이기 위한 방법이다.

제산 정규화도 마찬가지로 국소 영역 내에서 이루어지는 조작이지만 픽셀값의 분산을 추가적으로 필요로 한다. 영역 P_{ij} 안의 픽셀값의 분산은 (가중치를 적용한 평균을 사용하면) 다음과 같지만,

$$\sigma_{ij}^2 = \sum_{(p,q) \in P_{ij}} w_{pq} (x_{i+p, j+q} - \bar{x}_{ij})^2$$

감산 정규화를 거친 입력 이미지의 픽셀값을 다시 분산의 제곱근인 표준편차로 나눈다.

$$z_{ij} = \frac{x_{ij} - \bar{x}_{ij}}{\sigma_{ij}}$$

이때, 이 계산을 그대로 수행하면 명암의 차이가 적은(대비가 작은) 국소 영역일수록 작은 명암의 차이도 증폭되어서 이미지의 노이즈가 강조되는 결과가 일어난다. 그러므로 이 계산이 입력 이미지 내에서 대비가 큰 부분에만 적용되도록 하기 위해 다음과 같은 처리를 한다. 먼저, 상수 c 를 설정하고 영역 내 픽셀의 명암에 대한 표준편차가 ($\sigma_{ij} < c$)보다 작은 때에는 픽셀값을 c 로 나누어 다음과 같이 하거나,

$$z_{ij} = \frac{x_{ij} - \bar{x}_{ij}}{\max(c, \sigma_{ij})}$$

식 6-4

같은 효과가 σ_{ij} 의 값에 따라 연속적으로 변하는 다음 수식을 적용한다.

$$z_{ij} = \frac{x_{ij} - \bar{x}_{ij}}{\sqrt{c + \sigma_{ij}^2}}$$

감산 정규화 및 식 6-4에 따른 계산 정규화의 예를 그림 6-10에 나타내었다. 이 정규화의 결과, 이미지의 픽셀값은 음수를 가질 수도 있게 된다. 그림에서는 편의상 픽셀값의 범위가 [0, 255]가 되도록 픽셀값을 선형 변환하여 표시하였다.

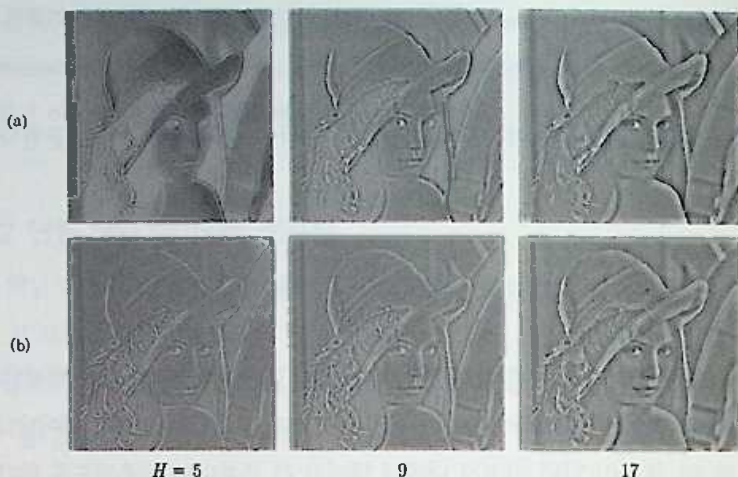


그림 6-10 (a) 감산 정규화, (b) 제산 정규화(식 6-4를 $c = 1.0$ 로 적용한 경우). 256×256 픽셀인 그레이 스케일 이미지를 $[0, 1]$ 로 명암값을 정규화하여 5×5 , 9×9 , 17×17 의 세 가지 크기로 각각을 정규화하였다. 시각화를 위해 결과를 256단계의 평균이 128이고, 최댓값이 255, 최솟값이 0이 되도록 정규화하였다.

6.6.3 다채널 이미지의 정규화

다채널 이미지를 정규화하기 위해 앞에서 설명한 그레이 스케일 이미지의 정규화를 채널마다 적용할 수도 있지만, 채널 간의 상호작용을 고려하는 경우가 일반적이다. 그 경우에 화소 값의 평균과 분산을 구하는 대상이 모든 채널에 걸친 국소 영역 P_{ij} 안의 픽셀의 집합으로 바뀌게 된다. K 채널로 이루어진 이미지 x_{ijk} 를 대상으로 할 때, 가중치를 곱한 평균을 다음과 같이 계산한다.

$$\bar{x}_{ij} = \frac{1}{K} \sum_{k=0}^{K-1} \sum_{(p,q) \in P_{ij}} w_{pqk} x_{i+p,j+q,k}$$

감산 정규화는 픽셀 (i, j) 마다 다르지만, 채널 간에는 공유되는 $\bar{x}_{ij}$ 를 빼서 다음과 같이 계산한다.

$$z_{ijk} = x_{ijk} - \bar{x}_{ij}$$

이미지의 모든 채널에 걸친 국소 영역 P_{ij} 의 분산은 다음과 같이 계산하며,

$$\sigma_{ij}^2 = \frac{1}{K} \sum_{k=0}^{K-1} \sum_{(p,q) \in P_{ij}} w_{pqk} (x_{i+p,j+q,k} - \bar{x}_{ij})^2$$

제산 정규화는 다음과 같이 된다.

$$z_{ijk} = \frac{x_{ijk} - \bar{x}_{ij}}{\max(c, \sigma_{ij})}$$

그레이 스케일 이미지와 마찬가지로 분모를 $\sqrt{c + \sigma_{ij}^2}$ 로 할 수도 있다.

다채널 이미지의 콘트라스트 정규화에서는 이렇게 국소 영역 내의 모든 채널의 픽셀을 대상으로 평균과 분산을 계산한다. 이렇듯, 채널 간의 상호작용을 고려하는 데에는 의미가 있어서 생물의 시각 특성(예를 들면, 내용의 차이에 민감하지만 밝기나 대비의 절대적 차이에는 둔감)을 설명하는 모형으로 삼기도 한다^[47, 63, 70].

6.7 기울기의 계산

합성곱층의 계산은 식 6-2의 합성곱 연산과 식 6-3의 활성화 함수 적용으로 구성된다. 6.4절에서 보았듯이, 앞부분의 계산이 조금 복잡하긴 하지만 결국 선형 계산에 지나지 않는다. 따라서 가중치 행렬 $\mathbf{W}$ 를 잘 정의하면 2장에서 다뤘던 앞먹임 신경망의 중간층(전결합층, fully-connected 층)과 마찬가지로 다음과 같이 나타낼 수 있다.

$$\begin{aligned} \mathbf{u}^{(l)} &= \mathbf{W}^{(l)} \mathbf{z}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{z}^{(l)} &= f^{(l)}(\mathbf{u}^{(l)}) \end{aligned}$$

단, $\mathbf{W}$ 는 크기 $H \times H \times K$ 인 M 개의 필터에 대한 계수 h_{pqkm} 을 식 6-2의 합성곱 인산을 제한하듯이 규칙적으로 반복시킨 것이다. 대부분의 성분이 0인 희소행렬이 된다.

이진과 계산도 전결합층에서와 기본적으로 같지만, 같은 필터의 계수가 몇 번이고 $\mathbf{W}^{(l)}$ 에 출현하는(가중치가 공유되는) 것을 고려해야 한다. 이를 표현하기 위해 필터의 계수 h_{pqkm} 으로부터 행렬 $\mathbf{W} = \mathbf{W}^{(l)}$ 을 만드는 과정을 다음과 같이 나타낸다. 먼저, 필터의 계수 h_{pqkm} 을 적당한 순서대로 늘어놓고, 성분 수가 $H \times H \times K \times M$ 개인 벡터 $\mathbf{h}$ 를 만든다. 다음으로, $\mathbf{h}$ 와 같은 길이를 가지며 $\mathbf{h}$ 와의 내적이 $l-1$ 층의 유닛 i 와 l 층의 유닛 j 사이의 가중치 w_{ij} 가 되는 벡터 $\mathbf{t}_{ji}$ 를 정의한다.

$$w_{ji} = \mathbf{t}_{ji}^T \mathbf{h}$$

$\mathbf{t}_{ji}$ 는 딱 하나의 성분만 1을 값으로 갖고 나머지 성분은 0이 되는 벡터다. 그리고 $\mathbf{t}_{ji}$ 의 성분을 t_{jir} 이라고 하고, r 이 고정되었을 때 $\mathbf{t}_{jir}$ 을 (j, i) 성분으로 갖는 행렬을 $\mathbf{T}_r$ 이라 한다.

그럼, 이 층 l 의 델타를 $\delta^{(l)}$ 이라 하고 전결합층에 대해 기울기를 계산하는 식을 형식적으로 적용하면, 이 층의 가중치 $\mathbf{W} = \mathbf{W}^{(l)}$ 의 기울기는 다음과 같이 주어진다.

$$\partial \mathbf{W} = \delta^{(l)} \mathbf{z}^{(l-1)T}$$

지금 위에서 서술한 대로 $\mathbf{W}$ 는 대부분의 성분이 0이며, 그렇지 않은 성분도 가중치 공유에 의해 같은 변수(필터의 계수)에 대응한다. 그래서 필터의 계수 $\mathbf{h}$ 에 대해 기울기 $\partial \mathbf{h}$ 를 변형해야 한다. 미분의 선형성으로부터 $\partial \mathbf{h}$ 의 성분 r 은 다음과 같이 나타낼 수 있다.

$$(\partial \mathbf{h})_r = \sum_{i,j} (\mathbf{T}_r \odot \partial \mathbf{W})_{ji}$$

⊙은 행렬의 성분 단위 곱을 나타내며, 합은 행렬의 모든 성분의 합을 말한다.

그리고 층 l 의 델타는 층 $l+1$ 의 델타 $\delta^{(l+1)}$ 및 층 $l+1$ 에 대해 마찬가지로 정의된 가중치 행렬 $\mathbf{W}^{(l+1)}$ 을 이용하여 다음과 같이 계산한다.

$$\delta^{(l)} = f^{(l)'}(\mathbf{u}^{(l)}) \odot (\mathbf{W}^{(l+1)\top} \delta^{(l+1)})$$

식 6-5

풀링층에는 학습의 대상이 되는 파라미터가 없으므로 기울기를 계산할 필요가 없지만, 보다 아래층에 전달할 델타에 대한 역전파 계산은 필요하다. 이를 위해서는 풀링층의 종류마다 $\mathbf{W}^{(l+1)}$ 을 정하고 식 6-5를 계산한다. 평균 풀링에서는 층 $l+1$ 의 유닛 j 의 크기 $H \times H$ 인 감수 영역(풀링 영역)을 P_j 라고 하고 다음과 같이 한다.

$$w_{ji}^{(l+1)} = \begin{cases} \frac{1}{|P_j|} & \text{if } i \in P_j \\ 0 & \text{otherwise} \end{cases}$$

앞에서 서술한 역전파 계산에서는 출력층에 속한 유닛의 델타에 $\mathbf{W}^{(l+1)\top}$ 를 곱하여 고르게 나누어지도록 한다. 최대 풀링에서는 층 $l+1$ 의 유닛 j 의 같은 감수 영역 내에서 앞먹임 계산을 할 때, 최댓값을 내놓는 입력층 유닛 i 에만 $w_{ji} = 1$ 로 설정하여 그 외의 유닛은 0으로 한다. 즉, 샘플마다 (앞먹임 계산 시점의 풀링 결과에 의해) 가중치가 변화한다. 역전파 계산에서는 최댓값을 내놓는 유닛의 델타가 그대로 전달된다.

6.8 실제 예: 물체 유형 인식

현재 CNN이 가장 눈에 띄는 성과를 내고 있는 물체 유형 인식을 통해 그 적용 예를 소개할 것이다. 구체적으로는 ILSVRC(ImageNet Large Scale Visual Recognition

Challenge)라는 콘테스트에서 주어진 과제 중 하나인, 입력 이미지 한 장에 찍힌 물체의 유형을 미리 정해진 1,000종의 유형 중에서 골라 답하는 문제를 다루어 볼 것이다. 훈련 데이터용으로 각 유형에 1,000장씩의 이미지가 주어지며, 샘플의 총 수는 100만 장이다. 대상이 되는 유형 1,000종으로부터 무작위로 추출된 100개를 표 6-1에 나타내었다. 학습은 지도 학습으로 실시된다.

표 6-1 1,000종류 중 무작위로 선택된 48종류의 카테고리 이름

bathtub	basset	earthstar	volleyball
banana	albatross	spindle	plastic bag
snow leopard	snail	odometer	racket
remote control	giant schnauzer	tennis ball	racket
ostrich	book jacket	mountain bike	Rhodesian ridgeback
little blue heron	studio couch	electric locomotive	yurt
screw	corn	pot	golden retriever
rock python	crib	yawl	dingo
lynx	Siberian husky	power drill	Indian cobra
ladybug	oxygen mask	greenhouse	African chameleon
pinwheel	nematode	plate	agama
rubber eraser	black swan	pencil sharpener	dhole

이 데이터로 표 6-2와 그림 6-11에 나타난 CNN을 사용한다. 이것은 2012년의 ILSVRC에서 우승한 CNN과 거의 같은 것으로, 물체 유형 인식이라는 고난이도의 과제에서 CNN의 유효함을 널리 알린 기념비적인 신경망이다^[39].

이 신경망은 다섯 개의 합성곱층, 세 개의 풀링층, 두 개의 콘트라스트 정규화층 및 세 개의 전결합층으로 구성되어 있다. 앞으로 수행할 학습을 통해 결정되는 파라미터를 가지는 것은 합성곱층과 전결합층뿐이다. fc6과 fc7층의 유닛은 3.5.3절에서 다뤘던 드롭아웃($p = 0.5$)을 적용한다.

표 6-2 층 유형/이름의 conv, pool, norm, fc는 각각 합성곱층, 풀링층, 정규화층, 전결합층을 나타낸다. 함수는 각 층에서 사용되는 활성화 함수의 예를 든다. ReL은 램프 함수를 나타낸다.

층 유형/이름	패치	스트라이드	출력 맵 크기	함수	파라미터
data	-	-	$227 \times 227 \times 3$	-	
conv1	11×11	4	$55 \times 55 \times 96$	Rel	35k
pool1	3×3	2	$27 \times 27 \times 96$	-	0
norm1	5×5	1	$27 \times 27 \times 96$	-	0
conv2	5×5	1	$27 \times 27 \times 256$	Rel	614k
pool2	3×3	2	$13 \times 13 \times 256$	-	0
norm2	5×5	1	$13 \times 13 \times 256$	-	0
conv3	3×3	1	$13 \times 13 \times 384$	Rel	885k
conv4	3×3	1	$13 \times 13 \times 384$	Rel	1,327k
conv5	3×3	1	$13 \times 13 \times 256$	Rel	885k
pool5	3×3	2	$6 \times 6 \times 256$	-	0
fc6	-	-	$1 \times 1 \times 4096$	Rel	37,748k
fc7	-	-	$1 \times 1 \times 4096$	Rel	16,777k
fc8	-	-	$1 \times 1 \times 4096$	softmax	4,096k

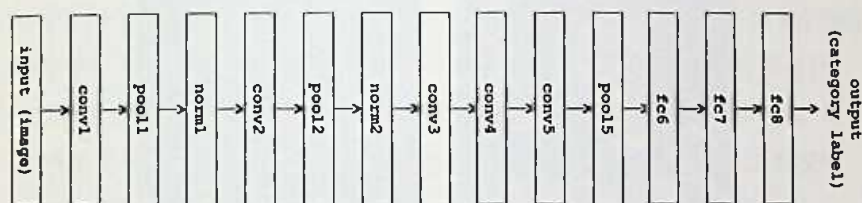


그림 6-11 ILSVRC2012에서 크리제브스키(Krizhevsky)의 연구진이 제안한 CNN의 구조. 표 6-2도 참조할 것

이 신경망을 랜덤하게 초기화하고, 미니배치의 크기를 128로 하여 확률적 경사 하강법을 수행하였을 때의 학습곡선을 그림 6-12에 나타내었다. 약 200,000개의 미니배치, 이미지 개수로는 $200,000 \times 128 / 1,000,000$ (총 샘플 수)이므로 약 25에포크 정도에서 수렴했다는 것을 알 수 있다. 그리고 훈련 샘플의 이미지(테스트

데이터도 마찬가지로는 사이즈를 강제로 256×256 으로 조정한 후, 일부 태두리를 제거한 중앙의 227×227 크기로 잘라서 그림 6-13(b)에 표시한 훈련 샘플의 평균 이미지를 빼서 입력층에 입력하였다.

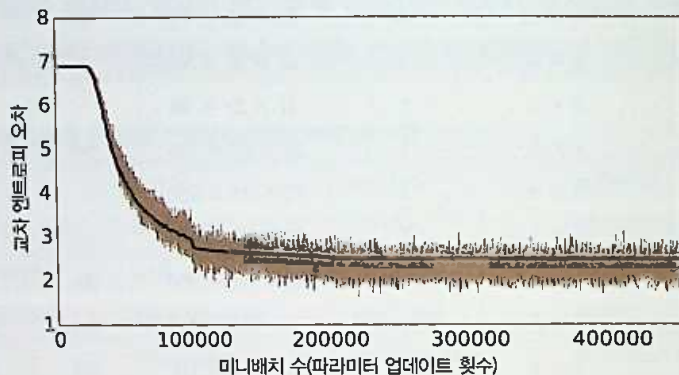


그림 6-12 학습곡선의 추이. 세로축이 교차 엔트로피, 가로축이 처리한 미니배치 수다. 변화가 심한 가는 선은 훈련 오차이며, 굵은 선이 테스트 오차다(1000 미니배치 간격으로 플로팅하여 매끈하게 보임).

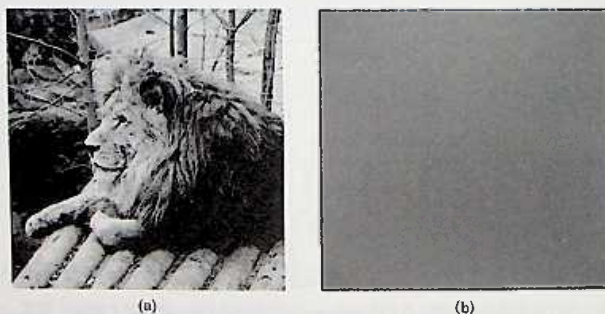


그림 6-13 (a) 테스트에 사용된 이미지(크리에이티브 커먼즈 라이선스(CC BY-SA 2.0)에 따라 이용, <http://lic.kr/elpadawan/8238633021>). (b) 입력 전에 이미지에서 빼는 평균영상

학습이 끝난 후의 CNN에 그림 6-13(a)의 이미지를 입력하였을 때 각 합성곱층, 풀링층, 정규화층에서 출력된 값을 시각화한 것을 그림 6-14부터 6-19까지 나타내었다. 그림 6-20은 그 후에 전결합층의 출력에 드롭아웃을 거친 것을 시각화한 것이다. 그림 6-20(a)에서 fc6층, (c)에서 fc7층 모두 출력이 0이 나온

유닛은 아주 많지 않고, 활성화되는 양상도 그리 희소한 상태는 아니다. 그림 6-20(e)에 나온 fc8층의 출력, 특히 (f)에 나온 소프트맥스 함수 출력값은 입력된 이미지의 유형인 'lion'에서 크게 튀고 있어 입력 이미지가 'lion'으로 정확하게 분류되었음을 나타내고 있다.

그림 6-21에는 학습이 끝난 CNN에 훈련 데이터에 없는 새로운 이미지를 입력한 테스트의 결과를 표시하였다. 모든 이미지에서 출력된 유형의 우도(소프트맥스 함수의 출력값) 상위 다섯 가지 중에 정답 유형이 포함되어 있다. 예를 들면, 'lion'이나 'zebra'는 높은 확신도(confidence)로 정확하게 인식되고 있다. 그리고 'japanese spaniel'을 'papillon'으로, 'crane'을 'snowplow'(제설차)로 각각 오답을 내었다. 전자의 두 가지는 사람이 보기에 간단하여 틀릴 것 같지 않은 이미지이며, 후자의 두 가지는 유형 분류 자체가 애매한, 사람이 분류해도 쉽지 않은 이미지다. CNN은 이렇듯, 그저 인식 성능이 높은 것뿐만 아니라 착각을 포함하는 행동 자체가 사람의 시각에 가까운 성향도 보인다.

물체 유형 인식을 위한 CNN은 정확도 향상을 위해 층수를 늘리는 것이 트렌드다⁶⁵⁾. 2014년 ILSVRC에서 2위에 오른 옥스포드 대학의 CNN VGG의 자세한 사항을 표 6-3에 나타내었다. 이 CNN은 모든 필터의 크기가 3×3 인 것이 특징이며, 학습으로 변화하는 가중치를 가진 층이 19개에 이르러, 먼저 서술한 2012년의 CNN의 두 배가 넘는 깊이로 만들어졌다. 그에 따라 파라미터 수도 약 두 배로 증가하였다. 더 큰 메모리를 갖춘 최신 GPU 덕에 이러한 대규모 확장이 가능해졌다. 그리고 VGG의 학습은 일부 층을 속아낸 11개의 합성곱층으로 시작하여, 여러 단계에 걸쳐 서서히 층수를 추가하는 방법으로 수행되었다. 각 단계마다 충분한 수의 에포크를 밟아 학습을 진행한 후, 새로운 층을 추가하는 방식으로 진행하였다. 또, 앞서 얘기한 크리제브스키의 신경망과 달리 VGG는 콘트라스트 정규화층을 전혀 사용하지 않는다. 문헌에서는 콘트라스트 정규화층이 성능 향상에 도움이 되지 않았기 때문이라고 설명하고 있다.



그림 6-14 conv1층의 96개 모든 필터($11 \times 11 \times 3$)(위), 그림 6-13(u)와 이미지 입력 시의 출력 맵 (55×55 픽셀, 시각화를 위해 명암값에 감마 보정 $(z/z_{max})^{2/5}$ 를 적용하였다)(아래)

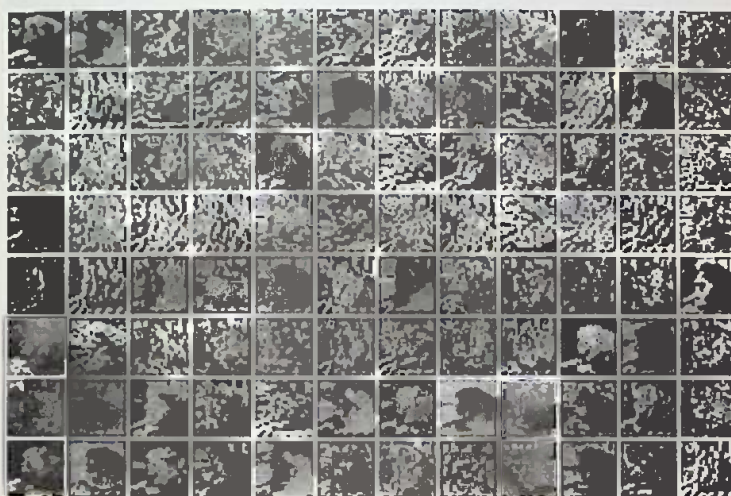
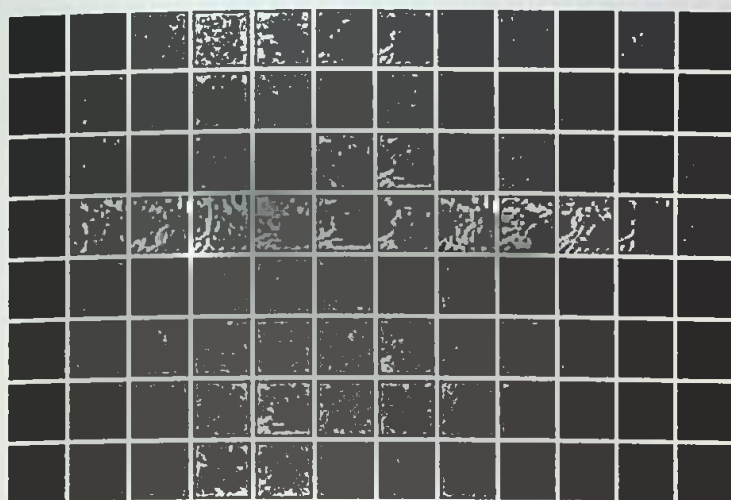


그림 6-15 pool1층의 출력 맵(27×27)(위), norm1층의 출력 맵(27×27 픽셀)(아래), 그림 6.13(㉔)의 이미지를 입력할 때의 것

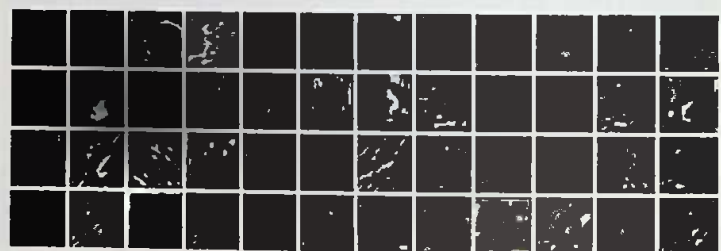
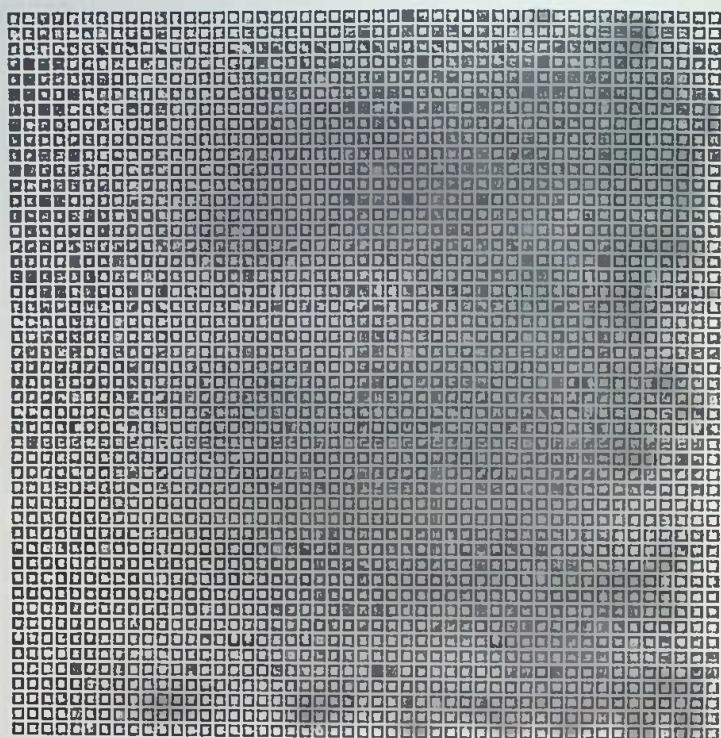


그림 6-16 conv2층의 256개 모든 필터($5 \times 5 \times 48$ 크기) 중 48개(그림의 매트릭스의 열이 각 필터에 대응)(위), 그림 6-13의 이미지를 입력할 때의 48개 필터의 출력 맵(27×27 픽셀)(아래)

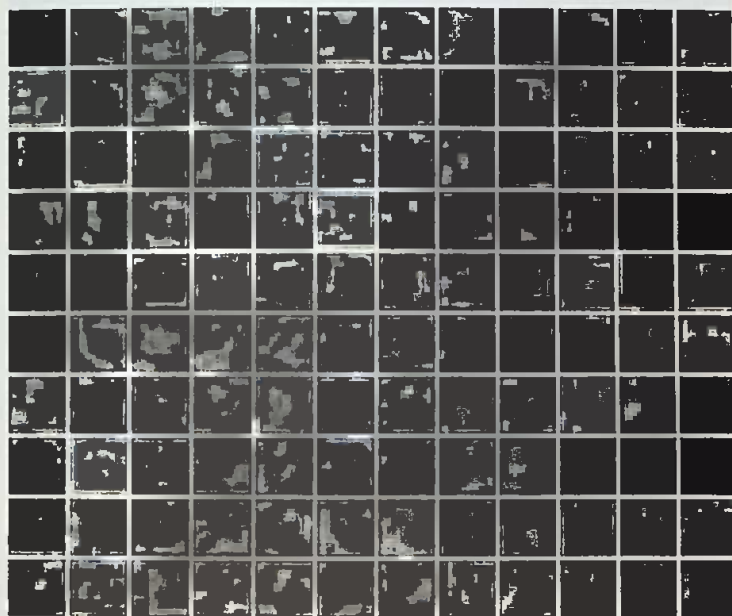
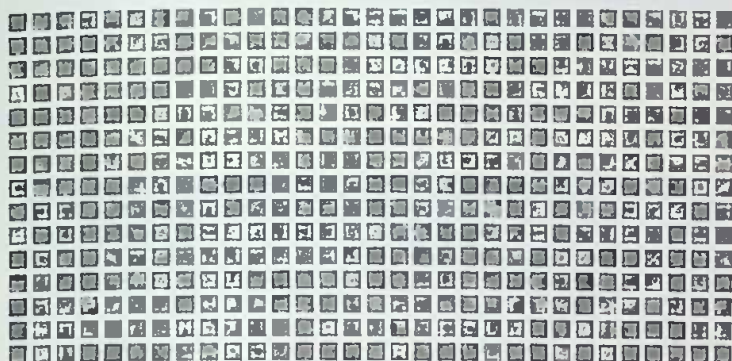


그림 6-17 conv3층의 384개 모든 필터($3 \times 3 \times 256$ 크기)의 일부(그림의 매트릭스의 열이 각 필터에 대응)(위). 그림 6-13a의 이미지를 입력할 때 그림에 나온 128개 필터의 출력 맵(13×13 픽셀)(아래)

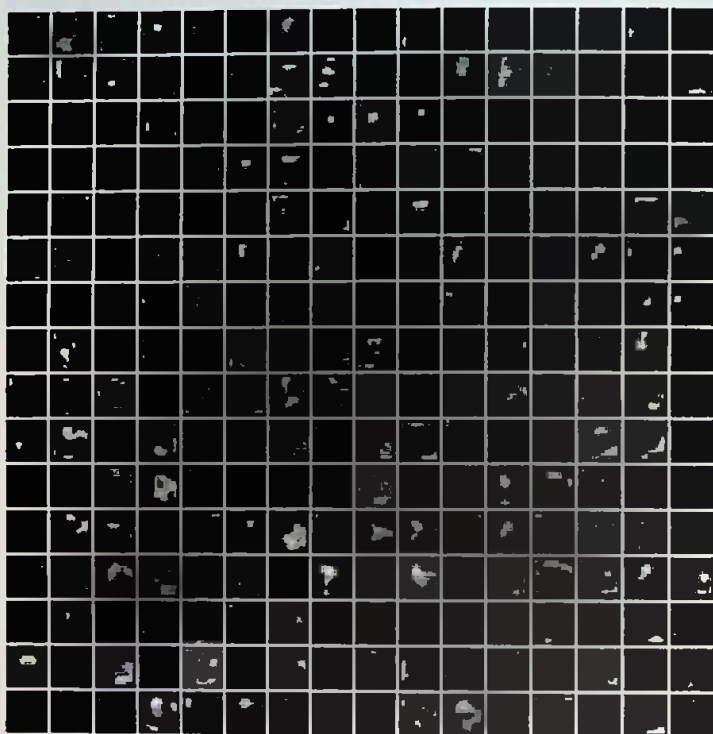


그림 6-18 그림 6-13(a)의 이미지 입력 시 conv5층의 256개 모든 출력 채널(각각 13×13)

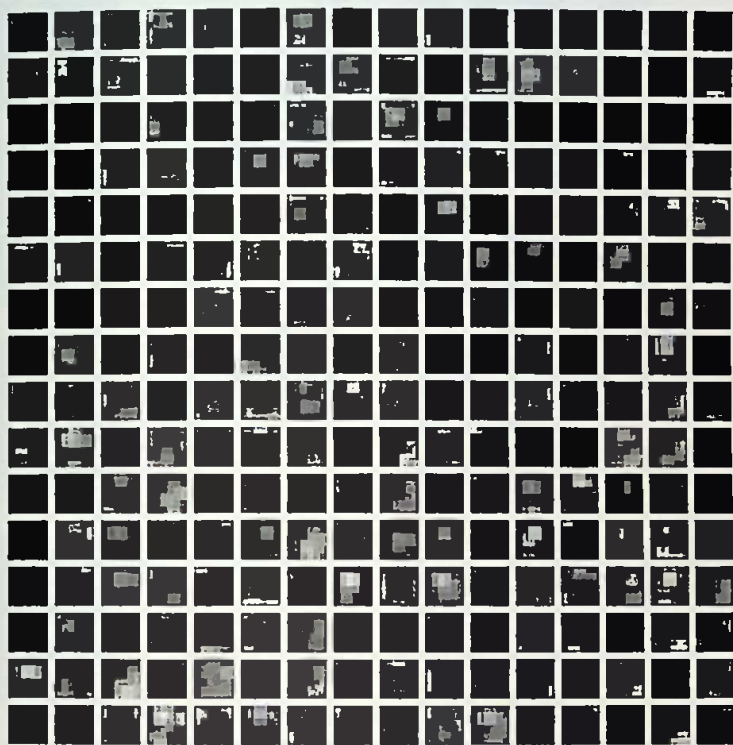


그림 6-19 그림 6-13(a)의 이미지 입력 시 pool5층의 256개 모든 출력 채널(각각 6×6)

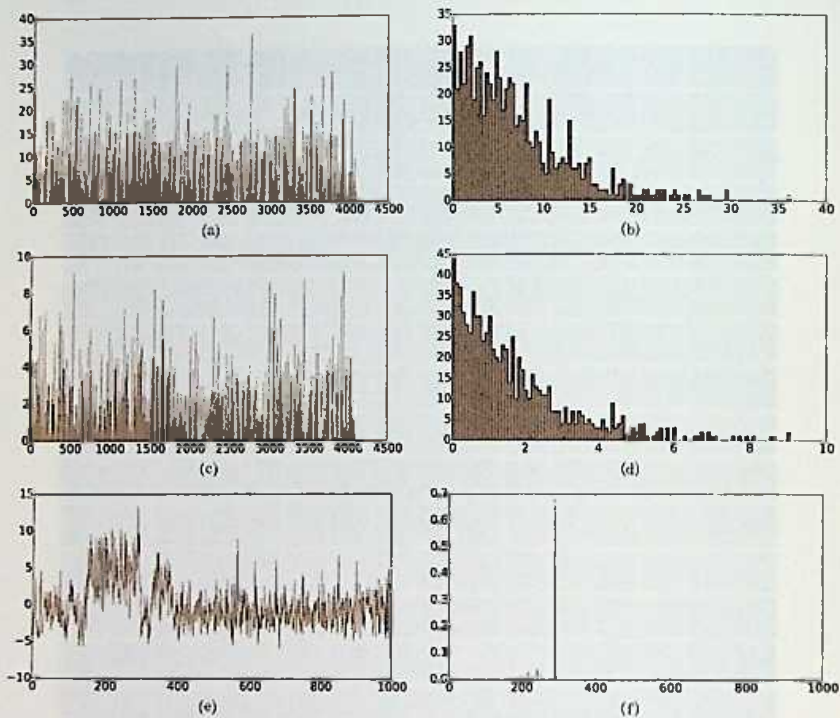


그림 6-20 그림 6-13(a)의 이미지 입력 시 전결합층의 출력. (a) fc6층의 출력. (b) (a)의 히스토그램. (c) fc7층의 출력. (d) (c)의 히스토그램. (e) fc8층의 출력(소프트맥스 함수 적용 전). (f) 소프트맥스 함수 적용 후, 적용 후에 카테고리 'lion'의 유닛 값이 피크를 그리는 것을 볼 수 있다.

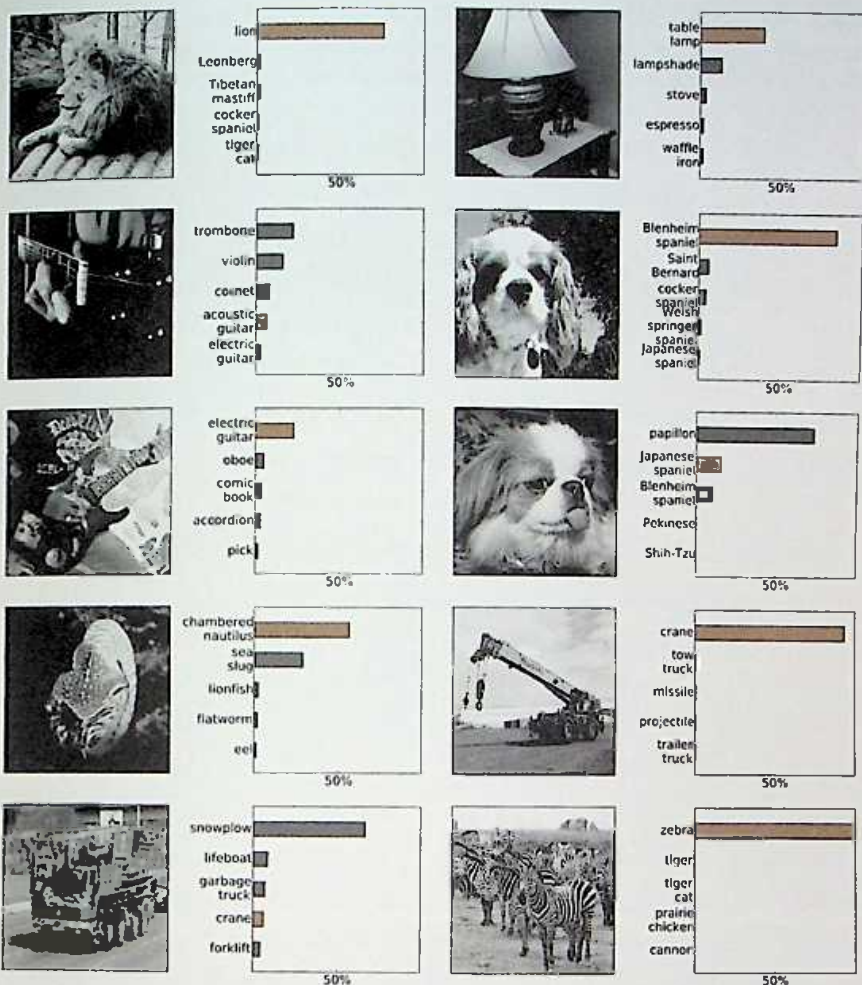


그림 6-21 학습 후 CNN의 테스트 결과*. 상위 다섯 개의 예측 카테고리에 대한 소프트맥스 층의 출력력을 나타낸다.

* 모든 이미지는 크리에이티브 커먼즈 라이선스(CC BY-SA 2.0)에 따라 이용하였다. 왼쪽 위부터 오른쪽 아래 순으로 <https://flickr.com/photos/elpadawan/8238633021>, <https://flickr.com/photos/38009628@N08/10085782733>, <https://flickr.com/photos/gi/167282325>, <https://flickr.com/photos/14511253@N04/4531941062>, <https://flickr.com/photos/monavelion/5032771365>, <https://flickr.com/photos/lostintexas/482312645>, <https://flickr.com/photos/wbaiv/13976917152>, <https://flickr.com/photos/elisfancub/3195695695391>, <https://flickr.com/photos/didbygraham/5527607622>, <https://flickr.com/photos/malczyk/5638610203>

표 6-3 2014년 ILSVRC에서 2위의 성적을 거둔 옥스포드 대학의 VGG^[65]. 표의 형식은 표 6-2를 참조

층 유형/이름	패치	슬라이드	슬릭 맵 크기	함수	파라미터
data	-	-	$224 \times 224 \times 3$	-	
conv1	3×3	1	$224 \times 224 \times 64$	Rel	576
conv2	3×3	1	$224 \times 224 \times 64$	Rel	37k
pool2	2×2	2	$112 \times 112 \times 64$	-	0
conv3	3×3	1	$112 \times 112 \times 128$	Rel	74k
conv4	3×3	1	$112 \times 112 \times 128$	Rel	147k
pool4	2×2	2	$56 \times 56 \times 128$	-	0
conv5	3×3	1	$56 \times 56 \times 256$	Rel	295k
conv6	3×3	1	$56 \times 56 \times 256$	Rel	590k
conv7	3×3	1	$56 \times 56 \times 256$	Rel	590k
conv8	3×3	1	$56 \times 56 \times 256$	Rel	590k
pool8	2×2	2	$28 \times 28 \times 256$	-	0
conv9	3×3	1	$28 \times 28 \times 512$	Rel	1,180k
conv10	3×3	1	$28 \times 28 \times 512$	Rel	2,359k
conv11	3×3	1	$28 \times 28 \times 512$	Rel	2,359k
conv12	3×3	1	$28 \times 28 \times 512$	Rel	2,359k
pool12	2×2	2	$14 \times 14 \times 512$	-	0
conv13	3×3	1	$14 \times 14 \times 512$	Rel	2,359k
conv14	3×3	1	$14 \times 14 \times 512$	Rel	2,359k
conv15	3×3	1	$14 \times 14 \times 512$	Rel	2,359k
conv16	3×3	1	$14 \times 14 \times 512$	Rel	2,359k
pool16	2×2	2	$7 \times 7 \times 512$	-	0
fc17	-	-	$1 \times 1 \times 4096$	Rel	102,760k
fc18	-	-	$1 \times 1 \times 4096$	Rel	16,777k
fc19	-	-	$1 \times 1 \times 1000$	softmax	4,096k

7

재귀 신경망

재귀 신경망(RNN)은 음성이나 언어, 동영상과 같은 연속열 데이터를 다루는 신경망이다. 이들 데이터는 일반적으로 크기가 샘플마다 두서너개 다르며, 그리고 연속열 내의 요소의 순서(문맥)에 의미가 있는 것이 특징이다. RNN은 이러한 특징을 잘 다루도록 만들어졌다. 이런 경우에는 연속열 데이터를 다루는 추론 문제에 대한 개요를 설명하고, 가장 기본적인 RNN과 그 학습 방법을 설명한다. 더불어 RNN의 일종으로 더 긴 컨텍스트를 포착할 수 있는 장 단기 기억(LSTM)과 RNN을 사용하여 일련의 연속열과 크기가 다른 연속열을 추론(측정)하는 커넥션리스트 세기열 분류법(CRC)에 대해서도 다룬다.



7.1 연속열 데이터의 분류

이번 장에서는 연속열 데이터의 분류 문제를 논하겠다. 연속열 데이터란, 각각의 요소에 순서가 있는 모임으로 주어지는 데이터로서 음성, 동영상, 텍스트 등을 다음과 같은 형태로 표현할 수 있다.

$$x^1, x^2, x^3, \dots, x^T$$

연속열의 길이 T 는 일반적으로 가변적이다. 데이터의 연속열 내 순서를 인덱스 $t = 1, 2, 3, \dots$ 으로 나타내며, 앞으로는 t 를 편의상 시각(時刻)이라 부르기로 한다. 그러나 t 는 시간에 대응하는 경우가 많을 뿐 항상 그런 것은 아니다. 낱말의 요소가 순서를 가지며, 그 순서에 의미가 숨어있는 경우에 속하는 데이터를 대상으로 한다.



We can get an idea of the quality of the learned feature vectors by displaying them in a 2-D map.

단어	We	can	get	...	the	learned	?
입력	x^1	x^2	x^3		x^{t-1}	x^t	x^{t+1}
출력		y^1	y^2		y^{t-2}	y^{t-1}	y^t

그림 7-1 문장이 그 중간까지만 주어졌을 때 그 다음 단어를 예측하는 문제. t 번째까지의 단어 $x^1, \dots, x^t$ 를 입력받아 $t+1$ 번째 단어 y^t 를 예측한다.

연속열 데이터를 다루는 추정 문제의 예로, 문장이 일부 주어졌을 때 다음 단어를 예측하는 문제를 들 수 있다. 그림 7-1처럼 문장이 t 번째의 단어까지 주어졌을 때 $t+1$ 번째 단어를 예측하는 것이 목표다. M 개의 단어로 구성된 사전으로부터 각 단어를 레이블 $l = 1, 2, \dots, M$ 으로 나타내기로 한다. 주어진 문장의 t 번째 단어를 변수 x^t , 예측할 단어를 y^t 라 하고, 이 두 가지는 단어의 레이블을 값으로 취하도록 한다. 단어의 예측은 그림 7-1에서 보듯이, 입력 연속열 $x^1, x^2, \dots, x^t$ 로부터 y^t 를 예측하는 문제로 볼 수 있다. 문장은 단어의 자유로운 조합이며,

무수하게 많은 조합을 만들 수 있지만, 한편으로 각 단어는 이전 단어의 연속열에 강하게 영향을 받기도 한다. 앞으로 설명할 재귀 신경망(Recurrent Neural Network)(이하 RNN)*은 이러한 단어 간의 의존 관계, 즉 문맥을 잘 학습하여 단어를 높은 정확도로 예측하는 것이 가능하다^[49, 50]. 이것은 언어 모형이라고도 불린다.

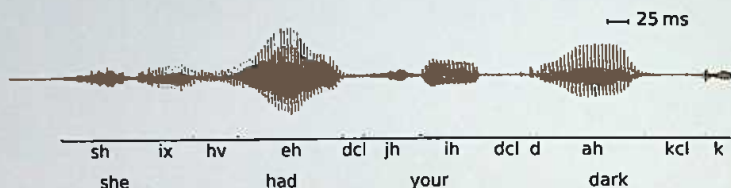


그림 7-2 'She had your dark (suit) ...'를 발화하였을 때의 음성 신호와 매 순간에 발생되고 있는 음소를 나타낸 것(TIMIT 데이터 세트(<https://catalog.ldc.upenn.edu/LDC93S11>)). 이를테면, 시간축을 10ms 간격으로 $t = 1, 2, 3, \dots$ 으로 나누었을 때, 각 t 의 위치에서 음성 신호를 25ms의 길이로 잘라내어 이것을 주파수 분석한 (x')가 RNN에 입력되는 연속열 데이터가 된다.

다른 예로, 발화를 기록한 음성 신호로부터 화자가 매 순간에 발한 음소(phoneme)를 추정하는 문제가 있다. 음소란, 그림 7-2에 나타난 /shi/나 /eh/와 같이 모음과 자음으로 나뉘진 발화의 최소 단위다. 음소의 가짓수는 유한하며, 영어의 경우 약 60개 정도가 된다. 음성 신호는 실숫값을 갖는 시간 τ 의 함수 $s(\tau)$ 로 주어지나, 짧은 시간 간격 Δt ($\sim 10\text{ms}$)로 이산화시킨 시각 $\tau = t\Delta t$ 마다 순간적인 주파수분포를 추출하여 이것을 자질벡터 x' 로 나타낸다. 음소의 추정은 예를 들면, 입력 연속열 $x^1, x^2, \dots$ 로부터 음소의 종류(레이블)의 연속열 $y^1, y^2, \dots$ 를 추정하는 문제로 형식화할 수 있다(그림 7-2). 발화는 목의 성대나 입의 근육을 사용하여 일어나는 것으로, 시간축 방향에 강한 의존 관계(문맥)가 있기 때문에 시각 t 에서 발생하는 음소 y^t 는 그 이전의 입력 $x^{t-k}, x^{t-k+1}, \dots, x^t$ 의 영향을 받는다고 보는 것이 자연스럽다**.

* 재귀형 신경망 또는 순환 신경망이라고 번역되는 경우도 있으나 같은 것을 가리킨다.

** 뒤의 음소로부터 영향도 있을 수 있다. 실제로, RNN의 확장을 통해 양방향의 의존 관계를 모두 학습하도록 하는 것도 가능하다.

앞에서는 음성 신호로부터 Δ 마다 자질 x' 를 추출하여 이와 같은 시각의 음소 레이블 y' 를 추정한다고 설명하였다. 그림 7-2에 나타난 것처럼 음소는 그 길이가 가변적이다. 따라서 정답이 되는 y' 의 연속열은 하나의 음소가 여러 개의 프레임에 걸쳐 계속되는 모습, 예를 들면 다음과 같은 식이 된다.

sh, sh, sh, sh, sh, sh, ix, ix, ix, ix, hv, hv, hv, ...

이상적인 출력은 이런 잉여성을 갖는 출력이 아니라 sh, ix, hv, ...와 같은 것일 것이다. 그러기 위해서는 입력 연속열과 길이가 다른 연속열을 출력할 수 있는 능력이 필요하다. 그러나 RNN의 구조만으로는 그렇게 할 수 없다. 이것을 가능하게 하는 방법은 7.6절에서 설명하겠다.

7.2 RNN의 구조

재귀 신경망(RNN)은 내부에 (방향이 있는) 순환경로를 가진 신경망을 통틀어 부르는 말이다. RNN은 그러한 구조 덕에 정보를 일시적으로 기억하고 그에 따라 반응을 동적으로 변화시킬 수 있다. 그로 인해 연속열 데이터 안에 존재하는 '문맥'을 포착하고 먼저 설명한 것과 같은 분류 문제도 잘 처리할 수가 있다. 이것이 앞먹임 신경망과의 가장 큰 차이이다.

RNN에는 Elman 신경망^[14], Jordan 신경망^[31], 시간 지연 신경망^[40], 에코 상태 신경망^[37] 등 다양한 종류가 있지만, 여기에서는 그림 7-3과 같은 앞먹임 신경망과 기본적으로 같지만 중간층 유닛의 출력이 다시 자기 자신과 연결되는 '귀환로'를 갖는 단순한 신경망을 주로 논하도록 한다.

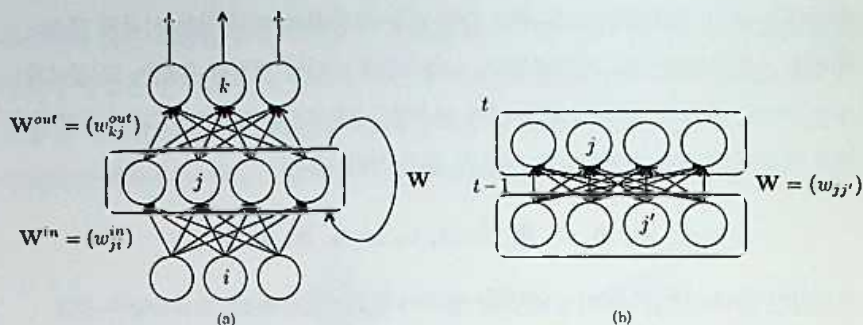


그림 7-3 (a) 재귀 신경망(RNN)의 개요. 중간층이 자기 자신으로 이어지는 귀환로를 가지며, 그 외에는 2층 앞먹임 신경망과 같은 구조를 갖는다. (b) 귀환로 결합의 상세. 현 시각 t 에서 유닛의 출력이 다음 시각 $t + 1$ 에서 유닛의 입력이 된다.

이 RNN의 동작은 각각의 시각 t 마다 하나의 입력 x' 를 입력받아, 또 동시의 하나의 출력 y' 를 출력하는 것이다. 신경망 내부에 있는 귀환로에 의해 출력을 계산할 때, RNN이 이전에 입력받은 모든 입력(입력의 히스토리)이 관여된다. 앞먹임 신경망이 입력 하나에 대해 하나의 출력을 내놓는 매핑을 모형화하였다면, RNN은 이론상 과거의 모든 입력에 대해 하나의 출력을 내놓는 매핑을 모형화했다고 할 수 있다(그림 7-4).

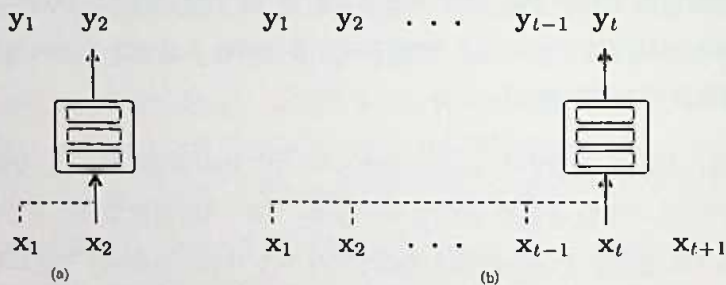


그림 7-4 RNN은 데이터의 연속열을 입력받아 출력 연속열을 생성한다((a), (b)는 시각 2와 t 에서의 입출력 상태). 매 시각마다 받아들이는 입력은 하나이지만, 해당 시각의 출력에는 이론상 그 시점까지의 모든 입력이 반영된다.

RNN의 출력층은 앞먹임 신경망과 같은 방법으로 설계한다. 즉, 예를 들면 분류 문제에는 대상이 되는 클래스 수만큼의 유닛을 가지고 소프트맥스 함수를 활성화 함수로 사용한다. 오차함수도 마찬가지다. 출력 연속열 $y^1, \dots, y^T$ 의 목표가 되는 $d^1, \dots, d^T$ 를 상정하여, 이 목표 연속열과 입력 연속열이 짝으로 구성되는 훈련 데이터에 대해 다음의 식으로 오차함수를 삼는다.

$$E(w) = - \sum_n \sum_t \sum_k d_{nk}^t \log y_k^t(x_n; w)$$

식 7-1

여기서 d_{nk}^t 는 n 번째 샘플의 시각 t 에서의 목표 출력(클래스 k 의 우도, 보통 0이나 1이다), y_k^t 는 그와 비교할 RNN의 출력, x_n 은 n 번째 샘플 입력 연속열이다. 연속열의 길이는 샘플마다 달라도 상관없다.

RNN은 앞먹임 신경망과 마찬가지로^[32] 연속열 데이터에 대해 만능성을 갖는다고 알려져 있다^[23]. 즉, 중간층이 충분한 수의 유닛을 갖는다면 임의의 연속열로부터 다른 임의의 연속열로의 매핑을 원하는 정확도로 근사할 수 있는 능력을 갖는 것이 증명되어 있다.

또, 연속열 데이터 전체가 일괄로 주어진 경우에는 그 연속열 데이터를 역순($t = T, T-1, \dots, 1$)으로 RNN에 입력하는 것도 가능하다. 특히, 원래 순서대로 연속열을 입력받는 RNN과 역순으로 연속열을 입력받는 RNN을 만들어 양쪽의 출력층을 통합하여 구성한 신경망을 양방향 RNN(bidirectional RNN)이라 부른다^[62]. 연속열 데이터가 시시각각 주어지는 온라인 추정을 해야 할 경우에는 이 방법을 쓸 수 없지만, 그렇지 않은 경우에는 정방향 입력만 주어지는 RNN보다 성능이 더 좋다고 알려져 있다^[20].

7.3 순전파 계산

그림 7-4와 같이 입력 연속열로부터 출력 연속열을 얻는 계산 방법을 논해 보자. 먼저, 기호를 다음과 같이 정의한다. 입력층, 중간층, 출력층에 속한 각 유닛의 인덱스를 i, j, k 로 나타내기로 한다(그림 7-3). 각 유닛은 시각 $t = 1, 2, \dots$ 마다 서로 다른 상태를 가지므로 신경망으로의 입력을 $\mathbf{x}^t = (x_i^t)$, 중간층 유닛의 입력과 출력을 각각 $\mathbf{u}^t = (u_j^t)$, $\mathbf{z}^t = (z_j^t)$, 출력층 유닛의 입력과 출력을 각각 $\mathbf{v}^t = (v_k^t)$, $\mathbf{y}^t = (y_k^t)$ 이라 하기로 한다. 또, 목표 출력을 $\mathbf{d}^t = (d_k^t)$ 라 한다. 입력층과 중간층 사이의 가중치를 $\mathbf{W}^{(in)} = (w_{ji}^{(in)})$, 중간층과 중간층 사이의 귀환로 결합의 가중치를 $\mathbf{W} = (w_{jj'})$, 중간층과 출력층 사이의 가중치를 $\mathbf{W}^{(out)} = (w_{kj}^{(out)})$ 로 표기한다. 가중치는 시각 t 와는 무관하며, (학습에 의해 업데이트되지만) 순전파 계산 중에는 상수인 것에 주의하라. 바이어스는 4장에서와 마찬가지로, 항상 1을 출력하는 특별한 유닛 하나를 한 층 아래에 갖추어 이와 각 유닛 간의 결합 가중치를 바이어스로 간주하는 방법을 택한다(그림 4-1).

RNN의 귀환로는 중간층의 출력을 스스로의 입력으로 되돌리는 것으로, 이 사이의 모든 유닛 간에 결합이 존재한다. 그림 7-3(b)에서 보듯이, 시각 $t - 1$ 에서 중간층에 임의의 유닛 j' 에서 시각 t 에 중간층에 속한 임의의 유닛 j 로, 가중치 $w_{jj'}$ 인 결합이 존재한다. 중요한 것은 이 귀환로로 들어오는 입력이 한 시각 전의 신호라는 것이다. 따라서 시각 t 에 대한 중간층 각 유닛의 입력은 같은 시각에 입력층으로부터 전해지는 것과 시각 $t - 1$ 에 중간층에서 나온 출력이 피드백된 것의 합으로 이루어진다.

$$u_j^t = \sum_i w_{ji}^{(in)} x_i^t + \sum_{j'} w_{jj'} z_{j'}^{t-1}$$

7-2

중간층의 바이어스는 $w_{j0}^{(in)} (x_0^t = 1 \text{로 고정})$ 이 주어진다(바이어스를 사용하지 않는 경우도 있음). 여기서 중간층의 출력은 통상적인 활성화 함수 f 를 거쳐 다음과 같이 계산한다.

$$z_j^t = f(u_j^t)$$

위의 두 식을 정리하면 다음으로 쓸 수 있다.

$$\mathbf{z}^t = \mathbf{f}(\mathbf{W}^{(in)}\mathbf{x}^t + \mathbf{W}\mathbf{z}^{t-1})$$

식 7-3

$t = 1$ 에서부터 시작하여 t 가 1씩 커질 때마다 입력 연속열 $\mathbf{x}^1, \mathbf{x}^2, \dots$ 로 식 7-3을 반복하여 계산하는 과정을 통해 임의의 시각 t 에 중간층의 상태 $\mathbf{z}^t$ 를 구할 수 있다. 그러나 $t = 1$ 일 때의 초기값 $\mathbf{z}^0 = (\mathbf{z}_j^0)$ 가 주어져야 하며, 보통 $\mathbf{z}_j^0 = 0$ 으로 시작한다*.

한편, RNN의 출력 $\mathbf{y}^t$ 를 아래와 같이 계산한다. 먼저, 출력층에 속한 각각의 유닛이 받는 입력은 중간층의 출력 $\mathbf{z}^t$ 가 주어졌을 때 다음과 같다(출력층의 바이어스는 $u_{k0}^{(out)}$ 이다).

$$v_k^t = \sum_j w_{kj}^{(out)} z_j^t$$

식 7-4

그리고 출력층의 활성화 함수는 3장의 앞먹임 신경망처럼 적용하려는 문제에 따라 선택한다(오차함수도 마찬가지로). 활성화 함수를 f^{out} 이라고 할 때, 위의 내용을 다시 정리하면 다음으로 나타낼 수 있다.

$$\mathbf{y}^t = \mathbf{f}^{(out)}(\mathbf{v}^t) = \mathbf{f}^{(out)}(\mathbf{W}^{(out)}\mathbf{z}^t)$$

식 7-5

* 0이 아닌 초기값을 설정하는 것으로, 노이즈에 대해서도 안정적이 된다는 연구도 있다⁷⁷⁾.

7.4 역전파 계산

RNN을 학습하는 데는 앞먹임 신경망과 같이 확률적 경사 하강법을 사용한다. 어떤 경우든 각 층의 가중치에 대한 오차의 미분을 계산해야 한다. 두 가지의 방법이 알려져 있는데, 첫 번째는 RTRL(RealTime Recurrent Learning)^[56], 또 한 가지는 BPTT(BackPropagation Through Time)다. 전자는 메모리 효율이 좋은 한편, 후자는 계산 속도가 빠르며 좀 더 간단하다. 여기서는 BPTT에 대하여 설명하겠다.

BPTT는 RNN을 시간 방향으로 펼쳐 앞먹임 신경망과 같은 형태로 만든 후, 역전파와 계산을 수행한다. 시간 방향으로 펼친다는 것은 매 시각마다의 RNN의 각 층이 별개인 것처럼 간주하는 것이다. 구체적으로 말하자면, 그림 7-5와 같이 매 시각의 RNN을 옆으로 늘어놓아 중간층의 귀환로를 연속되는 시각의 중간층 유닛으로의 결합으로 표현한다. 이렇게 전개한 신경망은 모순 없이 순수한 앞먹임 신경망이 되며, 그에 따라 역전파를 통해 기울기를 계산할 수 있게 된다.

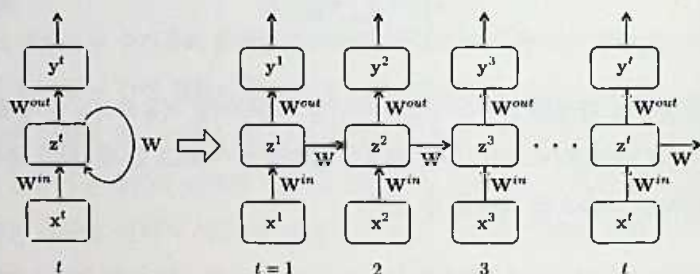


그림 7-5 RNN을 시간 방향으로 전개해서 앞먹임 신경망으로 전환한 예

4장에서 설명한 바와 같이, 보통의 앞먹임 신경망에서 $l+1$ 번째 층과 l 번째 층의 델타. 다시 말해 $\delta_j^{(l)} \equiv \partial E / \partial u_j^{(l)}$ 의 역전파는 다음과 같이 계산했다.

$$\delta_j^{(l)} = \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)} f'(u_j^{(l)})$$

7-6

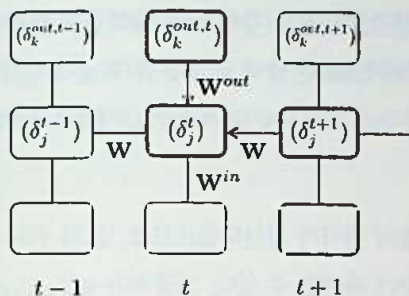


그림 7-6 시간 방향으로 전개한 RNN에서 델타의 역전파 계산

이것을 그림 7-5에 실린 신경망에 시각 t 일 때의 중간층 유닛의 델타를 계산하는 데 적용해 보겠다. 이를 위해 먼저 시각 t 의 출력층 유닛 k 의 델타를 다음과 같이 쓰고,

$$\delta_k^{out,t} \equiv \frac{\partial E}{\partial v_k^t}$$

같은 시각의 중간층 유닛 j 의 델타를 다음과 같이 나타낸다.

$$\delta_j^t \equiv \frac{\partial E}{\partial u_j^t}$$

시각 t 의 중간층 유닛 j 는 그림 7-6에서 보듯이 t 의 출력층 유닛과 $t+1$ 의 중간층 유닛과 연결된다. 따라서 δ_j^t 는 식 7-6을 이에 적용하면 다음과 같이 계산하게 된다.

$$\delta_j^t = \left(\sum_k w_{kj}^{out} \delta_k^{out,t} + \sum_{j'} w_{j'j} \delta_{j'}^{t+1} \right) f'(u_j^t) \quad \text{식 7-7}$$

각 시각 $t = 1, \dots, T$ 에 대한 출력층의 델타 $\delta_k^{out,t}$ 가 주어지면, $t = T$ 부터 시작해서 t 를 감소시켜 가며 식 7-7을 반복 계산하는 과정을 통해 각 t 에 대한 중간층의 δ_j^t 를 계산할 수 있다. 이때 $t = T+1$ 에 대한 델타는 아직 계산할 수

없으므로 $\delta_j^{T+1} = 0$ 으로 한다. 각 시각에 대한 출력층의 델타 $\delta_k^{out,t}$ 는 순전파 계산 시점에 이미 계산된 RNN의 출력 y^t 와 문제 유형에 적합한 출력 형식을 갖는 목표 출력 d^t 에 오차함수(즉, 회귀 문제에는 제곱오차, 유형분류 문제에는 교차 엔트로피)를 적용하여 계산된다.

이렇게 각 시각에 각 층의 델타가 계산되었으므로, 오차 E 의 각 층의 가중치에 대한 미분을 다음과 같이 계산할 수 있다. 가중치는 w_{ji}^{in} , $w_{jj'}$, w_{kj}^{out} 의 세 가지가 있다. w_{ji}^{in} 은 식 7-2에 나와 있듯이, 각 t 의 중간층 유닛 j 의 입력 u_j^t 에만 포함되므로 다음과 같이 된다.

$$\frac{\partial E}{\partial w_{ji}^{in}} = \sum_{t=1}^T \frac{\partial E}{\partial u_j^t} \frac{\partial u_j^t}{\partial w_{ji}^{in}} = \sum_{t=1}^T \delta_j^t x_i^t$$

$w_{jj'}$ 역시 식 7-2와 같이 각 t 에 대한 중간층의 유닛 j 의 입력에만 포함되므로 다음과 같이 된다.

$$\frac{\partial E}{\partial w_{jj'}} = \sum_{t=1}^T \frac{\partial E}{\partial u_j^t} \frac{\partial u_j^t}{\partial w_{jj'}} = \sum_{t=1}^T \delta_j^t z_{j'}^{t-1}$$

w_{kj}^{out} 은 식 7-4와 같이 각 t 의 출력층 유닛 k 의 입력 v_k^t 에만 포함되므로 다음과 같이 된다.

$$\frac{\partial E}{\partial w_{kj}^{out}} = \sum_{t=1}^T \frac{\partial E}{\partial v_k^t} \frac{\partial v_k^t}{\partial w_{kj}^{out}} = \sum_{t=1}^T \delta_k^{out,t} z_j^t$$

이제 위의 내용을 요약하겠다. 주어진 것은 입력 연속열 $x^1, \dots, x^T$ 과, 목표 출력 연속열 $d^1, \dots, d^T$ 이다. 먼저, $t = 1, \dots, T$ 의 순서로 순전파 계산을 통해 출력 연속열 $y^1, \dots, y^T$ 를 계산한다. 다음으로, 각 시각 t 에서 y^t 와 d^t 의 차이를 구해서 출력층의 각 유닛 k 에 대해(미래 시각 $T+1$ 에 대한 델타 $\delta_j^{T+1} = 0$ 으로 한 후)

$t = T, T-1, \dots, 1$ 의 순서로 역전파 계산을 수행하여 중간층의 델타를 순서대로 계산한다. 그런 다음, 가중치의 오차 기울기를 계산한 후에 각 가중치를 통상적인 경사 하강법으로 업데이트한다.

7.5 장·단기 기억

7.5.1 RNN의 기울기 소실 문제

여기까지 설명한 것과 같이 RNN은 연속열 데이터의 문맥을 포착하여 추정하는 것이 가능하다. 이때 포착할 수 있는 문맥의 길이, 즉 현재 시각으로부터 얼마나 먼 과거의 입력까지 반영할 수 있는지가 중요하다. 복잡한 문제일수록 요구되는 문맥의 길이가 길 것이다. 이론상으로는 과거의 모든 입력 이력이 고려되어야 할 테지만, 실제로 RNN의 출력에 반영시킬 수 있는 것은 기껏해야 과거 10시각 정도로 알려져 있다.

이러한 한계는 앞먹임 신경망의 기울기 소실 문제와 같은 원인에서 비롯된다. 4.5절에서 밝혔듯이, 층수가 많은 신경망에는 역전파법으로 기울기를 계산할 때 층을 거슬러 올라감에 따라 기울기 값이 폭발적으로 커지거나 0으로 소멸해 버리기 쉬운 성질이 있다. 위에서 설명한 것처럼 RNN의 역전파 계산은 시간 방향으로 전개되므로 층수가 매우 많은 앞먹임 신경망의 역전파 계산으로 바꿔 생각할 수 있다. RNN의 본래 층수가 적다고 해도 역전파 계산 시에는 많은 층수를 다루는 것과 마찬가지로, 그에 따라 기울기 값도 발산하거나 소멸하거나 둘 중 하나가 되기 쉽다. RNN에서 이 이상 긴 연속 열을 다루기 어려운 것은 이 때문이다. 이는 위에서 설명한 기본적인 RNN으로는 단기적인 기억은 실현할 수 있지만 더 장기적인 기억을 실현하는 것은 어렵다는 말도 된다.

7.5.2 LSTM의 개요

앞의 문제를 해결하기 위해서 장기적인 기억을 실현하기 위한 방법 몇 가지가 제안되었다. 그중에서도 가장 성공적인 시도가 이제 설명할 장·단기 기억(Long Short-Term Memory)(이하 LSTM)이다^[20, 30]. LSTM은 앞서 설명한 기본적인 RNN에 비해서 중간층의 각 유닛이 메모리 유닛이라 불리는 요소로 구성된 구조다. 그 외의 구조는 기존 RNN과 완전히 같다.

메모리 유닛 하나의 내부 구조를 그림 7-7에 나타내었다. 중앙에 메모리 셀(그림 안의 기호 a)이 있고, 그 주위에 다섯 개의 유닛(그림 안 기호 b~f)가 있다. 메모리 유닛 내에서 이들이 그림과 같은 신경망을 구성한다.

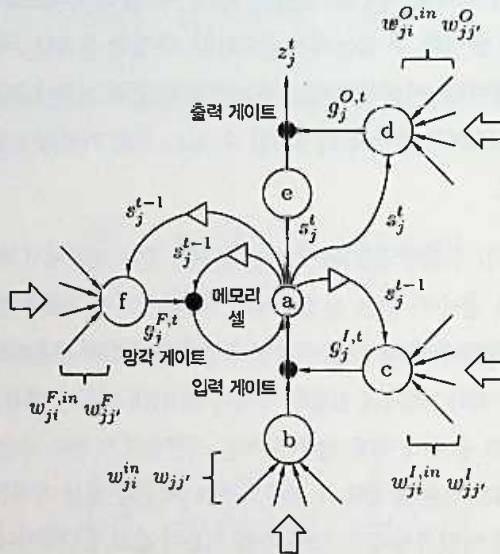


그림 7-7 LSTM의 메모리 유닛 하나의 구조. 상태를 기억하는 메모리 셀 a와 입력, 출력, 망각 게이트의 값을 결정하는 유닛 c, d, f와 외부 입력을 받아들이는 유닛 b가 있다. 큰 화살표는 외부로부터의 입력을 나타내며, 각각 입력층으로부터 오는 입력과 중간층(모든 메모리 유닛)의 출력이 되돌아오는 것을 합친 것이다. 유닛 b, c, d, f는 모두 이 입력을 받는다. 유닛 e는 메모리 셀의 출력에 활성화 함수를 적용한다. 삼각형은 시간 지연을 나타낸다.

메모리 셀 a는 상태 s_j^t 를 유지하며, 이를 한 시각 후의 자기 자신에게 귀환시키는 것으로 기억을 구현한다. 이 귀환로에는 중간에 망각 게이트가 있어서 유닛 f의 출력이 게이트 값 $g_j^{F,j} \in [0, 1]$ 을 갖는다. s_j^t 에 $g_j^{F,t}$ 을 곱한 값이 전달되어 $g_j^{F,t}$ 가 1에 가까우면 현재 상태가 기억되며, 0에 가까우면 리셋(망각)된다.

메모리 유닛의 외부로부터 입력은 유닛 b가 받고, 그 출력이 메모리 셀에 입력된다. 그 사이에는 입력 게이트가 있어서 유닛 c의 출력이 게이트 값 $g_j^{I,t} \in [0, 1]$ 이 된다. 유닛 b는 보통 RNN의 중간층 유닛 하나에 해당하며, 그 출력에 $g_j^{I,t}$ 를 곱한 것이 메모리 셀에 전달된다.

메모리 유닛에서 외부로의 출력은 메모리 셀에서 유닛 e를 통해 나간다. 유닛 d와 출력 사이에도 출력 게이트가 있어서 유닛 d의 출력은 게이트 값 $g_j^{O,t} \in [0, 1]$ 가 된다. 그 값이 1에 가까우면 메모리 셀의 출력이 외부로 전달되며, 0에 가까우면 블록된다.

지금까지 설명한 구조는 단기기억밖에 구현하지 못했던 RNN의 한계를 조금이나마 극복하기 위한 것이다. 단순한 케이스로, 망각 게이트를 1(오픈)로 두고 입력 게이트를 0(클로즈)으로 해두면, 메모리 셀의 상태는 영원히 그대로 남아있게 된다. 물론, 같은 상태를 그저 오랫동안 유지하기만 하면 되는 것이 아니므로 적절하게 이들 게이트를 열고 닫아야 한다. 만약 그 조절이 적절하다면, 긴 문맥의 정보를 포착하여 더 정밀한 추정을 구현할 수 있게 된다.

7.5.3 순전파 계산

지금까지 설명한 동작을 다음과 같은 식으로 나타내도록 하겠다. j 번째 메모리 유닛 안의 메모리 셀(그림 7-7의 (a))은 변수 s_j^t 를 유지한다. 메모리 셀의 귀환로는 변수 s_j^t 의 내용을 한 시각 후의 시점으로 전달한다.

$$s_j^t = g_j^{F,t} s_j^{t-1} + g_j^{I,t} f(u_j^t)$$

식 7-8

여기서 첫 번째 항은 한 시각 전으로부터 전달된 상태에 대한 항이며, 두 번째 항은 이 메모리 유닛 j 가 받아들인 입력에 대한 항이다. 후자는 보통 RNN과 마찬가지로 입력층과 이전 시각의 중간층으로부터 다음과 같은 입력을 받는다.

$$u_j^t = \sum_i w_{ji}^{in} x_i^t + \sum_{j'} w_{jj'} z_{j'}^{t-1}$$

식 7-8의 두 개의 항은 망각 게이트 값 $g_j^{F,t}$ 와 입력 게이트 값 $g_j^{I,t}$ 으로 구성되는데, 이 두 값은 각각 다음과 같이 계산된다.

$$g_j^{F,t} = f(u_j^{F,t}) = f \left(\sum_i w_{ji}^{F,in} x_i^t + \sum_{j'} w_{jj'}^F z_{j'}^{t-1} + w_j^F s_j^{t-1} \right)$$

$$g_j^{I,t} = f(u_j^{I,t}) = f \left(\sum_i w_{ji}^{I,in} x_i^t + \sum_{j'} w_{jj'}^I z_{j'}^{t-1} + w_j^I s_j^{t-1} \right)$$

w_j^F 는 메모리 셀에서 망각 게이트 사이의 유닛과의 가중치이며(그림 7-7의 b), w_j^I 는 메모리 셀에서 입력 게이트 사이의 유닛과의 가중치다(그림 7-7의 c). 이들은 조금 뒤에 설명할 출력 게이트에서도 같은 구조로 연결되는 결합과 함께 피프홀(peephole) 결합이라 부른다.

메모리 유닛의 출력은 식 7-8의 s_j^t 를 이용하여 다음과 같이 계산한다.

$$z_j^t = g_j^{O,t} f(s_j^t)$$

이때 $g_j^{O,t}$ 는 출력 게이트의 값으로 다음과 같이 계산한다.

$$g_j^{O,t} = f(u_j^{O,t}) = f \left(\sum_i w_{ji}^{O,in} x_i^t + \sum_{j'} w_{jj'}^O z_{j'}^{t-1} + w_j^O s_j^t \right)$$

출력 게이트에서만 s_j^{t-1} 가 아닌 s_j^t 를 계산하므로 이에 주의해야 한다. 각 게이트의 값 $g_j^{F,t}, g_j^{I,t}, g_j^{O,t}$ 는 계산에 사용되는 활성화 함수 f 를 로지스틱 시그모이드 함수로 해서 그 치역을 $[0, 1]$ 로 제한한다. 메모리 유닛의 출력 z_j^t 은 바로 다음 시각의 세 게이트를 제어하기 위한 입력이 되는 역할 외에도 식 7-4에 따라 출력층 유닛의 입력이 되기도 하며, 식 7-5에 따라 시각 t 의 신경망의 출력 y^t 를 결정한다.

7.5.4 역전파 계산

LSTM의 구조는 일견 복잡해 보이지만, 일반적인 RNN과 마찬가지로 경사 하강법을 통해 학습시킬 수 있다. 기울기 계산도 마찬가지로 역전파법을 통해 계산한다. 이때 메모리 유닛 안의 변수 중에 전체 출력 z_j^t 만이 다른 메모리 유닛에 귀환하며, 나머지는 메모리 유닛 안의 요소끼리의 연결임에 주의한다.

역전파 계산은 다음과 같이 수행한다. 먼저, 출력 게이트 값 $g_j^{O,t}$ 를 출력할 유닛의 델타로 본다. 일반적인 앞먹임 신경망에 있어 $l+1$ 층에서 l 층의 델타에 대한 역전파는 식 4-10에 의해 다음과 같이 계산했었다.

$$\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} \frac{\partial u_k^{(l+1)}}{\partial u_j^{(l)}}$$

순전파 계산 시에 $g_j^{O,t}$ 가 주어지는 유닛의 출력은 셀의 출력과 곱해져 $z_j^t = g_j^{O,t} f(s_j^t)$ 가 출력층 및 다음 시각의 메모리 유닛으로 전달된다. 전달받는 쪽의 출력층 한 개 유닛에 들어가는 총 입력은 $v_k^t = \sum_j w_{kj}^{out} z_j^t$ 일 때 다음이 된다.

$$\frac{\partial v_k^t}{\partial u_j^{O,t}} = w_{kj}^{out} f'(u_j^{O,t}) f(s_j^t)$$

또 다른 전달받는 쪽인 다음 시각의 메모리 유닛의 총 입력도 마찬가지로 계산할 수 있기 때문에 $g_j^{O,t}$ 를 출력하는 유닛의 델타는 다음으로 계산할 수 있다.

$$\epsilon_j^t = \sum_k w_{kj}^{out} \delta_k^{out,t} + \sum_{j'} w_{j'j} \delta_{j'}^{t+1}$$

$$\delta_j^{O,t} = f'(u_j^{O,t}) f(s_j^t) \epsilon_j^t$$

셀 바로 뒤에서 활성화 함수를 적용시키는 유닛 ($s_j^t \rightarrow f(s_j^t)$)의 델타도 같은 방법을 적용하여 다음으로 계산할 수 있다.

$$\tilde{\delta}_j^t = g_j^{O,t} f'(s_j^t) \epsilon_j^t$$

다음 차례로 메모리 셀을 살펴보자. 메모리 셀은 총 입력 s_j^t 를 입력받아 항등사상인 활성화 함수를 거쳐 s_j^t 를 출력하는 '유닛'이라 볼 수 있다. 이 '유닛'의 출력이 연결되는 곳은 외부 출력, 셀 자신으로 돌아오는 귀환로, 입력, 망각, 그리고 출력의 세 게이트를 제어하는 각 유닛을 향한 피프홀(peephole) 결합 등 총 다섯 곳이 있다. 각각의 델타를 역전파하는 것으로 이 '유닛'의 델타는 다음과 같이 계산된다.

$$\delta_j^{cell,t} = \tilde{\delta}_j^t + g_j^{F,t+1} \delta_j^{F,t+1} + w_j^F \delta_j^{I,t+1} + w_j^I \delta_j^{F,t+1} + w_j^O \delta_j^{O,t}$$

셀에는 입력을 받아들이는 유닛과 입력 게이트와 망각 게이트로 연결되는 두 개의 유닛 등 총 세 개의 출력이 입력된다. 셀로부터 각각의 유닛으로의 역전파는 입력을 받는 유닛에 대해서 다음이 되고,

$$\delta_j^t = g_j^{I,y} f'(u_j^t) \delta_j^{cell,t}$$

망각 게이트를 제어하는 유닛에 대해서는 다음으로 계산되고,

$$\delta_j^{F,t} = f'(u_j^{F,t}) s_j^{t-1} \delta_j^{cell,t}$$

입력 게이트를 제어하는 유닛에는 다음으로 계산된다.

$$\delta_j^{l,t} = f'(u_j^{l,t})f(u_j^t)\delta_j^{cell,t}$$

세 가지 식 모두 처음에 설명한 출력 게이트에 대한 계산과 같은 방법으로 유도할 수 있다.

7.6 입력과 출력의 연속열 길이가 다른 경우

7.6.1 은닉 마르코프 모델

RNN(LSTM도 포함)은 각 시각 t 마다 하나의 입력 x^t 를 받아 하나의 출력 y^t 를 내놓는다. 이 동작은 입력 연속열과 다른 길이를 가진 연속열을 추정해야 하는 경우, 예를 들어 7.1절에서 설명했던 음성 신호로부터 음소의 열을 추정하는 문제와 같은 경우에는 적합하지 않다. 목표는 sh, ix, hv, ...와 같은 연속열을 얻는 것이지만, RNN의 출력은 sh, sh, sh, sh, ix, ix, ix, hv, hv, ...처럼 같은 음소가 불필요한 반복을 얻게 되기 때문이다.

은닉 마르코프 모델(Hidden Markov Model)(이하 HMM)은 이런 문제를 다루는 가장 일반적인 방법이다. HMM은 내부 상태를 숨겨진 변수로 가지고, 이 변수가 시각과 함께 확률적으로 변화한다. 동시에 HMM은 현재의 내부 상태에 기초한 확률적 관측을 생성한다. 그럼, 이 관측을 시간순으로 늘어놓은 연속열 데이터가 있다고 생각해 보자. 예를 들어, 음소 인식 문제라면 각 음소에 대한 음소의 시작, 중간, 끝을 나타내는 세 가지 내부 상태를 가지고, 음성 신호의 자질값으로 관측하는 HMM을 생각해 보기로 한다. 그러면 내부 상태와 그 자질값의 관계를 가우시안 혼합 모델(Gaussian mixture model) 등으로 표현하고, 훈련 데이터를 이용해서 파라미터를 결정한다. 실제 인식은 주어진 자질값의 연속열 $x, \dots, x^T$ 에 대해, 그 연속열을 생성할 확률이 가장 높은 HMM을 특정한 후에 그 모형에 따른 음소를 추정 결과로 삼는다.

최근 이러한 프레임 안에서 가우시안 혼합 모델 대신에 딥 뉴럴넷(앞먹임 신경망)을 사용하면, 자질값으로부터 HMM의 내부 상태를 직접 추정하는 방법이 높은 성능을 보인다는 보고가 있었다^[27]. 연속열 데이터 인식을 목적으로 이렇게 신경망을 HMM과 결합하는 방법은 하이브리드법으로 알려져 있다. 앞먹임 신경망 대신 RNN을 사용하는 방법도 연구되고 있다.

그리고 입력된 연속열에 하나의 정답만을 출력하는 유형의 연속열 분류 문제도 있다. 예를 들면, 'Hello', 'Excuse me', 'I am sorry', 'Thank you', 'Good bye' 중 하나를 발화한 경우의 음성 신호로부터 발화의 내용이 이 중 어느 것인지를 추정하는 경우다. 이 문제를 RNN의 틀과 맞추기 위해서, 예를 들어 출력 연속열 $y^1, \dots, y^T$ 를 시간 방향으로 기하평균을 구하여 이 값을 클래스의 확률로 간주한다. 이 경우에 학습은 매 시각마다 목표 출력을 모두 같게 하는 것이면 된다(샘플 n 의 목표 출력이 모든 t 에서 $d_n^t = d_n$ 이 되는 것).

7.6.2 커넥셔니스트 시계열 분류

입력과 출력 사이에 연속열의 길이가 서로 다른 경우의 분류 문제를, 위에서 설명한 HMM을 쓰지 않고 신경망만을 사용해서 해결하려는 방법이 커넥셔니스트 시계열 분류(Connectionist Temporal Classification)(이하 CTC)^[20, 21, 22]. CTC는 RNN의 출력에 대한 해석을 바꾸어 입력 연속열과 길이가 다른 출력 연속열을 다룰 수 있도록 해준다. 어떠한 RNN(LSTM 및 양방향성 RNN을 포함)에도 적용할 수 있다.

먼저, 위에서 설명한 음성을 다루는 문제에서의 음소처럼 인식 대상이 되는 레이블의 집합을 L 이라 하고, 여기에 '공백'을 나타내는 레이블 ' $\sqcup$ '를 추가한 집합을 L' 이라 하자. 즉, $L' = \{a, b, c, \sqcup\}$ 이고, 정답인 레이블 연속열이 길이가 4인 $l = 'cbab'$ 였다고 하자. 이 연속열의 잉여 정보를 포함하는 표현은 ' $c \sqcup b \sqcup \sqcup aab$ '나 ' $\sqcup c \sqcup ba \sqcup \sqcup b$ '와 같이 무수히 많다. 조금 전의 두 가지 예는 입력 연속열의 길이가 $T = 8$ 일 때 일반적인 RNN이 출력할 수 있는 연속열에 해당한다.

이렇게 잉여 정보를 포함하는 연속열 π 와 그렇지 않은 원래의 연속열 l 은 다대일의 사상 $l = \mathcal{B}(\pi)$ 으로 연결된다. 다시 말해, $\mathcal{B}(c \sqcup b \sqcup \sqcup aab) = \mathcal{B}(\sqcup c \sqcup ba \sqcup \sqcup b) = 'cbab'$ 와 같다. 하나의 레이블 연속열 l 에 대해 l 의 모든 잉여성을 갖는 한 연속열의 집합을 다음으로 나타내기로 한다.

$$\mathcal{B}^{-1}(l) = \{\pi | \mathcal{B}(\pi) = l\}$$

또, 이 집합의 원소 π 를 l 에 대한 파스(parse)라고 부르기로 한다.

하나의 l 에 대한 parse $\pi \in \mathcal{B}^{-1}(l)$ 은 그림 7-8과 같이 나타낼 수 있다. 여기서 l' 은 그림 7-8(b)와 같이 l 의 모든 레이블 사이의 위치와 연속열의 시작, 끝 위치에 공백을 삽입한 것이다. (따라서 l' 의 길이는 항상 $2|l| + 1$ 이 된다). 그림 7-8(c)에서 $t = 1 \sim 8$ 까지 화살표를 따라가는 경로 하나가 파스 하나를 나타내며, 그림의 화살표를 전부 조합하면, $l = 'cbab'$ 에 대해 길이 $T = 8$ 인 파스를 모두 포함하게 된다.

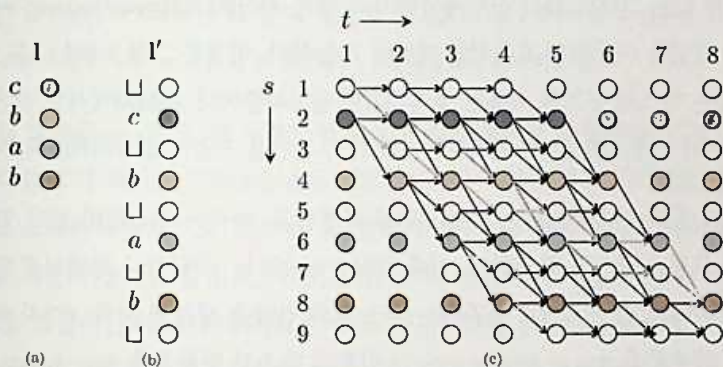


그림 7-8 CTC에서 다루는 레이블 연속열과 파스. (a) 추정해야 하는 잉여성이 없는 레이블 연속열 l . (b) (a)에 기계적으로 공백 레이블을 삽입한 l' . (c) 길이 $T = 8$ 인 경우의 파스 π , π 는 같은 길이 T 의 입력 연속열에 대한 RNN의 출력 연속열에 해당한다. 잉여성을 제거하면 l 과 일치하는 파스 ($\pi = \mathcal{B}(l)$)는 화살표의 경로 조합의 수만큼 존재한다. (d) 파란선의 파스는 $\sqcup c \sqcup ba \sqcup \sqcup b$

CTC를 적용하는 RNN은 출력층이 $|L'|$ 개의 유닛을 가지며, 그 활성화 함수가 소프트맥스인 신경망이다. 그리고 (지금까지와 마찬가지로) RNN의 시각 t 에 대한 출력 y_t^k (y_t^k 의 k 성분)은 해당 시각의 정답 레이블이 k 일 확률로 해석한다. 이때 연속열

안의 레이블의 출현이 서로 독립이라고 하면, 입력 연속열 $\mathbf{X} = (\mathbf{x}^1, \dots, \mathbf{x}^T)$ 에 대해 하나의 파스 $\pi = [\pi_1, \dots, \pi_T]$ 가 정답일 확률은 각 시각의 π_t 가 정답 레이블일 확률을 시간 방향으로 곱한 값인 다음으로 주어진다.

$$p(\pi|\mathbf{X}) = \prod_{t=1}^T y_{\pi_t}^t$$

CTC에서 핵심이 되는 아이디어는 하나의 레이블 1을 실현하는 모든 파스 $\pi \in \mathcal{B}^{-1}(1)$ 에 대해 $p(\pi|\mathbf{X})$ 를 더해서 1의 확률을 다음과 같이 계산하는 것이다.

$$p(1|\mathbf{X}) = \sum_{\pi \in \mathcal{B}^{-1}(1)} p(\pi|\mathbf{X})$$

식 7-9

이 분포를 계산할 수 있으면 이를 최대로 하는 1을 추정의 답으로 선택하면 된다.

식 7-9를 있는 그대로 계산하면 계산량이 크지만, 전진후진법(forward backward method)이라는 방법을 사용하면 효율적인 계산이 가능하다. 이것은 그림 7-8에 나온 것과 같은 파스의 구조를 이용한 동적계획법 알고리즘이다. HMM에서도 같은 아이디어가 이용되고 있다. 여기서는 중요한 부분만을 간결하게 정리하도록 하겠다.

지금부터 입력 연속열이 이미 주어졌다고 가정하고, 따라서 연속열의 길이 T 는 앞으로 고정된 값으로 간주한다. 이때 그림 7-8(c)의 (s, t) 를 하나 정해서 $1'$ 안의 s 번째 레이블 l_s 를 시각 t 에 통과하는 파스 π 의 집합을 생각해 보자. t 까지 이르는 파스의 앞부분을 $\pi_{1:t} = (\pi_1, \dots, \pi_t)$, t 이후의 파스의 뒷부분을 $\pi_{t:T} = (\pi_t, \dots, \pi_T)$ 로 나타낸다. 그리고 (s, t) 를 통과하는 모든 파스의 t 에서의 앞부분의 집합 $\pi_{1:t}$ 에 대한 확률 $p(\pi_{1:t})$ 의 합을 $\alpha_{s,t}$, (s, t) 를 통과하는 모든 파스의 t 에서의 뒷부분의 집합 $\pi_{t:T}$ 에 대한 확률 $p(\pi_{t:T})$ 의 합을 $\beta_{s,t}$ 로 나타낸다.

$\alpha_{s,t}$ 와 $\beta_{s,t}$ 는 모두 재귀적으로 계산할 수 있다. 그림 7-8처럼 모든 파스는 시각 $t = 1$ 일 때 $s = 1$ 이나 $s = 2$ 에서 출발한다. 그렇기 때문에 $t = 1$ 일 때의 $\alpha_{s,t}$ 는 $\alpha_{1,1} =$

y_b^1 . $\alpha_{2,1} = y_{l_1}^1$ 또는 $\alpha_{s,1} = 0 (\forall s > 2)$ 가 결정한다. $t > 1$ 일 때는 $\alpha_{s,t-1}$ 을 이용해서 l'_s 이 공백인 경우에는(또는 l'_s 은 공백이 아니지만 $l'_{s-2} = l'_s$, 즉 공백을 끼고 같은 레이블이 연속되는 경우에는) 다음과 같이 계산하고,

$$\alpha_{s,t} = y_{l'_s}^t (\alpha_{s-1,t-1} + \alpha_{s,t-1})$$

그 외의 경우에는 다음과 같이 계산할 수 있다*.

$$\alpha_{s,t} = y_{l'_s}^t (\alpha_{s-2,t-1} + \alpha_{s-1,t-1} + \alpha_{s,t-1})$$

위의 계산을 $t = 2$ 부터 $t = T$ 까지 순서대로 반복하면 모든 $\alpha_{s,t}$ 를 계산할 수 있다. $\beta_{s,t}$ 에 대한 계산도 같은 방법으로 이루어지지만 방향이 역순이다. 그리고 위의 식대로 계산을 수행하면 값이 너무 작아지므로 여기에 로그를 취하는 등의 조치가 필요하다. 이렇게 $\alpha_{s,t}$ (그리고 $\beta_{s,t}$)의 값을 구하면, 식 7-9의 분포는 $p(l|X) = \alpha_{|l|,T} + \alpha_{|l|+1,T}$ 와 같이 계산할 수 있다(그림 7-8에서 볼 수 있듯이, 파스가 유효하려면 마지막 시각 $t = T$ 일 때 $|l|$ 혹은 $|l| - 1$ 에 도달해야 하기 때문에)

CTC의 학습은 $\alpha_{s,t}$ 와 $\beta_{s,t}$ 를 이용하여 다음과 같은 절차로 수행한다. 먼저, 연속열 X 에 대한 정답 레이블 연속열을 d 로 정의한다. d 는 위의 1과 마찬가지로 잉여적인 정보를 포함하지 않으므로 일반적으로 입력 연속열보다 짧은 길이의 연속열이다. 그리고 이들의 쌍의 집합 $\{(X_n, d_n)\}$ 을 만든 뒤, 이를 훈련 샘플로 사용하여 RNN을 학습시킨다. MLE에 의해 RNN의 파라미터를 결정하고, 오차함수로 X 에 대한 레이블 연속열의 사후확률에 대해 음의 로그를 취한 우도를 선택한다.

$$E(w) = - \sum_n \log p(d_n | X_n)$$

이때 $p(d|X)$ 는 식 7-9와 같은 것이다($l = d$ 인 것으로 한다).

* 이 두 가지 케이스가 그림 7-8(c)의 각 노드로 들어오는 엣지 수(2나 3)에 대응하는 것에 주의한다.

오차함수의 기울기는 BPTT법으로 계산하지만, 그 시작점으로 각 시각 t 에서의 RNN의 출력에 대한 미분이 필요하다. 유닛 k 의 출력 y_k^t 에 대한 미분은 $\partial E / \partial y_k^t = -(1/p(d|X)) \partial p(d|X) / \partial y_k^t$ 이므로, $p(d|X)$ 를 y_k^t 로 미분할 필요가 있다. 여기서 $\alpha_{s,t}$ 와 $\beta_{s,t}$ 를 이용한다. 이 두 변수는 (s, t) 를 지나가는 파스(parse)의 전반부와 후반부 각각에 대한 확률의 총합이었다. 따라서 $\alpha_{s,t}$ 와 $\beta_{s,t}$ 의 곱은 (s, t) 를 지나가는 모든 파스에 걸쳐 같은 확률의 총합이 된다.

$$\alpha_{s,t} \beta_{s,t} = \sum_{\pi \in \mathcal{B}^{-1}(d) \text{ and } \pi_t = d'_s} p(\pi|X) \quad \text{식 7-10}$$

따라서 임의의 t 에 대하여 모든 $s \in [1, |d'|]$ 에 대해 합을 구하면 $p(d|X)$ 를 구할 수 있다.

$$p(d|X) = \sum_{s=1}^{|d'|} \alpha_{s,t} \beta_{s,t} \quad \text{식 7-11}$$

이때 d' 은 l' 과 마찬가지로 d 안에 각 레이블 사이의 위치 및 연속열 맨 처음과 맨 끝에 공백 레이블을 추가한 것이다. 목적하는 미분 $\partial p(d|X) / \partial y_k^t$ 는 식 7-11과 7-10으로부터 계산할 수 있다. 그 결과로 다시 출력층의 델타 $\delta_k^{out,t} = \partial E / \partial v_k^t$ 을 계산한다. 그 다음에는 보통의 BPTT를 적용하는 것으로 RNN의 어떤 층이라도 기울기를 구할 수 있다(자세한 내용은 문헌 20을 참조). RNN을 학습한 후에 주어진 입력 연속열 X 에 대하여 레이블 연속열 l 을 다음과 같이 추정한다.

$$\hat{l} = \underset{l}{\operatorname{argmax}} p(l|X)$$

이때 이 식의 정확한 값을 구하는 효율적인 계산 방법은 알려져 있지 않으며, 근사치에 대한 계산을 수행한다. 가장 단순한 방법은 입력 연속열에서 최대 우도를 갖는 파스를 다음으로 구한 후에 $\hat{l} = \mathcal{B}(\hat{\pi})$ 으로 하는 방법이다.

$$\hat{\pi} = \operatorname{argmax}_{\pi} p(\pi|X)$$

$\hat{\pi}$ 은 RNN의 각 시각 l 에서 최대 우도를 갖는 레이블의 연속열을 만드는 방법으로 간단히 구할 수 있지만, 그렇게 계산한 $\hat{\pi}$ 이 최적의 값임이 보장되지 않는다. 더 많은 계산량을 허용하는 대신 더 정밀한 $\hat{\pi}$ 을 구하는 방법도 알려져 있다^[20].

THE UNIVERSITY OF CHICAGO
LIBRARY
1100 EAST 58TH STREET
CHICAGO, ILL. 60637
U.S.A.

THE UNIVERSITY OF CHICAGO
LIBRARY

THE UNIVERSITY OF CHICAGO
LIBRARY

THE UNIVERSITY OF CHICAGO
LIBRARY

THE UNIVERSITY OF CHICAGO
LIBRARY

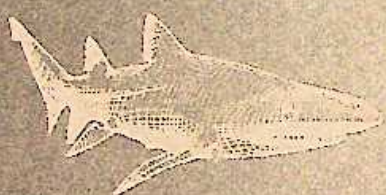
THE UNIVERSITY OF CHICAGO
LIBRARY

THE UNIVERSITY OF CHICAGO
LIBRARY

8

볼츠만 머신

볼츠만 머신은 유닛 간의 결합이 쌍방향인 신경망이다. 신경망의 행동을 정확하게 기술하는 것이 가장 큰 특징으로 일반적으로 데이터의 생성 모델로 쓰인다. 보통의 결정론적 신경망과 마찬가지로 신경망의 가중치나 바이어스를 통하여 결정되지만, 그 방식은 다르다. 현재 딥 러닝의 성공은 제약 볼츠만 머신을 사용한 DBN의 연구가 계기가 되었다.



8.1 데이터의 생성 모델

그림 8-1과 같이 숫자 필기 인식 이미지와 같은 데이터 x 의 집합을 살펴보자(이들 이미지를 어떻게 벡터 x 로 바꾸는지에 대해서는 그림 5-3을 참조).

$$x_1, x_2, \dots, x_N$$

그림 8-1과 같이 x 에는 숫자의 종류는 물론 쓴 사람 등의 차이에 의한 변동이 있지만, 그 폭은 벡터 x 의 공간 전체에 걸치지는 않는다. 숫자로 제한되어 있는 만큼 오히려 변동 폭은 상당히 제한되어 '치우쳐' 있다고 할 수 있다. 이 치우침을 포착하여 나타내기 위해 낱낱의 데이터 x 가 어떤 미지의 확률분포 $p_g(x)$ 를 따라 생성(generate)된다고 하자.

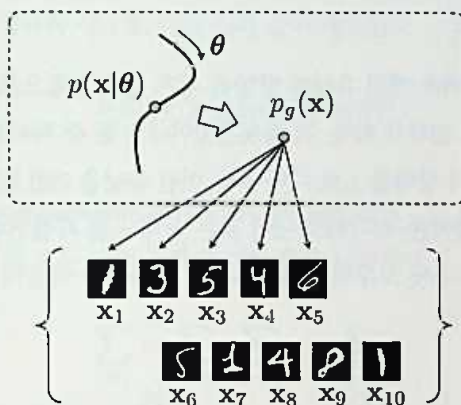


그림 8-1 데이터 생성의 확률적 구조와 그 모형의 이미지. 아직 알지 못하는 분포 $p_g(x)$ 로부터 데이터가 생성되고 있다. 모델분포 $p(x|\theta)$ 는 파라미터 θ 를 선택하여 이 분포를 근사한다.

실제의 $p_g(x)$ 를 알 수 없으므로 분포를 나타내는 적당한 함수 $p(x|\theta)$ 를 만들어서 이것으로 $p_g(x)$ 를 모델링한다. $p(x|\theta)$ 는 자유로운 파라미터 θ 를 가지며, 이 변수를 바꾸어서 다양한 분포를 나타낼 수 있다. 이러한 표현의 범위 내에서 θ 를 조절하여 실제 분포 $p_g(x)$ 에 가까워지도록 한다. 파라미터 θ 는 데이터를 사용해 구하는데, 가장 일반적인 방법은 최대우도추정(maximum likelihood estimation), 다시 말해 모델분포

$p(\mathbf{x}|\boldsymbol{\theta})$ 에서 $\mathbf{x}_1, \dots, \mathbf{x}_N$ 가 생성되었다고 할 때 가장 있을 법한 $\boldsymbol{\theta}$ 를 추정값으로 선택하는 방법이다. 즉, 동일한 $p(\mathbf{x}|\boldsymbol{\theta})$ 에서 데이터 $\mathbf{x}_1, \dots, \mathbf{x}_N$ 가 독립적으로 생성되었을 때 이들이 동시에 일어날 확률을 $\boldsymbol{\theta}$ 의 함수로 간주한 우도함수(likelihood function)를 상정하고, 이를 최대화하는 $\boldsymbol{\theta}$ 를 파라미터의 추정값으로 삼는다.

$$L(\boldsymbol{\theta}) = \prod_{n=1}^N p(\mathbf{x}_n|\boldsymbol{\theta})$$

식 8-1

8.2 볼츠만 머신

8.2.1 확률적 구조

그림 8-2(a)와 같이 여러 개의 유닛이 방향을 갖지 않은 결합으로 연결된 그래프(무향 그래프)가 있다고 하자. 각 유닛은 0이나 1 둘 중 하나의 값을 상태로 가지며, i 번째 유닛의 상태를 x_i 로 나타내자. 이런 유닛을 이진 유닛(binary unit)이라 부른다. 볼츠만 머신이란, 각각의 유닛이 갖는 상태 x_i 를 확률변수로 보았을 때, 그래프에 M 개 있는 유닛 모두의 상태 $\mathbf{x} = [x_1 \dots x_M]$ 이 다음의 확률분포에 의해 주어지는 것을 말한다.

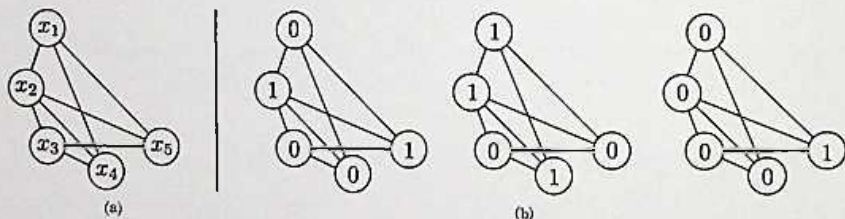


그림 8-2 볼츠만 머신의 예. (a) 각 유닛의 상태를 확률변수 x_i 로 나타낸다. (b) 그래프 전체의 상태의 조합은 2의 '유닛 수' 제곱만큼 존재한다.

$$p(\mathbf{x}|\theta) = \frac{1}{Z(\theta)} \exp \{-\Phi(\mathbf{x}, \theta)\}$$

식 8-2

$\Phi(\mathbf{x}, \theta)$ 를 에너지 함수(energy function)라 부르며, 다음과 같이 정의한다.

$$\Phi(\mathbf{x}, \theta) = - \sum_{i=1}^M b_i x_i - \sum_{(i,j) \in \mathcal{E}} w_{ij} x_i x_j$$

식 8-3

$\mathcal{E}$ 은 그래프에서 유닛 사이를 연결하는 결합(edge)이다. θ 는 $\Phi(\mathbf{x}, \theta)$ 에 포함된 $\{b_i | i = 1, \dots, M\}$ 과 $\{w_{ij} | (i,j) \in \mathcal{E}\}$ 이며, b_i 를 바이어스, w_{ij} 를 가중치라 부른다. 또한, 이 유닛 간의 결합은 방향성을 갖지 않으므로 $w_{ij} = w_{ji}$ 가 성립한다.

식 8-2의 $Z(\theta)$ 는 모델분포가 확률분포가 되기 위한 조건인 $\sum_{\mathbf{x}} p(\mathbf{x}|\theta) = 1$ 을 만족시키기 위한 정규화 상수로 다음 식과 같이 정의된다.

$$z(\theta) = \sum_{\mathbf{x}} \exp \{-\Phi(\mathbf{x}, \theta)\}$$

식 8-4

이를 분배함수(partition function)라 부른다. 여기서 $\mathbf{x}$ 의 합 $\sum_{\mathbf{x}}$ 는 다음과 같이 $\mathbf{x} = [x_1 \dots x_M]^T$ 의 모든 값의 조합에 대한 합을 나타낸다.

$$\sum_{\mathbf{x}} = \sum_{x_1=0,1} \sum_{x_2=0,1} \cdots \sum_{x_M=0,1}$$

$\mathbf{x}$ 는 각 성분이 이진 값(0 혹은 1)을 취하므로 조합하면 모두 합해 2^M 가지가 존재한다. 이 합에 대한 표기는 앞으로 쓰일 것이다.

식 8-2에서 8-4까지에 의해 정의되는 $p(\mathbf{x}|\theta)$ 는 2^M 가지 $\mathbf{x}$ 의 조합에 각각 발생 확률을 부여한다. 그리고 식 8-2와 같은 형태의 분포를 볼츠만 분포(Boltzmann distribution) 혹은 깁스 분포(Gibbs distribution)라 부른다. 분포의 구체적인 형태는 에너지 함수 $\Phi(\mathbf{x}, \theta)$ 에 의해 정해진다. 지수 함수가 갖는 단조성으로 인해 $\Phi(\mathbf{x}, \theta)$ 값이 작으면 작을수록 $\mathbf{x}$ 의 발생 확률이 높아진다. 식 8-3에서 보듯이 $\Phi(\mathbf{x}, \theta)$

에는 두 개의 항이 있는데, 첫 번째 항은 각 유닛에 대해 유닛 단독의 상태 x_i 로 결정되며, 두 번째 항은 두 개의 유닛의 상태 (x_i, x_j) 의 조합으로 결정된다. 에너지 함수의 값은 이 두 개 항의 그래프 전체에 대한 합이며, 이 합이 작게 되는 상태 $\mathbf{x}$ 가 발생하기 쉽고 그렇지 않은 상태는 발생하기 어렵게 된다.

8.2.2 학습

이 모델분포 $p(\mathbf{x}|\theta)$ 가 데이터의 진짜 분포 $p_g(\mathbf{x})$ 와 가능한 한 가까워지도록 데이터의 집합 $\mathbf{x}_1, \dots, \mathbf{x}_N$ 을 이용하여 최대우도추정을 행한다. 이어서 데이터 $\mathbf{x}_n$ 은 각 성분이 0 혹은 1의 두 가지 값만을 갖도록 한다(실숫값을 갖는 경우는 뒤에 설명하겠다). 식 8-1에 나온 우도함수 $L(\theta)$ 을 최대로 하는 θ 를 구하는 것이 목표이지만, $L(\theta)$ 대신에 $\log L(\theta)$ 를 최대화하여도 무방하다. 이 로그 우도함수 (log-likelihood function)는 식 8-2를 고쳐 다음과 같이 된다.

$$\log L(\theta) = \sum_{n=1}^N \log p(\mathbf{x}_n|\theta) = \sum_{n=1}^N \{-\Phi(\mathbf{x}_n, \theta) - \log Z(\theta)\}$$

이 로그 우도함수의 파라미터 θ , 즉 바이어스 b_i 및 가중치 w_{ij} 에 대한 기울기는 각각 다음과 같다.

$$\begin{aligned} \frac{\partial \log L(\theta)}{\partial b_i} &= \sum_{n=1}^N x_{ni} - NE_{\theta}[x_i] && \text{식 8-5a} \\ \frac{\partial \log L(\theta)}{\partial w_{ij}} &= \sum_{n=1}^N x_{ni}x_{nj} - NE_{\theta}[x_i x_j] && \text{식 8-5b} \end{aligned}$$

이때 x_{ni} 와 x_{nj} 는 각각 $\mathbf{x}_n$ 의 i 번째 성분과 j 번째 성분을 나타낸다. $E_{\theta}[\cdot]$ 은 θ 를 지정한 모델분포에 대한 기댓값으로, $E_{\theta}[x_i] = \sum_{\mathbf{x}} x_i p(\mathbf{x}|\theta)$, $E_{\theta}[x_i x_j] = \sum_{\mathbf{x}} x_i x_j p(\mathbf{x}|\theta)$ 이다. 모든 파라미터에 대해 식 8-5의 미분(기울기)이 0이 되는 θ 를 구한다.

표기의 편의를 위해 경험분포(empirical distribution)를 도입한다.

$$q(\mathbf{x}) \equiv \frac{1}{N} \sum_{n=1}^N \delta(\mathbf{x}, \mathbf{x}_n)$$

$\delta(\cdot)$ 은 다음과 같이 정의된다.

$$\delta(\mathbf{x}, \mathbf{y}) = \begin{cases} 0 & \mathbf{x} \neq \mathbf{y} \\ 1 & \mathbf{x} = \mathbf{y} \end{cases}$$

이 경험분포 $q(\mathbf{x})$ 를 사용하면 표본평균을 이 분포에 대한 기댓값으로 나타낼 수 있다. 예를 들면, 식 8-5b의 첫 번째 항을 N 으로 나눈 것을 다음의 식으로 다시 쓸 수 있다.

$$\frac{1}{N} \sum_{n=1}^N x_{ni} x_{nj} = \sum_{\mathbf{x}} x_i x_j q(\mathbf{x}) \quad \text{식 8-6}$$

$q(\mathbf{x})$ 에 대한 기댓값을 $\langle \cdot \rangle_{data}$, $p(\mathbf{x}|\theta)$ 에 대한 기댓값 $E_{\theta}[\cdot]$ 을 $\langle \cdot \rangle_{model}$ 로 나타내면, 식 8-5의 양변에 N 으로 나눈 것을 다음과 같이 고칠 수 있다.

$$\frac{1}{N} \frac{\partial \log L(\theta)}{\partial b_i} = \langle x_i \rangle_{data} - \langle x_i \rangle_{model} \quad \text{식 8-7a}$$

$$\frac{1}{N} \frac{\partial \log L(\theta)}{\partial w_{ij}} = \langle x_i x_j \rangle_{data} - \langle x_i x_j \rangle_{model} \quad \text{식 8-7b}$$

가중치 w_{ij} 와 바이어스 b_i 를 식 8-7에서 주어진 기울기 방향으로 업데이트하는 것을 반복하면, 우도함수를 최대화하는 기울기 상승법(목적함수의 부호를 반전한 경사 하강법)을 실행할 수 있다. 하지만 두 번째 항의 기댓값 $\langle \cdot \rangle_{model}$ 을 계산하는 것이 쉽지 않다. 이 값은 $\sum_{\mathbf{x}}$ 모양의 합을 포함하며, 이것은 위에 설명한 바와 같이 그래프의 유닛 수 M 에 대해 2^M 가지의 조합에 대한 합이기 때문이다. 조합의 수가

$M = 10$ 일 때는 1,048,576, $M = 30$ 이면 1,073,741,824가 되는 것과 같이 M 의 증가와 함께 폭발적으로 성장한다. 그 때문에 많은 수의 유닛을 가진 대규모 볼츠만 머신의 $\langle \cdot \rangle_{model}$ 을 직접 계산하는 것은 현실적이지 않다.

8.3 깁스 샘플링

어떤 파라미터 θ 가 주어졌을 때 모델분포 $p(\mathbf{x}|\theta)$ 로부터 $\mathbf{x}$ 를 샘플링하는(분포에 따르는 $\mathbf{x}$ 를 무작위로 생성시키는) 것을 생각해 보자. 만약 이러한 샘플링을 할 수 있다면, 샘플을 자유롭게 만들어 낼 수 있어 여러모로 편리하다. 예를 들면, $\mathbf{x}$ 의 샘플을 여러 개 사용하여 조금 전에 설명한 것과 같은 계산이 어려운 기댓값 $\langle x_i x_j \rangle_{model}$ 을 근사적으로 구할 수도 있게 된다. 일반적으로 다변수 확률분포로부터 이를 따르는 변수를 생성하는 것은 간단한 일이 아니지만, 볼츠만 머신으로 지역적 마르코프성(Markov性)이라 불리는 성질을 이용하여 깁스 샘플링(Gibbs sampling)이라는 편리한 방법을 사용할 수 있다.

유닛 i 이외의 모든 유닛의 변수를 열거한 벡터를 $\mathbf{x}_{-i}$ 라고 하고, 조건부 분포 $p(x_i|\mathbf{x}_{-i}, \theta)$. 다시 말해 $\mathbf{x}_{-i}$ 의 값이 정해졌을 때 변수 x_i 의 분포를 살펴보기로 한다. 조건부 분포의 정의에 따라 이 값은 다음과 같이 쓸 수 있다.

$$p(x_i|\mathbf{x}_{-i}, \theta) = \frac{p(\mathbf{x}, \theta)}{\sum_{x_i=0,1} p(\mathbf{x}, \theta)}$$

이 식에 식 8-2를 대입하면 분모와 분자로 몇 개 항이 소거되고 다음과 같이 된다.

$$p(x_i|\mathbf{x}_{-i}, \theta) = \frac{\exp \left\{ \left(b_i + \sum_{j \in \mathcal{N}_i} w_{ij} x_j \right) x_i \right\}}{1 + \exp \left(b_i + \sum_{j \in \mathcal{N}_i} w_{ij} x_j \right)}$$

식 8-8

이때 $\mathcal{N}_i$ 는 유닛 i 와 결합을 갖는 유닛의 집합이다. 이렇게 유닛 i 외의 모든 유닛의 상태 $\mathbf{x}_{-i}$ 의 값이 정해진 조건부 분포 $p(x_i | \mathbf{x}_{-i}, \theta)$ 는 $\mathcal{N}_i$ 의 유닛의 상태가 정해진 조건부 분포 $p(\{x_j | j \in \mathcal{N}_i\}, \theta)$ 으로 주어진다.

이런 방법으로 각 x_i 의 조건부 분포에 대한 효율적인 계산이 가능하므로, 유닛 i 를 제외한 모든 유닛의 상태가 주어졌을 때 x_i 를 샘플링하는 것은 간단하다. 구체적으로 밝히자면, 먼저 식 8-8에 따라 주어진 $\mathbf{x}_{-i}$ 로부터 확률 $p(x_i = 1 | \mathbf{x}_{-i}, \theta)$ 를 계산한다. 다음으로, 구간 $[0, 1]$ 의 균등난수를 생성하여 그 값이 이 확률을 밑돌면 $x_i = 1$, 그렇지 않으면 $x_i = 0$ 으로 한다. 이렇게 얻은 x_i 는 식 8-8의 조건부 분포를 따르게 된다.

깁스 샘플링은 이렇게 변수 x_i 의 샘플링을 각 변수 ($i = 1, \dots, M$)에 대해서 순서대로 반복하여 원래의 결합분포 $p(\mathbf{x} | \theta)$ 를 따르는 $\mathbf{x}$ 의 값을 샘플링하는 방법이다. 전체적인 순서는 다음과 같다. 먼저, 각 변수를 적당한 값으로(무작위로 0 혹은 1의 값을 부여하여) 초기화한 뒤에 이를 $\mathbf{x}^{(0)}$ 이라 한다. 그런 다음, 각 성분 x_i 에 대해 $i = 1, \dots, M$ 의 순서대로 위에서 설명한 샘플링을 한 후, 모든 성분에 대해 완료하면 $i = 1$ 로 되돌아가 이를 반복한다. 그리고 t 바퀴째($t = 1, 2, \dots$)의 $x_i^{(t)}$ 는 다음으로부터 샘플링한다.

$$p(x_i | x_1^{(t)}, \dots, x_{i-1}^{(t)}, x_{i+1}^{(t-1)}, \dots, x_M^{(t-1)})$$

즉, x_i 를 샘플링할 때 그 이외의 변수는 가장 새로운 값으로 설정한다.

t 가 충분히 커지면 이렇게 얻게 되는 $\mathbf{x}^{(t)}$ 는 모델분포 $p(\mathbf{x} | \theta)$ 를 따르게 되는 것으로 알려져 있다^[51]. 처음 반복을 시작할 때, 그러니까 t 가 작은 동안에는 그 정확도가 충분하지 않다. 또, 여러 개의 샘플을 얻기 위해 t 번째와 t' 번째의 샘플을 추출할 때는 t 와 t' 의 간격을 충분히 뒤야 한다. t 와 t' 가 너무 가까우면 $\mathbf{x}^{(t)}$ 와 $\mathbf{x}^{(t')}$ 가 서로 영향을 미치게 되기 때문이다.

깁스 샘플링은 정확도를 높이기 위해 반복 횟수 t 를 충분히 크게 해야 하므로 계산 비용이 비싼 방법이지만, $p(x|\theta)$ 를 따르는 변수를 생성할 수 있는 유용한 수단이다. $p(x|\theta)$ 를 따르는 변수를 생성하여 이 분포에 대한 기댓값 $\langle \cdot \rangle_{model}$ 이나 주변분포 등을 근사할 수 있다. 앞으로 설명할 제약 볼츠만 머신의 학습에도 깁스 샘플링에 기초한 방법이 사용된다.

8.4 은닉 변수를 갖는 볼츠만 머신

8.4.1 확률적 구조

지금까지는 그래프 유닛의 모든 상태가 데이터 x 의 모든 성분과 대응하는 경우를 논하였다. 이번에는 그림 8-3과 같이 그래프가 직접적으로 데이터와 관계하지 않는 유닛을 가지는 경우를 논하겠다. 유닛의 종류를 나타내기 위해 유닛의 상태를 두 종류의 변수 v 와 h 를 써서 나타내기로 하자. v 는 데이터 x 를 직접 나타내며($v = x$), h 는 x 와는 직접적으로 관계되지 않는다. v 와 같은 변수를 가시 변수(visible variable), h 와 같은 변수를 은닉 변수(hidden variable)라고 부른다. 이는 그래프의 거동은 v 와 h 만으로 표현되지만, h 의 값은 외부로부터 관측이 불가능하고 v 만 관측할 수 있다(이렇게 관찰할 수 있는 v 는 생성된 데이터라고도 볼 수 있다)는 의미다.

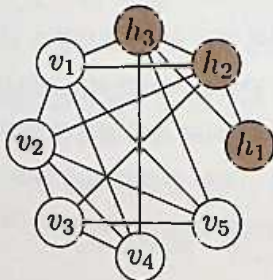


그림 8-3 은닉 변수를 갖는 볼츠만 머신의 예. v_1, v_2, v_3, v_4, v_5 가 가시 변수, h_1, h_2, h_3 이 은닉 변수다.

은닉 변수를 갖는 볼츠만 머신에도 상태의 확률분포는 은닉 변수를 갖지 않는 경우와 마찬가지로 정의된다. 표기의 편의를 위해 $\mathbf{v}$ 와 $\mathbf{h}$ 의 성분을 순서대로 열거하여 하나의 벡터 $\mathbf{z}$ 를 만든다. 이를테면, $\mathbf{v} = [v_1 v_2 v_3]^T$, $\mathbf{h} = [h_1 h_2]^T$ 일 때 $\mathbf{z} = [z_1 z_2 z_3 z_4 z_5]^T = [v_1 v_2 v_3 h_1 h_2]^T$ 가 된다는 것이다. 이 변수 $\mathbf{z}$ 를 사용하면 에너지 함수는 식 8-3과 마찬가지로 다음과 같이 나타낼 수 있다.

$$\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) = \Phi(\mathbf{z}, \boldsymbol{\theta}) = - \sum_i b_i z_i - \sum_{i,j} w_{ij} z_i z_j$$

모든 유닛의 상태는 다음의 확률분포에 의해 기술된다.

$$p(\mathbf{v}, \mathbf{h} | \boldsymbol{\theta}) = p(\mathbf{z} | \boldsymbol{\theta}) = \frac{1}{Z(\boldsymbol{\theta})} \exp \{ -\Phi(\mathbf{z}, \boldsymbol{\theta}) \}$$

이때 $Z(\boldsymbol{\theta}) = \sum_{\mathbf{z}} p(\mathbf{z} | \boldsymbol{\theta})$ 이다.

은닉 변수를 가지지 않는 것과 비교해서, 은닉 변수를 갖는 볼츠만 머신은 일반적으로 더 높은 자유도를 가질 수 있다. 은닉 변수가 없는 경우에는 모델분포의 자유도는 데이터 $\mathbf{x}$ 의 차원(次元)과 항상 같다. 하지만 은닉 변수가 있으면 더 복잡한 데이터의 생성분포를 더 높은 정확도로 근사할 수 있는 가능성이 있다. 실제로, (앞으로 설명할 RBM에 비해) 은닉 변수의 수를 충분히 늘리면 임의의 분포를 나타낼 수 있음이 증명되어 있다^[58].

8.4.2 학습

은닉 변수를 갖는 볼츠만 머신의 학습을 은닉 변수를 갖지 않는 경우와 마찬가지로 최대우도추정에 따라 고찰해 보자. $\mathbf{v} = \mathbf{x}$ 이므로 아래에서는 각 데이터를 ($\mathbf{x}_n$ 가 아니라) $\mathbf{v}_n$ 로 표기한다. 데이터 $\mathbf{v}_1, \dots, \mathbf{v}_N$ 에 대한 우도를 계산하기 위해서 모델분포 $p(\mathbf{v}, \mathbf{h} | \boldsymbol{\theta})$ 에 데이터와 관계없는 은닉 변수가 포함되어 있으므로 먼저 은닉 변수 $\mathbf{h}$ 를 다음과 같이 주변화(marginalize)한다.

$$p(\mathbf{v}|\boldsymbol{\theta}) = \sum_{\mathbf{h}} p(\mathbf{v}, \mathbf{h}|\boldsymbol{\theta})$$

식 8-9

이것은 $\mathbf{v}$ 만의 분포이며, 데이터 $\mathbf{x}$ 의 모델분포 $p(\mathbf{x}|\boldsymbol{\theta})$ 에 해당한다. 따라서 우도함수는 다음이 되고,

$$L(\boldsymbol{\theta}) = \prod_{n=1}^N p(\mathbf{v}_n|\boldsymbol{\theta})$$

여기에 식 8-9를 대입하여 로그를 취하면 다음의 식이 된다.

$$\log L(\boldsymbol{\theta}) \propto \frac{1}{N} \sum_{n=1}^N \left[\log \sum_{\mathbf{h}} \exp \{ -\Phi(\mathbf{v}_n, \mathbf{h}, \boldsymbol{\theta}) \} - \log Z(\boldsymbol{\theta}) \right]$$

식 8-10

식 8-10을 $\boldsymbol{\theta}$ 로 미분한다. 우변 $[\cdot]$ 안의 첫 번째 항을 w_{ij} 로 미분하면 다음이 된다(편의상 $\mathbf{v}_n$ 을 $\mathbf{v}$ 로 표기하였다).

$$\begin{aligned} \frac{\partial}{\partial w_{ij}} \log \sum_{\mathbf{h}} \exp \{ -\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \} &= \frac{\sum_{\mathbf{h}} z_i z_j \exp \{ -\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \}}{\sum_{\mathbf{h}} \exp \{ -\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) \}} \\ &= \sum_{\mathbf{h}} z_i z_j p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) \end{aligned}$$

식 8-11

마지막의 $p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta})$ 은 $\mathbf{v}$ 가 정해져 있을 때의 $\mathbf{h}$ 의 조건부 분포로, 조건부 분포의 정의에 따라 유도되었다. 한편, 식 8-10의 우변 $[\cdot]$ 안 두 번째 항을 w_{ij} 로 미분하는 것은 이전과 같다. 따라서 식 8-6과 같이 표본평균을 경험분포의 기댓값으로 바꿔 쓰면, 로그 우도의 미분은 은닉 변수가 없을 때와 같이 다음의 형태가 된다.

$$\frac{\partial \log L(\boldsymbol{\theta})}{\partial w_{ij}} \propto \langle z_i z_j \rangle_{data} - \langle z_i z_j \rangle_{model}$$

식 8-12

이때 여기서의 기댓값 $\langle \cdot \rangle_{data}$ 는 $\langle g(\mathbf{v}, \mathbf{h}) \rangle_{data} = \sum_{\mathbf{v}} \sum_{\mathbf{h}} g(\mathbf{v}, \mathbf{h}) p(\mathbf{v} | \mathbf{h}, \theta) q(\mathbf{v})$ 를 뜻한다. 바이어스에 대한 미분도 마찬가지로 유도할 수 있으며, 이들의 기울기 방향으로 파라미터를 업데이트하는 경사 하강법을 적용할 수 있다. 다만, 은닉 변수가 없는 경우에서 어려웠던 두 번째 항의 기댓값과 함께, 첫 번째 항에도 상태의 수가 폭발적으로 증가하는 조합의 계산이 필요하기 때문에 계산이 훨씬 어려워진다.

8.5 제약 볼츠만 머신

8.5.1 확률적 구조

제약 볼츠만 머신(Restricted Boltzmann Machine)이라 RBM은 은닉 변수를 포함하는 볼츠만 머신의 일종으로, 유닛 간의 결합에 특별한 제약이 가해진 것을 말한다. 구체적으로는 그림 8-4와 같이 가시 변수의 유닛과 은닉 변수의 유닛 사이에만 결합이 존재하고, 가시 변수의 유닛끼리나 은닉 변수의 유닛끼리의 결합은 가지지 않는 것과 같은 것이다. 단층으로 구성된 앞먹임 신경망과 똑같이 생겼지만, 이것은 볼츠만 머신이므로 결합이 쌍방향(무향 에지)이다. 앞으로는 가시 변수의 유닛을 가시 유닛, 은닉 변수의 유닛을 은닉 유닛으로 부르기로 한다.

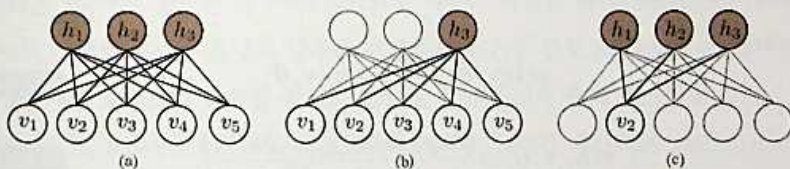


그림 8-4 (a) 제약 볼츠만 머신(RBM)의 예. v_1, v_2, v_3, v_4, v_5 가 가시 변수, h_1, h_2, h_3 이 은닉 변수다. (b) 가시 변수가 결정되면 각 은닉 유닛의 상태의 확률이 정해진다. (c) 이번에는 반대로 은닉 변수가 결정되면 가시 유닛의 상태의 확률이 정해진다.

RBM은 이 특별한 구조 덕에 상태의 분포함수가 특별한 모양을 가지게 된다.

가시 유닛을 인덱스 i 로, 은닉 유닛을 인덱스 j 로 각각 나타내겠다. RBM의 에너지 함수는 다음이 된다.

$$\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) = - \sum_i a_i v_i - \sum_j b_j h_j - \sum_i \sum_j w_{ij} v_i h_j \quad \text{식 8-13}$$

모델의 파라미터는 가시 변수와 같은 수의 바이어스 $\{a_i\}$, 은닉 변수와 같은 수의 바이어스 $\{b_j\}$, 그리고 이 두 가지의 조합과 같은 수의 가중치 $\{w_{ij}\}$ 의 세 가지로 구성된다. 모든 변수의 확률분포는 지금까지와 마찬가지로 $p(\mathbf{v}, \mathbf{h}|\boldsymbol{\theta}) = (1/Z(\boldsymbol{\theta})) \exp \{-\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})\}$ 의 형태로 주어진다.

8.5.2 조건부 분포

RBM은 그 구조 덕에 변수의 조건부 분포를 간단하게 계산할 수 있는 성질을 갖는다. 가시 변수의 값이 정해져 있을 때의 은닉 변수의 조건부 분포 $p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta})$ 는 정의에 의해 다음과 같다.

$$p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) = \frac{p(\mathbf{v}, \mathbf{h}|\boldsymbol{\theta})}{\sum_{\mathbf{h}} p(\mathbf{v}, \mathbf{h}|\boldsymbol{\theta})}$$

여기에 $p(\mathbf{v}, \mathbf{h}|\boldsymbol{\theta}) = (1/Z(\boldsymbol{\theta})) \exp \{-\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta})\}$ 와 식 8-13을 대입하여 계산하면 다음을 얻을 수 있다.

$$p(\mathbf{h}|\mathbf{v}, \boldsymbol{\theta}) = \prod_j p(h_j|\mathbf{v}, \boldsymbol{\theta}) \quad \text{식 8-14a}$$

$$p(h_j|\mathbf{v}, \boldsymbol{\theta}) = \frac{\exp \{(b_j + \sum_i w_{ij} v_i) h_j\}}{1 + \exp (b_j + \sum_i w_{ij} v_i)} \quad \text{식 8-14b}$$

이들 식이 의미하듯이, RBM에서 은닉 유닛 하나의 상태 h_j 는 다른 은닉 유닛의 상태와 독립이다. 그러므로 가시 변수의 값이 모두 정해져 있으면 은닉 변수의 확률분포가 각각 개별로 정해진다(그림 8-4 (b)). 반대로, 은닉 변수의 값이 모두

정해져 있을 때에는 가시 변수의 분포가 정해진다(그림 8-4 (c)).

식 8-14b에서 $h_j = 1$ 이 될 확률은 로지스틱 함수 $\sigma(x) = 1/(1 + \exp(-x))$ 을 사용하여 다음과 같이 간단히 나타낼 수 있다.

$$p(h_j = 1 | \mathbf{v}, \boldsymbol{\theta}) = \sigma \left(b_j + \sum_i w_{ij} v_i \right) \quad \text{식 8-15}$$

이와 반대 방향, 즉 은닉 유닛의 상태가 정해졌을 때의 가시 유닛의 거동도 마찬가지로 된다. 다시 말해, $\mathbf{h}$ 가 정해졌을 때의 $v_i = 1$ 일 조건부확률은 다음이 된다.

$$p(v_i = 1 | \mathbf{h}, \boldsymbol{\theta}) = \sigma \left(a_i + \sum_j w_{ij} h_j \right) \quad \text{식 8-16}$$

이진 변수 x 가 확률 p 로 1, $1 - p$ 로 0을 값으로 가질 때, 이 변수의 분포는 베르누이 분포(Bernoulli distribution)다. 이것으로부터 지금 설명하는 RBM을 베르누이 RBM(Bernoulli RBM)이라 부르기도 한다.

8.5.3 RBM과 자기부호화기

RBM은 5장에서 설명한 자기부호화기와도 비슷한 역할을 한다. RBM으로 가시층 유닛의 상태로 은닉층 유닛의 상태를 계산하는 조건부 분포 $p(h_j | \mathbf{v}_i; \boldsymbol{\theta})$ 은 앞먹임 신경망에서 로지스틱 함수를 활성화 함수로 갖는 유닛의 출력 계산식과 같다. 마찬가지로, RBM에서 은닉층 상태에서부터 가시층 상태를 계산하는 $p(v_i | \mathbf{h}_j; \boldsymbol{\theta})$ 는 가시층의 유닛이 같은 활성화 함수를 가졌다고 상정했을 때의 출력에 대한 계산과 같다. 앞으로 설명하겠지만, RBM의 최적화 계산은 데이터 $\mathbf{v}$ 를 가시층에 설정하고 은닉층의 상태를 계산하여 다시 은닉층의 상태를 고정하고 가시층의 상태를 계산하는 과정을 반복한다. 이 계산은 자기부호화기의 순전파 계산과 매우

비슷하다. RBM과 자기부호화기의 차이는 학습(최적화) 방법에 있다. 자기부호화기는 입력 $\mathbf{x}$ 와 출력 $\hat{\mathbf{x}}$ 의 값이 직접 가까워지도록 파라미터를 결정하였지만, RBM에서는 가시층의 상태분포가 데이터의 생성분포에 가까워지도록 학습이 이루어진다. 그렇지만 학습에 결과로 얻어지는 파라미터가 비슷해지는 일도 드물지 않게 일어난다.

자기부호화기는 학습 후에 신경망에 데이터를 입력하여 얻는 중간층의 출력을 그 데이터의 표현으로 보고, 어떤 다른 목적으로 이용하는 것을 고려했었다. 이를테면, 데이터 그 자체가 아니라 데이터를 입력했던 때의 중간층 출력을 클래스 분류에 이용해서 분류 성능을 향상시키는 것을 예로 들 수 있다. RBM도 $p(\mathbf{h}|\mathbf{v}; \theta)$ 에 따라서 이 확률분포로부터 계산한 은닉층의 상태를 입력 $\mathbf{v}$ 의 표현으로 간주하고, 그와 같은 추론에 이용하는 것이 가능하다. 그런 경우 RBM은 단순히 데이터의 확률분포를 추정하는 것뿐만 아니라, 데이터가 생성하는 구조 자체를 포착하기 위해서 쓰고 있다고도 생각할 수 있다. 데이터 생성 과정의 핵심적인 정보, 즉 은닉층의 상태를 불필요한 정보를 뺀 본질적 정보로 보고 추론에 이용하는 것이다.

8.6 RBM의 학습

8.6.1 깁스 샘플링을 사용한 기울기 계산

데이터 $\{\mathbf{v}_n | n = 1, \dots, N\}$ 으로부터 RBM의 파라미터 $\{a_i\}$, $\{b_j\}$, $\{w_{ij}\}$ 를 결정하는 방법을 설명하겠다. RBM의 로그 우도에 대한 미분은 은닉 변수가 있는 경우의 로그 우도의 미분(식 8-11)을 위에서 설명한 은닉 변수의 조건부 분포가 갖는 독립성을 이용하여 다음 식에 따라 구할 수 있다.

$$\begin{aligned}\frac{1}{N} \frac{\partial \log L}{\partial w_{ij}} &= \frac{1}{N} \sum_{n=1}^N \left\{ \sum_{h_j=0,1} v_{ni} h_j p(h_j | \mathbf{v}_n) \right\} - \sum_{\mathbf{v}, \mathbf{h}} v_i h_j p(\mathbf{v}, \mathbf{h} | \boldsymbol{\theta}) \\ &= \frac{1}{N} \sum_{n=1}^N v_{ni} p(h_j = 1 | \mathbf{v}_n) - \sum_{\mathbf{v}, \mathbf{h}} v_i h_j p(\mathbf{v}, \mathbf{h} | \boldsymbol{\theta})\end{aligned}\quad \text{식 8-17a}$$

마찬가지로, 바이어스에 대한 미분은 다음이 된다.

$$\frac{1}{N} \frac{\partial \log L}{\partial a_i} = \frac{1}{N} \sum_{n=1}^N v_{ni} - \sum_{\mathbf{v}, \mathbf{h}} v_i p(\mathbf{v}, \mathbf{h} | \boldsymbol{\theta}) \quad \text{식 8-17b}$$

$$\frac{1}{N} \frac{\partial \log L}{\partial b_j} = \frac{1}{N} \sum_{n=1}^N p(h_j = 1 | \mathbf{v}_n) - \sum_{\mathbf{v}, \mathbf{h}} h_j p(\mathbf{v}, \mathbf{h} | \boldsymbol{\theta}) \quad \text{식 8-17c}$$

이들의 기울기의 첫째 항을 $\langle \cdot \rangle_{data}$, 두 번째 항을 $\langle \cdot \rangle_{model}$ 이라 하면, 경사 하강법에 의한 가중치와 바이어스의 업데이트 식은 다음과 같이 쓸 수 있다.

$$\Delta w_{ij} = \epsilon (\langle v_i h_j \rangle_{data} - \langle v_i h_j \rangle_{model}) \quad \text{식 8-18a}$$

$$\Delta a_i = \epsilon (\langle v_i \rangle_{data} - \langle v_i \rangle_{model}) \quad \text{식 8-18b}$$

$$\Delta b_j = \epsilon (\langle h_j \rangle_{data} - \langle h_j \rangle_{model}) \quad \text{식 8-18c}$$

모든 파라미터의 업데이트 식이 $\langle \cdot \rangle_{data}$ 와 $\langle \cdot \rangle_{model}$ 두 항으로 이루어져 있다.

RBM의 경우에는 $\langle \cdot \rangle_{data}$, 즉 식 8-17의 첫 번째 항은 어렵지 않게 계산할 수 있으나, 두 번째 항인 $\langle \cdot \rangle_{model}$ 은 통상적인 볼츠만 머신과 마찬가지로 쉽게 계산할 수 없다. 여기서 깃스 샘플링을 사용하여 근사적으로 계산하는 방법을 고려해 보자. 은닉 유닛끼리, 그리고 가시 유닛끼리 결합을 갖지 않는 성질 덕에, 소위 말하는 블록 샘플링과 같은 방법을 사용할 수 있어서 어느 정도 효율적인 계산이 가능하다.

구체적인 방법은 먼저 $\mathbf{v}$ 에 랜덤한 초깃값 $\mathbf{v}^{(0)}$ (각 성분마다 0 혹은 1의 값을 랜덤하게 배정한 것)을 설정한다. 다음으로, 식 8-15를 사용하여 $p_j \equiv p(h_j = 1 | \mathbf{v}^{(0)})$ 의 값을

산출하여 무작위로 이진 변수 h_j 를 샘플링한다. 즉, 구간 $[0, 1]$ 의 균등난수가 p_j 보다 작으면 $h_j = 1$ 로 하고, 그렇지 않으면 $h_j = 0$ 으로 한다. 이 샘플링을 각 j 에 대해 순서대로 시행하여 얻어진 이진 벡터를 $\mathbf{h}^{(0)}$ 으로 나타낸다. 이어서 이번엔 반대로 $\mathbf{h}^{(0)}$ 로부터 식 8-16을 이용하여 $p_i \equiv p(v_i = 1 | \mathbf{h}^{(0)})$ 의 값을 구하고, 균등난수를 이용하여 같은 방법으로 이진 변수 v_i 를 각 가시 유닛에 대해 샘플링해서 그 결과를 $\mathbf{v}^{(1)}$ 로 나타낸다. 위와 같이 $\mathbf{v}$ 와 $\mathbf{h}$ 를 서로 엇갈려가며 샘플링하는 절차를 다음의 순서로 반복한다.

$$\mathbf{v}^{(0)} \rightarrow \mathbf{h}^{(0)} \rightarrow \mathbf{v}^{(1)} \rightarrow \dots \rightarrow \mathbf{v}^{(T)} \rightarrow \mathbf{h}^{(T)}$$

충분히 큰 T 에 이른 $\mathbf{v}^{(T)}$ 와 $\mathbf{h}^{(T)}$ 는 $p(\mathbf{v}, \mathbf{h} | \theta)$ 로부터 샘플링 결과에 가까운 것이 보장된다.

이 방법으로 $\mathbf{v}$ 와 $\mathbf{h}$ 의 샘플 여러 쌍 $\{(\mathbf{v}, \mathbf{h})\}$ 을 생성하여 이 값들에 기초한 평균값을 원하는 기댓값의 추정값으로 이용한다. 다시 말해,

$$\langle v_i h_j \rangle_{model} \approx \overline{v_i h_j}, \quad \langle v_i \rangle_{model} \approx \overline{v_i} \text{이며, } \langle h_j \rangle_{model} \approx \overline{h_j} \text{이다.}$$

8.6.2 대조적 발산(CD)

파라미터의 업데이트 정도를 위와 같이 깁스 샘플링에 의해 계산하면, 특히 $\langle \cdot \rangle_{model}$ 항에 충분한 정밀도를 얻기 위해 반복 횟수 T 를 충분히 크게 해야 해서 상당한 계산 비용이 든다. 그러나 방법을 약간 수정하는 것만으로 적은 반복 횟수로 파라미터의 더 나은 업데이트 값을 계산할 수 있는 방법이 알려져 있다^[25]. 이 방법을 대조적 발산(Contrastive Divergence)(이하 CD)이라 부른다^[3, 25, 26].

CD의 절차는 단순하다. 맨 처음에 가시 유닛에 설정하는 $\mathbf{v}^{(0)}$ 을 (랜덤하게 초기화하는 것이 아니라) 훈련 샘플 $\mathbf{v}^{(0)} = \mathbf{v}_n$ 으로 하여 깁스 샘플링과 같이 은닉 변수와 가시 변수의 샘플링을 T 번 반복한다. T 는 보통 크지 않게, $T = 1$ 로 해도 무방하다.

앞으로 샘플링을 T 번 시행하는 CD를 CD_T 로 표기하겠다*. 이렇게 얻어진 $\mathbf{v}^{(T)}$ 및 $\mathbf{h}^{(T)}$ 를 사용하여 같은 방법으로 기댓값을 근사한다.

CD_T 의 구체적인 계산은 아래와 같다. 그리고 지금까지는 파라미터를 한 번 업데이트하는 데에 묵시적으로 N 개의 훈련 샘플 모두를 사용하는 것을 상정하였으나, 앞먹임 신경망의 확률적 경사 하강법과 같이 소수의 샘플로 이루어진 미니배치 단위의 업데이트를 수행하는 것이 보다 효과적이다. 샘플 하나 단위로 업데이트하는 경우를 보면, 샘플 $\mathbf{v}_n$ 을 하나 골라 $\mathbf{v}^{(0)} \equiv \mathbf{v}_n$ 으로 설정하여 위에서 설명한 순서 $\mathbf{v}^{(0)} \rightarrow \mathbf{h}^{(0)} \rightarrow \dots \mathbf{v}^{(T)} \rightarrow \mathbf{h}^{(T)}$ 대로 반복적으로 생성한 후에 다음과 같이 파라미터를 업데이트한다.

$$\Delta w_{ij} = \epsilon \left(v_i^{(0)} p_j^{(0)} - v_i^{(T)} p_j^{(T)} \right) \quad \text{식 8-19a}$$

$$\Delta a_i = \epsilon \left(v_i^{(0)} - v_i^{(T)} \right) \quad \text{식 8-19b}$$

$$\Delta b_j = \epsilon \left(p_j^{(0)} - p_j^{(T)} \right) \quad \text{식 8-19c}$$

이때 $p_j^{(t)} = p(h_j = 1 | \mathbf{v}^{(t)}, \boldsymbol{\theta})$ 이다. 파라미터의 업데이트를 미니배치 단위로 하는 경우에는 미니배치에 포함되는 샘플 하나하나에 대해 식 8-19의 우변 값을 계산하고, 그 평균을 파라미터의 업데이트 값으로 한다. 미니배치의 크기 및 생성 방법은 3.3절에서 논한 내용을 따른다.

위와 같이 CD는 적은 횟수의 샘플링으로 기댓값을 근사하는 방법이라 볼 수 있다. 단, 식 8-19에서 구하게 되는 파라미터의 업데이트 방향은 원래 값인 로그 우도의 기울기에 비하면 좋은 근삿값이 되지 못한다. 실제로 이 업데이트 방향은 로그 우도의 기울기가 아니라, 대조적 발산이라 불리는 별도의 목적함수의 함숫값을 작게 하는 방향이라 볼 수 있다. 자세한 내용을 알고 싶거나 왜 대조적

* 그리고 CD_1 은 연산량만 놓고 보면 가장 낮지만, 일반적으로는 $CD_T (T > 1)$ 의 결과가 더 낫다. 또 충분한 횟수를 반복한 깃스 샘플링이 T 가 작은 경우의 CD_T 보다도 결과가 뛰어나다.

발산값을 최소화하는 것으로 목표한 최적화가 가능한지는 문헌 [25]를 참조하기 바란다.

8.6.3 CD의 실제 적용

식 8-19에는 몇 가지 변경할 수 있는 여지가 있다. 문헌에 따라서는 식 8-19a 및 식 8-19c의 $p_j^{(0)}$ 가 샘플값 $h_j^{(0)}$ 로 치환된다^[3]. 단, 식 8-17에 있듯이 엄밀히 따지면 $p_j^{(0)}$ 가 정확하다고 할 수 있다. 또, 식 8-19a와 8-19b의 샘플값 $v_i^{(T)}$ 는 그 생성에 사용된 $p_i^{(T)}$ 로 치환해도 무방하며, 그렇게 했을 때 최적화가 약간 빨라진다는 지적도 있다^[26].

3장에서 설명한 확률적 경사 하강법에서 부가적으로 사용되는 방법 대부분이 RBM에서도 유효하다. 그중 하나로 기울기 감쇠를 들 수 있다. 로그 우도에 최적화항 $-\sum_{i,j} w_{ij}^2/2$ 를 더한다. 그 결과, 가중치의 업데이트 값은 다음이 된다.

$$\Delta w_{ij}^{(t+1)} = w_{ij}^{(t+1)} - w_{ij}^{(t)} = \epsilon \left(v_j^{(0)} p_j^{(0)} - v_i^{(T)} p_j^{(T)} - \lambda w_{ij}^{(t)} \right)$$

바이어스는 간혹 큰 값을 가져야 할 필요도 있으므로 바이어스에는 가중치 감쇠를 적용하지 않는다. 자주 사용되는 λ 의 값은 0.01에서 0.00001 사이이다. 또, 모멘텀이 유효한 경우가 있다. 모멘텀을 사용하는 경우의 업데이트 값은 다음과 같이 수정된다.

$$\Delta w_{ij}^{(t+1)} = w_{ij}^{(t+1)} - w_{ij}^{(t)} = \epsilon \left(v_j^{(0)} p_j^{(0)} - v_i^{(T)} p_j^{(T)} \right) + \mu \Delta w_{ij}^{(t)}$$

바이어스도 마찬가지다. μ 의 값은 0.5에서 0.9 사이의 값이 자주 사용된다.

5.4절에서 설명한 희소 규제화도 RBM에 적용할 수 있다. 은닉 유닛의 활성도를 억제하여 보다 적은 은닉 유닛으로 입력 샘플을 나타낼 수 있도록 해준다.

자기부호화기와 마찬가지로, 유닛 j 의 평균 활성도를 $\hat{\rho}_j$, 그 목표값을 ρ 로 하여

이 두 분포 사이의 클랙-라이블러 발산(아래 식)에 가중치(β 라 한다)를 곱한 값을 구한다.

$$E_{sparse} = -\rho \log \left(\frac{\rho}{\hat{\rho}_j} \right) - (1 - \rho) \log \left(\frac{1 - \rho}{1 - \hat{\rho}_j} \right)$$

그리고 이를 로그 우도에 더한 뒤에 기울기 방향을 수정한다. 업데이트 값은 다음과 같다.

$$\begin{aligned} \Delta w_{ij} &= \epsilon \left(v_i^{(0)} p_j^{(0)} - v_i^{(T)} p_j^{(T)} - \beta (\partial E_{sparse}) v_i^{(0)} \right) \\ \Delta b_j &= \epsilon \left(p_j^{(0)} - p_j^{(T)} - \beta (\partial E_{sparse}) \right) \end{aligned}$$

이때 ∂E_{sparse} 는 다음과 같이 정의한다.

$$\partial E_{sparse} = \left(-\frac{\rho}{\hat{\rho}_j} + \frac{1 - \rho}{1 - \hat{\rho}_j} \right) f'(p_j^{(0)})$$

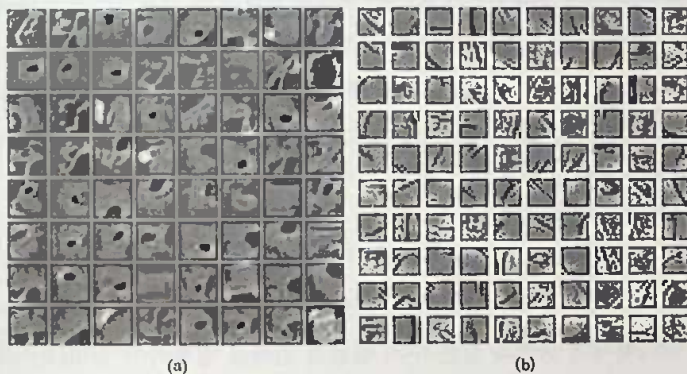


그림 8-5 (a) 베르누이 RBM에 의한 숫자 필기 인식(MNIST)의 학습 결과, (b) 가우시안-베르누이 RBM에 의한 자연 이미지의 패치(CIFAR-10)의 학습 결과

그림 8-5(a)에 베르누이 RBM과 8.7.1절에서 설명한 가우시안 베르누이 RBM에 각각 최소 규제화를 적용하여 숫자 손글씨와 자연 이미지 배치의 집합을 대상으로 학습한 결과를 나타내었다. 그림은 각 은닉 유닛의 가중치를 이미지화한 것이다.

8.6.4 지속적 CD

보통의 CD는 두 번째 항의 기댓값 $\langle \cdot \rangle_{model}$ (에 해당하는 것)을 계산하는 데에 매번 새로운 훈련 샘플 $\mathbf{v}_n$ 을 시작점으로 하여 $\mathbf{h}$ 를 샘플링하고 그 다음에 $\mathbf{v}$ 를 샘플링한다. 다시 말하면, $\mathbf{v}^{(0)}$ 으로 시작하여 $\mathbf{v}^{(0)} \rightarrow \mathbf{h}^{(0)} \rightarrow \mathbf{v}^{(1)}$ 순으로 샘플링을 시행(CD_T 라면 이를 T 번 반복)한다. 문헌에 따라서는 이 기댓값을 $\mathbf{v}_n$ 의 재현 $\mathbf{v}^{(1)}$ 에 관한 통계량이라는 의미로 $\langle \cdot \rangle_{recon}$ 으로 나타내는 경우가 있다. 첫 번째 항의 $\langle \cdot \rangle_{data}$ 를 계산하는 데 사용한 것과 같은 샘플 $\mathbf{v}_n$ 을 사용하는 것이 특징이다.

지속적 CD(Persistent CD)(이하 PCD)는 CD를 개량한 것으로, 보다 깃스 샘플링과 비슷한 방법으로 $\langle \cdot \rangle_{model}$ 을 구한다. PCD에서는 이 기댓값을 구하기 위해 샘플링을 할 때 매번 새로운 훈련 샘플 $\mathbf{v}_n$ 을 시작점으로 하는 것이 아니라, 바로 전에 파라미터를 업데이트했던 당시의 샘플링한 $\mathbf{v}$ 를 시작점으로 $\mathbf{h}$ 와 $\mathbf{v}$ 를 샘플링하고 $\langle \cdot \rangle_{model}$ 을 구한다(CD_T 와 같이 이 샘플링을 여러 번 반복해도 무방하다). 이렇게 PCD로는 $\langle \cdot \rangle_{data}$ 의 평가와 $\langle \cdot \rangle_{model}$ 의 평가가 서로 분리되어 독립적으로 행해진다. 이름의 유래는 $\mathbf{h}$ 와 $\mathbf{v}$ 의 샘플링 연쇄가 파라미터 업데이트 시점마다가 아닌, 지속적으로(persistent하게) 이뤄지기 때문이다.

파라미터의 업데이트를 미니배치 단위로 하는 경우, $\langle \cdot \rangle_{data}$ 의 평가에는 CD와 마찬가지로 미니배치가 포함하는 모든 샘플을 사용하고, $\langle \cdot \rangle_{model}$ 의 평가에는 (이러테른) 미니배치의 샘플과 같은 수의 샘플링 연쇄를 병행하여 실행/유지해 두고 이것들로 평가한다. 이때 샘플링을 파라미터 업데이트 시마다 한 번만 하게 하면 계산 시간은 CD_1 과 같지만, 정밀도는 CD_{10} 에 필적할 만하다고 알려져 있다.

8.7 그 외의 유닛

여기까지 상태가 0과 1 두 가지 값을 갖는 이진 유닛만을 대상으로 하였으나, 0과 1 이외의 다른 상태를 가질 수 있거나 값이 정해지는 방법이 다른 유닛이 몇 가지 있다.

8.7.1 가우시안 유닛

가시층이 이진 유닛으로 이루어진 경우, 입력 데이터가 이진 값을 갖는 것만 대상으로 할 수 있어서 응용 범위가 제한된다. 그래서 연속값을 가질 수 있는 유닛으로 가우시안 유닛(Gaussian unit)이 고안되었다^[16, 29]. 특히, 가시층에 이 유닛을 사용하면 연속값을 갖는 데이터를 적용할 수 있게 된다. RBM의 가시층에 가우시안 유닛을 사용하고 은닉층에는 그대로 이진 유닛을 사용한 것을 가우시안-베르누이 RBM(Gaussian-Bernoulli RBM)이라 부른다. 이 RBM은 다음과 같은 에너지 함수를 갖는다.

$$\Phi(\mathbf{v}, \mathbf{h}, \boldsymbol{\theta}) = \sum_i \frac{(v_i - a_i)^2}{2\sigma_i^2} - \sum_j b_j h_j - \sum_i \sum_j w_{ij} \frac{v_i}{\sigma_i} h_j$$

여기서 σ_i 는 뒤에 설명할 가우시안 분포의 표준편차다(이번 장에서는 로지스틱 함수를 $\sigma(x)$ 로 나타내고 있으나, 이와는 무관하다). 그리고 가시층과 은닉층이 모두 이진 유닛으로 구성된 가장 기본적인 RBM을 앞에서 말한 대로 베르누이 RBM이라 부른다.

가우시안-베르누이 RBM에서 가시 변수 v_i 의 조건부 분포 $p(v_i|\mathbf{h})$ 를 다음과 같이 계산한다.

$$p(v_i|\mathbf{h}) \propto \exp \left\{ -\frac{(v_i - a_i - \sum_j w_{ij} h_j)^2}{2\sigma_i^2} \right\}$$

즉, 이 경우 가시 변수 v_i 는 평균 $a_i + \sum_j w_{ij}h_j$, 분산 σ_i^2 인 가우스 분포를 따른다. 은닉 변수 h_j 의 조건부 분포 $p(h_j|\mathbf{v})$ 는 지금까지의 RBM과 같다. σ_i 는 아직 알지 못하는 파라미터로서 데이터로부터 최대우도추정을 할 수도 있지만, 훈련 데이터의 평균을 0, 분산을 1로 바꾸는 정규화를 거친 후에 $\sigma_i = 1$ 로 고정시키는 것이 일반적이다. 최적화는 베르누이 RBM과 같이 CD로 수행한다. 파라미터의 업데이트 식은 다음과 같다.

$$\Delta w_{ij} = \epsilon \left(v_i^{(0)} p_j^{(0)} - p_i^{(T)} p_j^{(T)} \right) \quad \text{식 8-20a}$$

$$\Delta a_i = \epsilon \left(v_i^{(0)} - p_i^{(T)} \right) \quad \text{식 8-20b}$$

$$\Delta b_j = \epsilon \left(p_j^{(0)} - p_j^{(T)} \right) \quad \text{식 8-20c}$$

그림 8-5(b)는 가우시안-베르누이 RBM을 사용하여 자연 이미지의 패치를 학습한 결과다*.

그리고 가시층, 은닉층 모두 가우시안 유닛으로 구성하는 것은 이론적으로는 가능하지만, 학습시키기가 어려운 관계로 일반적으로는 사용되지 않는다.

8.7.2 ReLU

유닛 하나로 보다 많은 정보를 표현하기 위해 이항유닛(binomial unit)⁶⁹⁾이 고안되었다⁶⁹⁾. 이항유닛은 동일한 파라미터를 갖는 여러 개의 이진 유닛으로 표현된다. 이들 이진 유닛은 동일한 가중치와 바이어스를 공유하며, 따라서 같은 입력을 받는다. 그 때문에 상태에 대한 조건부확률 p 도 완전히 같게 되지만, 유닛

* 그레이 스케일로 변환한 CIFAR-10 데이터 세트로부터 무작위로 추출한 10×10 픽셀의 패치 50,000장을 훈련 샘플로 하고, 5.5절의 영위상(ZCA) 백색화란 적용한 후에 RBM으로 학습하였다. RBM의 은닉 유닛 수는 100, 희소 규제화를 적용하였으며($\rho = 0.01$, $\beta = 1.0$), 모멘텀 $\mu = 0.5$, 가중치 감쇠 $\lambda = 0.001$ 을 사용하였다. 최적화는 CD_1 로, 가중치값 $\sigma = 0.01$ 의 가우스 분포로 초기화한 후, 학습률 $\epsilon = 0.001$ 로 1샘플 단위 업데이트를 2,000,000회 실시하였다.

각각의 상태는 확률적으로 결정된다. 이항유닛의 상태는 이들 이진 유닛의 상태의 합으로 정의된다. 이항유닛 하나를 K 개의 이진 유닛으로 나타낼 때, 그 상태에 대한 기댓값은 Kp , 분산은 $Kp(1-p)$ 가 된다.

이 이항유닛을 확장하여 더 높은 표현 능력을 갖게 한 것이 정규화 선형유닛(Rectified Linear Unit)이다.^[52](ReLU라는 약자로 자주 쓰인다). 먼저, 위와 마찬가지로 동일한 가중치와 바이어스를 갖는 이진 유닛이 있다고 하자. 단, 이번에는 유닛의 개수 제한이 없으며, 유닛의 바이어스에 서로 다른 오프셋 $-0.5, -1.5, -2.5, \dots$ 을 더한다. 새로운 유닛의 상태에 대한 초깃값은 각 이진 유닛이 상태 1을 갖는 조건부확률의 합 $\sum_i \sigma(x - i + 0.5)$ 로 나타낼 수 있다($\sigma(x)$ 는 로지스틱 함수로, x 는 각 이진 유닛이 받아들이는 총 입력이다). 이 기댓값은 다음과 같이 근사할 수 있으며,

$$\sum_{i=1}^{\infty} \sigma(x - i + 0.5) \approx \log(1 + e^x)$$

식 8-21

우변은 램프 함수에 가까운 형태를 갖는다. 기댓값을 이렇게 근사할 수 있으므로 이 유닛의 상태는 총 입력 x 에 대해 결정되는 $\log(1 + e^x)$ 에 노이즈 값을 더한 뒤 정수화한 것이라 볼 수 있다. 노이즈 값을 더하는 이유는 식 8-21이 본래의 기댓값이며, 이 값이 실제로 실현되는 값도 일정 수준의 분산을 따라 흩어져 분포할 것이기 때문이다. 그리고 그 분산은 $\sigma(x)$ 가 된다.

이 유닛의 상태를 정의 그대로 계산해서 결정하는 것은 계산 비용이 크다. 그래서 상태가 정숫값을 갖도록 제약을 조금 풀고, 식 8-21의 우변을 램프 함수로 근사하면 이 유닛의 상태는 근사적으로 $\max(0, x + N(0, \sigma(x)))$ 로 주어진다. 이때 $N(0, V)$ 는 평균 0, 분산 V 인 가우스 분포를 따르는 확률분포다. 문헌^[52]에서는 이것을 노이즈 첨가 램프 함수라고 부른다. 또, 이 유닛을 RBM의 은닉층에 사용하면 이진 유닛보다 영상 인식 테스트에서 좋은 결과를 얻을 수 있다는 사실이 보고되어 있다. 앞먹임 신경망에서도 시그모이드 함수보다 램프 함수를 활성화 함수로 사용했을 때 좋은 결과를 내는 사례가 많다.

8.8 딥 빌리프 네트워크

딥 빌리프 네트워크(Deep Belief Network)(이하 DBN)는 그림 8-6처럼 유닛을 층 모양으로 늘어놓아 최상층의 층간만 무향 에지(undirected edge)로 연결하고, 그 외의 층간은 아래를 향한 유향 에지(directed edge)로 연결한 구조를 갖는다. 정보는 위에서 아래로 한 방향으로 전달되어, 맨 아래층인 가시층의 상태 v 가 결정되는 구조다. 유향 에지를 포함하므로 볼츠만 머신이 아니며, 별도의 이름을 갖게 되었다. RBM에 비하여 은닉층의 수를 늘린 것으로, 더 복잡한 데이터 생성 구조를 모형화할 수 있으리라 기대되고 있다. 그리고 RBM이나 자기부호화기처럼 입력 데이터를 가시층에 설정했을 때 결정되는 맨 위층의 상태를 그 데이터의 표현으로 간주하여 클래스 분류 등의 추론을 수행한다.

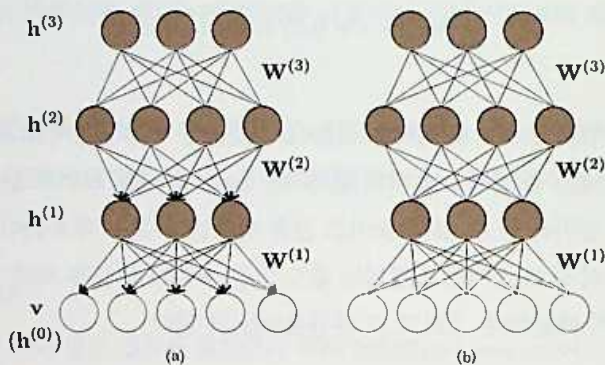


그림 8-6 (a) 딥 빌리프 네트워크(DBN), (b) 딥 볼츠만 머신(DBM)

최하위층인 가시층으로부터 위로 올라가며 층 번호를 $l = 0, 1, \dots, L$ 로 매기고, 각 층의 유닛의 상태를 $h^{(l)}$ 로 표기하면 DBN의 모든 유닛의 결합 확률분포는 다음에 따라 주어진다.

$$p(h^{(0)}, \dots, h^{(L)}; \theta) = \left(\prod_{l=0}^{L-2} p(h^{(l)} | h^{(l+1)}) \right) p(h^{(L-1)}, h^{(L)})$$

$l = 0$ 은 가시층이며, $\mathbf{h}^{(0)} = \mathbf{v}$ 로 한다. $l = 0$ 으로부터 $l = L - 2$ 까지의 층은 인접하는 위층과 유향 에지로 연결되어 있고, 그 사이의 조건부 분포는 다음으로 주어진다.

$$p(\mathbf{h}_i^{(l)} = 1 | \mathbf{h}^{(l+1)}) = \sigma(b_i^{(l)} + \sum_j w_{ij}^{(l+1)} h_j^{(l+1)})$$

가장 위층은 RBM과 구조가 같으므로 $l = L - 1$ 과 $l = L$ 층의 결합분포 $p(\mathbf{h}^{(L-1)}, \mathbf{h}^{(L)})$ 은 RBM의 분포함수와 같다(식 8-13의 $\mathbf{v}, \mathbf{h}$ 를 $\mathbf{h}^{(L-1)}, \mathbf{h}^{(L)}$ 로 치환한 식이 된다).

DBN은 은닉 변수 간에 결합이 존재하며 층수도 더 많으므로, 은닉 변수의 조건부 분포 $p(\mathbf{h} | \mathbf{v})$ 를 RBM처럼 쉽게 계산할 수는 없다. 그래서 RBM의 층간 계산을 모방하여 서로 인접한 층 사이의 조건부 분포를 다음과 같이 근사한다.

$$p(h_j^{(l)} = 1 | \mathbf{h}^{(l-1)}) = \sigma(b_j^{(l)} + \sum_i w_{ij}^{(l-1)} h_i^{(l-1)})$$

이를 이용하여 $\mathbf{h}^{(0)} = \mathbf{v}$ 로 하고 $l = 1, \dots, L$ 의 순서대로 상태를 계산한다. 다시 말해, 심층 앞먹임 신경망의 순전파 계산과 같은 방식으로 계산하는 것이 된다. 실제로, 앞으로 설명할 방법으로 최적화를 마친 후에 DBN은 그림 8-7처럼 앞먹임 신경망으로 전환하여 이용하게 된다^[29]. 이때 랜덤하게 가중치를 초기화한 층을 가장 위층에 한 층 이상 추가하여(분류 문제라면 소프트맥스 층이 필요), 그 이외 층은 DBN을 학습할 때 얻은 가중치를 초깃값으로 삼아 역전파법에 기초한 경사 하강법으로 지도 학습시킨다.

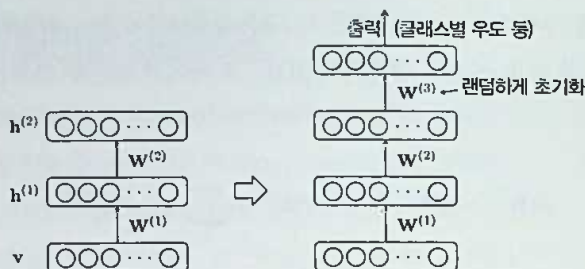


그림 8-7 앞먹임 신경망으로 전환되는 DBN. 전환한 뒤에 역전파법과 경사 하강법으로 지도 학습을 실시한다.

DBN은 5.6절에서 설명한 다층 앞먹임 신경망과 같은 방식으로 사전훈련을 해서 학습시킨다^{[28]*}. 주어진 데이터 $v^1, \dots, v_n$ 에 대해 먼저 가장 아래층($l = 0$ 과 $l = 1$ 사이의 파라미터를 $h^{(0)} (= v)$ 과 $h^{(1)}$ 가 별개의 RBM의 가시 변수와 은닉 변수인 것처럼 최적화한다. 그런 다음, 이렇게 구한 파라미터를 써서 데이터 $v_1, \dots, v_n$ 의 샘플 각각에 대해 층 $l = 1$ 의 상태 $h_1^{(1)}, \dots, h_n^{(1)}$ 을 생성한다. 그리고 한 층 위층($l = 1$ 과 2 사이의 파라미터를 또 다른 별개의 RBM의 가시 변수와 은닉 변수로 보고 앞에서 생성한 $h_1^{(1)}, \dots, h_n^{(1)}$ 는 데이터처럼(가시층의 상태) 사용해서 최적화한다.

8.9 딥 볼츠만 머신

딥 볼츠만 머신(Deep Boltzmann Machine)(이하 DBM)^[60]은 DBN의 유향 에지를 모두 무향 에지로 치환한 볼츠만 머신으로, RBM을 쌓아올린 것으로 볼 수 있다. DBM은 DBN과 마찬가지로 최하층이 가시층이다. 은닉층을 두 개 갖는 RBM을 예로 들어 에너지 함수를 식으로 쓰면 다음과 같다.

* 연구된 순서는 DBN의 사전훈련이 먼저다. 다층 신경망의 사전훈련은 당초에는 DBN의 학습 방법으로 발표되었다.

$$\Phi(\mathbf{v}, \mathbf{h}^{(1)}, \mathbf{h}^{(2)}) = - \sum_i a_i v_i - \sum_j b_j^{(1)} h_j^{(1)} - \sum_i \sum_j w_{ij}^{(1)} v_i h_j^{(1)} \\ - \sum_k b_k^{(2)} h_k^{(2)} - \sum_j \sum_k w_{jk}^{(2)} h_j^{(1)} h_k^{(2)}$$

DBM에서는 은닉 유닛끼리의 결합이 있으므로 RBM과 같이 각 유닛의 상태에 대한 조건부 분포를 계산할 수는 없다. 그래서 DBM에서는 평균장 근사(mean field approximation)라는 방법을 사용하여 각 유닛의 상태를 계산한다. 평균장 근사란, DBM처럼 변수 사이에 의존 관계가 없는 그래프 모델(graph model)에 대해 이들 변수가 서로 독립이라고 가정하고 각 변수의 주변분포를 근사적으로 계산하는 방법이다.

두 개의 은닉층을 갖는 DBM을 예로 들면, 조건부 분포 $p(\mathbf{h}^{(1)}, \mathbf{h}^{(2)}|\mathbf{v})$ 를 다음과 같은 분포로 근사한다.

$$q(\mathbf{h}^{(1)}, \mathbf{h}^{(2)}|\mathbf{v}) = \prod_j q(h_j^{(1)}|\mathbf{v}) \prod_k q(h_k^{(2)}|\mathbf{v})$$

이 근사분포는 다른 층의 은닉 변수($h_j^{(1)}$ 와 $h_k^{(2)}$)가 (사실은 그렇지 않지만) 서로 독립임을 나타낸다. 이러한 형태로 나타낼 수 있는 분포는 무수히 많지만, 평균장 근사는 그중 가장 진짜 $p(\mathbf{h}^{(1)}, \mathbf{h}^{(2)}|\mathbf{v})$ 에 가까운 분포를 최적화 계산을 통해 찾아낸다. 이때 두 분포의 유사도는 쿨백-라이블러 발산으로 측정한다.

지금 논하고 있는 은닉 변수는 이진 변수이므로, 각 은닉 변수의 분포 ($q(h_j^{(1)} = 1|\mathbf{v})$ 등)는 변수의 값이 1일 확률 $\mu_j^{(1)} = q(h_j^{(1)} = 1|\mathbf{v})$ 로 완전히 나타낼 수 있다($q(h_j^{(1)} = 0|\mathbf{v}) = 1 - \mu_j^{(1)}$ 이므로). 따라서 $q(\mathbf{h}^{(1)}, \mathbf{h}^{(2)}|\mathbf{v})$ 는 은닉층에 속하는 각각의 유닛에 의해 정의되는 $\mu_j^{(l)}$ 를 조절 가능한 파라미터로 갖는다고 볼 수 있다. 그에 따라 이야기는 다시 이 근사가 가장 정확해지도록 이들 파라미터를 결정하는 것으로 귀결된다. 결과만 먼저 밝히자면, 고정점 방정식이라 불리는 다음 식을 반복하여 계산하면서 $\mu_j^{(l)}$ 를 서서히 업데이트한다.

$$\mu_j^{(l)} = \sigma \left(b_j^{(l)} + \sum_i w_{ij}^{(l)} \mu_i^{(l-1)} + \sum_k w_{jk}^{(l+1)} \mu_k^{(l+1)} \right)$$

구체적으로, 각 층 각 유닛의 $\{\mu_j^{(l)}\}$ 의 현재 추정값으로 우변을 계산한 뒤, 구한 값을 새로운 $\mu_j^{(l)}$ 로 삼는 과정을 이 값이 수렴할 때까지 반복한다. 얻어진 $\mu_j^{(l)}$ 는 $h_j^{(l)}$ 가 (다른 은닉 변수의 상태와는 관계없이) 1이 될 확률이다.

DBM의 학습(최적화)은 DBN과 마찬가지로 층마다 RBM의 학습 방법을 반복하는 사전훈련, 그리고 사전훈련 후의 미세조정 두 단계로 구성된다. 이때 문헌^[60]에 따르면, 맨 아래층(가시층) 및 최상층의 유닛은 지금까지와 마찬가지로이지만, 그 외의 은닉층 l 에서는 조건부 분포가 약간 수정된다. 구체적인 방법은, 예를 들어 층 $l = 1$ 인 경우에 가시층의 조건부 분포는 다음이 되어 지금까지와 같지만,

$$p(v_i = 1 | \mathbf{h}^{(l)}) = \sigma \left(a_i + \sum_j w_{ij}^{(l)} h_j^{(l)} \right)$$

중간층 l 의 조건부 분포는 다음과 같이 달라지게 된다.

$$p(h_j^{(l)} = 1 | \mathbf{h}^{(l-1)}) = \sigma \left(b_j^{(l)} + 2 \sum_j w_{ij}^{(l)} h_i^{(l-1)} \right)$$

이 차이는 DBM에서는 층 l 이 $l + 1$ 과 $l - 1$ 양쪽에서 입력을 받는다는 조건을 반영한 것이다.

이렇게 결정한 파라미터를 초깃값으로 한 뒤, DBM에 대한 미세조정을 거친다.

8.4.2절에서 다루었던 은닉 변수를 포함하는 볼츠만 머신의 일반적인 최적화 방법, 다시 말해 식 8-12의 기울기 방향으로 파라미터를 수정하는 경사 하강법을 통해 계산한다. $\langle z_i z_j \rangle_{data}$ 를 계산하는 데 필요한 조건부 분포 $p(\mathbf{h} | \mathbf{v})$ 는 위와 같이 평균장 근사로 계산한다. 샘플 하나의 기울기를 계산할 때 $\mu_j^{(l)} = p(h_j^{(l)} = 1 | \mathbf{v})$

로 표기하고, 각 항은 $\langle z_i z_j \rangle_{data} = -v_i \mu_j^{(l)}$, $\langle z_i \rangle_{data} = -v_i$, $\langle z_j \rangle_{data} = -\mu_j^{(l)}$ 이 된다. 한편, $\langle z_i z_j \rangle_{model}$ 은 RBM과 마찬가지로 CD를 통해 계산한다. 이때 CD의 반복 횟수는 단독 RBM보다는 늘려야 한다고 알려져 있다.

또, DBN과 마찬가지로 이렇게 학습한 DBM은 앞먹임 신경망으로 전환할 수 있다. 방법도 DBN과 기본적인 열개는 같으나 문헌^[60]에서는 입력을 그림 8-8과 같은 형태로 확장할 것을 제안하고 있다. 원래 DBM의 하위은닉층 $h^{(1)}$ 은 입력층 v 및 상위은닉층 $h^{(2)}$ 양쪽으로부터 정보를 받아들인다. 앞먹임 신경망으로 전환된 후의 해당 층도 마찬가지로 두 가지의 입력을 받게 하려는 의도다.

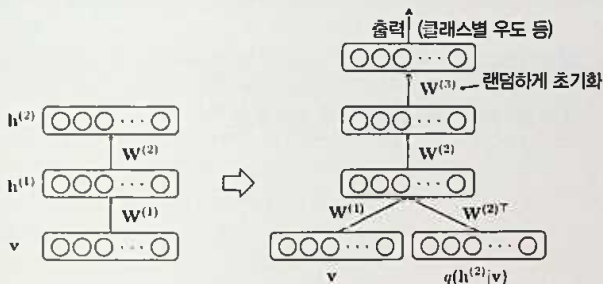


그림 8-8 앞먹임 신경망으로 전환되는 DBM. 은닉층이 위아래층 양쪽에서 입력을 받는 것을 반영하여 입력을 확장하는 경우가 있다.

8.10 성능 비교

숫자 손글씨 데이터 세트 MNIST를 대상으로 위의 방법대로 DBN과 DBM을 학습시킨 뒤, 앞먹임 신경망으로 전환하여 확률적 경사 하강법으로 미세 조정된 결과로, 숫자인식 성능을 측정하여 표 8-1에 나타내었다. DBN 및 DBM은 2층 구조의 신경망으로, 입력층은 $28 \times 28 = 784$ 아래층부터 순서대로 1,000개, 500개의 유닛을 갖는 두 개의 은닉층으로 구성되었다. 전환한 뒤의 앞먹임

신경망은 여기에 10개의 유닛으로 구성된 출력층을 추가한다. DBM에서는 그림 8-8과 같이 입력을 확장한 구조를 갖도록 한다. 표에서 보듯이, 앞먹임 신경망의 가중치를 랜덤하게 초기화했던 경우와 비교하여, DBN이나 DBM으로 비지도 학습을 수행하여 얻은 파라미터를 초깃값으로 사용하는 방법이 더 좋은 성능을 보였으며, DBN과 DBM이 이 태스크에 유용하다는 것을 보여준다.

표 8-1 같은 구조를 갖는 2층 구조의 앞먹임 신경망을 서로 다른 방법으로 훈련시킨 후 MNIST 필기인식 데이터로 성능을 측정한 결과

학습 방법 및 사용 기법	에러율(%)
가중치의 무작위 초기화 ^[60]	1.6
커널 SVM ^[12]	1.4
DBN을 이용한 가중치 초기화 ^[28]	1.2
DBN을 이용한 가중치 초기화 + 그림 8-8과 같은 입력 확장 ^[60]	0.95

진솔한 서평을 올리주세요

이 책이나 이미 읽은 제이펍의 다른 책이 있다면, 책의 장단점을 잘 보여주는 솔직한 서평을 올려주세요. 매월 다섯 분을 선별하여 원하시는 제이펍 도서 1부씩을 선물해드리겠습니다.

■ 서평 이벤트 참여 방법

- 제이펍의 책을 읽고 자신의 블로그나 인터넷 서점에 서평을 올린다.
- 서평이 작성된 URL을 적어 아래의 계정으로 메일을 보낸다.
review.jpub@gmail.com

■ 서평 당선자 발표

매월 첫 주 제이펍 홈페이지(www.jpub.kr) 및 페이스북(www.facebook.com/jeipub)에 공지하고 당선된 분에게는 개별 연락을 드리겠습니다.



참고문헌

- 1 P. Baldi and K. Hornik. Neural networks and principal component analysis: Learning from examples without local minima. *Neural Networks*, 2:53-58, 1989.
- 2 A. Bell and T. Sejnowski. The "independent components" of natural scenes are edge filters. *Vision Research*, 37(23):3327-3338, 1997.
- 3 Y. Bengio. Learning deep architectures for ai. *Foundations and Trends in Machine Learning*, 2(1):1-127, 2009.
- 4 Y. Bengio, P. Lamblin, D. Popovici and H. Larochelle. Greedy layer-wise training of deep networks. In *Proc. NIPS*, 2006.
- 5 P. Berkes and L. Wiskott. Slow feature analysis yields a rich repertoire of complex cell properties. *Journal of Vision*, 5(6):579-602, 2005.
- 6 Y-Lan Boureau, F. Bach, Y. LeCun and J. Ponce. Learning mid-level features for recognition. In *Pro. CVPR*, 2010.
- 7 Y-Lan Boureau, J. Ponce and Y. LeCun. A theoretical analysis of feature pooling in visual recognition. In *Pro. ICML*, 2010.
- 8 C. F. Cadieu, H. Ha, D. L. K. Yamins, N. Pinto, D. Ardila, E. A. Solomon, N. J. Majaj and J. J. DiCarlo. Deep Neural Networks Rival the Representation of Primate IT Cortex for Core Visual Object Recognition. *arXiv*, 2014.
- 9 D. C. Cireşan, U. Meier, J. Masci, L. M. Gambardella and J. Schmidhuber. Flexible, high performance convolutional neural networks for image classification. In *Pro. IJCAI*, 2011.
- 10 D. C. Cireşan, U. Meier and J. Schmidhuber. Multicolumn deep neural networks for image classification. In *Pro. CUPR*, 2012.
- 11 A. Coates, H. Lee, A. Y. Ng. An analysis of single-layer networks in unsupervised feature learning. In *Pro. AISTATS*, 2011.

- [12] D. Decoste and Schoölkopf. Training invariant support vector machines. *Machine Learning*, 46(1/3):161-190, 2002.
- [13] J. Duchi, E. Hazan and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121-2159, 2011.
- [14] J. L. Elman. Finding structure in time. *Cognitive Science*, 14:179-221, 1990.
- [15] C. Farabet, C. Couprie, L. Najman and Y. LeCun. Learning hierarchical features for scene labeling. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2013.
- [16] Y. Freund and D. Haussler. Unsupervised learning of distributions on binary vectors using two layer networks. Technical Reports, Santa Cruz, CA, USA, 1994.
- [17] K. Fukushima and S. Miyake. Neocognitron: A new algorithm for pattern recognition tolerant of deformations and shifts in positions. *Pattern Recognition*, 15:455-469, 1982.
- [18] X. Glorot, A. Bordes and Y. Bengio. Deep sparse rectifier neural networks. In *Pro. AISTAT*, 2011.
- [19] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio. Maxout networks. *arXiv*, 2013.
- [20] A. Graves. *Supervised sequence labelling with recurrent neural networks*. Ph.D thesis, Technische Universität München, 2008.
- [21] A. Graves, S. Fernández, F. Gomez and J. Schmidhuber. Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks. In *Pro. ICML*, 2006.
- [22] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke and J. Schmidhuber. A novel connectionist system for unconstrained handwriting recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(5):855-868, 2009.
- [23] B. Hammer. On the approximation capability of recurrent neural networks. *Neurocomputing*, 31(1-4):107-123, 2000.
- [24] D. Hebb. *The Organization of Behavior*. Wiley, 1949.
- [25] G. E. Hinton. Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14(8):1771-800, 2002.
- [26] G. E. Hinton. A Practical Guide to Training Restricted Boltzmann Machines. Technical Reports, 2010.
- [27] G. E. Hinton, L. Deng, D. Yu, G. E. Dahl, A. Mohamed, N. Jaitly, A. Senior and V. Vanhoucke. Deep Neural Networks for Acoustic Modeling in Speech Recognition. *IEEE Signal Processing Magazine*, 29, 2012.
- [28] G. E. Hinton, S. Osindero and Y. The. A fast learning algorithm for deep belief nets. *Neural Computation*, 18:1527-1544, 2006.
- [29] G. E. Hinton and R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313:504-507, 2006.
- [30] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735-1780, 1997.

- [31] C. Holk. Neural networks as cybernetic systems, 2nd and revised edition, 2006.
- [32] K. Hornik, M. Stinchcombe and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2:359-366, 1989.
- [33] D. H. Hubel and T. N. Wiesel. Receptive fields, binocular interactions and functional architecture in the cat's visual cortex. *Journal of Physiology*, 160:106-154, 1962.
- [34] D. H. Hubel and T. N. Wiesel. Receptive fields and functional architecture of monkey striate cortex. *The Journal of physiology*, 195(1):215-243, 1968.
- [35] 伊里正夫, 藤野和建. 数値計算の常識. 共立出版, 1985.
- [36] M. Ito and H. Komatsu. Representation of angles embedded within contour stimuli in area v2 of macaque monkeys. *The Journal of Neuroscience*, 24(13):3313-3324, 2004.
- [37] H. Jaeger. The "echo state" approach to analysing and training recurrent neural networks. Technical Report GMF Report 148, German National Research Center for Information Technology, 2001.
- [38] K. Jarrette, K. Kavukcuoglu, M. Ranzato and Y. LeCun. What is the best multi-stage architecture for object recognition. In *Pro. ICCV*, 2009.
- [39] A. Krizhevsky, I. Sutskever and G. E. Hinton. ImageNet Classification with Deep Convolution Neural Networks. In *Pro. NIPS*, 2012.
- [40] K. J. Lang, A. H. Waibel and G. E. Hinton. A time-delay neural network architecture for isolated word recognition. *Neural Networks*, 3(1):23-43, 1990.
- [41] Q. V. Le, M. Ranzato, R. Monga, M. Devin, K. Chen, G. S. Corrado, J. Dean and A. Y. Ng. Building high-level features using large scale unsupervised learning. In *Pro. ICML*, 2012.
- [42] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541-551, 1989.
- [43] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. Gradient-based learning applied to document recognition. *Proceeding of the IEEE*, 86(11):2278-2324, 1998.
- [44] Y. LeCun, L. Bottou, G. Orr and K. Muller. Efficient backprop. In *Neural Networks: Tricks of the trade*. Springer, 1998.
- [45] H. Lee, C. Ekanadham and A. Y. Ng. Sparse deep belief net model for visual area v2. In *Pro. NIPS*, 2008.
- [46] H. Lee, R. Grosse, R. Ranganath and A. Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *Pro. ICML*, 2009.
- [47] S. Lyu and E. P. Simoncelli. Nonlinear image representation using divisive normalization. In *Pro. CVPR*, 2008.
- [48] W. S. McCulloch and W. Pitts. A logical calculus of ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4):115-133, 1943.
- [49] T. Mikolov, S. Kombrink, L. Burget, J. Cernocky and S. Khudanpur. Extensions of recurrent neural network based language model. In *Pro. ICASP*, 2011.

- [50] T. Mikolov, M. Karafiat, J. Cernocky and S. Khudanpur. Recurrent neural network based language model. In *Pro. Interspeech*, 2010.
- [51] K. P. Murphy. *Machine Learning: A probabilistic perspective*. MIT Press, 2012.
- [52] V. Nair and G. E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Pro. ICML*, 2010.
- [53] B. A. Olhausen and D. J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381:607-609, 1996.
- [54] N. Pinto, D. D. Cox and J. J. DiCarlo. Why is real-world visual object recognition hard? *PLoS computational biology*, 4(1):0151-0156, 2008.
- [55] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery. *Numerical Recipes 3rd Edition: The Art of Scientific Computing*, Cambridge University Press, 2007.
- [56] A. J. Robinson and F. Fallside. The utility driven dynamic error propagation network. Technical Report CUED/F-INFENG/TR.1, Cambridge University Engineering Department, 1987.
- [57] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386-408, 1958.
- [58] N. L. Roux and Y. Bengio. Representational power of restricted Boltzmann machines and deep belief networks. *Neural Computation*, 20(6):1631-1649, 2008.
- [59] D. E. Rumelhart and J. McClelland. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*. MIT Press, 1986.
- [60] R. Salakhutdinov and G. E. Hinton. Deep Boltzmann Machines. In *Pro. AISTATS*, 448-455, 2009.
- [61] A. M. Saxe, P. W. Koh, Z. Chen, M. Bhand, B. Suresh and A. Y. Ng. On random weights and unsupervised feature learning. In *Pro. ICML*, 2010.
- [62] M. Schuster and K. K. Paliwal. Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing*, 45:2673-2681, 1997.
- [63] O. Schwartz and E. P. Simoncelli. Natural signal statistics and sensory gain control. *Nature Neuroscience*, 4(8):819-825, 2001.
- [64] P. Y. Simard, D. Steinkraus and J. Platt. Best practice for convolutional neural networks applied to visual document analysis. In *Pro. ICDAR*, 2003.
- [65] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv*, 2014.
- [66] N. Srebro and A. Shraibman. Rank, trace-norm and max-norm. In *Pro. COLT 2005*, 545-560, Springer, 2005.
- [67] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929-1958, 2014.

- 68 R. Szeliski. *Computer Vision: Algorithms and Applications*, Springer, 2010.
- 69 Y. Teh and G. E. Hinton. Rate-coded restricted Boltzmann machines for face recognition. In *Pro. NIPS*, 2000.
- 70 P. C. Teo and D. J. Heeger. Perceptual image distortion. In *Proc. SPIE, Human Vision, Visual Processing and Digital Display V*, 2179:127-141, 1994.
- 71 P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio and P. A. Manzagol. Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion. *Journal of Machine Learning Research*, 11:3371-3408, 2010.
- 72 L. Wan, M. Zeiler and S. Zhang. Regularization of neural networks using dropconnect. In *Pro. ICML*, 2013.
- 73 P. Werbos. Backpropagation through time: What it does and how to do it. *Proceedings of the IEEE*, 78(10):1550-1560, 1990.
- 74 R. J. Williams and D. Zisner. Gradient-based learning algorithms for recurrent networks and their computational complexity. In U. Chauvin and D. E. Rumelhart, editors, *Back-propagation: Theory, Architectures and Applications*, 433-486. Lawrence Erlbaum Publishers, 1995.
- 75 D. L. K. Yamins, H. Hong, C. F. Cadieu, E. A. Solomon, D. Seibert, and J. J. DiCarlo. Performance-optimized hierarchical models predict neural responses in higher visual cortex. In *Proc. the National Academy of Sciences of the United State of America*, 2014.
- 76 M. D. Zeiler and R. Fergus. Stochastic pooling for regularization of deep convolutional neural networks. In *Proc. ICLR*, 2013.
- 77 H. G. Zimmermann, R. Grothmann, A. M. Schaefer and C. Tietz. Identification and forecasting of large dynamical systems by dynamical consistent neural networks. In S. Haykin, J. Principe and J. Sejnowski and T. McWhirter, editors, *New directions in statistical signal processing: From Systems to Brain*, 203-242, MIT Press, 2006.



찾아보기

A	
activation function	13
AdaGrad	47
autoencoder	5, 74
average pooling	113

B	
backpropagation	3, 55
BackPropagation Through Time(BPDT)	144
batch learning	33
Bernoulli distribution	175
Bernoulli RBM	175
bias	13
bidirectional RNN	141
binary unit	164
binomial unit	184
Boltzmann distribution	165

C	
CD	178
code	74
complex cell	101
Connectionist Temporal Classification(CTC)	154
Contrastive Divergence	178
Convolutional Neural Network(CNN)	4, 101
convolution layer	104
cross entropy	27

D	
data augmentation	45
DBM	188
DBN	186
decode	74
deep autoencoder	94
Deep Belief Network(DBN)	4, 186
Deep Boltzmann Machine	188
deep learning	7
delta	61
denoising autoencoder	95
difference approximation	68
divisive normalization	116
drop-out	40

E	
early stopping	37
empirical distribution	167
encode	74
energy function	165
error function	21

F	
feature	76
feedforward neural network	13
filter	105
forward backward method	156

fully-connected	101
fully-connected layer	104

G

Gaussian-Bernoulli RBM	183
Gaussian mixture model	153
Gaussian unit	183
generalization error	36
generate	163
Gibbs distribution	165
Gibbs sampling	168
gradient	32
gradient descent method	32

H

hidden layer	18
Hidden Markov Model	153
HMM	153

I

input layer	18
internal layer	18

L

learning curve	37
learning rate	32
likelihood	24
likelihood function	164
Local Contrast Normalization(LCN)	104, 115
logistic function	15
logistic sigmoid function	15
log-likelihood function	166
Long Short-Term Memory(LSTM)	148
Lp pooling	113

M

map	110
maximum likelihood estimation	24, 163
maxout	18
max pooling	112
mean field approximation	189

minibatch	35
model averaging	45
momentum	48
multi-layer perceptron	13

N

negative log likelihood	27
normalization of data	43

O

object category recognition	103
on-center	90
output layer	18
overcomplete	81
overfitting	37
overlearning	37

P

partition function	165
Persistent CD(PCD)	182
phoneme	138
pooling layer	104
pre-training	5
Principal Component Analysis(PCA)	79, 89

R

RealTime Recurrent Learning(RTRL)	144
receptive field	101
rectified linear function	16
Rectified Linear Unit	185
Recurrent Neural Network(RNN)	7, 138
regression	22
regularization	38
Restricted Boltzmann Machine(RBM)	5, 173

S

sigmoid function	16
simple cell	101
softmax function	26
sparse autoencoder	81
sparse coding	6

stacked autoencoder	93
standardization of data	43
stochastic gradient descent	33
stride	108
subtractive normalization	116

T

test data	37
test error	37
training data	21
training error	36
training samples	21

V

vanishing gradient problem	69
visual cortex	101

W

weight	13
weight decay	39
weight sharing	76, 112
weight tying	76, 112
whitening	87

Z

zero-padding	107
Zero-phase Component Analysis(ZCA)	89
Zero-phase Whitening	89

ㄱ

가우시안-베르누이 RBM	183
가우시안 유닛	183
가우시안 혼합 모델	153
가중치	13
가중치 감쇠	39
가중치 공유	76, 112
가중치 상한	39
감산 정규화	116
감수 영역	101
경사 하강법	32
강학습분포	167

과완비	81
과잉적합	37
과적합	37
과학습	37
교차 엔트로피	27
국소 콘트라스트 정규화	104, 115

규제화	38
기울기	32
기울기 소실 문제	69
깁스 분포	165
깁스 샘플링	168

C

다층 퍼셉트론	13
단순 세포	101
대조적 발산	178
데이터 정규화	43
데이터 표준화	43
데이터 확장	45
델타	61
드롭아웃	40
디노이징 자기부호화기	95
딤 러닝	7
딤 볼츠만 머신	188
딤 빌리프 네트워크	4, 186

ㄹ

램프 함수	16
로그 우도함수	166
로지스틱 시그모이드 함수	15

ㅁ

맥스아웃	18
맵	110
모델 평균	45
모멘텀	48
물체 카테고리 인식	103
미니배치	35

ㅂ

바이어스	13
배치학습	33

백색화	87
베르누이 RBM	175
베르누이 분포	175
복잡 세포	101
복호화	74
볼츠만 분포	165
부호	74
부호화	74
분배함수	165

人

사전훈련	5
생성	163
소프트맥스 함수	26
스트라이드	108
시각 피질	101
시그모이드 함수	16
심층 자기부호화기	94
심층학습	7

○

앞먹임 신경망	13
양방향 RNN	141
에너지 함수	165
역전파법	3, 55
영위상 백색화	89
영위상 성분 분석	89
오차함수	21
온센터	90
우도	24
우도함수	164
은닉 마르코프 모델	153
은닉층	18
음소	138
이진 유닛	164
이항유닛	184
일반화 오차	36
입력층	18

π

자기부호화기	5, 74
장·단기기억	148

재귀 신경망	7, 138
전결합	101
전결합층	104
진진후진법	156
정규화 선형유닛	185
제로 매딩	107
재산 정규화	116
제약 볼츠만 머신	5, 173
조기종료	37
조기중단	37
주성분 분석	79, 89
중간층	18
지속적 CD	182

夫

차분근사	68
최대우도법	24
최대우도추정	163
최대 풀링	112
출력층	18

ㄱ

커넥서니스트 시계열 분류	154
---------------	-----

ㅊ

테스트 데이터	37
테스트 오차	37
특징	76

ㅍ

평균장 근사	189
평균 풀링	113
풀링층	104
필터	105

ㅎ

학습곡선	37
학습률	32
합성곱 신경망	4, 101
합성곱층	104

확률적 경사 하강법	33	훈련 샘플	21
활성화 함수	13	훈련 오차	36
회귀	22	희소 부호화	6
훈련 데이터	21	희소 자기부호화기	81

